

Cardiac Computational Modeling

From Ion Channels to Tissue Solvers

A Visual Textbook for Engine V5.4

Cardiac Computational Modeling Research

February 2026

PART I

Single Cell Dynamics

From the squid giant axon to the human ventricular myocyte — how ion channels generate the electrical signals that drive every heartbeat.

The Hodgkin–Huxley Revolution

In 1952, Alan Hodgkin and Andrew Huxley did something no one thought possible: they wrote down a mathematical model of the nerve impulse that was so precise it predicted experiments that hadn't been done yet. Every cardiac cell model used today — from the simplest to the most complex — is a direct descendant of their work.

1.1 The Membrane as a Circuit

A cell membrane is, at its core, a thin lipid bilayer — roughly 7 nm of oily molecules separating two conducting salt solutions. This structure has a remarkable electrical consequence: the membrane acts as a *capacitor*. Like two metal plates separated by an insulator, the membrane stores charge — positive ions lining up on one side, negative on the other.

But the membrane is not a perfect insulator. Embedded within it are *ion channels* — protein pores that allow specific ions (Na^+ , K^+ , Ca^{2+} , Cl^-) to flow through when open. Each channel type acts like a variable resistor: its conductance changes with voltage, time, and chemical signals. At any instant, the total current flowing across the membrane is the sum of the capacitive current (charging the membrane) and the ionic current (flowing through channels):

$$I_m = C_m \frac{dV_m}{dt} + I_{\text{ion}}(V_m, t) \quad (1.1)$$

Here $C_m \approx 1 \mu\text{F}/\text{cm}^2$ is the specific membrane capacitance (remarkably constant across all cell types), V_m is the transmembrane potential, and I_{ion} is the total ionic current — the quantity that Hodgkin and Huxley set out to understand.

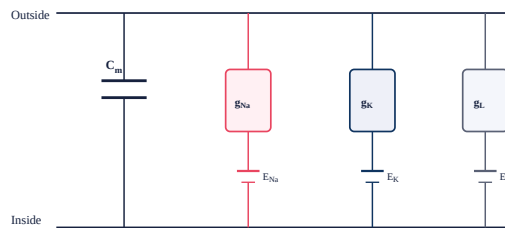


Figure 1.1. The Hodgkin–Huxley equivalent circuit. The membrane capacitance C_m sits in parallel with voltage-dependent conductances for Na^+ , K^+ , and a leak current. Each ionic branch has a variable conductance g_x in series with a Nernst battery E_x .

The key insight of Figure 1.1 is that each ionic species contributes its own current $I_x = g_x(V_m - E_x)$, where E_x is the *reversal potential* — the voltage at which the electrochemical gradient for that ion vanishes. For sodium, $E_{\text{Na}} \approx +65$ mV; for potassium, $E_{\text{K}} \approx -88$ mV. The enormous 150 mV difference between these two batteries is the energy source that powers every action potential.

Intuition

Worked example — the potassium battery. Where does $E_K \approx -88$ mV actually come from? It is the *Nernst potential*: the voltage at which the electrical pull on K^+ exactly cancels its tendency to diffuse down the concentration gradient. For a single ionic species,

$$E_K = \frac{RT}{zF} \ln \frac{[K^+]_o}{[K^+]_i}$$

At body temperature $T = 310$ K (37 °C), with $R = 8.314 \text{ J}\cdot\text{mol}^{-1}\text{K}^{-1}$, $F = 96485 \text{ C}\cdot\text{mol}^{-1}$, and valence $z = +1$, the prefactor is

$$\frac{RT}{zF} = \frac{(8.314)(310)}{96485} = 0.0267 \text{ V} = 26.7 \text{ mV}.$$

Plugging in intracellular $[K^+]_i = 138$ mM and extracellular $[K^+]_o = 4$ mM:

$$E_K = 26.7 \times \ln \frac{4}{138} = 26.7 \times \ln(0.0290) = 26.7 \times (-3.541) = -94.6 \text{ mV}.$$

So the potassium battery sits near -95 mV — even more negative than the cell's resting potential. (Textbooks quote anywhere from -88 to -95 mV depending on the exact concentrations assumed; either way, the roughly 150 mV climb up to $E_{Na} \approx +65$ mV is the energy every action potential spends.)

Intuition

Think of each ion channel as a door with a bouncer who checks the voltage. At rest ($V_m \approx -85$ mV), the Na^+ doors are nearly all shut and the K^+ doors are mostly open. The cell sits quietly at a voltage close to E_K . But if something pushes V_m above about -55 mV, the Na^+ doors suddenly swing open — and the positive feedback loop that creates an action potential begins.

1.2 Gating Variables: m , h , and n

Hodgkin and Huxley's genius was not the circuit diagram — that was known. It was their decision to treat each conductance as a product of independent "gates," each opening and closing with its own voltage-dependent kinetics. For the squid axon, they found:

$$g_{Na} = \bar{g}_{Na} \cdot m^3 \cdot h, \quad g_K = \bar{g}_K \cdot n^4$$

Here $\bar{g}_{Na}$ and $\bar{g}_K$ are the *maximal conductances* — what you'd measure if every channel were fully open. The gating variables m , h , and n each range between 0 (all gates closed) and 1 (all gates open). They obey first-order kinetics:

$$\frac{dx}{dt} = \alpha_x(V_m)(1 - x) - \beta_x(V_m)x, \quad x \in \{m, h, n\} \quad (1.2)$$

where α_x and β_x are voltage-dependent rate constants that Hodgkin and Huxley measured experimentally from the squid giant axon. This equation says something beautifully simple: each gate transitions between a closed state (population $1 - x$) and an open state (population x), with forward rate α_x and backward rate β_x .

It is often more revealing to rewrite equation (1.2) in *relaxation form*:

$$\frac{dx}{dt} = \frac{x_{\infty}(V_m) - x}{\tau_x(V_m)} \quad (1.3)$$

where $x_{\infty} = \alpha_x / (\alpha_x + \beta_x)$ is the *steady-state* value (where the gate would end up if voltage were held constant forever) and $\tau_x = 1 / (\alpha_x + \beta_x)$ is the *time constant* (how fast it gets there). This form makes the physics transparent: the gate exponentially approaches its target $x_{\infty}(V_m)$ with time constant $\tau_x(V_m)$.

△ Key Insight

The relaxation form (1.3) is not just prettier notation — it is the foundation of the **Rush–Larsen method**, the standard numerical integrator for gating variables in cardiac simulations. We will meet it again in Chapter 8 when we discuss operator splitting.

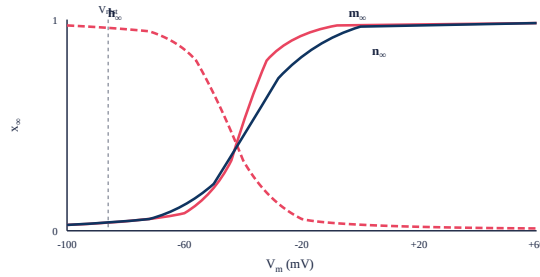


Figure 1.2. Steady-state gating curves for the HH model. At rest, $m_{\infty} \approx 0$ (Na^+ activation off), $h_{\infty} \approx 0.95$ (Na^+ inactivation nearly fully available), and $n_{\infty} \approx 0.04$ (K^+ essentially closed). The Na^+ activation curve m_{∞} is notably steeper than n_{∞} , explaining why sodium channels open faster than potassium channels — the key to the action potential upstroke.

What do these three gates mean physically? The variable m is the sodium *activation* gate: it controls how quickly Na^+ channels open when the membrane depolarizes. The variable h is the sodium *inactivation* gate: it shuts Na^+ channels back down, even if the voltage stays high. The variable n is the potassium *activation* gate: it opens K^+ channels slowly during depolarization to repolarize the cell.

The powers (m^3 and n^4) reflect the cooperativity of channel gating — Hodgkin and Huxley interpreted them as the number of independent “particles” that must simultaneously be in the right position for the channel to conduct. Modern structural biology has confirmed this: voltage-gated channels have four voltage-sensing domains, and the sodium channel’s fast gate involves three of them.

1.3 The Sodium Hypothesis and Positive Feedback

The action potential is a self-amplifying event, and understanding *why* it fires requires tracing a feedback loop. Suppose a small stimulus pushes V_m slightly above the threshold of about -55 mV. Here is what happens:

Step 1: The small depolarization causes m_{∞} to increase (the m_{∞} curve in Figure 1.2 is steeply rising near threshold). Gates start opening: m rises toward m_{∞} .

Step 2: As m increases, the sodium conductance $g_{\text{Na}} = \bar{g}_{\text{Na}} m^3 h$ grows rapidly. Na^+ ions rush into the cell (driven by the large electrochemical gradient $V_m - E_{\text{Na}} \approx -55 - 65 = -120$ mV).

Step 3: The inward Na^+ current further depolarizes the membrane, which causes even more m -gates to open — and the process accelerates explosively.

Intuition

This is *positive feedback*: depolarization opens Na^+ gates, which cause more depolarization, which opens more gates. It is the biological equivalent of a spark igniting gasoline. Once started, the upstroke is essentially unstoppable — V_m shoots from -85 mV toward $E_{\text{Na}} \approx +65$ mV in less than 1 ms. This is the "all-or-nothing" nature of the action potential.

What stops this runaway? Two mechanisms, operating on different timescales:

Fast shutdown — inactivation (h): While m opens in under 0.5 ms, the inactivation gate h closes with a time constant of 1–5 ms. As $h \rightarrow 0$, the product $m^3 h \rightarrow 0$ even though m is large. The Na^+ current self-terminates. This is the fundamental asymmetry: m is fast, h is slower.

Slow recovery — potassium (n): The K^+ gate n opens even more slowly ($\tau_n \approx 5$ –10 ms). As n^4 grows, the outward K^+ current $I_K = \bar{g}_K n^4 (V_m - E_K)$ overwhelms the now-extinct Na^+ current and pulls V_m back toward E_K .

1.4 Activation vs. Inactivation: The Refractory Period

After an action potential, the cell enters a *refractory period* during which it cannot fire again. This is essential for ordered wave propagation — without it, waves could travel backward and the heart would fibrillate. The refractory period has a clear gating-variable explanation:

Immediately after the action potential, $h \approx 0$ (Na^+ channels are inactivated) and n is still elevated (K^+ channels are open). Even if a new stimulus arrives, there are no Na^+ channels available to open — the h gate needs time to recover (h must return toward $h_{\infty}(V_{\text{rest}}) \approx 0.6$). This recovery takes 5–10 ms in nerve, and 200–300 ms in ventricular cardiac cells.

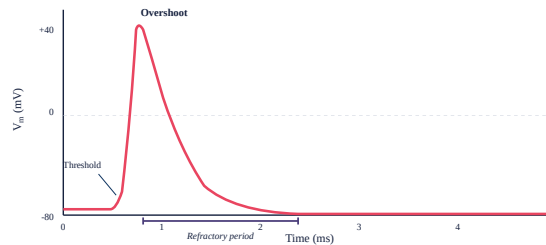


Figure 1.3. The nerve action potential (squid axon). The upstroke is driven by Na^+ influx ($m^3 h$ activating), the repolarization by Na^+ inactivation ($h \rightarrow 0$) and delayed K^+ activation (n^4 increasing). The refractory period corresponds to the time needed for h to recover.

Engine V5.4 Connection

In Engine V5.4, every ionic model inherits from the `IonicModel` abstract base class (`cardiac_sim/ionic/base.py`), which requires: `compute_Iion(V, ionic_states)` to compute total ionic current, `step(V, ionic_states, dt)` to advance the full model, and separate methods `compute_gate_steady_states()` and `compute_gate_time_constants()` so that the Rush–Larsen solver can update gates externally. V is always stored separately from ionic states. The Hodgkin–Huxley structure — membrane current as sum of conductances times driving forces, gating variables following first-order kinetics — is exactly the pattern this interface encodes. Understanding HH means understanding the API contract of every model in the engine.

The complete Hodgkin–Huxley system is four coupled ordinary differential equations: one for V_m (the circuit equation) and three for the gates (m , h , n). It was the first biophysically detailed cell model and re-

mains the conceptual template for all that followed. But for the heart, we need something quite different — as we'll see in the next chapter.

From Neurons to Heart Cells

The nerve action potential is a brief spark — over in 2 milliseconds. The cardiac action potential is a sustained flame, lasting 200–400 ms. This chapter explains why the heart demands such a profoundly different electrical signature, and what new biophysics makes it possible.

2.1 Why the Heart Needs a Longer Action Potential

A nerve fiber has one job: transmit a binary signal from point A to point B as fast as possible. It fires and recovers quickly so it can fire again. But a cardiac myocyte has a very different job: it must *contract*, and contraction takes time. The heart needs the electrical signal to outlast the mechanical event so that every cell stays activated long enough to squeeze in synchrony.

If the cardiac AP were as short as a nerve spike, the cell would repolarize before it finished contracting. Worse, a new electrical impulse could arrive and re-activate parts of the heart while other parts were still contracting — leading to chaotic, uncoordinated contraction. The long AP duration serves as a *built-in safety mechanism*: by keeping the cell depolarized for hundreds of milliseconds, it prevents premature re-excitation.

Intuition

Think of it this way: the nerve AP is like a camera flash — bright, instantaneous, done. The cardiac AP is like holding a light switch on for a deliberate period. The heart needs this "held" state because mechanical contraction is slow compared to electrical activation. The electrical plateau is the heart's way of saying "stay contracted until I tell you to relax."

2.2 The L-type Calcium Current and the Plateau

So how does the heart stretch a 2 ms spike into a 300 ms plateau? The answer is a current that neurons barely use: the *L-type calcium current*, I_{CaL} . This inward current activates during depolarization (like Na^+), but it inactivates *much more slowly* — on a timescale of 50–200 ms rather than 1–2 ms.

During the plateau phase (Phase 2 of the cardiac AP), the membrane sits at roughly 0 to +20 mV because two opposing forces are nearly balanced:

$$\underbrace{I_{\text{CaL}}}_{\text{inward, depolarizing}} \approx - \underbrace{I_{\text{Kr}} + I_{\text{Ks}}}_{\text{outward, repolarizing}}$$

The L-type Ca^{2+} current holds the voltage up, while slowly activating delayed-rectifier K^+ currents (I_{Kr} and I_{Ks}) try to pull it down. This tug-of-war creates a nearly flat plateau that can last over 200 ms. Eventually, as I_{CaL} inactivates (both via voltage and via rising intracellular Ca^{2+}), the K^+ currents win and repolarization accelerates.

Key Insight

The plateau is a *fragile equilibrium*. A small change in any current during Phase 2 can dramatically alter AP duration. This is why drug-induced QT prolongation (blocking I_{Kr}) is so dangerous — it tips the balance toward inward current, lengthens the plateau, and can trigger the lethal arrhythmia *torsades de pointes*. Understanding this balance is central to computational drug safety testing.

The L-type calcium current also serves a second critical role: it triggers the release of a much larger amount of calcium from the sarcoplasmic reticulum (SR) — the cell's internal calcium store. This process, called *calcium-induced calcium release* (CICR), amplifies the Ca^{2+} signal roughly 10-fold and drives actual muscle contraction. We will see this calcium handling machinery in detail when we study the TTP06 model in Chapter 5.

2.3 Minimal Cardiac Models: FitzHugh–Nagumo to Fenton–Karma

Before diving into the full complexity of modern ionic models, it is instructive to see how the cardiac AP can be captured with just 2–3 variables. These minimal models sacrifice biophysical detail for mathematical tractability, but they capture the essential shape.

The FitzHugh–Nagumo Model (2 variables)

In 1961, Richard FitzHugh realized that the HH equations could be distilled to their essence. The key insight: the fast variable (Na^+ activation) and the slow variable (recovery) are what matter most. He proposed:

$$\frac{dv}{dt} = v - \frac{v^3}{3} - w + I_{\text{ext}}, \quad \frac{dw}{dt} = \epsilon(v + a - bw) \quad (2.1)$$

Here v is a rescaled voltage, w is a slow recovery variable (combining h and n into a single "how recovered is the cell" variable), and $\epsilon \ll 1$ ensures that w moves much slower than v . The cubic nonlinearity $v - v^3/3$ creates the bistability needed for excitation: v has two stable branches (resting and excited) and the system jumps between them.

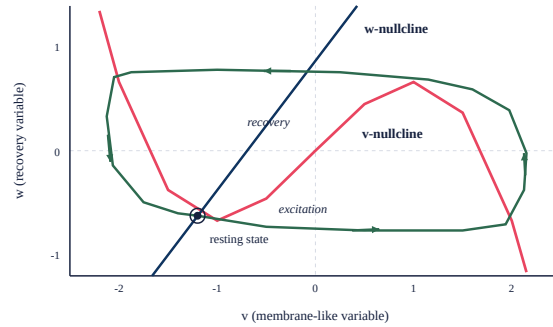


Figure 2.1. The FitzHugh–Nagumo phase plane. The cubic v -nullcline (red, where $dv/dt = 0$, i.e. $w = v - v^3/3$) is N-shaped; the straight w -nullcline (blue, where $dw/dt = 0$) crosses it once, at the stable resting state — the fixed point. A supra-threshold stimulus throws the state rightward across the unstable middle branch: the fast excitation upstroke, in which v jumps while w barely moves. The slow recovery variable w then climbs, carrying the trajectory up the right branch, back across the top, and slowly down the left branch to rest. The wide, flat loop is the phase-plane signature of one action potential.

FitzHugh–Nagumo captures excitability, threshold behavior, and refractoriness — but it produces a triangular AP, not the flat plateau of a cardiac cell.

Intuition

Worked example — one excitation, stepped by hand. Take the standard parameters $a = 0.7$, $b = 0.8$, $\epsilon = 0.08$, no applied current ($I_{\text{ext}} = 0$), and forward-Euler steps of $\Delta t = 0.1$. The cell rests at the fixed point $(v, w) \approx (-1.2, -0.62)$ — the intersection in Figure 2.1. A brief stimulus kicks the voltage variable above threshold to $v_0 = 0$, leaving $w_0 = -0.60$.

Step 1.

$$dv/dt = v_0 - v_0^3/3 - w_0 = 0 - 0 - (-0.60) = +0.600$$

$$dw/dt = \epsilon(v_0 + a - b w_0) = 0.08(0 + 0.7 - 0.8(-0.60)) = 0.08(1.18) = 0.0944$$

$$v_1 = 0 + 0.1(0.600) = 0.060, \quad w_1 = -0.60 + 0.1(0.0944) = -0.591$$

Step 2. $dv/dt = 0.650$, $dw/dt = 0.099 \Rightarrow v_2 = 0.125$, $w_2 = -0.581$.

Step 3. $dv/dt = 0.705$, $dw/dt = 0.103 \Rightarrow v_3 = 0.196$, $w_3 = -0.570$.

In three steps v climbed by **0.196** while w moved only **0.030** — a **6.5×** split in speed. That gap is ϵ : the voltage races up the cubic toward the excited branch ($v \approx +2$) in a fast upstroke while the recovery variable barely stirs. Only later, once w has slowly accumulated, does it overtake the balance and drag v back down — the recovery limb of the loop in Figure 2.1.

The Fenton–Karma Model (3 variables)

In 1998, Flavio Fenton and Alain Karma introduced a three-variable model specifically designed for cardiac tissue. The key addition: a slow inward current that produces the plateau.

$$\frac{dV}{dt} = -(I_{\text{f}}(V, v) + I_{\text{so}}(V) + I_{\text{si}}(V, w)) \quad (2.2)$$

The three currents map directly to cardiac physiology: I_{f} is a fast inward current (like Na^+ creating the upstroke), I_{so} is a slow outward current (like K^+ causing repolarization), and I_{si} is a slow inward current (like Ca^{2+} sustaining the plateau). With just three variables (V, v, w) , the Fenton–Karma model reproduces realistic AP shapes, spiral waves, and even alternans — all phenomena seen in real cardiac tissue.

Engine V5.4 Connection

Minimal models like Fenton–Karma are not toys. In Engine V5.4, they serve as essential *validation benchmarks*: if the monodomain solver produces the correct spiral-wave dynamics with a 3-variable model, you can have confidence in the diffusion numerics before scaling up to TTP06 (18 variables) or ORd (41 variables). They are also used for parameter sweeps and machine-learning studies where millions of simulations are needed.

The progression from HH (4 variables, nerve) → FitzHugh–Nagumo (2 variables, generic excitable) → Fenton–Karma (3 variables, cardiac) → full ionic models (18–40+ variables, human ventricle) is the story of matching model complexity to the scientific question being asked. In the next chapter, we zoom in on the full cardiac action potential and dissect each phase.

Anatomy of the Cardiac Action Potential

The ventricular action potential is traditionally divided into five phases (0 through 4). Each phase has a dominant current, a clear biophysical story, and direct consequences for the heart's electrical and mechanical function. Understanding these phases is the Rosetta Stone for reading any cardiac cell model. Chapter 5 provides the complete mathematical formulations for all 12 membrane currents in the TTP06 model; here we build the qualitative intuition first.

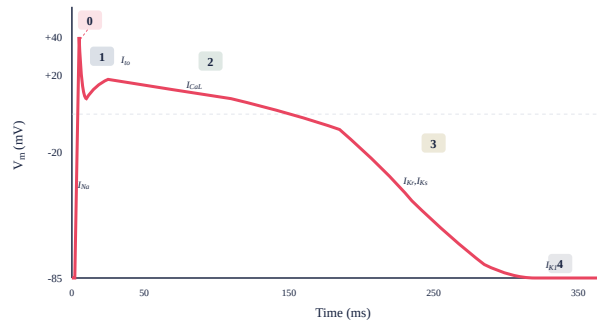


Figure 3.1. The five phases of the human ventricular action potential with dominant currents. Phase 0: Na^+ upstroke. Phase 1: transient outward K^+ (notch). Phase 2: Ca^{2+} - K^+ plateau. Phase 3: K^+ -driven repolarization. Phase 4: resting potential maintained by I_{K1} .

3.1 Phase 0: The Sodium Upstroke

Phase 0 is the fastest event in cardiac electrophysiology. When an arriving wavefront brings V_m above threshold (≈ -65 mV), sodium channels undergo the same explosive positive feedback we saw in the HH model: m -gates open rapidly, Na^+ floods inward, and V_m rockets from -85 mV to about $+40$ mV in roughly 1–2 ms.

The maximum rate of rise, $dV/dt_{\max}$, is a clinical measure of sodium channel availability. In healthy ventricular tissue, $dV/dt_{\max} \approx 200\text{--}300$ V/s. This is also what determines *conduction velocity* — the faster the upstroke, the larger the local current source that can depolarize neighboring cells, and the faster the wave propagates. In the monodomain equation (Chapter 7), this translates directly to the source term.

3.2 Phase 1: Early Repolarization (I_{to})

Immediately after the peak, a brief notch appears — a small dip in voltage before the plateau. This is caused by the *transient outward* potassium current I_{to} , which activates rapidly during depolarization and inactivates within tens of milliseconds.

Phase 1 may look like a minor detail, but it has major consequences. The depth of the notch varies across the ventricular wall: epicardial cells (outer surface) have large I_{to} and a deep notch, while endocardial cells (inner surface) have little I_{to} and almost no notch. This *transmural gradient* in AP morphology creates voltage differences across the wall that are visible on the ECG — the J-wave and T-wave both depend on it.

3.3 Phase 2: The Plateau — A Delicate Balance

The plateau is the heart's signature. For 150–250 ms, the membrane voltage hovers between -10 and $+20$ mV, sustained by the near-cancellation of two opposing current groups:

Inward (depolarizing): The L-type calcium current I_{CaL} is the dominant plateau current. Ca^{2+} ions enter through slowly inactivating channels, holding the voltage up. A smaller late sodium current (I_{NaL}) also contributes, especially in disease states.

Outward (repolarizing): Two delayed-rectifier potassium currents slowly activate: I_{Kr} (rapid component, carried by hERG channels) and I_{Ks} (slow component). The inward-rectifier I_{K1} is largely blocked at plateau voltages due to its voltage dependence — it will become important again in Phase 3–4.

△ Key Insight

The plateau balance explains why the cardiac AP duration (APD) is so sensitive to perturbations. Block 30% of I_{Kr} (as many common drugs do), and the plateau suddenly lacks enough repolarizing current. APD extends, the QT interval on the ECG lengthens, and the risk of arrhythmia rises. This is why *in silico* drug testing with models like TTP06 and ORd has become a regulatory focus (the CiPA initiative).

3.4 Phase 3: Repolarization — The Potassium Takeover

As I_{CaL} gradually inactivates (driven both by voltage-dependent inactivation and by rising intracellular Ca^{2+} concentration), the balance tips irreversibly toward outward current. The K^+ currents I_{Kr} and I_{Ks} , which have been slowly building, now dominate. As the voltage falls, I_{K1} — the strong inward rectifier that was blocked at plateau voltages — begins to unblock and adds its large outward conductance to the repolarization.

Phase 3 repolarization is *regenerative*: as V_m falls, more I_{K1} unblocks, which pulls V_m down further, which unblocks more I_{K1} . This positive feedback (now outward) ensures that repolarization, once started, proceeds rapidly to completion. The membrane returns to its resting potential of about -85 mV.

3.5 Phase 4: Rest and the Role of I_{K1}

At rest, the membrane potential sits near $E_K \approx -88$ mV because I_{K1} dominates the conductance. The "inward rectifier" name refers to a peculiar property: I_{K1} passes large outward current at voltages just above E_K , but essentially zero current at plateau voltages. This voltage-dependent block (caused by intracellular magnesium and polyamines plugging the channel pore) is what allows the plateau to exist — without it, K^+ would short-circuit the plateau and the AP would collapse.

🌱 Intuition

Think of I_{K1} as a two-state switch. At rest, it is "on" — a strong anchor holding V_m near E_K . During the plateau, it switches "off" — stepping aside to let the delicate Ca^{2+}/K^+ balance play out. Then in Phase 3, it switches back "on" to slam the door shut and restore the resting potential. No other current has this dual role.

With the five phases clearly in mind, we now turn to the single most important ion in cardiac physiology: calcium. The next chapter is devoted entirely to the intracellular calcium machinery — from the nanoscale architecture of the dyadic cleft to the pathological remodeling that drives heart failure and arrhythmia.

Intracellular Calcium: The Master Regulator

Calcium is the singular ion that links electrical excitation to mechanical contraction. But this link is not simple: it involves nanoscale signaling domains, stochastic release events, multi-protein regulatory complexes, and feedback loops that span timescales from microseconds to minutes. This chapter provides a modern, in-depth account of cardiac calcium handling — from the architecture of the dyadic cleft to the pathological remodeling seen in heart failure.

4.1 The Dyadic Cleft — Where Electrical Meets Mechanical

Excitation-contraction (EC) coupling occurs at *dyads* — nanoscopic junctions formed where transverse tubules (T-tubules) invaginate to within 10–20 nm of the junctional sarcoplasmic reticulum (jSR). This tiny gap — the *dyadic cleft* — is the reaction chamber where the electrical signal is transduced into a calcium signal. During an action potential, calcium concentration within the cleft surges from a resting ~ 100 nM to ~ 10 μ M, a 100-fold increase confined to a space smaller than a virus particle.

The cleft is held together by **junctophilin-2 (JPH2)**, a structural protein that physically anchors the T-tubule membrane to the jSR membrane. JPH2 maintains the critical 10–20 nm separation required for efficient local control coupling. As we will see in Section 4.7, the loss of JPH2 in heart failure is one of the earliest and most consequential structural failures.

Calcium Release Units and Nanoscale Organization

The functional unit of calcium release is the *calcium release unit* (CRU), also called a *couplon*. A CRU consists of a cluster of type-2 ryanodine receptors (RyR2) on the jSR membrane, positioned directly beneath a group of L-type calcium channels (Ca v 1.2) on the T-tubule membrane. A typical human ventricular myocyte contains approximately 10,000–20,000 CRUs.

Recent super-resolution imaging — using dSTORM (direct stochastic optical reconstruction microscopy), DNA-PAINT, and most recently MINFLUX microscopy — has revealed the nanoscale architecture of these clusters in unprecedented detail. Key findings include:

RyR2 cluster heterogeneity: A typical CRU contains 4–20 RyR2 tetramers (16–80 individual subunits), but cluster sizes vary widely. RyR2 channels function cohesively as a CRU only when their nearest-neighbor distance (NND) is less than ~ 100 – 150 nm. Beyond this threshold, coupling between channels is lost and the release event becomes unreliable.

Sparse channel activation: Despite clusters containing dozens of channels, only 2–4 RyR2 channels typically open simultaneously during a single elementary release event under basal conditions. Beta-adrenergic stimulation increases this to 5–8 channels, providing a mechanism for graded calcium release amplification.

L-type channel organization: Super-resolution studies using STED and DNA-PAINT (2024) revealed that Ca v 1.2 and Ca v 1.3 channels form loose clusters of ~ 9 molecules within ~ 120 nm domains, arranged in an irregular pattern with significant intermolecular spacing — not densely packed. This loose arrangement reflects the functional need for positioning (close to RyR2) rather than crowding.

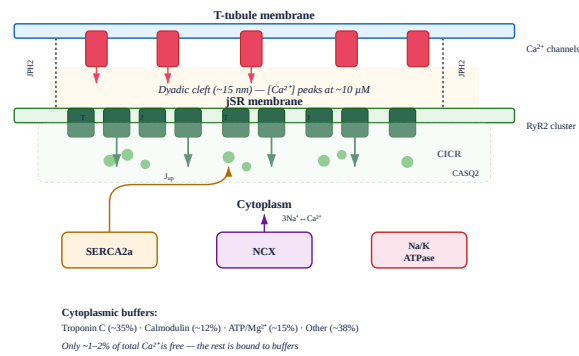


Figure 4.1. Architecture of the dyadic cleft. L-type Ca²⁺ channels on the T-tubule membrane face RyR2 clusters on the jSR, separated by ~15 nm. Junctophilin-2 (JPH2) anchors the two membranes. Triadin (T) and junctin (J) link RyR2 to luminal CASQ2 polymers. SERCA2a pumps Ca²⁺ back into the SR; NCX extrudes Ca²⁺ across the sarcolemma. Only 1–2% of cytoplasmic Ca²⁺ is free.

4.2 Calcium-Induced Calcium Release — Local Control Theory

Calcium Sparks: The Elementary Release Event

A *calcium spark* is the optical signature of the concerted opening of a small group (typically 2–10) of RyR2 channels within a single CRU. Discovered by Cheng, Lederer, and Bhatt in 1993 using confocal line-scanning and the fluorescent indicator fluo-3, sparks are now understood as the fundamental building blocks of the whole-cell calcium transient.

Each spark has well-defined characteristics: a peak amplitude of 0.5–2 μM above baseline, a spatial half-width of 1–3 μM , a rise time of 2–5 ms, and a full-width-at-half-maximum (FWHM) duration of 15–50 ms. During a normal heartbeat, the action potential triggers ~10,000 sparks nearly simultaneously across the myocyte; their spatial and temporal summation produces the global calcium transient that drives contraction.

Intuition

Think of CRUs as thousands of tiny fireworks launchers spread throughout the cell. When the action potential arrives, it does not light them all at once with a single fuse. Instead, each launcher has its own independent trigger — an L-type channel — that fires with a certain probability (~30–40% per beat). The global calcium transient is the *sum* of thousands of probabilistic, independent sparks. This is why calcium release is *graded*: more L-type current recruits more sparks, producing a larger transient — without the all-or-nothing behavior you might expect from positive feedback.

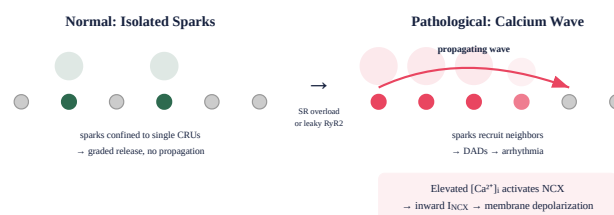


Figure 4.2. The spark-to-wave transition. Left: Under normal conditions, individual sparks (green) are spatially isolated — positive feedback is confined within each CRU. Right: With SR overload or leaky RyR2, sparks recruit neighboring CRUs, generating a propagating calcium wave that activates NCX and triggers DADs.

Local Control vs. Common Pool

The *local control* theory, established in the 1990s and continuously refined through 2025, resolves a paradox that troubled the field for decades: how can CICR be both *high-gain* (a tiny calcium trigger from L-

type channels produces a large SR release) and *graded* (release is proportional to trigger size, not all-or-nothing)?

The answer is spatial isolation. Each RyR2 cluster senses *local* calcium concentration within its dyadic cleft ($\sim 10\ \mu\text{M}$ during activation), not the global cytosolic concentration ($\sim 1\ \mu\text{M}$). Positive feedback — calcium from one RyR2 opening its neighbors — is confined within the CRU because the cleft's tiny volume ($\sim 10^{-18}\ \text{L}$) means even picoampere currents produce micromolar concentrations locally, while rapid diffusion dilutes the signal before it reaches neighboring CRUs $\sim 2\ \mu\text{m}$ away.

This contrasts with the older *common pool* model, which assumed a single well-mixed cytoplasmic calcium compartment. Common pool models predict all-or-nothing release — once any RyR2 opens, the resulting calcium elevation should trigger all other RyR2s. Experimentally, this is not what happens: sparks occur at discrete locations without propagating, and the total release scales smoothly with L-type current magnitude. Local control explains both observations.

SR Load: The Central Regulator of Release

The SR calcium content ($[\text{Ca}^{2+}]_{\text{SR}}$, typically 1–2 mM total, of which only $\sim 20\text{--}50\ \mu\text{M}$ is free — the rest bound to calsequestrin) is the single most powerful determinant of release properties. Spark frequency increases *exponentially* with SR load: doubling the load can increase spontaneous release 5–10-fold. Spark amplitude increases more modestly, while the probability that a given L-type channel opening will trigger a spark (the *coupling fidelity*) rises steeply with load.

This load-dependence creates an inherent negative feedback: release depletes the store, which reduces the probability of further release. In healthy cells, this feedback stabilizes the system. In disease, when SR load is chronically elevated or RyR2 sensitivity is pathologically increased, the feedback can be overwhelmed — sparks merge into waves, and arrhythmias follow.

The Spark-to-Wave Transition

Under pathological conditions — elevated SR load, increased diastolic calcium, enhanced RyR2 sensitivity — isolated sparks can recruit their neighbors, generating *calcium waves* that propagate through the myocyte at $\sim 100\ \mu\text{m/s}$. The wave front advances by saltatory conduction: a spark at one CRU raises local calcium enough to trigger sparks at neighboring CRUs $\sim 2\ \mu\text{m}$ away. The critical radius for wave propagation is $\sim 2.7\ \mu\text{m}$ — below this, the calcium elevation dissipates before reaching the next CRU.

Calcium waves are the primary trigger for *delayed afterdepolarizations* (DADs). As the wave sweeps through the cell, the elevated cytoplasmic calcium activates the sodium-calcium exchanger (NCX) in forward mode, generating a net inward current that depolarizes the membrane. If this depolarization reaches threshold during diastole, a triggered action potential fires — the cellular mechanism behind many clinical arrhythmias.

4.3 The Ryanodine Receptor — Gatekeeper of the SR

RyR2 is the largest known ion channel ($\sim 2.2\ \text{MDa}$ as a homotetramer) and the primary gate controlling calcium release from the SR. Its regulation involves a remarkable convergence of signals: cytoplasmic calcium (activation), luminal calcium (load sensing), phosphorylation by multiple kinases, oxidative modifications, and protein–protein interactions.

Phosphorylation by PKA and CaMKII

Two kinases dominate RyR2 regulation, each targeting a distinct serine residue with different physiological and pathological consequences:

PKA at Serine 2808: During beta-adrenergic stimulation (exercise, stress), PKA phosphorylates RyR2 at S2808, increasing the channel's sensitivity to calcium activation. The channel opens more frequently and

requires less trigger calcium. This is the acute "fight-or-flight" enhancement of calcium release. Phosphatase 1 (PP1) rapidly reverses the phosphorylation when adrenergic tone subsides.

CaMKII at Serine 2814: CaMKII, activated by calcium/calmodulin, phosphorylates RyR2 at S2814 with more potent effects on spark frequency than PKA. Critically, CaMKII also increases the channel's sensitivity to *luminal* calcium, amplifying the load-dependent release mechanism. This creates a positive feedback loop: more calcium → more CaMKII activation → more RyR2 phosphorylation → more calcium release.

▲ Key Insight

Recent work (2023–2025) has established that CaMKII phosphorylation at S2814 — not PKA phosphorylation at S2808 — is the primary driver of pro-arrhythmic calcium leak in chronic heart failure. A 2025 study in *Nature Cardiovascular Research* found an unexpected twist: preventing RyR2 phosphorylation reduced calcium leak but paradoxically reactivated the sodium current I_{Na} , suggesting a previously unknown crosstalk between RyR2 status and sarcolemmal channel regulation. This has implications for anti-arrhythmic drug design.

Luminal Calcium Sensing: The CASQ2–Triadin–Junctin Complex

RyR2 possesses an intrinsic ability to sense the calcium concentration *inside* the SR, independent of cytoplasmic calcium. This luminal sensing is mediated by a multi-protein complex anchored to the SR luminal face of RyR2:

Calsequestrin 2 (CASQ2) is the primary SR calcium storage protein, binding ~40–50 calcium ions per molecule across a broad affinity range ($K_d \approx 0.1$ –10 mM). At low luminal $[Ca^{2+}]$ (<1 mM), CASQ2 exists as a monomer and directly inhibits RyR2 via triadin and junctin — a molecular "brake" that prevents release from a depleted store. As luminal calcium rises through intermediate concentrations (3–5 mM), CASQ2 undergoes progressive oligomerization: monomer → dimer → polymer. The polymeric form dissociates from the triadin–junctin bridge, *relieving* RyR2 inhibition and permitting calcium release.

Triadin (primarily the 95 kDa junctional isoform) and **junctin** are integral SR membrane proteins that serve as molecular scaffolds linking CASQ2 to RyR2. Their ablation causes complete loss of functional CRUs, severe arrhythmias, and cardiomyopathy — underscoring their essential structural role. Mutations in triadin and junctin are now recognized causes of catecholaminergic polymorphic ventricular tachycardia (CPVT).

🌿 Intuition

The CASQ2 sensing mechanism works like a pressure gauge. When the SR "tank" is nearly empty (low luminal calcium), CASQ2 monomers bind to the RyR2 "valve" and hold it shut — preventing futile release from a depleted store. As the tank fills (SERCA pumps calcium back in), CASQ2 polymers form and detach from the valve, allowing it to open on the next trigger. This self-regulating mechanism ensures that release magnitude tracks store content — a form of automatic gain control.

The "Leaky RyR" Hypothesis and Heart Failure

Andrew Marks and colleagues demonstrated that RyR2 dysfunction is central to heart failure pathophysiology through a mechanism now termed the *leaky RyR hypothesis*. In failing hearts, chronic neurohormonal activation leads to persistent hyperphosphorylation of RyR2 at both S2808 (PKA) and S2814 (CaMKII). This, combined with oxidative modification (S-nitrosylation, sulfhydryl oxidation), increases the channel's diastolic open probability — creating a sustained *calcium leak* from the SR even when the cell should be at rest.

The consequences cascade: elevated diastolic $[Ca^{2+}]_i$ (~200–300 nM vs. normal 50–100 nM) impairs relaxation, activates calcium-dependent proteases (calpains), and drives maladaptive hypertrophic signaling

via calcineurin/NFAT. Simultaneously, the increased forward-mode NCX activity needed to extrude the leaked calcium brings excess sodium into the cell, raising $[Na^+]_i$ and further compromising SERCA efficiency. The leaked SR calcium is also unavailable for systolic release, weakening contraction.

Store-Overload-Induced Calcium Release (SOICR)

When SR calcium load exceeds a critical threshold ($[Ca^{2+}]_{SR} > \sim 1.5$ mM free), spontaneous calcium release occurs without any electrical trigger — a phenomenon called SOICR. At such high loads, CASQ2 inhibition is completely relieved, and the tiny amount of calcium that inevitably leaks through any channel is amplified by CICR into full-scale spontaneous waves.

SOICR is the cellular mechanism behind digitalis toxicity (Na/K-ATPase inhibition → sodium overload → reverse NCX → calcium overload → SOICR → arrhythmia), and it is a primary trigger for CPVT — a genetic arrhythmia syndrome caused by RyR2 or CASQ2 mutations that lower the load threshold for spontaneous release. Carvedilol, a nonselective beta-blocker, uniquely suppresses SOICR by directly reducing RyR2 open duration, independent of its beta-blocking properties — making it particularly effective for CPVT and heart failure arrhythmias.

4.4 SERCA2a and Phospholamban — The Pump and Its Brake

The sarco/endoplasmic reticulum calcium-ATPase 2a (SERCA2a) is the workhorse of diastolic calcium removal, responsible for 70–90% of cytoplasmic calcium clearance in human ventricular myocytes. Understanding its regulation is essential for understanding both normal relaxation and the diastolic dysfunction that characterizes heart failure.

SERCA2a Kinetics

SERCA2a transports 2 Ca^{2+} ions per ATP hydrolyzed, with a K_m of ~ 0.5 – 1 μM for calcium — meaning it operates at roughly half-maximal velocity at resting diastolic $[Ca^{2+}]$. The pump consumes an estimated 30–40% of the myocyte's total ATP budget during active beating — a staggering metabolic cost that underscores calcium cycling's importance. A typical human ventricular myocyte contains $\sim 1,000$ – $2,000$ SERCA2a molecules per unit SR surface.

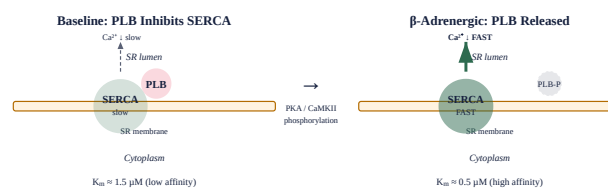


Figure 4.3. The SERCA2a/phospholamban pump-brake system. Left: At baseline, PLB inhibits SERCA2a by reducing its calcium affinity (high K_m), resulting in slow calcium reuptake. Right: After PKA phosphorylation (β -adrenergic stimulation), PLB dissociates, restoring SERCA2a's full affinity and dramatically accelerating calcium reuptake — the molecular basis of lusitropy.

Phospholamban: The Molecular Brake

Phospholamban (PLB) is a 52-amino-acid transmembrane protein that constitutively *inhibits* SERCA2a in its unphosphorylated state. It reduces SERCA2a's calcium affinity 2–3-fold (shifting K_m from ~ 0.5 μM to ~ 1.5 μM), dramatically slowing calcium reuptake at diastolic calcium concentrations. PLB is the reason that at baseline, SERCA2a operates far below its maximum capacity — a reserve that can be unlocked on demand.

During beta-adrenergic stimulation, PKA phosphorylates PLB at serine-16 (and CaMKII at threonine-17), causing PLB to dissociate from SERCA2a. This relieves inhibition and dramatically accelerates calcium reuptake — the molecular basis of the *lusitropic* (relaxation-enhancing) effect of sympathetic stimulation. The coordinated phosphorylation of both RyR2 (more release) and PLB (faster reuptake) ensures that the

enhanced systolic transient is followed by faster relaxation, allowing the heart to fill adequately at elevated heart rates.

△ Key Insight

The CUPID trials (Calcium Upregulation by Percutaneous Administration of Gene Therapy in Cardiac Disease) attempted to restore SERCA2a expression in failing hearts using AAV1-delivered gene therapy. Early-phase results were encouraging, but the pivotal CUPID 2 trial (2016) did not meet its primary endpoint. Current research has shifted toward upstream approaches: rather than adding more pump protein, investigators are targeting phosphatase-1 inhibitors (e.g., AB-1002) to relieve PLB-mediated inhibition of the *endogenous* SERCA2a — a more physiologically regulated strategy.

4.5 The Sodium-Calcium Exchanger — Dual-Mode Transport

The sodium-calcium exchanger NCX1 is a bidirectional transporter that exchanges 3 Na^+ for 1 Ca^{2+} across the sarcolemma. The direction of transport depends on the electrochemical gradients of both ions and the membrane potential — making NCX a voltage-sensitive, context-dependent player in calcium homeostasis.

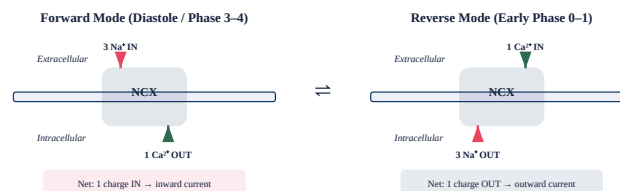


Figure 4.4. The two operating modes of the $\text{Na}^+/\text{Ca}^{2+}$ exchanger (NCX). Left: Forward mode — dominant during diastole and late repolarization — extrudes Ca^{2+} while importing Na^+ , generating a net inward (depolarizing) current. Right: Reverse mode — briefly active during early depolarization — imports Ca^{2+} and exports Na^+ , generating a net outward current. The reversal potential depends on both $[\text{Na}^+]$ and $[\text{Ca}^{2+}]$ gradients plus V_m .

Forward Mode (Calcium Extrusion)

Under normal diastolic conditions, NCX operates in *forward mode*: 3 Na^+ ions enter the cell down their concentration gradient, providing the energy to extrude 1 Ca^{2+} against its gradient. This accounts for ~ 10 – 30% of diastolic calcium removal (with SERCA2a handling the remainder). Because the exchange is electrogenic (net transfer of one positive charge inward), forward-mode NCX generates a small depolarizing current — normally negligible, but significant when calcium is elevated.

Reverse Mode and Arrhythmogenesis

When intracellular sodium is elevated (as in heart failure or digitalis toxicity) or the membrane is strongly depolarized, NCX reverses: Na^+ exits and Ca^{2+} enters. This reverse-mode operation contributes to calcium overload. More importantly, during spontaneous calcium waves, the large forward-mode NCX current (extruding the wave-elevated calcium) generates substantial inward current — 100 – 500 pA when summed over the cell — sufficient to depolarize the membrane and trigger delayed afterdepolarizations (DADs). This is the primary mechanism linking calcium instability to electrical arrhythmia.

Recent cryo-EM structures of human NCX1 (2023) revealed distinct inactivated and activated conformations, with a sodium-dependent inactivation mechanism that serves as an intrinsic "brake" when intracellular sodium is chronically elevated. A 2024 study showed that gain-of-function NCX1 mutations paradoxically depress contractility by excessive calcium extrusion, demonstrating that too much of either forward or reverse NCX activity is maladaptive.

The Calcium Clock in Pacemaker Cells

In sinoatrial node (SAN) cells, a fundamentally different calcium cycling mechanism contributes to automaticity. During diastole, spontaneous calcium sparks from the SR activate NCX in forward mode, generating a ramp-like inward current that gradually depolarizes the membrane toward threshold. This "calcium clock" works in concert with the classical "membrane clock" (funny current I_f , T-type calcium current) to set heart rate. Disruption of the calcium clock — through RyR2 inhibition or SERCA2a depression — slows or abolishes pacemaker activity, demonstrating that spontaneous calcium cycling is not merely a byproduct but an active driver of the heartbeat.

4.6 Calcium Buffering — The 1% Rule

Perhaps the most underappreciated fact in cardiac calcium biology is that **only ~1–2% of total intracellular calcium exists as free Ca^{2+} ions**. The remaining 98–99% is bound to proteins. This massive buffering has profound consequences for calcium signaling: it drastically slows effective calcium diffusion (to ~10–30 $\mu\text{m}^2/\text{s}$, roughly 10× slower than free aqueous diffusion) and shapes the kinetics of every calcium-dependent process.

The buffer capacity β — defined as the ratio of total calcium change to free calcium change — ranges from ~20 to 100 depending on the cell compartment. During a typical beat, free $[\text{Ca}^{2+}]_i$ rises from ~100 nM to ~1–2 μM , a 10–20-fold increase. But the total calcium flux required to achieve this is 20–100 times larger, because most of the released calcium is immediately captured by buffers.

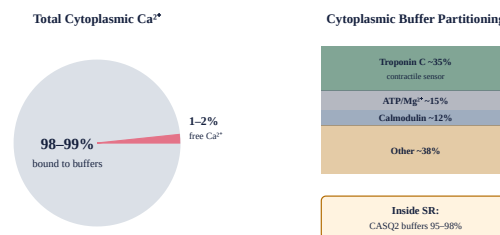


Figure 4.5. Calcium buffering in the cardiomyocyte. Left: Only 1–2% of total cytoplasmic calcium is free; the rest is bound to buffer proteins. Right: Partitioning among the major cytoplasmic buffers. Troponin C (the contractile sensor) is the largest single buffer pool. Inside the SR, calsequestrin-2 (CASQ2) binds 95–98% of luminal calcium.

The Major Buffers

Troponin C (~35% of cytoplasmic buffering): The regulatory calcium sensor of the contractile apparatus. Each troponin C molecule has 4 calcium-binding sites (2 regulatory with $K_d \approx 1\text{--}10 \mu\text{M}$, 2 structural with $K_d \approx 100 \text{ nM}$). With ~10–15 million troponin C molecules per myocyte, this represents the largest single buffer pool. Calcium binding to the regulatory sites triggers the conformational cascade (tropomyosin shift → myosin binding site exposure → cross-bridge cycling) that produces contraction. Thus troponin C is simultaneously a buffer, a sensor, and the effector of the entire contractile process.

Calmodulin (~12% of buffering): A ubiquitous calcium sensor with 4 EF-hand binding motifs (K_d range 100 nM to 5 μM). Calmodulin activates CaMKII, calcineurin, and numerous other enzymes — serving as the cell's primary calcium-to-kinase signal transducer. With ~1–2 million molecules per myocyte and faster on/off kinetics than troponin C, calmodulin is particularly responsive to rapid calcium transients.

Calsequestrin 2 (~95–98% of SR luminal buffering): Inside the SR, CASQ2 binds ~40–50 calcium ions per molecule, maintaining total SR calcium at 1–2 mM while keeping free SR calcium at only ~20–50 μM . This enormous buffer capacity allows the SR to function as a high-capacity store without the osmotic stress that millimolar free calcium would create.

Other contributors: ATP and its magnesium complexes (~15%), mitochondrial uptake (<5% acutely but important for long-term calcium homeostasis), and various smaller calcium-binding proteins collectively account for the remaining buffering capacity.

Mathematical Note

In computational models, calcium buffering is typically handled by either *rapid-equilibrium approximation* (assuming buffer binding/unbinding is instantaneous, valid when buffer kinetics are much faster than calcium transient kinetics) or *dynamic buffering ODEs* (explicitly tracking bound and free buffer pools). TTP06 uses a rapid-equilibrium approach for most buffers, while ORd tracks selected buffer states dynamically. The choice matters: dynamic buffering adds state variables but captures the delay between calcium rise and buffer saturation — important for accurate spark modeling.

4.7 T-Tubule Remodeling — When Structure Fails

The T-tubule network consists of regular sarcolemmal invaginations ($\sim 0.2\text{--}0.5\ \mu\text{m}$ diameter) that penetrate deep into the myocyte at $\sim 2\ \mu\text{m}$ intervals, aligned with the Z-lines of the sarcomere. This regular architecture ensures that every CRU across the cell volume is within $\sim 1\ \mu\text{m}$ of a T-tubule, enabling near-synchronous calcium release during the action potential.

T-Tubule Loss in Heart Failure

One of the earliest and most consequential structural changes in the failing myocardium is progressive loss and disorganization of the T-tubule network. Advanced heart failure can show 50–80% reduction in T-tubular density, with the remaining tubules becoming tortuous, dilated, and misaligned.

The primary mechanism is **calpain-mediated cleavage of junctophilin-2**. Elevated diastolic calcium in failing cells activates the protease calpain, which cleaves JPH2 — the very protein that anchors T-tubules to the jSR. Once this anchor is lost, the T-tubule membrane retracts from the SR surface, physically separating L-type channels from RyR2 clusters.

Orphaned RyR2 and Dyssynchronous Release

As T-tubules withdraw, the RyR2 clusters they once faced become *orphaned* — physically uncoupled from their L-type channel triggers. These orphaned RyR2s cannot be activated by the normal CICR mechanism (the L-type channel calcium signal no longer reaches them). Instead, they can only be activated secondarily, by the slower wave-like diffusion of calcium from neighboring intact dyads. The result is *dyssynchronous calcium release*: surface dyads fire promptly with the action potential, while interior orphaned clusters fire with a variable delay of 10–100 ms.

This dyssynchrony has dual consequences. First, the contractile force is reduced — not all sarcomeres are activated simultaneously. Second, the orphaned RyR2s, no longer under the tight local control of a nearby L-type channel, often exhibit increased sensitivity to luminal calcium and are prone to spontaneous opening. They become sources of diastolic calcium leak and spontaneous waves — contributing to both systolic dysfunction and arrhythmia susceptibility.

Engine V5.4 Connection

From a modeling perspective, the T-tubule remodeling story highlights a limitation of standard ionic models like TTP06 and ORd: they assume a single, well-coupled dyadic compartment. Capturing the spatial heterogeneity of heart failure — some CRUs coupled, some orphaned, some partially coupled — requires either multi-compartment extensions or spatially explicit subcellular models. This remains an active area of computational research, with multiscale models (2021–2024) integrating nanoscale dyadic dynamics into whole-cell and tissue-level simulations.

4.8 Emerging Frontiers: Imaging, Modeling, and Therapy

Super-Resolution Imaging of Living Cardiomyocytes

The past five years have seen transformative advances in visualizing calcium handling machinery in intact, living cells. **MINFLUX microscopy** (2024–2025) can now resolve individual RyR2 subunits and determine their 3D orientation within clusters — approaching atomic-level structural information in the native cellular environment. Live-cell photoactivated localization microscopy (PALM, 2022) has correlated nanoscale RyR2 cluster configuration with functional calcium spark properties, revealing that heart failure induces fragmentation of large RyR2 clusters into numerous smaller groupings with increased nearest-neighbor distances and impaired cooperative gating.

Next-Generation Calcium Indicators

Genetically encoded calcium indicators continue to evolve. The jRCaMP1b family (2023) achieves submillisecond rise times with the highest sensitivity yet — enabling real-time nanodomain calcium imaging. The NEMOer sensor family (2025) was specifically designed for endoplasmic/sarcoplasmic reticulum calcium imaging, overcoming the limited dynamic range of previous luminal indicators. These tools are beginning to provide direct experimental access to the $[Ca^{2+}]_{SR}$ dynamics that models have long predicted but experiments could not resolve.

Graph-Theoretic Prediction of Spark Fidelity

A striking computational advance (2023) uses graph theory to predict calcium spark probability directly from nanoscale cluster architecture. By constructing an adjacency graph of RyR2 positions within a CRU (nodes = channels, edges = functional coupling), researchers showed that the *maximum eigenvalue* of this adjacency matrix predicts spark probability with remarkable accuracy. This bridges the gap between structure (super-resolution images) and function (calcium spark recordings), explaining why some RyR2 clusters reliably produce sparks while others are functionally "silent."

With this deep understanding of calcium handling in hand, we are now prepared to examine the quantitative ionic models that encode these mechanisms computationally. The next chapter dissects the ten Tusscher–Panfilov 2006 model — 18 state variables that capture the calcium cycling machinery alongside every ionic current we have discussed.

The ten Tusscher–Panfilov 2006 Model

TTP06 is the workhorse model for human ventricular electrophysiology. With 18 state variables — 12 gating variables, 5 ionic concentrations, and the SR-release recovery variable R' — it captures every phase of the action potential we dissected in Chapter 3, plus the calcium cycling machinery that drives contraction.

5.1 18 State Variables: What and Why

A natural question when encountering TTP06 for the first time is: *why 18 variables?* The answer lies in the variety of currents and intracellular processes that shape the cardiac AP. Each current requires gates, and each ion species requires a concentration variable to track changes over multiple beats.

The 18 states fall into two natural groups:

12 gating variables (each between 0 and 1, following Hodgkin–Huxley-style kinetics):

Gate	Current	Role	Speed
m	I_{Na}	Na^+ activation	< 1 ms
h, j	I_{Na}	Na^+ fast and slow inactivation	1–100 ms
d	I_{CaL}	Ca^{2+} activation	1–5 ms
f, f_2	I_{CaL}	Ca^{2+} voltage inactivation	20–300 ms
f_{Ca}	I_{CaL}	Ca^{2+} -dependent inactivation	2–50 ms
r	I_{to}	Transient outward activation	1–5 ms
s	I_{to}	Transient outward inactivation	5–50 ms
x_{r1}, x_{r2}	I_{Kr}	Rapid delayed rectifier	10–100 ms
x_s	I_{Ks}	Slow delayed rectifier	100–500 ms

5 concentration variables + the SR-release recovery variable R' (dynamic, changing each beat):

Variable	Description	Typical Value
$[\text{Na}^+]_i$	Intracellular sodium	~ 10 mM
$[\text{K}^+]_i$	Intracellular potassium	~ 138 mM
$[\text{Ca}^{2+}]_i$	Cytoplasmic calcium (free)	~ 0.0001 mM
$[\text{Ca}^{2+}]_{\text{SR}}$	SR calcium store	~ 1 – 3 mM
$[\text{Ca}^{2+}]_{\text{ss}}$	Subspace calcium (near L-type channels)	~ 0.0001 mM
R'	SR release recovery variable	~ 1

The distinction between $[\text{Ca}^{2+}]_i$, $[\text{Ca}^{2+}]_{\text{ss}}$, and $[\text{Ca}^{2+}]_{\text{SR}}$ reflects the *compartmentalization* of calcium handling. Calcium entering through L-type channels first reaches a tiny subspace near the SR release channels, triggering CICR locally before diffusing into the bulk cytoplasm.

5.2 The Complete Current Inventory

TTP06 models 12 distinct membrane currents plus 4 internal calcium fluxes. Every membrane current contributes to the total ionic current:

$$I_{\text{ion}} = \underbrace{I_{\text{Na}} + I_{\text{CaL}}}_{\text{depolarizing}} + \underbrace{I_{\text{to}} + I_{\text{Kr}} + I_{\text{Ks}} + I_{\text{K1}} + I_{\text{pK}}}_{\text{repolarizing}} + \underbrace{I_{\text{NaK}} + I_{\text{NaCa}} + I_{\text{pCa}}}_{\text{pumps/exchangers}} + \underbrace{I_{\text{bNa}} + I_{\text{bCa}}}_{\text{background}} \quad (5.1)$$

This equation is the heart of the model: at every timestep, the simulator evaluates all 12 terms, sums them, and uses the total to update V_m . The sign convention follows physiology — positive current is outward (repolarizing), negative current is inward (depolarizing). The table below provides a complete reference, mapping each current to its AP phase, physiological role, and clinical significance.

Current	Type	Ion	AP Phase	Direction	Clinical Relevance
I_{Na}	Voltage-gated channel	Na^+	0 (upstroke)	Inward	Conduction velocity; Class I antiarrhythmics
I_{CaL}	Voltage-gated channel	Ca^{2+}	2 (plateau)	Inward	EC coupling trigger; Ca^{2+} channel blockers (verapamil)
I_{to}	Voltage-gated channel	K^+	1 (notch)	Outward	Transmural heterogeneity; Brugada syndrome
I_{Kr}	Voltage-gated channel	K^+	3 (repolarization)	Outward	Primary drug target; Long QT syndrome (hERG block)
I_{Ks}	Voltage-gated channel	K^+	3 (repolarization reserve)	Outward	Rate adaptation; LQT1; adrenergic modulation
I_{K1}	Inward rectifier	K^+	4 (rest) / 3 (late)	Outward	Resting potential; Andersen-Tawil syndrome (LQT7)
I_{pK}	Voltage-gated channel	K^+	2 (plateau)	Outward	Small contribution to repolarization reserve
I_{NaK}	ATP-driven pump	Na^+ / K^+	All	Net outward	Gradient maintenance; digitalis target
I_{NaCa}	Exchanger	$\text{Na}^+ / \text{Ca}^{2+}$	2–3 / 4	Both*	Calcium extrusion; DAD generation; heart failure
I_{pCa}	ATP-driven pump	Ca^{2+}	All	Outward	Fine calcium homeostasis
I_{bNa}	Background leak	Na^+	All	Inward	Resting Na^+ influx; balanced by I_{NaK}
I_{bCa}	Background leak	Ca^{2+}	All	Inward	Resting Ca^{2+} influx; balanced by pumps

* I_{NaCa} operates in *forward mode* (inward, extruding Ca^{2+}) during late repolarization and diastole, and in *reverse mode* (outward, bringing Ca^{2+} in) briefly during early depolarization.

5.3 Voltage-Gated Depolarizing Currents

Fast Sodium Current (I_{Na}) — Phase 0

The fast sodium current is responsible for the action potential upstroke and determines conduction velocity in tissue. It follows the classical HH formulation with three activation gates (m) and two inactivation gates (fast h and slow j):

$$I_{Na} = \bar{g}_{Na} m^3 h j (V_m - E_{Na}) \quad (5.2)$$

The maximal conductance $\bar{g}_{Na} = 14.838$ nS/pF is one of the largest in the model. The m^3 activation is ultrafast ($\tau_m < 1$ ms), providing the explosive upstroke. The dual inactivation (h fast, j slow) ensures that sodium channels close within a few milliseconds and remain closed throughout the plateau — creating the absolute refractory period. Physiologically, I_{Na} determines the maximum rate of rise $dV/dt_{max} \approx 250\text{--}350$ V/s, which is a direct clinical measure of sodium channel availability. Class I antiarrhythmic drugs (lidocaine, flecainide) work by reducing $\bar{g}_{Na}$, slowing conduction.

L-Type Calcium Current (I_{CaL}) — Phase 2

The L-type calcium current sustains the plateau and triggers excitation-contraction coupling. Unlike I_{Na} , it uses the Goldman-Hodgkin-Katz (GHK) formulation rather than an Ohmic driving force, because Ca^{2+} has a very low intracellular concentration and a 2+ valence:

$$I_{CaL} = P_{CaL} d f f_2 f_{Ca} \cdot 4 \frac{F^2}{RT} (V_m - 15) \frac{0.25 [Ca^{2+}]_{ss} e^{2F(V_m-15)/RT} - [Ca^{2+}]_o}{e^{2F(V_m-15)/RT} - 1} \quad (5.3)$$

Note three features that distinguish this from a simple Ohmic current. First, TTP06 uses a *permeability* $P_{CaL} = 3.98 \times 10^{-5}$ cm/s rather than a conductance — the natural parameter for the GHK formulation. Second, the voltage is shifted by -15 mV ($V_m - 15$), reflecting the experimental observation that the current-voltage relationship of L-type channels peaks near $+15$ mV rather than 0 mV. Third, the intracellular calcium carries an activity coefficient of 0.25, while the extracellular side uses bare $[Ca^{2+}]_o$ — this asymmetry accounts for the different ionic environments on each side. The four gates represent distinct biophysical mechanisms: d is voltage activation ($\tau_d \sim 1\text{--}5$ ms), f and f_2 are voltage-dependent inactivation (slow, $\tau \sim 20\text{--}300$ ms — this determines how long the plateau lasts), and f_{Ca} is *calcium-dependent inactivation* (CDI), which responds to the local subspace calcium $[Ca^{2+}]_{ss}$. CDI is the primary negative feedback mechanism that limits calcium entry: as CICR raises subspace calcium, f_{Ca} inactivates the channel, shutting off the trigger.

Intuition

The GHK equation replaces the simple $(V_m - E_{Ca})$ driving force with a nonlinear function of voltage and concentrations on both sides. Why? For ions with large concentration gradients ($[Ca^{2+}]_o/[Ca^{2+}]_i \approx 20,000$) and multivalent charge, the Ohmic approximation badly overestimates current at large depolarizations. The GHK formula — derived from the Nernst-Planck electrodiffusion equation — accounts for the exponential concentration profile across the membrane and gives physically correct saturation behavior.

5.4 Voltage-Gated Repolarizing Currents

Transient Outward Current (I_{to}) — Phase 1

The transient outward K^+ current creates the Phase 1 notch. It activates rapidly during depolarization (r gate, $\tau \sim 1\text{--}5$ ms) and inactivates within tens of milliseconds (s gate, $\tau \sim 5\text{--}50$ ms):

$$I_{to} = \bar{g}_{to} r s (V_m - E_K) \quad (5.4)$$

The maximal conductance $\bar{g}_{to}$ varies dramatically between cell types: 0.294 nS/pF in epicardial cells (deep notch) vs. 0.073 nS/pF in endocardial cells (minimal notch). This transmural gradient is the single largest source of AP shape variation across the ventricular wall and produces the J-point deflection on the surface ECG. In Brugada syndrome, enhanced I_{to} in right ventricular epicardium creates an exaggerated notch that can trigger reentrant arrhythmias.

Rapid Delayed Rectifier (I_{Kr}) — Phase 3

I_{Kr} is the primary repolarization current, carried by hERG (Kv11.1) channels. It has unusual kinetics — fast inactivation at positive voltages, rapid recovery from inactivation during repolarization — that produce a characteristic "resurgent" current during Phase 3:

$$I_{Kr} = \bar{g}_{Kr} \sqrt{[K^+]_o / 5.4} x_{r1} x_{r2} (V_m - E_K) \quad (5.5)$$

The $\sqrt{[K^+]_o / 5.4}$ factor captures the experimental observation that hERG conductance scales with the square root of extracellular potassium. The two gating variables x_{r1} (activation, $\tau \sim 10\text{--}100$ ms) and x_{r2} (inactivation, fast) together create the voltage-dependent rectification that makes I_{Kr} small during the plateau but large during Phase 3. Clinically, I_{Kr} is the most important current in cardiac pharmacology: hERG block is the leading cause of drug-induced Long QT syndrome, and the CiPA initiative focuses heavily on predicting hERG interactions *in silico*.

Slow Delayed Rectifier (I_{Ks}) — Repolarization Reserve

I_{Ks} activates very slowly ($\tau_{x_s} \sim 100\text{--}500$ ms) and provides a "safety net" when I_{Kr} is compromised:

$$I_{Ks} = \bar{g}_{Ks} x_s^2 (V_m - E_{Ks}) \quad (5.6)$$

Note that I_{Ks} uses the potassium–sodium reversal potential E_{Ks} (computed from the Goldman equation for K^+ and Na^+ permeability) rather than the pure potassium reversal E_K , because the KvLQT1/minK channel complex has slight Na^+ permeability. The x_s^2 activation gives the current a sigmoid-like voltage dependence with a slower foot than I_{Kr} . Under normal conditions, I_{Ks} contributes modestly to repolarization. Its importance emerges when I_{Kr} is blocked — then I_{Ks} becomes the backup repolarization pathway. This is the concept of *repolarization reserve*: a healthy heart can tolerate partial block of one K^+ channel because others compensate. Loss of repolarization reserve (e.g., combined $I_{Kr} + I_{Ks}$ reduction) dramatically increases arrhythmia risk.

Inward Rectifier (I_{K1}) — Phase 4 and Late Phase 3

$$I_{K1} = \bar{g}_{K1} \sqrt{[K^+]_o / 5.4} x_{K1,\infty}(V_m) (V_m - E_K) \quad (5.7)$$

I_{K1} is unique among TTP06 currents: it has *no dynamic gating variable*. Its rectification is instantaneous, captured entirely by the steady-state function $x_{K1,\infty}(V_m)$, which transitions from ~ 1 below -80 mV to ~ 0 above -40 mV. Physiologically, this reflects the block of Kir2.1 channels by intracellular Mg^{2+} and polyamines at depolarized voltages. The result is a current that behaves as a strong "anchor" at rest (holding V_m near $E_K \approx -88$ mV), vanishes during the plateau (allowing the ICaL/IKr balance to play out), and then reactivates during Phase 3 to accelerate the final phase of repolarization.

Plateau Potassium Current (I_{pK})

$$I_{pK} = \bar{g}_{pK} \frac{1}{1 + e^{(25 - V_m)/5.98}} (V_m - E_K) \quad (5.8)$$

I_{pK} is a small outward K^+ current active during the plateau. It has no dynamic gates — the sigmoidal voltage dependence is instantaneous. Its contribution to total current is minor ($\bar{g}_{pK} = 0.0146$ nS/pF), but it participates in the repolarization reserve and affects APD at the margins.

5.5 Pumps, Exchangers, and Background Currents

The six voltage-gated currents above shape the AP waveform on a beat-to-beat basis. But the cell must also maintain its ionic gradients over thousands of beats. This is the job of the pumps, exchangers, and background leak currents — a group that is often overlooked but is essential for long-term stability.

Na^+/K^+ ATPase (I_{NaK}) — The Gradient Keeper

$$I_{NaK} = \frac{P_{NaK} [K^+]_o [Na^+]_i}{([K^+]_o + K_{mK})([Na^+]_i + K_{mNa})} \cdot \frac{1}{1 + 0.1245 e^{-0.1 F V_m / RT} + 0.0353 e^{-F V_m / RT}} \quad (5.9)$$

The Na^+/K^+ -ATPase pumps 3 Na^+ ions out and 2 K^+ ions in per ATP hydrolyzed, generating a net outward current. This pump is what maintains the sodium and potassium gradients that all voltage-gated currents depend on. Without it, gradients would run down within minutes and the cell would depolarize. The current is voltage-dependent (less effective at positive voltages), saturates with intracellular Na^+ ($K_{mNa} = 40$ mM) and extracellular K^+ ($K_{mK} = 1$ mM), and is the target of cardiac glycosides (digoxin/digitalis): by blocking I_{NaK} , intracellular Na^+ rises, which reduces the driving force for the Na^+/Ca^{2+} exchanger, indirectly increasing intracellular Ca^{2+} and strengthening contraction (positive inotropy).

Na^+/Ca^{2+} Exchanger (I_{NaCa}) — The Bidirectional Bridge

$$I_{NaCa} = \frac{k_{NaCa} ([Na^+]_i^3 [Ca^{2+}]_o e^{F V_m / RT} - \alpha [Na^+]_o^3 [Ca^{2+}]_i e^{(\gamma-1) F V_m / RT})}{(K_{m,Na}^3 + [Na^+]_o^3)(K_{m,Ca} + [Ca^{2+}]_o)(1 + k_{sat} e^{(\gamma-1) F V_m / RT})} \quad (5.10)$$

The NCX exchanges 3 Na^+ for 1 Ca^{2+} and can operate in either direction depending on the electrochemical gradients. The numerator contains two terms: the first (forward) drives Na^+ in and Ca^{2+} out; the second (reverse, scaled by $\alpha = 2.5$) drives Na^+ out and Ca^{2+} in. The factor α sets the relative strength of re-

verse-mode transport. The denominator has three components: the K_m saturation terms for Na^+ and Ca^{2+} , and a voltage-dependent saturation factor $1 + k_{\text{sat}} e^{(\gamma-1)FV_m/RT}$ that prevents the exchanger from running at unphysically high rates at extreme potentials. In **forward mode** (dominant during late repolarization and diastole): Na^+ enters, Ca^{2+} is extruded $\rightarrow$ net inward current. This is the primary mechanism for removing calcium from the cytoplasm after each beat (contributing $\sim 28\%$ of calcium removal in human ventricular myocytes, with SERCA handling the remaining $\sim 70\%$). In **reverse mode** (brief, during early depolarization): Na^+ exits, Ca^{2+} enters $\rightarrow$ net outward current. The voltage-dependent partition factor $\gamma \approx 0.35$ determines the relative influence of voltage on the forward vs. reverse rates.

I_{NaCa} is critically important in disease. In heart failure, elevated $[\text{Na}^+]_i$ reduces forward-mode NCX activity, impairing calcium removal and contributing to diastolic dysfunction. In conditions of calcium overload, the inward current generated by forward-mode NCX during diastole can trigger *delayed afterdepolarizations* (DADs) — a major arrhythmia mechanism (Chapter 4, Section 4.5).

Sarcolemmal Ca^{2+} Pump (I_{pCa})

$$I_{\text{pCa}} = \frac{\bar{g}_{\text{pCa}} [\text{Ca}^{2+}]_i}{K_{\text{pCa}} + [\text{Ca}^{2+}]_i} \quad (5.11)$$

The plasma membrane Ca^{2+} -ATPase (PMCA) provides a secondary route for calcium extrusion — slower than NCX but with higher affinity ($K_{\text{pCa}} = 0.0005$ mM). It contributes only ~ 1 – 2% of total calcium removal per beat but is important for fine-tuning the diastolic calcium level over many beats.

Background Leak Currents (I_{bNa} , I_{bCa})

$$I_{\text{bNa}} = \bar{g}_{\text{bNa}} (V_m - E_{\text{Na}}), \quad I_{\text{bCa}} = \bar{g}_{\text{bCa}} (V_m - E_{\text{Ca}}) \quad (5.12)$$

These small Ohmic currents represent the passive leak of Na^+ and Ca^{2+} through non-specific channels. Their conductances are tiny ($\bar{g}_{\text{bNa}} = 2.9 \times 10^{-4}$, $\bar{g}_{\text{bCa}} = 5.92 \times 10^{-4}$ nS/pF), but they serve a critical bookkeeping role: they provide the steady inward Na^+ and Ca^{2+} leak that the pumps (I_{NaK} , I_{pCa} , NCX) must counterbalance. Without these background currents, the model would slowly drift to non-physiological concentrations over hundreds of beats.

△ Key Insight

The balance between background leak currents and pump/exchanger currents is what determines the long-term stability of ionic concentrations. If you modify pump parameters (e.g., reducing I_{NaK} to simulate ischemia), you must simulate hundreds of beats to reach a new steady state — the first few beats will look normal, but gradual Na^+ accumulation will progressively alter the AP shape. This is why cardiac modelers must always run to steady state before collecting data.

5.6 Currents During Each AP Phase — A Visual Map



Figure 5.1. Activity map of all 12 TTP06 membrane currents across the five AP phases. Bar width represents the temporal extent; opacity represents relative magnitude. Red bars are inward (depolarizing), blue bars are outward (repolarizing), gray bars are housekeeping currents active throughout the cycle.

Mathematical Note

All gating variables in TTP06 satisfy the same relaxation ODE we derived in equation (1.3): $\frac{dx}{dt} = (x_\infty - x)/\tau_x$. This uniformity is what makes the Rush–Larsen method (Chapter 9) so powerful: you can integrate every gate with the same exponential formula $x^{n+1} = x_\infty + (x^n - x_\infty)e^{-\Delta t/\tau_x}$, regardless of how stiff the kinetics are. The concentration variables ($[Na^+]_i$, $[K^+]_i$, etc.) do *not* follow relaxation kinetics and must be integrated with Forward Euler or a similar method.

5.7 Endo, Epi, and M-Cells: Transmural Heterogeneity

One of TTP06's most important features is its ability to represent three distinct cell types across the ventricular wall, each with a different AP morphology:

Epicardial cells (outer wall): Large I_{to} → deep Phase 1 notch → shorter AP. **Endocardial cells** (inner wall): Small I_{to} → minimal notch → moderate AP duration. **M-cells** (mid-wall): Small I_{Ks} → very long plateau → longest AP duration.

These differences arise from changing just a few maximal conductance values ($\bar{g}_{to}$, $\bar{g}_{Ks}$) and some kinetic parameters. The fact that the same model structure, with minor parameter adjustments, reproduces three qualitatively different AP shapes is a testament to how well TTP06 captures the underlying biophysics.

Engine V5.4 Connection

In Engine V5.4, the TTP06 implementation lives in the `cardiac_sim/ionic/ttp06/` package: `model.py` (the `TTP06Model` class), `currents.py` (all 12 current functions), `gating.py` (steady-states and time constants), `calcium.py` (concentration update equations), and `parameters.py` (the `StateIndex` enum and `TTP06Parameters` dataclass). V is stored separately; the 18 ionic states live in a tensor of shape `(n_cells, 18)`. The `cell_type` parameter (`CellType.ENDO`, `CellType.EPI`, `CellType.M_CELL`) selects endo/epi/M-cell variants by switching maximal conductances and I_{to} kinetics. A single tissue simulation can represent the full transmural gradient by assigning different cell types to different spatial regions — essential for realistic ECG simulation.

5.8 Calcium Handling and CICR

The calcium subsystem is perhaps the most physiologically important part of TTP06, because calcium is the link between electrical excitation and mechanical contraction. The model tracks calcium through three compartments connected by fluxes:

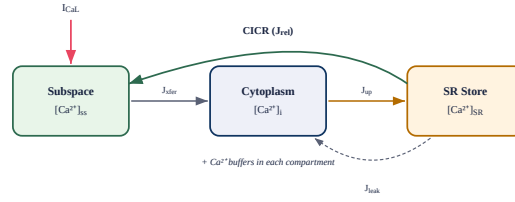


Figure 5.2. Calcium cycling in TTP06. The L-type current I_{CaL} brings Ca^{2+} into the subspace, triggering CICR from the SR (J_{rel}). Ca^{2+} diffuses to the cytoplasm (J_{diff}), drives contraction, then is pumped back into the SR by SERCA (J_{up}).

The CICR mechanism works like a trigger and amplifier:

Trigger: L-type calcium channels open during Phase 2, admitting a small amount of Ca^{2+} into the subspace. This raises $[Ca^{2+}]_{ss}$.

Amplification: The elevated $[Ca^{2+}]_{ss}$ activates ryanodine receptors (RyR) on the SR membrane, which release a much larger bolus of Ca^{2+} from the SR store into the subspace. This is the "release" flux J_{rel} .

Diffusion and contraction: Ca^{2+} diffuses from the subspace to the bulk cytoplasm (J_{diff}), where it binds to troponin C and triggers the contractile machinery of the sarcomere.

Recovery: The SERCA pump (J_{up}) actively transports Ca^{2+} from the cytoplasm back into the SR, refilling the store for the next beat. A small leak (J_{leak}) allows some Ca^{2+} to trickle back out.

Getting this calcium balance right is critical: if the SR is overloaded, spontaneous Ca^{2+} release can trigger delayed afterdepolarizations (DADs) and arrhythmias. If the SR is depleted, contractile force drops and the AP shape distorts. The three-compartment model in TTP06 captures these dynamics with reasonable fidelity while remaining computationally tractable for tissue-scale simulations.

The O'Hara–Rudy 2011 Model

The O'Hara–Rudy dynamic (ORd) model was built from an unprecedented dataset of undiseased human ventricular myocyte recordings. With 41 state variables, 15 membrane currents, and a CaMKII signaling pathway that confers rate adaptation, it represents the current gold standard for human ventricular electrophysiology simulation. This chapter provides the same comprehensive treatment given to TTP06 in Chapter 5 — every current, every equation, organized by physiological function.

6.1 41 State Variables: What and Why

ORd has more than twice the state variables of TTP06 (41 vs. 18). The additional complexity arises from three sources: (1) separate fast and slow inactivation gates for most currents, (2) CaMKII-phosphorylated variants of several gates, and (3) two additional ionic compartments (subspace sodium and potassium). Every state variable has a clear biophysical justification.

29 gating variables (each between 0 and 1, following Hodgkin–Huxley-style kinetics):

Gate(s)	Current	Role	CaMKII variant?
m	I_{Na}	Fast Na^+ activation	No
h_f, h_s	I_{Na}	Fast and slow inactivation	h_{sp} (slow, phosphorylated)
j	I_{Na}	Recovery from inactivation	j_p
m_L	I_{NaL}	Late Na^+ activation	No
h_L	I_{NaL}	Late Na^+ inactivation	h_{Lp}
d	I_{CaL}	Ca^{2+} activation	No
f_f, f_s	I_{CaL}	Fast and slow voltage inactivation	f_{fp}
$f_{Ca,f}, f_{Ca,s}$	I_{CaL}	Fast and slow Ca^{2+} -dependent inactivation	$f_{Ca,fp}$
j_{Ca}	I_{CaL}	Ca^{2+} -dependent recovery	No
n_{Ca}	I_{CaL}	CaMKII phosphorylation transition	—
a	I_{to}	Transient outward activation	a_p
i_F, i_S	I_{to}	Fast and slow inactivation	i_{Fp}, i_{Sp}
$x_{r,f}, x_{r,s}$	I_{Kr}	Fast and slow activation	No
x_{s1}, x_{s2}	I_{Ks}	Slow delayed rectifier gates	No
x_{K1}	I_{K1}	Inward rectifier (now dynamic!)	No

A key design principle of ORd is the *CaMKII-phosphorylated population* concept. For currents modulated by CaMKII (I_{Na} , I_{NaL} , I_{CaL} , I_{to}), the model maintains separate gating variables for the phosphorylated channel population. The total current is then a weighted average: $(1 - \Phi) \cdot I_{normal} + \Phi \cdot I_{phosphorylated}$, where Φ is the CaMKII-dependent phosphorylation fraction. This doubles the gate count for affected currents but allows rate-dependent modulation of channel kinetics.

8 concentration variables (dynamic, changing each beat):

Variable	Description	Typical Value	New vs. TTP06?
$[\text{Na}^+]_i$	Intracellular sodium (bulk)	~ 7 mM	Same concept
$[\text{Na}^+]_{ss}$	Subspace sodium	~ 7 mM	New
$[\text{K}^+]_i$	Intracellular potassium (bulk)	~ 145 mM	Same concept
$[\text{K}^+]_{ss}$	Subspace potassium	~ 145 mM	New
$[\text{Ca}^{2+}]_i$	Cytoplasmic calcium (free)	~ 0.0001 mM	Same concept
$[\text{Ca}^{2+}]_{ss}$	Subspace calcium	~ 0.0001 mM	Same concept
$[\text{Ca}^{2+}]_{n\text{sr}}$	Network SR calcium	~ 1.2 mM	Split from $[\text{Ca}^{2+}]_{\text{SR}}$
$[\text{Ca}^{2+}]_{j\text{sr}}$	Junctional SR calcium	~ 1.2 mM	Split from $[\text{Ca}^{2+}]_{\text{SR}}$

Additional state variables:

Variable	Description	Role
$\text{CaMK}_{\text{trap}}$	CaMKII trapped (autophosphorylated) fraction	Long-term kinase memory; drives rate adaptation
V_m	Membrane potential	Primary electrical variable
$J_{\text{rel,NP}}$	SR release flux (non-phosphorylated)	RyR release dynamics
$J_{\text{rel,CaMK}}$	SR release flux (CaMKII-phosphorylated)	Enhanced RyR release

The most important structural differences from TTP06 are the addition of subspace Na^+ and K^+ compartments (reflecting the fact that these ions also have local concentrations near the membrane), the split of the SR into network and junctional compartments (SERCA pumps into the nSR; RyR releases from the jSR), and the CaMKII trapped state that provides long-term "memory" of recent heart rate.

Intuition

Why does ORd track subspace Na^+ and K^+ ? In TTP06, only Ca^{2+} had a subspace compartment because its concentration can change dramatically during a single beat ($\times 10$ – 100 locally). Na^+ and K^+ concentrations change by much less ($< 1\%$), so a single bulk compartment seemed sufficient. But ORd found that even small Na^+ gradients near the membrane affect NCX and Na^+/K^+ -ATPase activity — the subspace compartments improve accuracy when these transporters are the focus.

6.2 The Complete Current Inventory

ORd models 15 distinct membrane currents (the sodium–calcium exchanger I_{NaCa} is split into junctional and subspace components below, so equation (6.1) lists 16 terms). Every current contributes to the total ionic current:

$$\begin{aligned}
 I_{\text{ion}} = & \underbrace{I_{\text{Na}} + I_{\text{NaL}} + I_{\text{CaL}}}_{\text{depolarizing}} + \underbrace{I_{\text{to}} + I_{\text{Kr}} + I_{\text{Ks}} + I_{\text{K1}} + I_{\text{Kb}}}_{\text{repolarizing}} \\
 & + \underbrace{I_{\text{NaK}} + I_{\text{NaCa,i}} + I_{\text{NaCa,ss}} + I_{\text{pCa}}}_{\text{pumps/exchangers}} + \underbrace{I_{\text{Na,b}} + I_{\text{Ca,b}}}_{\text{background}} + \underbrace{I_{\text{ClCa}} + I_{\text{Clb}}}_{\text{chloride}}
 \end{aligned} \tag{6.1}$$

Compared to TTP06's 12 currents, ORd adds I_{NaL} (late sodium), splits I_{NaCa} into two populations, adds chloride currents (I_{ClCa} , I_{Clb}), and replaces the plateau potassium current I_{pK} with a background potas-

sium current I_{Kb} . The L-type calcium current is further decomposed into three ionic components ($I_{CaL,Ca}$, $I_{CaL,Na}$, $I_{CaL,K}$). The table below provides the complete reference.

Current	Type	Ion	AP Phase	Direction	Clinical Relevance
I_{Na}	Voltage-gated channel	Na^+	0 (upstroke)	Inward	Conduction velocity; Class I antiarrhythmics
I_{NaL}	Voltage-gated channel	Na^+	2 (plateau)	Inward	NEW: APD modulation; ranolazine target; LQT3
I_{CaL}	Voltage-gated channel	Ca^{2+} / Na^+ / K^+	2 (plateau)	Inward	EC coupling trigger; 3 ionic components; CaMKII-modulated
I_{to}	Voltage-gated channel	K^+	1 (notch)	Outward	Transmural heterogeneity; Brugada syndrome
I_{Kr}	Voltage-gated channel	K^+	3 (repolarization)	Outward	hERG block; drug-induced LQTS; CiPA assessment
I_{Ks}	Voltage-gated channel	K^+	3 (repolarization reserve)	Outward	Rate adaptation (CaMKII); LQT1; β -adrenergic
I_{K1}	Inward rectifier	K^+	4 (rest) / 3 (late)	Outward	Resting potential; now with dynamic gate
I_{Kb}	Background leak	K^+	All	Outward	Replaces TTP06's I_{pK} ; small repolarization
I_{NaK}	ATP-driven pump	Na^+ / K^+	All	Net outward	Thermodynamic cycle model; digitalis target
$I_{NaCa,i}$	Exchanger	Na^+ / Ca^{2+}	2–3 / 4	Both*	Bulk cytoplasm NCX; calcium extrusion
$I_{NaCa,ss}$	Exchanger	Na^+ / Ca^{2+}	2–3 / 4	Both*	NEW: Subspace NCX; local Ca^{2+} control
I_{pCa}	ATP-driven pump	Ca^{2+}	All	Outward	Fine calcium homeostasis (PMCA)
I_{Nab}	Background leak	Na^+	All	Inward	Resting Na^+ influx; balanced by I_{NaK}
I_{Cab}	Background leak	Ca^{2+}	All	Inward	Resting Ca^{2+} influx; GHK formulation
I_{ClCa}	Ca^{2+} -activated channel	Cl^-	1–2	Inward†	NEW: Contributes to Phase 1 notch and plateau
I_{Clb}	Background leak	Cl^-	All	Inward†	NEW: Background Cl^- leak

* I_{NaCa} operates in forward mode (inward, extruding Ca^{2+}) during late repolarization and diastole, and in reverse mode (outward, bringing Ca^{2+} in) briefly during early depolarization. † Cl^- currents are depolarizing (inward by convention) because Cl^- efflux = inward current, with $E_{Cl} \approx -40$ mV.

6.3 Voltage-Gated Depolarizing Currents

Fast Sodium Current (I_{Na}) — Phase 0

The fast sodium current retains the classical HH structure from TTP06, but with two key upgrades: refitted kinetics from undiseased human data, and CaMKII phosphorylation of the slow inactivation gate. The total current blends non-phosphorylated and phosphorylated channel populations:

$$I_{Na} = \bar{g}_{Na} m^3 [(1 - \Phi_{Na}) h j + \Phi_{Na} h_p j_p] (V_m - E_{Na}) \quad (6.2)$$

Here Φ_{Na} is the CaMKII phosphorylation fraction for the sodium channel, and the composite inactivation gate h is a weighted sum of fast and slow components: $h = A_{hf} h_f + (1 - A_{hf}) h_s$, with the weight A_{hf} depending on voltage (fast inactivation dominates at depolarized potentials). The phosphorylated population uses $h_p = A_{hf} h_f + (1 - A_{hf}) h_{sp}$, where h_{sp} has a shifted voltage dependence — CaMKII phosphorylation slows recovery from inactivation, contributing to rate-dependent sodium channel availability. The maximal conductance $\bar{g}_{Na} = 75$ mS/ μ F is substantially larger than TTP06's 14.838 nS/pF (note different units), reflecting the refitting to human data.

Late Sodium Current (I_{NaL}) — Phase 2

The late sodium current is the most important addition in ORd relative to TTP06. It represents a small but persistent Na^+ influx that flows throughout the plateau — generated by sodium channels that briefly re-open or fail to fully inactivate:

$$I_{NaL} = \bar{g}_{NaL} m_L [(1 - \Phi_{NaL}) h_L + \Phi_{NaL} h_{Lp}] (V_m - E_{Na}) \quad (6.3)$$

The maximal conductance $\bar{g}_{NaL} = 0.0075$ mS/ μ F is roughly 1/10,000th of peak I_{Na} , but because I_{NaL} flows for 200+ ms (vs. <2 ms for peak I_{Na}), it carries significant charge and contributes approximately 5% of APD. The activation gate m_L is similar to the fast sodium m gate but with slightly slower kinetics. The inactivation gate h_L has a time constant of ~ 200 ms — orders of magnitude slower than the fast sodium h gate — explaining why this current persists through the plateau.

CaMKII phosphorylation of I_{NaL} (h_{Lp}) shifts the inactivation, increasing the late current at faster heart rates. This creates a positive feedback loop: faster rate $\rightarrow$ more Ca^{2+} entry $\rightarrow$ more CaMKII activation $\rightarrow$ more I_{NaL} $\rightarrow$ more Na^+ entry $\rightarrow$ potentially dangerous if unchecked. Clinically, this pathway is relevant to arrhythmia in heart failure.

△ Key Insight

I_{NaL} is the primary target of **ranolazine**, an antianginal drug that selectively inhibits late sodium current without significantly affecting peak I_{Na} . This specificity is impossible to study computationally without explicitly modeling I_{NaL} — one of the main reasons TTP06 is inadequate for drug safety assessment. In Long QT syndrome type 3 (LQT3), gain-of-function mutations in SCN5A increase I_{NaL} by 2–5 fold, dangerously prolonging the plateau. ORd can reproduce this pathology by simply scaling $\bar{g}_{NaL}$.

L-Type Calcium Current (I_{CaL}) — Phase 2

The L-type calcium current in ORd is substantially more complex than in TTP06. It is decomposed into three ionic components, reflecting the imperfect selectivity of the Cav1.2 channel:

$$I_{\text{CaL}} = I_{\text{CaL,Ca}} + I_{\text{CaL,Na}} + I_{\text{CaL,K}} \quad (6.4)$$

Each component uses the GHK flux equation with its respective permeability. The calcium component (by far the largest) is:

$$I_{\text{CaL,Ca}} = P_{\text{Ca}} d \left[(1 - \Phi_{\text{CaL}}) f j_{\text{Ca}} f_{\text{Ca}} + \Phi_{\text{CaL}} f_p j_{\text{Ca}} f_{\text{Ca,p}} \right] \cdot \Psi_{\text{Ca}}(V_m, [\text{Ca}^{2+}]_{\text{ss}}) \quad (6.5)$$

where Ψ_{Ca} is the GHK flux: $\Psi_{\text{Ca}} = 4 \frac{F^2}{RT} V_m \frac{[\text{Ca}^{2+}]_{\text{ss}} e^{2FV_m/RT} - 0.341 [\text{Ca}^{2+}]_e}{e^{2FV_m/RT} - 1}$, and the composite gates are: $f = A_{ff} f_f + (1 - A_{ff}) f_s$ (voltage inactivation, blending fast and slow components) and $f_{\text{Ca}} = A_{\text{Ca},f} f_{\text{Ca},f} + (1 - A_{\text{Ca},f}) f_{\text{Ca},s}$ (calcium-dependent inactivation).

The CaMKII-phosphorylated population (Φ_{CaL}) uses separate gates f_p and $f_{\text{Ca,p}}$ with shifted kinetics — phosphorylation facilitates channel reopening and slows CDI, increasing calcium influx at faster rates. This CaMKII-mediated enhancement of I_{CaL} is counterbalanced by increased SERCA activity (also CaMKII-dependent), maintaining calcium homeostasis while strengthening contraction — a key component of the positive force-frequency relationship.

The Na^+ and K^+ components follow the same gating but use sodium and potassium permeabilities ($P_{\text{Na}} = 0.00125 P_{\text{Ca}}$ and $P_{\text{K}} = 3.574 \times 10^{-4} P_{\text{Ca}}$) and their respective GHK fluxes. Though individually small, these components contribute to the ion balance over many beats.

Mathematical Note

Why split I_{CaL} into three components? In TTP06, only Ca^{2+} permeation was modeled because Ca^{2+} is the dominant charge carrier. But when tracking Na^+ and K^+ concentrations dynamically over thousands of beats (as ORd does with its subspace compartments), even the small Na^+ and K^+ fluxes through L-type channels contribute to concentration drift. Ignoring them causes slow, insidious accumulation errors that manifest as AP duration changes after hundreds of beats — exactly the kind of subtle bug that plagues long-running simulations.

6.4 Voltage-Gated Repolarizing Currents

Transient Outward Current (I_{to}) — Phase 1

The transient outward current in ORd has separate fast and slow inactivation gates (replacing TTP06's single s gate), plus CaMKII-phosphorylated variants:

$$I_{\text{to}} = \bar{g}_{\text{to}} \left[(1 - \Phi_{\text{to}}) a i + \Phi_{\text{to}} a_p i_p \right] (V_m - E_{\text{K}}) \quad (6.6)$$

where $i = A_{iF} i_F + (1 - A_{iF}) i_S$ blends fast and slow inactivation, with the weight A_{iF} depending on voltage. The CaMKII-phosphorylated inactivation $i_p = A_{iF} i_{Fp} + (1 - A_{iF}) i_{Sp}$ has shifted kinetics. The maximal conductance $\bar{g}_{\text{to}}$ retains the dramatic transmural gradient: 0.02 mS/ μF in endocardium vs. 0.08 mS/ μF in epicardium and M-cells.

Rapid Delayed Rectifier (I_{Kr}) — Phase 3

I_{Kr} in ORd uses refitted kinetics from undiseased human hERG data. Like TTP06, it has two activation components, but ORd blends fast and slow activation:

$$I_{Kr} = \bar{g}_{Kr} \sqrt{[K^+]_o / 5.4} x_r r_{Kr}(V_m) (V_m - E_K) \quad (6.7)$$

Here $x_r = A_{x_r,f} x_{r,f} + (1 - A_{x_r,f}) x_{r,s}$ is a weighted blend of fast and slow activation, and $r_{Kr}(V_m)$ is an instantaneous rectification factor that captures the voltage-dependent inactivation of hERG channels. The $\sqrt{[K^+]_o / 5.4}$ factor preserves the extracellular K^+ dependence from TTP06. Notably, I_{Kr} is *not* CaMKII-modulated in ORd — its kinetics are purely voltage-dependent. This is the key current for the CiPA (Comprehensive *in vitro* Proarrhythmia Assay) initiative, which uses ORd as its reference model for predicting drug-induced arrhythmia risk.

Slow Delayed Rectifier (I_{Ks}) — Repolarization Reserve

$$I_{Ks} = \bar{g}_{Ks} K_{s,Ca} x_{s1} x_{s2} (V_m - E_{Ks}) \quad (6.8)$$

A key difference from TTP06 is the calcium-sensitivity factor $K_{s,Ca} = 1 + 0.6 / (1 + (3.8 \times 10^{-5} / [Ca^{2+}]_i)^{1.4})$, which increases I_{Ks} when intracellular calcium is elevated. At resting calcium (~ 100 nM), $K_{s,Ca} \approx 1$ and the current is near baseline. During systolic calcium peaks (~ 1 μ M), $K_{s,Ca}$ increases to ~ 1.6 , boosting repolarization reserve precisely when the cell needs it most. Combined with CaMKII modulation of $\bar{g}_{Ks}$ at faster heart rates, this creates robust rate-dependent enhancement of repolarization reserve. The reversal potential E_{Ks} remains a mixed Na^+/K^+ potential, computed from the Goldman equation. The maximal conductance shows strong transmural variation: $\bar{g}_{Ks} = 0.0034$ mS/ μ F (baseline) in endocardial cells, scaled **1.4 $\times$** in epicardial cells, with reduced values in M-cells explaining the prolonged M-cell APD and the T-wave morphology on the ECG.

Inward Rectifier (I_{K1}) — Phase 4 and Late Phase 3

$$I_{K1} = \bar{g}_{K1} \sqrt{[K^+]_o} x_{K1} r_{K1}(V_m) (V_m - E_K) \quad (6.9)$$

Unlike TTP06 — where I_{K1} used only an instantaneous steady-state function — ORd gives I_{K1} a *dynamic* gating variable x_{K1} with its own time constant (though still fast, $\tau \sim 1\text{--}10$ ms). The instantaneous rectification factor $r_{K1}(V_m)$ captures the polyamine block at depolarized voltages. This formulation better reproduces the terminal repolarization phase and the resting membrane potential measured in human ventricular myocytes.

Background Potassium Current (I_{Kb})

$$I_{Kb} = \bar{g}_{Kb} x_{Kb}(V_m) (V_m - E_K) \quad (6.10)$$

I_{Kb} replaces TTP06's I_{pK} . The gating is instantaneous (a sigmoidal function of voltage). Like its predecessor, it provides a small outward current during the plateau.

6.5 Pumps, Exchangers, and Background Currents

Na^+/K^+ ATPase (I_{NaK}) — Thermodynamic Cycle Model

The Na^+/K^+ -ATPase in ORd represents the most significant single-equation change from TTP06. Instead of the Hill-type empirical formulation, ORd uses a thermodynamically constrained cycle model that explic-

itly accounts for the pump's conformational states:

$$I_{\text{NaK}} = 30 \frac{z_{\text{Na}} \cdot a_1 \cdot k_{1p} - z_{\text{Na}} \cdot b_1 \cdot k_{1m}}{1 + \frac{a_1 k_{1p}}{b_2 k_{2m}} + \frac{b_1 k_{1m}}{a_2 k_{2p}} + \frac{a_2 k_{2p}}{b_3 k_{3m}} + \frac{b_2 k_{2m}}{a_3 k_{3p}} + \frac{a_3 k_{3p}}{b_4 k_{4m}} + \frac{b_3 k_{3m}}{a_4 k_{4p}}} \quad (6.11)$$

The rate constants a_i , b_i , k_{ip} , k_{im} depend on $[\text{Na}^+]_i$, $[\text{K}^+]_o$, ATP, and V_m through microscopic detailed balance. This formulation satisfies the second law of thermodynamics — the pump cannot generate flux against its electrochemical gradient without ATP hydrolysis, a constraint that the simpler TTP06 formulation does not enforce. The practical consequence is more accurate pump behavior under extreme conditions (ischemia, low ATP, hyperkalemia) where the driving forces are close to equilibrium.

Na⁺/Ca²⁺ Exchanger — Split Populations

ORD explicitly models two NCX populations operating in different calcium environments:

$$I_{\text{NaCa}} = I_{\text{NaCa,i}} + I_{\text{NaCa,ss}} \quad (6.12)$$

where $I_{\text{NaCa,i}}$ (80% of NCX) senses bulk cytoplasmic $[\text{Ca}^{2+}]_i$ and $[\text{Na}^+]_i$, while $I_{\text{NaCa,ss}}$ (20% of NCX) senses subspace $[\text{Ca}^{2+}]_{ss}$ and $[\text{Na}^+]_{ss}$. Each uses a thermodynamic formulation:

$$I_{\text{NaCa,x}} = 0.8 \bar{g}_{\text{NaCa}} \cdot \frac{k_{3p} k_{4pp} [\text{Na}^+]_x^3 - k_{3pp} k_{4p} [\text{Ca}^{2+}]_x}{(1 + Q_{10})(h_1 \cdot h_2 + h_3)(1/h_7 + h_8)} \quad (6.13)$$

The split is physiologically motivated: NCX molecules near the dyadic cleft "see" a very different calcium signal (high, fast transient from CICR) than those in the bulk cytoplasm (lower, slower rise). In TTP06's single-population NCX, forward-mode extrusion is driven by a spatially averaged $[\text{Ca}^{2+}]_i$ that underestimates the local signal near the release sites. The subspace NCX population responds to the actual local calcium in the dyad, providing more accurate calcium cycling and a better match to experimental measurements of NCX current during the action potential.

Sarcolemmal Ca²⁺ Pump (I_{pCa})

$$I_{\text{pCa}} = \frac{\bar{g}_{\text{pCa}} [\text{Ca}^{2+}]_i}{K_{m,\text{pCa}} + [\text{Ca}^{2+}]_i} \quad (6.14)$$

The PMCA formulation is essentially unchanged from TTP06: a Michaelis–Menten-style saturating function of cytoplasmic calcium. It contributes roughly 1–2% of total calcium extrusion per beat.

Background Leak Currents (I_{Nab} , I_{Cab})

$$I_{\text{Nab}} = \bar{g}_{\text{Nab}} (V_m - E_{\text{Na}}) \quad (6.15)$$

$$I_{\text{Cab}} = P_{\text{Cab}} \cdot \Psi_{\text{Ca}}(V_m, [\text{Ca}^{2+}]_i) \quad (6.16)$$

The sodium background current retains TTP06's Ohmic formulation. However, the calcium background current in ORd switches from an Ohmic to a GHK formulation (Ψ_{Ca}), providing more physically correct behavior at the very low intracellular Ca^{2+} concentrations where the Ohmic approximation breaks down.

Chloride Currents (I_{ClCa} , I_{Clb}) — New in ORd

$$I_{\text{ClCa}} = \bar{g}_{\text{ClCa}} \frac{[\text{Ca}^{2+}]_{\text{ss}}}{K_{d,\text{ClCa}} + [\text{Ca}^{2+}]_{\text{ss}}} (V_m - E_{\text{Cl}}) \quad (6.17)$$

$$I_{\text{Clb}} = \bar{g}_{\text{Clb}} (V_m - E_{\text{Cl}}) \quad (6.18)$$

ORd introduces chloride currents, absent from TTP06. The calcium-activated chloride current I_{ClCa} is gated by subspace calcium and contributes to the Phase 1 notch alongside I_{to} . With $E_{\text{Cl}} \approx -40$ mV (between rest and plateau), this current is depolarizing at rest and repolarizing during the plateau, adding nuance to the Phase 1–2 transition. The background chloride leak I_{Clb} is small but ensures chloride balance.

⚙ Engine V5.4 Connection

Engine V5.4's ORd implementation omits the chloride currents (I_{ClCa} and I_{Clb}). Their combined contribution is typically <1% of total ionic current, and many benchmark ORd implementations (including the CiPA reference) also exclude them. If chloride sensitivity is needed for a specific study, these currents can be added as post-hoc corrections without modifying the core state variable system.

6.6 The CaMKII Signaling Pathway

The calcium/calmodulin-dependent protein kinase II (CaMKII) signaling pathway is the largest conceptual addition in ORd and the primary mechanism underlying rate adaptation. CaMKII is activated by calcium/calmodulin binding and can autophosphorylate to maintain activity even after calcium levels fall — creating a molecular "memory" of recent heart rate.

The active CaMKII fraction in ORd is:

$$\text{CaMK}_{\text{active}} = \text{CaMK}_{\text{bound}} + \text{CaMK}_{\text{trap}} \quad (6.19)$$

where $\text{CaMK}_{\text{bound}} = \text{CaMK}_0 (1 - \text{CaMK}_{\text{trap}}) / (1 + K_m / [\text{Ca}^{2+}]_{\text{ss}})$ is the fraction bound to calcium/calmodulin (instantaneous, set by current subspace Ca^{2+}), and $\text{CaMK}_{\text{trap}}$ is the autophosphorylated fraction that decays slowly:

$$\frac{d \text{CaMK}_{\text{trap}}}{dt} = \alpha_{\text{CaMK}} \text{CaMK}_{\text{bound}} \text{CaMK}_{\text{active}} - \beta_{\text{CaMK}} \text{CaMK}_{\text{trap}} \quad (6.20)$$

The time constant for CaMKII trapping is on the order of seconds — much slower than any gating variable. This creates a hierarchical timescale structure:

Milliseconds: Gate kinetics (m, h, d, f, etc.) — respond within a single AP. **Hundreds of milliseconds:** Concentration changes ($[\text{Ca}^{2+}]_i$, $[\text{Na}^+]_i$) — settle over a few beats. **Seconds to minutes:** CaMKII trapping — requires dozens to hundreds of beats to equilibrate.

Active CaMKII phosphorylates four main targets in ORd:

Target	Effect of CaMKII Phosphorylation	Consequence for AP
I_{CaL}	Slowed voltage inactivation; enhanced recovery	Increased Ca^{2+} entry → stronger contraction
I_{NaL}	Slowed inactivation → increased late Na^+	Slight APD prolongation at fast rates
RyR / J_{rel}	Increased open probability and release fraction	Larger Ca^{2+} transient; potential for diastolic leak
SERCA / J_{up}	Faster Ca^{2+} reuptake into SR	Faster relaxation; maintains SR load at fast rates

The net effect of CaMKII at faster heart rates is: more calcium entry per beat (via enhanced I_{CaL}), faster calcium reuptake (via enhanced SERCA), larger SR release (via enhanced RyR sensitivity), and slightly increased late sodium influx. The enhanced SERCA activity dominates the APD effect — by removing calcium from the cytoplasm faster, it reduces the duration of the calcium transient and the plateau currents that depend on it, resulting in APD shortening. This is how ORd achieves rate adaptation.

△ Key Insight

CaMKII makes ORd simulations significantly more expensive than TTP06. With 41 state variables (vs. 18), the per-cell cost roughly doubles. But the bigger issue is the slow CaMKII equilibration time: reaching steady state after a rate change requires 300–600 beats (2–5 minutes of simulated time), compared to 50–100 beats for TTP06. For tissue-scale studies where CaMKII effects are not the primary focus, TTP06 remains the pragmatic choice.

6.7 Currents During Each AP Phase — A Visual Map

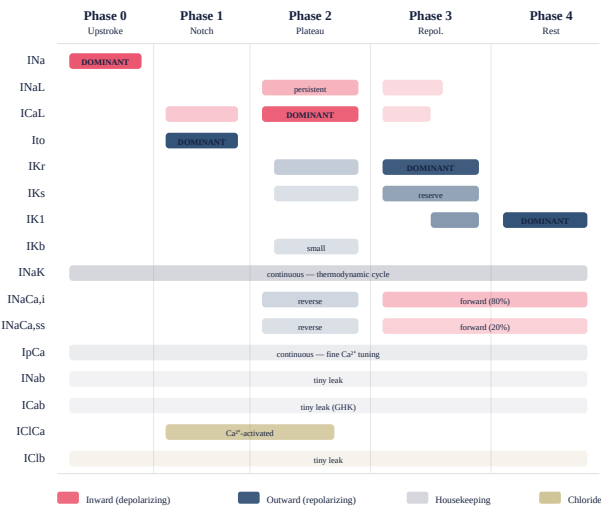


Figure 6.1. Activity map of all ORd membrane currents across the five AP phases. Bar width represents temporal extent; opacity represents relative magnitude. Red bars are inward (depolarizing), blue bars are outward (repolarizing), gray bars are housekeeping currents, gold bars are chloride currents. Compare with Figure 5.1 for TTP06 — note the addition of I_{NaL} (persistent through the plateau), split NCX populations, and chloride currents.

Mathematical Note

As in TTP06, all gating variables in ORd satisfy the relaxation ODE $dx/dt = (x_\infty - x)/\tau_x$. The Rush–Larsen exponential integrator (Chapter 9) applies to every gate, regardless of CaMKII phosphorylation status. The key numerical difference is that ORd has 29 gates vs. TTP06's 12, so the Rush–Larsen evaluation cost roughly doubles. The concentration ODEs remain non-relaxation and require explicit or implicit integration.

6.8 Endo, Epi, and M-Cells: Transmural Heterogeneity

Like TTP06, ORd supports three transmural cell types. However, ORd's transmural differences are more nuanced, affecting more parameters:

Parameter	Endocardial	Epicardial	M-Cell	Physiological Basis
$\bar{g}_{to}$	0.02	0.08 (4×)	0.08 (4×)	Large epi/M-cell notch; minimal endo notch
P_{CaL}	1×	1.2×	2.5×	Enhanced plateau in M-cells
$\bar{g}_{Kr}$	1×	1.3×	0.8×	M-cell APD prolongation
$\bar{g}_{Ks}$	0.0034	0.00476 (1.4×)	Reduced	M-cell APD prolongation (reduced repol. reserve)
$\bar{g}_{K1}$	1×	1.2×	1.3×	Resting potential variation
$\bar{g}_{NaL}$	1×	0.6×	1×	Reduced late Na^+ in epicardium
I_{NaCa}	1×	1.1×	1.4×	Enhanced Ca^{2+} extrusion in M-cells
I_{NaK}	1×	0.9×	0.7×	Reduced pump activity in M-cells

The M-cell phenotype in ORd arises from the combination of reduced I_{Ks} (less repolarization reserve), reduced I_{Kr} (0.8×), enhanced I_{CaL} (2.5×), and reduced I_{NaK} (0.7×). These changes collectively produce the longest APD of any cell type — a feature that is important for generating the T-wave on the surface ECG. The transmural APD gradient (endo < epi < M-cell) drives the voltage gradient that produces the terminal T-wave inscription.

Engine V5.4 Connection

In Engine V5.4, both TTP06 and ORd implement the same `IonicModel` interface. The `cell_type` parameter selects transmural variant by switching maximal conductances. A simulation can mix cell types spatially — endocardial cells on the inner surface, epicardial on the outer surface, M-cells in between — reproducing the full transmural gradient with a single model class.

6.9 Calcium Handling: Two SR Compartments

ORd refines the calcium subsystem beyond TTP06's three-compartment model (subspace, cytoplasm, SR) by splitting the SR into two functionally distinct regions:

Network SR (nSR): The bulk of the SR volume. SERCA pumps calcium from the cytoplasm into the nSR. Calcium diffuses freely within the nSR toward the junctional terminals.

Junctional SR (jSR): The terminal cisternae directly apposed to the T-tubule membrane, containing the RyR2 clusters. This compartment empties rapidly during CICR and must refill from the nSR between beats.

The calcium fluxes in ORd are:

J_{up} (SERCA2a, cytoplasm $\rightarrow$ nSR): CaMKII-enhanced pump with phospholamban relief. At faster rates, CaMKII increases the maximum pump rate, enabling faster reuptake.

J_{tr} (nSR $\rightarrow$ jSR diffusion): A simple first-order transfer with time constant $\tau_{\text{tr}} = 60$ ms. This creates a delay between SERCA refilling and jSR recovery — the jSR content lags behind the nSR, explaining why premature beats (before full jSR recovery) produce weaker calcium transients.

J_{rel} (RyR, jSR $\rightarrow$ subspace): CaMKII-enhanced release. The release flux is proportional to the jSR-subspace calcium gradient and is activated by subspace calcium (CICR trigger). CaMKII phosphorylation increases the RyR open probability, producing larger releases at faster rates. The ORd release model blends non-phosphorylated and CaMKII-phosphorylated populations, with the total release $J_{\text{rel}} = (1 - \Phi) J_{\text{rel,NP}} + \Phi J_{\text{rel,CaMK}}$.

J_{xfer} (subspace $\rightarrow$ cytoplasm diffusion): First-order transfer with time constant $\tau_{\text{xfer}} = 8$ ms. Combined with the subspace Na^+ and K^+ diffusion, this maintains the subspace as a distinct microenvironment that equilibrates with the bulk cytoplasm on a timescale of tens of milliseconds.

J_{leak} (nSR $\rightarrow$ cytoplasm): A small passive leak proportional to $[\text{Ca}^{2+}]_{\text{nSR}}$. This leak is the starting point for modeling diastolic SR calcium leak in heart failure — by increasing the leak rate, one can simulate the RyR2 dysfunction that drives spontaneous calcium waves and DADs.

6.10 TTP06 vs. ORd at a Glance

The following table summarizes the key differences between TTP06 and ORd for quick reference:

Feature	TTP06	ORd
State variables	18	41
Membrane currents	12	15 (incl. 2 chloride)
Data source	Mixed species/cell lines	Undiseased human ventricle
Late Na^+ current	Not modeled	Explicit I_{NaL}
CaMKII signaling	Not modeled	Full pathway; rate adaptation
SR compartments	1 (single SR)	2 (nSR + jSR)
NCX populations	1 (bulk)	2 (bulk + subspace)
Na^+/K^+ pump	Hill-type empirical	Thermodynamic cycle
Subspace ions	Ca^{2+} only	Ca^{2+} , Na^+ , K^+
Beats to steady state	$\sim 50\text{--}100$	$\sim 300\text{--}600$ (CaMKII)
Cost per cell per step	$1\times$ (baseline)	$\sim 2\text{--}2.5\times$
Best for	Wave dynamics, large-scale 3D, prototyping	Drug safety (CiPA), rate-dependent phenomena

⚙ Engine V5.4 Connection

Engine V5.4 implements both models behind the same `IonicModel` interface. Switching from TTP06 to ORd requires changing a single configuration parameter — the spatial solver, mesh, operator splitting strategy, and output pipeline remain identical. The `step(V, ionic_states, dt)` method handles the internal complexity, whether updating TTP06's 18 or ORd's 40 ionic state variables (V stored separately in both). The `RushLarsenSolver` calls each model's `compute_gate_steady_states()` and `compute_gate_time_constants()` to perform the exponential update, and `compute_concentration_rates()` for Forward Euler — all defined identically on the `IonicModel` ABC. This abstraction is what allows the same tissue solver to work with any ionic model.

With both ionic models thoroughly characterized — from 18-variable TTP06 to 41-variable ORd — we now have the single-cell building blocks that will be embedded at every spatial point in the tissue. In Part II, we add the spatial dimension: diffusion, the monodomain equation, and the numerical methods that make tissue-scale simulation possible.

PART II

Tissue-Level Monodomain Modeling

*Connecting millions of cells with diffusion — from the reaction–diffusion PDE
to the numerical solvers that make whole-heart simulation possible.*

The Monodomain Equation

A single cell model tells you what happens at one point. To simulate a wave of activation spreading across the heart, we need to couple millions of these cells through the physics of electrical diffusion. The monodomain equation is this coupling — and it is the equation that Engine V5.4 was built to solve.

7.1 From Single Cell to Tissue: Adding Diffusion

Imagine you have a row of light bulbs connected by wires. If you turn on one bulb in the middle, electrical current flows through the wires to its neighbors, and they start to glow too — then *their* neighbors light up, and so on down the line. That is exactly what happens in cardiac tissue. One cell depolarizes (the "bulb turns on"), and current flows through gap junctions (the "wires") to neighboring cells, triggering them in sequence. This chain reaction is the propagating action potential.

In Chapter 1, we wrote the membrane current equation for a single cell: $C_m dV_m/dt = -I_{\text{ion}}$. This says "the voltage changes because ions flow through channels." But in tissue, there is a second source of current: *electrotonic current* from neighboring cells, flowing through gap junctions.

When cell A depolarizes before cell B, a voltage gradient exists between them, and current flows from A to B through the intracellular pathway. This is just Ohm's law — voltage difference divided by resistance gives current — written in continuous form:

$$\mathbf{J}_i = -\mathbf{D}_i \nabla V_m$$

where $\mathbf{D}_i$ is the *effective intracellular conductivity tensor* — it encodes both the gap-junction coupling strength and the geometric anisotropy of the tissue. The divergence of this current, $\nabla \cdot (\mathbf{D}_i \nabla V_m)$, gives the net current arriving at each point from its neighbors. Adding this to the single-cell equation gives us the monodomain equation.

7.2 The Reaction–Diffusion Formulation

Before writing the equation, let us understand what it will say in plain language. At every point in the tissue, the voltage changes for exactly two reasons: (1) ion channels at that point open and close, pushing the voltage up or down — this is the *reaction*, the local chemistry; (2) current leaks in from depolarized neighbors or out to resting neighbors through gap junctions — this is the *diffusion*, the spatial coupling. The monodomain equation is simply the mathematical statement that "voltage change = reaction + diffusion."

Written out, the monodomain equation is:

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi I_{\text{ion}}(V_m, \mathbf{w}) + I_{\text{stim}} \quad (7.1)$$

This is a *reaction–diffusion* equation — one of the most important classes of PDEs in mathematical biology. The "reaction" is the ionic current I_{ion} (local chemistry at each cell), and the "diffusion" is $\nabla \cdot (\mathbf{D} \nabla V_m)$ (spatial coupling between cells). The stimulus current I_{stim} represents external pacing.

Intuition

The monodomain equation says: "at every point in the tissue, the voltage changes for two reasons — (1) ion channels open and close locally (the reaction), and (2) current leaks in from depolarized neighbors or out to resting neighbors (the diffusion)." These two processes compete: the reaction creates sharp voltage fronts, while the diffusion smooths them out. The balance determines the conduction velocity — typically 60–80 cm/s along fibers in healthy ventricular tissue.

The state vector $\mathbf{w}$ contains all the gating variables and concentrations from the ionic model (18 for TTP06, 41 for ORd). These evolve by their own ODEs at each spatial point:

$$\frac{\partial \mathbf{w}}{\partial t} = \mathbf{g}(V_m, \mathbf{w}) \quad (7.2)$$

Note that $\mathbf{w}$ has *no diffusion term* — gating variables do not diffuse from cell to cell. Only voltage (and in the bidomain model, extracellular potential) spreads spatially. This locality of the ionic state is what makes operator splitting so natural.

7.3 Conductivity, Fibers, and Anisotropy

Think of cardiac tissue like a bundle of straws, all pointing roughly the same direction. If you push water into one straw, it flows easily along the straw's length but very poorly sideways into neighboring straws. Heart muscle works the same way: myocytes are long, thin cells connected end-to-end by gap junctions. Electrical current zips along the fiber direction about *three times faster* than it crosses fibers. This directional preference is called *anisotropy*, and it shapes the entire wavefront: elliptical in a tissue slab, spiraling in the ventricle wall where fibers rotate transmurally from epicardium to endocardium.

Mathematically, a single number (scalar conductivity) cannot capture "fast in one direction, slow in another." We need a *tensor* — essentially a 2×2 or 3×3 matrix that converts a voltage gradient in any direction into the resulting current in that direction. If $\mathbf{f}$ is the local fiber direction (a unit vector), the conductivity tensor is:

$$\mathbf{D} = d_{\parallel} \mathbf{f} \mathbf{f}^T + d_{\perp} (\mathbf{I} - \mathbf{f} \mathbf{f}^T) \quad (7.3)$$

where $d_{\parallel}$ and $d_{\perp}$ are the conductivities along and across fibers, respectively. Typical values are $d_{\parallel} \approx 0.174$ and $d_{\perp} \approx 0.019$ mm²/ms, giving an anisotropy ratio of about 3:1 in conduction velocity (since $CV \propto \sqrt{d}$). In 3D, a sheet-normal direction adds a third axis with even lower conductivity.

7.4 Boundary Conditions: What Happens at the Edge?

So far we have written the monodomain equation and explained the conductivity tensor — but we have said nothing about what happens at the *boundary* of the tissue. Every PDE needs boundary conditions, and physically, the question is simple: **what happens to electrical current when it reaches the edge of the heart?**

Think of a swimming pool. Water circulates inside, but what happens at the walls? If the walls are solid concrete, no water escapes — that is a *Neumann* (zero-flux) condition. If the pool has an overflow drain that holds the water level fixed, that is a *Dirichlet* (fixed-value) condition. If the walls are porous and leak at a rate proportional to the water pressure, that is a *Robin* (mixed) condition. Cardiac tissue faces the same three choices for electrical current at its surface.

For the monodomain equation, the three boundary condition types are:

- **Neumann (zero flux):** $\mathbf{D} \nabla V_m \cdot \hat{\mathbf{n}} = 0$ — no current leaves the tissue. This is the default for isolated hearts and the overwhelmingly common choice in cardiac simulations.
- **Dirichlet (fixed voltage):** $V_m = V_{\text{boundary}}$ — the transmembrane potential is clamped at the boundary. Rare in practice (what physical setup would clamp V_m at the tissue surface?) but useful in test problems.
- **Robin (partial flux):** $\mathbf{D} \nabla V_m \cdot \hat{\mathbf{n}} + \alpha V_m = g$ — a weighted combination. Used when the tissue surface is partially coupled to an external medium.

Intuition

Why Neumann is the default. In a real heart, cardiac cells are sealed at the tissue surface by connective tissue (the epicardium and endocardium). No intracellular current escapes through these layers into the surrounding pericardial fluid. Mathematically, zero current flux across the boundary is exactly what the Neumann condition says. Nearly every monodomain simulation uses this condition on all boundaries.

Each boundary condition type has a different effect on the numerical discretization — it changes the stiffness matrix, the right-hand side vector, or both. The concrete formulations (ghost nodes for FDM, natural conditions for FEM, modified stencils) are developed in Chapter 8, Sections 8.6–8.7.

The situation becomes substantially more complex in the bidomain model, where *two* unknowns (V_m and ϕ_e) each need their own boundary condition. In particular, the "isolated tissue" case (Neumann on both) creates a null space in the linear system that requires special handling. We will tackle bidomain boundary conditions in Chapter 12.

Spatial Discretization

The monodomain PDE lives in continuous space — but computers work with discrete numbers. Spatial discretization replaces the continuous diffusion operator with a matrix that computes the same physics at a finite set of mesh points. This chapter develops three discretization methods (FDM, FEM, FVM) and shows how boundary conditions enter the discrete system.

 How to read this chapter

This is the longest chapter in the book — three discretization methods plus boundary conditions. You do not need all of it in one sitting. Route by what you are building:

- **Implementing FDM?** Read §8.1 (what discretization means physically) and §8.2 (the method and its stencils), then §8.7 for boundary conditions.
- **Adding FEM?** Go to §8.3 (weak form, mass and stiffness matrices).
- **Using FVM?** Go to §8.4 (flux balance across control volumes).
- **Not sure which method to pick?** §8.5 compares all three in one decision table.
- **Wiring a method into the full solver?** §8.6 assembles the pieces, from the continuous PDE to the matrix system.
- **Boundary conditions** (Neumann, Dirichlet) live in §8.7 and apply to all three methods.

8.1 Spatial Discretization: What It Means Physically

The continuous diffusion operator $\nabla \cdot (\mathbf{D} \nabla V_m)$ answers a simple physical question: *how much net electrical current is flowing into this point from its neighbors?* Spatial discretization replaces this continuous question with a discrete one: given the voltage at every mesh point, compute the net current arriving at each point from its neighbors. This computation turns out to be a matrix-vector product.

Think of it this way. If node i is at -85 mV (resting) and its neighbor j is at $+20$ mV (depolarized), there is a voltage difference of 105 mV driving current from j to i through the intracellular resistance. The amount of current depends on the conductivity between them and the distance. Summing up these pairwise contributions from all neighbors of node i gives the net diffusive current — and that sum is exactly one row of the matrix-vector product $\mathbf{K} \cdot \mathbf{V}$.

Three classical methods exist for building this matrix. Each approaches the problem from a different mathematical direction, but all three produce the same object: a *sparse matrix* that encodes how current flows through the tissue mesh. The matrix is sparse because each node couples only to its immediate neighbors — a node in the middle of a 3D mesh might have 6–26 neighbors out of millions of total nodes.

Method	Core Idea	Mesh Requirement	Best For
FDM	Replace derivatives with neighbor differences	Regular Cartesian grid	Tissue slabs, GPU computing, fast prototyping
FEM	Reformulate as integral (weak form)	Any: triangles, tetrahedra, hexahedra	Realistic heart geometries, curved boundaries
FVM	Balance fluxes across control-volume faces	Any with orthogonal dual mesh	Sharp conductivity jumps (scar/healthy boundary)

We will work through each method with the same running philosophy: *show the matrix*. For each method, we will build the actual stiffness matrix for a tiny example mesh, inspect its structure, and verify that it

makes physical sense. Then in Section 8.6, we will connect these matrices back to the ionic models from Chapters 5–6, showing how TTP06's 12 currents or ORD's 15 currents feed into the full system.

8.2 Finite Difference Method (FDM)

The Central Idea

FDM is the most direct approach: place nodes on a regular grid with uniform spacing h , and approximate the second derivative using the values at neighboring grid points. The fundamental approximation in 1D is:

$$d \frac{\partial^2 V}{\partial x^2} \Big|_i \approx \frac{d}{h^2} (V_{i-1} - 2V_i + V_{i+1})$$

This is second-order accurate (error $\sim O(h^2)$), derived from Taylor expansion. But the deeper insight is *geometric*: the expression $V_{i-1} - 2V_i + V_{i+1}$ measures how much V_i deviates from the average of its neighbors. If V_i is higher than the average $\frac{1}{2}(V_{i-1} + V_{i+1})$, the Laplacian is negative — current flows outward, pulling V_i down. If V_i is lower than the average, current flows inward, pushing V_i up. Diffusion always acts to smooth out voltage differences.

Worked Example: 5-Node 1D Cable

Consider a tiny 1D cardiac cable with 5 nodes, spacing $h = 0.2$ mm, and uniform conductivity $d = 0.1$ mm²/ms. The no-flux (Neumann) boundary condition means no current flows out of the cable ends.



Figure 8.1. A 5-node 1D cable with node 3 depolarized to +20 mV (wavefront). All other nodes at resting potential −85 mV.

The FDM stencil $[1, -2, 1]$ is applied at each interior node. At boundary nodes (0 and 4), the no-flux condition $\partial V / \partial x = 0$ modifies the stencil to $[-1, 1]$ (the "missing" neighbor is replaced by mirroring). Collecting all stencils into a matrix, we get the **stiffness matrix** $\mathbf{K}$:

$$\mathbf{K} = \frac{d}{h^2} \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix} = 2.5 \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix}$$

Three properties of $\mathbf{K}$ to notice: (1) it is *symmetric* — the coupling from node i to j equals the coupling from j to i ; (2) it is *tridiagonal* — each node couples only to its immediate neighbors; (3) every row sums to zero — applying a uniform voltage $\mathbf{V} = c\mathbf{1}$ gives $\mathbf{K} \cdot \mathbf{V} = \mathbf{0}$, meaning diffusion does nothing to a spatially constant voltage (as it should).

Now let us compute the diffusion term $-\mathbf{K} \cdot \mathbf{V}$ for the wavefront scenario $\mathbf{V} = [-85, -85, -85, +20, -85]^T$ (mV):

$$-\mathbf{K} \cdot \mathbf{V} = -2.5 \begin{pmatrix} 1(-85) + (-1)(-85) \\ (-1)(-85) + 2(-85) + (-1)(-85) \\ (-1)(-85) + 2(-85) + (-1)(20) \\ (-1)(-85) + 2(20) + (-1)(-85) \\ (-1)(20) + 1(-85) \end{pmatrix} = -2.5 \begin{pmatrix} 0 \\ 0 \\ -105 \\ 210 \\ -105 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 262.5 \\ -525 \\ 262.5 \end{pmatrix}$$

Read this result physically:

Node 2 (-85 mV, next to wavefront): receives $+262.5$ mV/ms of diffusive current — a large positive push from the depolarized node 3. This is the electrotonic current that will soon trigger node 2's sodium channels, propagating the wave to the left.

Node 3 ($+20$ mV, the wavefront): loses -525 mV/ms — current flows out to both resting neighbors. The depolarized cell is the "source" that drives the wavefront forward.

Node 4 (-85 mV, behind the wavefront): also receives $+262.5$ mV/ms — the same push as node 2. Node 2 feels the full second difference across two neighbors ($V_1 - 2V_2 + V_3 = 105$); node 4 is a boundary node whose one-sided zero-flux stencil gives $V_3 - V_4 = 105$. For this particular wavefront ($+20$ mV) and resting (-85 mV) pair the two happen to coincide, so both nodes are pulled equally toward the wave. Check: node 3 sheds $-525 = -(262.5 + 262.5)$ — diffusion conserves current, so the five entries sum to zero.

Nodes 0–1 (far from wavefront): zero diffusive current — all neighbors have the same voltage, so there is nothing to diffuse.

Intuition

This example reveals the core mechanism of wave propagation. The depolarized region (node 3) acts as a current source, pushing electrotonic current into the resting neighbors (nodes 2 and 4). When this current is strong enough to bring a neighbor above threshold (~ -65 mV), the neighbor's sodium channels fire explosively, depolarizing it and extending the wavefront. The matrix $\mathbf{K}$ encodes *all* of this neighbor-to-neighbor coupling.

The 5-Point and 9-Point Stencils (2D)

In two dimensions with isotropic conductivity d , the Laplacian is approximated by the **5-point stencil**:

$$d \nabla^2 V|_{i,j} \approx \frac{d}{h^2} (V_{i+1,j} + V_{i-1,j} + V_{i,j+1} + V_{i,j-1} - 4V_{i,j})$$

Same idea as 1D: the value at the center minus the average of its four axis-aligned neighbors. The stiffness matrix now has 5 nonzeros per row (the center node and its four neighbors). On an $N \times N$ grid, $\mathbf{K}$ is $N^2 \times N^2$ — potentially millions of rows — but with only $\sim 5N^2$ nonzero entries. This extreme sparsity is what makes large-scale cardiac simulation tractable.

When the conductivity tensor $\mathbf{D}$ has off-diagonal entries (because cardiac fibers are rotated relative to the grid axes):

$$\mathbf{D} = \begin{pmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{pmatrix}$$

the 5-point stencil can only handle the diagonal terms D_{xx} and D_{yy} . The cross-derivative $D_{xy} \partial^2 V / \partial x \partial y$ requires the **9-point stencil**, which includes diagonal neighbors:

$$D_{xy} \frac{\partial^2 V}{\partial x \partial y} \bigg|_{i,j} \approx \frac{D_{xy}}{4h^2} (V_{i+1,j+1} - V_{i+1,j-1} - V_{i-1,j+1} + V_{i-1,j-1})$$

In 3D, the full anisotropic operator requires a **27-point stencil** — the node plus all 26 neighbors in a $3 \times 3 \times 3$ cube. The stiffness matrix then has up to 27 nonzeros per row.



Figure 8.2. FDM stencils for the 2D diffusion operator. Left: The 5-point stencil uses axis-aligned neighbors only (isotropic or axis-aligned anisotropic). Right: The 9-point stencil adds diagonal neighbors (green) for the cross-derivative D_{xy} terms from rotated fiber fields.

Key Insight

When to use FDM: FDM is fastest, simplest to implement, and cache-friendly — ideal for regular geometries (tissue slabs, rectangular patches), GPU computation, and rapid prototyping. Its limitation is that it *requires a structured Cartesian grid*. You cannot fit a regular grid to the curved walls of the left ventricle without staircase artifacts at the boundary. For realistic anatomical geometries, use FEM or FVM instead.

8.3 Finite Element Method (FEM)

Why FEM? The Geometry Problem

Real hearts are not rectangular. The left ventricle is a thick-walled ellipsoidal shell with curved boundaries, trabeculations, and papillary muscles. To represent this geometry faithfully, we need meshes made of triangles (2D) or tetrahedra (3D) that can conform to any shape. FDM cannot do this. FEM can.

The mathematical trick that enables unstructured meshes is a change of perspective. Instead of approximating derivatives pointwise (as FDM does), FEM reformulates the PDE in *weak form* — multiplying by a test function and integrating. This seemingly abstract step has a profound practical consequence: it lowers the smoothness requirement on the solution, allowing us to use simple piecewise-linear "hat functions" on arbitrary triangulations.

Weak Form: What Integration by Parts Really Does

Before diving into the math, let us understand the idea in plain language. We want to ask: "does my approximate solution satisfy the PDE?" But checking this point-by-point (like FDM does) fails on triangular meshes because our piecewise-linear hat functions have *kinks* — the second derivative is undefined at element boundaries. So instead, we check the PDE *on average*, by multiplying both sides by a smooth "test function" $\mathbf{v}$ and integrating over the whole domain. If the average checks out for *every* possible test function, the solution must be correct. This is the weak form.

The magic trick is *integration by parts* (the multidimensional version of the product rule from calculus). It shifts one derivative from $\mathbf{V}_m$ onto the test function $\mathbf{v}$, so we only need first derivatives of both — and first derivatives of our hat functions are perfectly well-defined (they are just constant slopes on each triangle). This is why FEM can use simple piecewise-linear approximations on arbitrary meshes, while FDM needs smooth grids.

Now let us see this concretely. Start with the monodomain diffusion term. Multiply by a test function $\mathbf{v}$, integrate over the domain Ω , and apply integration by parts (the divergence theorem):

$$\underbrace{\int_{\Omega} \chi C_m \frac{\partial V_m}{\partial t} \mathbf{v} \, d\mathbf{x}}_{\text{time derivative}} + \underbrace{\int_{\Omega} \nabla V_m \cdot (\mathbf{D} \nabla \mathbf{v}) \, d\mathbf{x}}_{\text{diffusion (after IBP)}} = - \underbrace{\int_{\Omega} \chi I_{\text{ion}} \mathbf{v} \, d\mathbf{x}}_{\text{ionic current}} + \underbrace{\int_{\Omega} I_{\text{stim}} \mathbf{v} \, d\mathbf{x}}_{\text{stimulus}}$$

The crucial step is the integration by parts on the diffusion term. The original PDE contains $\nabla \cdot (\mathbf{D} \nabla V_m)$ — a second derivative of V_m . After integration by parts, we have $\nabla V_m \cdot (\mathbf{D} \nabla \mathbf{v})$ — only *first* derivatives of both V_m and $\mathbf{v}$. The boundary integral vanishes because of the no-flux condition (physically: no current flows out of the heart). This matters because first derivatives of piecewise-linear functions are well-defined (they are piecewise constant), while second derivatives are not.

From Integrals to Matrices

Expand the voltage as $V_m(\mathbf{x}, t) = \sum_j V_j(t) \varphi_j(\mathbf{x})$, where each φ_j is a "hat function" that equals 1 at node j and 0 at all other nodes. Choose $\mathbf{v} = \varphi_i$ for each node i . The integrals become matrix entries:

$$\mathbf{M} \dot{\mathbf{V}}_m + \mathbf{K} \mathbf{V}_m = \mathbf{F} \quad (8.1)$$

with two matrices and one vector:

Mass matrix $M_{ij} = \int_{\Omega} \chi C_m \varphi_i \varphi_j \, d\mathbf{x}$ — comes from the time derivative. Physically, it weights each node by the "volume" it represents. It is symmetric, positive definite, and sparse.

Stiffness matrix $K_{ij} = \int_{\Omega} \nabla \varphi_i \cdot (\mathbf{D} \nabla \varphi_j) \, d\mathbf{x}$ — encodes the diffusive coupling. This is the FEM analogue of the FDM matrix we built above. It is symmetric, positive semi-definite (row sums = 0 under Neumann BC), and sparse.

Load vector $F_i = - \int_{\Omega} \chi I_{\text{ion}} \varphi_i \, d\mathbf{x} + \int_{\Omega} I_{\text{stim}} \varphi_i \, d\mathbf{x}$ — contains the ionic current and stimulus. This is where Chapters 5–6 connect: at each node, TTP06 or ORd evaluates I_{ion} , which becomes an entry in $\mathbf{F}$.

Worked Example: 4-Node, 2-Triangle Mesh

Consider the simplest 2D mesh: a unit square $[0, h] \times [0, h]$ divided into two triangles. We use isotropic conductivity d and spacing h .

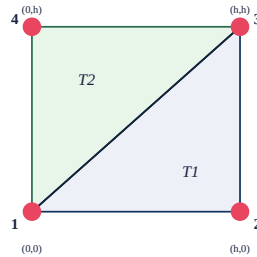


Figure 8.3. A minimal 2D FEM mesh: 4 nodes, 2 triangles (T_1 with nodes 1-2-3, T_2 with nodes 1-3-4). The diagonal splits the square into two right triangles of area $h^2/2$.

FEM assembles the global stiffness matrix element-by-element. For each triangle, we compute a small *element stiffness matrix* (3×3 for a triangle with 3 nodes), then "scatter" its entries into the correct positions in the global matrix.

For a triangle with vertices at $\mathbf{x}_a, \mathbf{x}_b, \mathbf{x}_c$ and area A , the gradients of the linear basis functions are constant vectors: $\nabla\varphi_a = \frac{1}{2A}(y_b - y_c, x_c - x_b)^T$ (and cyclic permutations). The element stiffness entry is:

$$K_{ab}^e = A \nabla\varphi_a \cdot (d \nabla\varphi_b) = \frac{d}{4A} [(y_b - y_c)(y_c - y_a) + (x_c - x_b)(x_a - x_c)]$$

Triangle T_1 (nodes 1, 2, 3) with vertices $(0, 0), (h, 0), (h, h)$, area $= h^2/2$:

$$\mathbf{K}^{T_1} = \frac{d}{2} \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix} \quad (\text{for nodes 1, 2, 3})$$

Triangle T_2 (nodes 1, 3, 4) with vertices $(0, 0), (h, h), (0, h)$, area $= h^2/2$:

$$\mathbf{K}^{T_2} = \frac{d}{2} \begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \\ -1 & -1 & 2 \end{pmatrix} \quad (\text{for nodes 1, 3, 4})$$

Global assembly: Add element matrices into the global 4×4 matrix. Node 1 appears in both triangles, so its diagonal entry gets contributions from both. Node 2 appears only in T_1 ; node 4 only in T_2 . The result:

$$\mathbf{K}_{\text{global}} = \frac{d}{2} \begin{pmatrix} 2 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{pmatrix}$$

Check: every row sums to zero (conservation under Neumann BC). The matrix is symmetric and positive semi-definite with a single zero eigenvalue (the constant mode). Physically, each node couples to its connected neighbors with equal weight $-d/2$ — which makes sense because the mesh is symmetric and the conductivity is isotropic.

Notice what happened during assembly: the element stiffness matrices *overlap* at shared nodes. Node 1's diagonal entry is $d/2 + d/2 = d$ (contributions from both triangles). Node 3's diagonal entry is also $d/2 + d/2 = d$. The off-diagonal entries come from whichever element(s) connect the two nodes. Nodes 2 and 4 are *not* directly connected by any element edge, so $K_{24} = 0$ — even though they are geometric neighbors across the diagonal.

The Mass Matrix: Consistent vs. Lumped

The mass matrix for a linear triangle with area A has entries:

$$M_{ab}^e = \frac{\chi C_m A}{12} \times \begin{cases} 2 & \text{if } a = b \\ 1 & \text{if } a \neq b \end{cases}$$

For our two-triangle mesh (each with area $h^2/2$), the assembled **consistent mass matrix** is:

$$\mathbf{M}_{\text{consistent}} = \frac{\chi C_m h^2}{12} \begin{pmatrix} 4 & 1 & 2 & 1 \\ 1 & 2 & 1 & 0 \\ 2 & 1 & 4 & 1 \\ 1 & 0 & 1 & 2 \end{pmatrix}$$

This matrix is full (no zero entries for connected nodes) and couples each node to its neighbors even for the time derivative. This coupling is mathematically correct but computationally inconvenient: for explicit time stepping, you would need to solve $\mathbf{M} \dot{\mathbf{V}} = \mathbf{b}$ at every timestep, which requires inverting $\mathbf{M}$.

The **lumped mass matrix** avoids this by concentrating each row's total mass onto the diagonal:

$$\mathbf{M}_{\text{lumped}} = \frac{\chi C_m h^2}{3} \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The diagonal entries reflect the area "owned" by each node: nodes 1 and 3 (shared by two triangles) each own $2 \times h^2/6 = h^2/3$ of area, while nodes 2 and 4 (each in one triangle) own $h^2/6$. With a lumped mass matrix, explicit time stepping becomes trivial: $\dot{\mathbf{V}}_i = \mathbf{b}_i / \mathbf{M}_{ii}$ — just a pointwise division, no linear solve required.

Mathematical Note

Mass lumping introduces an $O(h^2)$ error — the same order as the spatial discretization itself. So for P1 elements, lumping does not degrade the overall accuracy order. This is why lumped mass is the default choice in cardiac simulation: it preserves the convergence rate while eliminating the mass-matrix solve. The only case where consistent mass matters is when using higher-order elements (P2 or above), where lumping can reduce the accuracy benefit.

Element Types

For cardiac simulations, **P1 elements** (linear triangles in 2D, linear tetrahedra in 3D) are the workhorse, providing $O(h^2)$ accuracy with minimal overhead. The gradients are constant within each element, making the integrals trivial. **P2 elements** (quadratic) achieve $O(h^3)$ accuracy but at $\sim 8\times$ the memory cost. For the monodomain problem — where the ionic model, not the linear solve, is the bottleneck — P1 elements with mesh refinement are generally preferred.

Key Insight

When to use FEM: FEM is the method of choice for realistic anatomical heart geometries. It handles any mesh shape (triangles, tetrahedra, hexahedra), naturally accommodates curved boundaries, and has rigorous mathematical convergence theory. Its main cost is the assembly step (computing element stiffness matrices and scattering into the global matrix) and the need for a mass matrix. For structured Cartesian grids, FDM is faster; for problems dominated by sharp conductivity jumps, FVM may be more appropriate.

⚙ Engine V5.4 Connection

Engine V5.4 supports P1 FEM on triangular (2D) and tetrahedral (3D) meshes. The mass matrix can be either consistent or lumped (diagonal). The stiffness matrix $\mathbf{K}$ is assembled element-by-element and stored in CSR (Compressed Sparse Row) format. Both explicit solvers (which need only $\mathbf{KV}$ products) and implicit solvers (which solve $(\mathbf{M} + \alpha \mathbf{K})\mathbf{V} = \mathbf{b}$) operate through the same assembled matrices.

8.4 Finite Volume Method (FVM)

The Conservation Principle

FVM starts from the most physically intuitive formulation: conservation. Around each node $\mathbf{i}$, draw a *control volume* $\Omega_{\mathbf{i}}$ — the region of space "owned" by that node. Integrate the monodomain equation over $\Omega_{\mathbf{i}}$:

$$\chi C_m |\Omega_{\mathbf{i}}| \frac{dV_{\mathbf{i}}}{dt} = \underbrace{\sum_{j \in \mathcal{N}(\mathbf{i})} F_{ij}}_{\text{net flux from neighbors}} - \chi |\Omega_{\mathbf{i}}| I_{\text{ion},\mathbf{i}}$$

where $|\Omega_{\mathbf{i}}|$ is the volume (or area, in 2D) of the control cell, $\mathcal{N}(\mathbf{i})$ is the set of neighbors sharing a face with cell $\mathbf{i}$, and F_{ij} is the flux from cell $\mathbf{j}$ into cell $\mathbf{i}$. The key insight: the flux leaving cell $\mathbf{i}$ into cell $\mathbf{j}$ is *exactly equal and opposite* to the flux entering cell $\mathbf{j}$ from cell $\mathbf{i}$: $F_{ij} = -F_{ji}$. This ensures that total charge is conserved to machine precision — no charge is created or destroyed at any face.

Two-Point Flux Approximation (TPFA)

The flux between cells $\mathbf{i}$ and $\mathbf{j}$ is approximated using only the voltages at the two cell centers:

$$F_{ij} = \sigma_{ij} \frac{V_j - V_i}{|\mathbf{x}_j - \mathbf{x}_i|} A_{ij}$$

where A_{ij} is the area of the shared face and σ_{ij} is the effective conductivity at the face. When the two cells have different conductivities ($\sigma_{\mathbf{i}} \neq \sigma_{\mathbf{j}}$), the correct effective value is the **harmonic mean**:

$$\sigma_{ij} = \frac{2\sigma_{\mathbf{i}}\sigma_{\mathbf{j}}}{\sigma_{\mathbf{i}} + \sigma_{\mathbf{j}}}$$

This is the formula for two resistors in series — the face lies between the two cells, so the total resistance is the sum of half-cell resistances. Importantly, if one cell has zero conductivity (scar tissue), $\sigma_{ij} = 0$ regardless of the other cell's conductivity. This makes FVM naturally correct at sharp tissue boundaries.

Worked Example: 1D, 4-Cell FVM

Consider four control volumes in 1D with uniform cell width h and uniform conductivity σ . Each face has area 1 (unit cross-section). The flux from cell $\mathbf{j}$ to cell $\mathbf{i}$ is $F_{ij} = \sigma(V_j - V_i)/h$.

The balance equation for cell $\mathbf{i}$ (interior) is: $h\chi C_m dV_{\mathbf{i}}/dt = \sigma(V_{\mathbf{i}-1} - V_{\mathbf{i}})/h + \sigma(V_{\mathbf{i}+1} - V_{\mathbf{i}})/h$, which simplifies to:

$$\chi C_m \frac{dV_i}{dt} = \frac{\sigma}{h^2} (V_{i-1} - 2V_i + V_{i+1})$$

This is *identical* to the FDM formula! On a uniform 1D grid with uniform conductivity, FVM and FDM produce the same stiffness matrix. The difference emerges in two situations: (1) non-uniform grids, where FVM correctly accounts for variable cell sizes through the control-volume areas, and (2) discontinuous conductivity, where the harmonic mean gives the physically correct flux while FDM's arithmetic average does not.

⚠ Key Insight

When to use FVM: FVM excels when *conservation* is paramount and when conductivity has sharp jumps (e.g., scar tissue abutting healthy myocardium). The harmonic-mean flux ensures physically correct behavior at material interfaces. The trade-off is that the two-point flux approximation requires the mesh to be "K-orthogonal" (control-volume faces perpendicular to the line connecting cell centers) for full accuracy with anisotropic conductivity. On non-orthogonal meshes, more complex multi-point flux approximations are needed.

8.5 Choosing Your Method: A Practical Guide

The choice between FDM, FEM, and FVM is ultimately a practical engineering decision. No method is universally best — each has a domain where it excels. The following table summarizes the trade-offs:

Criterion	FDM	FEM	FVM
Mesh type	Structured Cartesian only	Any (triangles, tets, hex)	Any with dual mesh
Geometry fidelity	Staircase boundaries	Exact curved boundaries	Good with Voronoi cells
Implementation	Simplest (arrays + stencil)	Most complex (assembly)	Moderate (flux loops)
Anisotropy	9/27-pt stencil for off-diagonal	Natural (in weak form)	Needs K-orthogonal mesh
Conservation	Approximate	Approximate (exact in weak sense)	Exact by construction
Sharp interfaces	Poor (averaging artifacts)	Good (per-element D)	Excellent (harmonic mean)
GPU friendliness	Excellent (regular access)	Good (element-level parallelism)	Good (face-level parallelism)
Memory per node	Lowest	Moderate (M + K storage)	Low
Mass matrix	Not needed (M = I)	Required (can be lumped)	Diagonal by construction
Accuracy order (P1)	$O(h^2)$	$O(h^2)$	$O(h^2)$

In practice, the cardiac simulation community uses all three:

FDM dominates rapid prototyping, GPU-accelerated tissue slabs, and machine-learning training datasets where millions of simulations are needed on simple geometries.

FEM dominates patient-specific simulations on MRI-derived anatomical meshes — the standard for clinical applications and the default in major cardiac simulation packages (openCARP, FEniCS-based solvers, deal.II frameworks).

FVM is used for specialized problems involving sharp material interfaces (fibrosis modeling, scar characterization) and in Lattice Boltzmann-based solvers (Chapters 17–18) where conservation is the defining principle.

8.6 Connecting the Pieces: From PDE to Matrix System

We have spent the last five sections building spatial discretization tools — stiffness matrices from FDM, FEM, and FVM. Now comes the payoff: we will derive, step by step, the central matrix equation that the simulator actually solves. The goal is to show you exactly where every term comes from, so that equation (8.2) below feels *inevitable* rather than pulled from thin air.

Here is the roadmap. We start from the continuous PDE (equation 7.1), break it into pieces, replace each continuous piece with its discrete counterpart, and collect everything into one matrix equation. By the end, you will see that equation (8.2) is just a compact way of writing "voltage changes at every node because of ion channels (local) plus current from neighbors (spatial)."

Step 1: Start from the Continuous PDE

Recall the monodomain equation (7.1):

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi I_{\text{ion}}(V_m, \mathbf{w}) + I_{\text{stim}}$$

This is a continuous equation — it holds at every point $\mathbf{x}$ in the tissue simultaneously. A computer cannot handle "every point" (there are infinitely many), so we pick a finite set of N points (the mesh nodes) and ask: what does this equation look like at node i ?

Step 2: Sample at Node i (the Semi-Discrete Idea)

Let $V_{m,i}(t)$ denote the voltage at node i as a function of time. The time derivative is simple — it just becomes $dV_{m,i}/dt$, an ordinary derivative because we are tracking a single node's voltage over time. The ionic current is also simple — it only depends on local quantities, so it becomes $I_{\text{ion}}(V_{m,i}, \mathbf{w}_i)$ evaluated entirely at node i . The tricky term is the diffusion operator $\nabla \cdot (\mathbf{D} \nabla V_m)$, because it involves spatial derivatives — information from neighboring nodes. This is what sections 8.2–8.4 taught us to handle.

Step 3: Replace the Diffusion Operator with a Matrix Product

Every spatial discretization method does the same thing: it replaces the continuous diffusion operator at node i with a *weighted sum* of voltages at node i and its neighbors:

$$[\nabla \cdot (\mathbf{D} \nabla V_m)]_i \approx - \sum_{j=1}^N K_{ij} V_{m,j}$$

The minus sign is a convention: we define $\mathbf{K}$ so that it is *positive* semi-definite (the natural sign for a "stiffness" matrix). The entry K_{ij} encodes how strongly node j influences node i through diffusion — this is exactly what we computed in the FDM stencil (Section 8.2), the FEM element assembly (Section 8.3), or the FVM flux balance (Section 8.4). If node j is not a neighbor of node i , then $K_{ij} = 0$, which is why $\mathbf{K}$ is sparse.

In vector notation, gathering all N node voltages into $\mathbf{V}_m = (V_{m,1}, V_{m,2}, \dots, V_{m,N})^T$:

$$\text{Diffusion at all nodes} \approx -\mathbf{K} \mathbf{V}_m$$

Step 4: Handle the Time Derivative (the Mass Matrix)

For FDM and FVM, the time derivative is straightforward. At node i , the left-hand side of the PDE is simply $\chi C_m dV_{m,i}/dt$, and we can write $\chi C_m \dot{\mathbf{V}}_m$ for the whole vector. In matrix language, this is $\mathbf{M} \dot{\mathbf{V}}_m$ where $\mathbf{M}$ is a diagonal matrix with $M_{ii} = \chi C_m |\Omega_i|$ and $|\Omega_i|$ is the volume associated with node i (for FDM on a uniform grid, $|\Omega_i| = h^d$, so $\mathbf{M}$ is proportional to the identity).

For FEM, the mass matrix arises naturally from the weak form (Section 8.3). The time-derivative integral $\int_{\Omega} \chi C_m (\partial V_m / \partial t) \varphi_i dx$ produces the entry $M_{ij} = \int_{\Omega} \chi C_m \varphi_i \varphi_j dx$. This is a *consistent* mass matrix — it is not diagonal because the hat functions of neighboring nodes overlap. In practice, we often *lump* it (replace each row with its row sum on the diagonal) so that $\mathbf{M}$ becomes diagonal and trivially invertible, just like FDM's.

In all cases, $\mathbf{M}$ has the physical meaning: it weights each node's time derivative by the "volume of tissue" that node represents.

Step 5: Gather the Ionic Current into a Load Vector

The ionic current and stimulus terms are purely local — they don't involve spatial derivatives. At node i , they contribute:

$$F_i(t) = -\chi |\Omega_i| I_{\text{ion}}(V_{m,i}, \mathbf{w}_i) + |\Omega_i| I_{\text{stim},i}(t)$$

Here $I_{\text{ion}}(\mathbf{V}_m, \mathbf{w})$ is the total ionic current at node i — this is equation (5.1) for TTP06 (12 currents, 18 state variables) or equation (6.1) for ORd (15 currents, 40 state variables). The state vector $\mathbf{w}_i$ lives entirely at node i : none of its components (gates, concentrations, CaMKII states) diffuse. Collecting all N entries gives the *load vector* $\mathbf{F}(t)$.

The weighting factor $|\Omega_i|$ is the "volume" associated with node i : for FDM on a uniform grid, $|\Omega_i| = h^d$; for lumped FEM, it equals the lumped mass diagonal entry divided by χC_m ; for FVM, it is the control-volume size.

Step 6: Assemble the Semi-Discrete System

Now we put steps 3, 4, and 5 together. The continuous PDE at node i was:

$$\underbrace{\chi C_m \frac{dV_{m,i}}{dt}}_{\text{mass} \times \text{time deriv}} = \underbrace{-\sum_j K_{ij} V_{m,j}}_{\text{diffusion}} + \underbrace{(-\chi I_{\text{ion},i} + I_{\text{stim},i}) |\Omega_i|^{-1} \cdot |\Omega_i|}_{\text{ionic} + \text{stimulus}}$$

Stacking all N nodes into vectors and matrices:

$$\boxed{\mathbf{M} \dot{\mathbf{V}}_m = -\mathbf{K} \mathbf{V}_m + \mathbf{F}(t)} \quad (8.2)$$

This is it — the *semi-discrete* monodomain system. "Semi-discrete" because space has been discretized (the matrices $\mathbf{M}$ and $\mathbf{K}$) but time is still continuous (the time derivative $\dot{\mathbf{V}}_m$). Every term has a clear origin:

Term	Origin	What it encodes
M	Time-derivative integral (PDE left-hand side)	Volume weight per node; I for FDM, diagonal for lumped FEM/FVM
K	Diffusion operator $\nabla \cdot (\mathbf{D}\nabla\mathbf{V}_m)$	Spatial coupling: stencil (FDM), element assembly (FEM), flux balance (FVM)
F(t)	Ionic current + stimulus (PDE right-hand side)	Local chemistry from TTP06/ORD at each node; no spatial derivatives

Intuition

Equation (8.2) says exactly the same thing as the continuous PDE (7.1), but in a language a computer can execute: "at each node, the voltage rate of change equals the diffusive current arriving from neighbors (encoded by the matrix $-\mathbf{K}\mathbf{V}_m$) plus the ionic current produced locally (encoded by **F**)." The matrices are just bookkeeping — a structured way to store the neighbor weights that sections 8.2–8.4 computed.

One Full Timestep: How It All Fits Together

To make this concrete, here is what happens during a single timestep Δt using Godunov operator splitting (Chapter 9), FDM spatial discretization, and TTP06 ionic model, on a mesh with N nodes:

Step 1 — Reaction (at every node independently): For each node $i = 1, \dots, N$, evaluate TTP06's 12 currents from the current voltage $V_{m,i}^n$ and state variables $\mathbf{w}_i^n$:

$$I_{\text{ion},i} = I_{\text{Na},i} + I_{\text{CaL},i} + I_{\text{to},i} + \dots \quad (12 \text{ terms from eq. 5.1})$$

Update the voltage: $V_{m,i}^* = V_{m,i}^n + \Delta t (-I_{\text{ion},i}/C_m)$. Update the 18 state variables using Rush–Larsen for gates and Forward Euler for concentrations: $\mathbf{w}_i^{n+1} = \text{advance}(\mathbf{w}_i^n, V_{m,i}^n, \Delta t)$. This step is *embarrassingly parallel* — each node is independent.

Step 2 — Diffusion (the matrix-vector product): Starting from the reaction-updated voltages $\mathbf{V}_m^*$, solve: $\mathbf{M}(\mathbf{V}_m^{n+1} - \mathbf{V}_m^*)/\Delta t = -\mathbf{K}\mathbf{V}_m^{n+1}$ (for implicit diffusion) or $\mathbf{V}_m^{n+1} = \mathbf{V}_m^* + (\Delta t/\chi C_m)(-\mathbf{K}\mathbf{V}_m^*)$ (for explicit diffusion with lumped mass). For FDM, this is just the stencil applied to each grid point — 5 multiplies and adds in 2D.

Advance to next timestep: Set $\mathbf{V}_m^n \leftarrow \mathbf{V}_m^{n+1}$ and repeat. On a mesh with 1 million nodes, this means evaluating TTP06 one million times per timestep (Step 1), then performing one sparse matrix-vector product of size 10^6 (Step 2). With $\Delta t = 0.02$ ms and a simulation length of 500 ms, this is **25,000** timesteps $\times 10^6$ ionic evaluations = 2.5×10^{10} TTP06 calls. This is why efficient ionic model implementations and spatial discretizations matter.

Engine V5.4 Connection

This separation of spatial and temporal concerns is a deliberate design choice in Engine V5.4. All three discretizations (FDM, FEM, FVM) implement the `SpatialDiscretization` ABC, which exposes `apply_diffusion(V)` for explicit solvers and `get_diffusion_operators(dt, scheme)` for implicit solvers. You can switch from FDM to FEM without changing a single line of the solver code. The ionic model plugs in through the `IonicModel` interface — switching from TTP06 (18 ionic states + V) to ORD (40 ionic states + V) changes only the reaction step, not the diffusion step. Chapters 10 and 11 describe the time integrators; Chapter 14 shows how this framework extends to the bidomain's 2×2 block system.

8.7 Boundary Conditions for the Monodomain

Every PDE needs boundary conditions — rules that describe what happens at the edge of the domain. Without them, the problem has infinitely many solutions. Think of it this way: if you know that heat diffuses through a metal rod, you still cannot predict the temperature distribution unless you also know what is happening at the *ends* of the rod. Are they insulated? Are they held at a fixed temperature? Are they radiating into the surrounding air? Each choice gives a different answer.

For cardiac tissue, the same principle applies. The monodomain equation (7.1) tells us how the transmembrane voltage V_m evolves inside the tissue, but it says nothing about the tissue boundary — the surface where myocardium meets the surrounding medium (pericardium, blood, air in an *ex vivo* preparation, or simply the edge of a computational domain). We need a rule at every boundary point.

8.7.1 What Happens at the Edge of the Tissue?

There are three fundamental types of boundary conditions, each with a distinct physical interpretation for cardiac tissue:

BC Type	Mathematical Statement	Physical Meaning	When Used
Neumann	$\mathbf{D} \nabla V_m \cdot \hat{\mathbf{n}} = 0$ on $\partial\Omega$	No current leaves the tissue	Isolated heart, most simulations
Dirichlet	$V_m = V_0$ on $\partial\Omega$	Voltage is clamped by an external electrode	Voltage-clamp experiments, stimulus protocols
Robin (mixed)	$\mathbf{D} \nabla V_m \cdot \hat{\mathbf{n}} + \alpha (V_m - V_\infty) = 0$	Partial current leakage into a surrounding bath	Bath-coupled experiments, Langendorff preparations

Here $\hat{\mathbf{n}}$ is the outward unit normal to the boundary $\partial\Omega$, and $\mathbf{D}$ is the diffusion tensor from equation (7.1). The vast majority of monodomain simulations use **Neumann (zero-flux) boundary conditions**, because an isolated piece of cardiac tissue cannot leak current into empty space. This is the physically correct default.

Intuition

Why Neumann is the natural choice. In an isolated heart, the intracellular current has nowhere to go at the tissue surface — there are no cells beyond the boundary. So the current flux through the boundary must be zero: $\mathbf{D} \nabla V_m \cdot \hat{\mathbf{n}} = 0$. This is exactly the Neumann condition. Dirichlet BCs (clamped voltage) are used only in special experimental setups where an electrode physically holds the boundary voltage at a fixed value.

8.7.2 Neumann: The Natural Boundary Condition

The zero-flux condition $\mathbf{D} \nabla V_m \cdot \hat{\mathbf{n}} = 0$ on $\partial\Omega$ says that the component of the diffusive flux perpendicular to the boundary is zero. Let us see how this is implemented in each discretization framework.

Finite Difference Method (FDM). On a structured grid, the boundary lies along the edge of the grid. Consider the left boundary of a 1D cable with N nodes indexed $0, 1, \dots, N-1$. The Laplacian at node 0 requires the value at a "ghost node" at position -1 (one grid spacing outside the domain). The Neumann condition $\partial V_m / \partial x|_{x=0} = 0$ implies that $V_{-1} = V_1$ — the ghost value mirrors the first interior value. Substituting into the standard 3-point stencil at node 0 :

$$(\Delta V_m)_0 = \frac{V_{-1} - 2V_0 + V_1}{\Delta x^2} = \frac{2(V_1 - V_0)}{\Delta x^2} \quad (8.3)$$

The stencil at the boundary node uses a coefficient of **2** on the interior neighbor instead of **1**, and the ghost node disappears. In the stiffness matrix $\mathbf{K}$ from equation (8.2), this modifies only the first and last rows. The matrix remains symmetric — a crucial property for iterative solvers.

Finite Element Method (FEM). Here the Neumann condition is especially elegant. In the weak form (Section 8.1), we multiplied the PDE by a test function and integrated by parts. The integration by parts produced a boundary integral:

$$\int_{\partial\Omega} \mathbf{D} \nabla V_m \cdot \hat{\mathbf{n}} \phi_i dS$$

For Neumann BCs, this integral is exactly zero — it vanishes from the weak form without any special treatment. This is why Neumann is called the **natural boundary condition** in FEM: it is automatically satisfied by the variational formulation. You do not modify any matrix entries; you simply assemble the system as usual, and the zero-flux condition is imposed for free.

Finite Volume Method (FVM). The FVM formulation is based on flux balance across cell faces. At a boundary face, the Neumann condition says the flux is zero, so no contribution from that face enters the discrete equation. The implementation is straightforward: skip boundary faces (or set their flux contribution to zero) during assembly.

Worked Example: 5-Node Cable with Neumann BCs

Consider a 1D cable with 5 nodes ($i = 0, 1, 2, 3, 4$), spacing $\Delta x = 0.025$ cm, and isotropic diffusion coefficient $D = 0.001 \text{ cm}^2/\text{ms}$. With Neumann BCs at both ends, the modified discrete Laplacian (stiffness matrix divided by Δx^2) is:

$$\mathbf{K}_{\text{Neu}} = \frac{D}{\Delta x^2} \begin{pmatrix} -1 & 1 & & & \\ 1 & -2 & 1 & & \\ & 1 & -2 & 1 & \\ & & 1 & -2 & 1 \\ & & & 1 & -1 \end{pmatrix} \quad (8.4)$$

Notice the first and last rows: the diagonal entries are -1 instead of -2 , reflecting the ghost-node mirror condition from equation (8.3). Every row sums to zero — this is a mathematical signature of Neumann BCs. It means a spatially uniform voltage $V_m = \text{const}$ produces zero diffusive flux ($\mathbf{K} \mathbf{1} = \mathbf{0}$), which is physically correct: a flat voltage profile has no gradients, so there is no current flow.

Mathematical Note

The null space of Neumann. The all-ones vector $\mathbf{1}$ is in the null space of $\mathbf{K}_{\text{Neu}}$. For the monodomain, this is not a problem — the mass matrix $\mathbf{M}$ and the reaction term break the degeneracy. But for the bidomain's elliptic equation (Chapter 12), this null space becomes a significant issue: the extracellular potential ϕ_e is determined only up to an additive constant. We will return to this in Section 12.6.

The forward pointer to spectral methods is also worth noting: the cosine basis naturally satisfies Neumann BCs (Appendix C.10), which is why the DCT diagonalizes $\mathbf{K}_{\text{Neu}}$. The eigenvalues of equation (C.5) are precisely the eigenvalues of this matrix.

8.7.3 Dirichlet Boundary Conditions

The Dirichlet condition $V_m|_{\partial\Omega} = V_0$ fixes the voltage at the boundary. Physically, this represents an ideal electrode that clamps the transmembrane voltage to a prescribed value — for instance, a voltage-clamp experiment where one end of a fiber is held at -80 mV.

FDM implementation. For a Dirichlet node, we do not solve an equation — the value is known. We replace the equation at the boundary node with $V_{m,0} = V_0$. In matrix terms, we zero out the row and column corresponding to the boundary node, place **1** on the diagonal, and move the known value to the right-hand side:

$$\text{Row 0 : } \mathbf{1} \cdot V_{m,0} = V_0 \quad (8.5)$$

All other rows that reference $V_{m,0}$ have that contribution moved to the right-hand side vector **b**. For example, row **1** originally reads $(V_0 - 2V_1 + V_2)/\Delta x^2$; after Dirichlet elimination, it becomes $(-2V_1 + V_2)/\Delta x^2 + V_0/\Delta x^2$, where the last term is a known constant added to b_1 .

FEM implementation. Dirichlet BCs are called **essential boundary conditions** in FEM — they must be explicitly enforced. The standard approach is the same row/column elimination: set the diagonal to **1**, zero off-diagonal entries in that row and column, and adjust the right-hand side. An alternative is the **penalty method**: add a large number **P** (e.g., 10^{10}) to the diagonal entry and set $b_i = P \cdot V_0$, which forces $V_{m,i} \approx V_0$ without modifying the matrix structure.

Worked Example: Stimulus at One End

Using the same 5-node cable, suppose we apply a stimulus by clamping $V_{m,0} = 20$ mV (Dirichlet) while keeping the right end insulated (Neumann at $i = 4$). The modified system matrix becomes:

$$\mathbf{K}_{\text{mixed}} = \frac{D}{\Delta x^2} \begin{pmatrix} 1 & 0 & & & \\ 0 & -2 & 1 & & \\ & 1 & -2 & 1 & \\ & & 1 & -2 & 1 \\ & & & 1 & -1 \end{pmatrix}, \quad \mathbf{b}_{\text{mixed}} = \frac{D}{\Delta x^2} \begin{pmatrix} 20 \text{ mV} \\ 20 \text{ mV} \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad (8.6)$$

Row 0 enforces $V_{m,0} = 20$ mV (Dirichlet). Row 1 has the contribution from $V_{m,0}$ moved to the right-hand side ($b_1 = D \cdot V_0/\Delta x^2$). Rows 2–3 are standard interior stencils. Row 4 has the Neumann modification (-1 on diagonal instead of -2). This mixed system is the most common setup for 1D cable simulations with a stimulus electrode.

The sine basis naturally satisfies Dirichlet BCs (Appendix C.10), which is why the DST diagonalizes the Laplacian with clamped boundaries. The eigenvalues of equation (C.8) correspond to this case.

8.7.4 Summary and Forward Pointers

For the monodomain equation, boundary conditions are relatively straightforward — there is a single unknown (V_m) and a single condition per boundary segment. The table below summarizes the implementation strategies:

BC Type	FDM	FEM	Spectral
Neumann (zero-flux)	Ghost node mirroring (8.3)	Natural — automatic from weak form	DCT (Appendix C.10)
Dirichlet (fixed voltage)	Row elimination (8.5)	Essential — row elimination or penalty	DST (Appendix C.10)
Mixed (Neumann on some edges, Dirichlet on others)	Combine above per boundary segment	Combine above per boundary segment	DCT+DST on different axes (C.8)

The situation becomes substantially more complex for the bidomain model, where we have two unknowns (V_m and ϕ_e) and must specify boundary conditions for each independently. The interplay between intracel-

lular and extracellular BCs — and the consequences for the linear system's null space — is the subject of Section 12.6.

⚙ Engine V5.4 Connection

Engine V5.4 connection. All three spatial discretization classes (`FDMDiscretization`, `FEMDiscretization`, `FVMDiscretization`) handle Neumann BCs by default. For FDM, the stiffness matrix $\mathbf{K}$ is constructed with the modified boundary stencil from equation (8.3). For FEM, Neumann BCs emerge automatically from the assembly loop. Dirichlet BCs are enforced through row/column elimination in the `apply_dirichlet()` method, which modifies both the system matrix and the right-hand side vector. The spectral solvers (`DCTSolver`, `FTTSolver`) handle BCs implicitly through the choice of transform basis — no matrix modification is needed.

Operator Splitting: Divide and Conquer

The monodomain equation couples a stiff, nonlinear ODE system (ion channels) with a linear diffusion PDE. Trying to solve them simultaneously would be nightmarish. Operator splitting breaks the problem into two simpler subproblems — and it works remarkably well.

9.1 Why Split? The Stiffness Argument

Imagine you are cooking a stir-fry. The vegetables need to sizzle at high heat for 30 seconds (fast process), while the rice simmers gently for 20 minutes (slow process). Trying to cook both in the same pot at the same temperature is hopeless — you either burn the rice or undercook the vegetables. The smart approach: cook them in separate pots, each at the right temperature, then combine at the end. Operator splitting does exactly this for the monodomain equation: the ionic channels (fast, local, nonlinear) get their own "pot," and the diffusion (slow, global, linear) gets another.

The fundamental problem is *timescale separation*. During the upstroke (Phase 0), the sodium gate m changes with a time constant of ~ 0.1 ms, while the diffusion length scale requires timesteps of ~ 0.01 ms for stability. But during the plateau (Phase 2), everything moves slowly — $\tau \sim 50\text{--}200$ ms. Any method that tries to handle both regimes in a single system must either use tiny timesteps everywhere (wasting computation during the plateau) or use implicit methods for the full coupled system (requiring the solution of a massive nonlinear system at each step).

Operator splitting exploits the structure of the problem. Write equation (7.1) as:

$$\frac{\partial V_m}{\partial t} = \underbrace{\frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla V_m)}_{\mathcal{D}(V_m)} + \underbrace{\frac{-I_{\text{ion}}(V_m, \mathbf{w})}{C_m}}_{\mathcal{R}(V_m, \mathbf{w})}$$

Instead of solving $\partial V_m / \partial t = \mathcal{D} + \mathcal{R}$ as one equation, we solve them sequentially: first advance the reaction $\mathcal{R}$ (purely local ODE at each point), then advance the diffusion $\mathcal{D}$ (linear PDE, no ionic coupling).

9.2 Godunov (First-Order) Splitting

The simplest splitting method advances from t^n to $t^{n+1} = t^n + \Delta t$ in two stages:

Stage 1 — Reaction: At every spatial point, solve the ODE system $dV_m/dt = \mathcal{R}(V_m, \mathbf{w})$ and $d\mathbf{w}/dt = \mathbf{g}(V_m, \mathbf{w})$ from t^n to t^{n+1} , using V_m^n as the initial condition.

Stage 2 — Diffusion: Starting from the updated V_m^* (output of Stage 1), solve $\partial V_m / \partial t = \mathcal{D}(V_m)$ from t^n to t^{n+1} .

Godunov splitting is first-order accurate in time: the splitting error is $O(\Delta t)$. For cardiac simulations with $\Delta t = 0.01\text{--}0.05$ ms, this error is generally acceptable because the spatial and temporal discretization errors are of similar magnitude.

9.3 Strang (Second-Order) Splitting

For higher accuracy, Strang splitting uses a symmetric sequence:

Half-step reaction ($\Delta t/2$) $\rightarrow$ **Full-step diffusion** (Δt) $\rightarrow$ **Half-step reaction** ($\Delta t/2$).

The symmetry cancels the leading-order splitting error, yielding $O(\Delta t^2)$ accuracy. The cost is modestly higher (two reaction solves instead of one, though each is half as long), but the improved accuracy can al-

low larger timesteps for the same error tolerance.

Mathematical Note

Strang splitting is second-order because of a *symmetry argument*. If the error of $\mathcal{R} \rightarrow \mathcal{D}$ is $+c\Delta t^2$ for some constant c , then the error of $\mathcal{D} \rightarrow \mathcal{R}$ is $-c\Delta t^2$ (the error changes sign when you swap the order). By sandwiching — $\mathcal{R}(\Delta t/2) \rightarrow \mathcal{D}(\Delta t) \rightarrow \mathcal{R}(\Delta t/2)$ — the first-order errors cancel, leaving $O(\Delta t^3)$ local error and $O(\Delta t^2)$ global error.

9.4 The Rush–Larsen Method for Ionic Variables

During the reaction substep, we must integrate the gating ODEs $dx/dt = (x_\infty - x)/\tau_x$. Forward Euler would require very small substeps for gates with fast kinetics (small τ_x). The Rush–Larsen method side-steps this entirely by noting that for *fixed voltage*, the gating ODE has an exact exponential solution:

$$x^{n+1} = x_\infty(V_m^n) + (x^n - x_\infty(V_m^n)) e^{-\Delta t/\tau_x(V_m^n)} \quad (9.1)$$

This is unconditionally stable for any $\Delta t > 0$: even if $\tau_x = 0.01$ ms (the fastest Na^+ gate during the up-stroke), the exponential simply drives x to x_∞ without oscillation. For slow gates ($\tau_x \gg \Delta t$), the exponential reduces to Forward Euler, so no accuracy is lost.

Intuition

Rush–Larsen is like solving the "easy part" of the ODE exactly and only approximating the "hard part" (the voltage dependence of x_∞ and τ_x). Since these parameters change slowly compared to the gate dynamics during an upstroke, the approximation is excellent. It transforms the stiffest part of the problem — fast gating — from a numerical headache into a single exponential evaluation per gate per timestep.

Engine V5.4 Connection

In Engine V5.4, the splitting and ionic integration are cleanly separated. The `SplittingStrategy ABC` (`solver/splitting/base.py`) owns an `IonicSolver` and a `DiffusionSolver` and calls them in the appropriate order. `GodunovSplitting` calls `ionic_solver.step(state, dt)` then `diffusion_solver.step(state, dt)`. `StrangSplitting` calls ionic at `dt/2`, diffusion at `dt`, ionic at `dt/2`.

The `RushLarsenSolver` (`solver/ionic_time_stepping/rush_larsen.py`) is a subclass of the `IonicSolver ABC`. Its `step()` method follows a specific order: (1) compute I_{ion} from the current state, (2) compute x_∞ and τ_x from the *old* voltage, (3) Forward Euler on voltage, (4) Rush–Larsen exponential update on gates, (5) Forward Euler on concentrations. A separate `ForwardEulerIonicSolver` is available for debugging. Concentration variables always use Forward Euler because they do not follow relaxation kinetics.

Explicit Diffusion Solvers

After the reaction substep updates the ionic variables, the diffusion substep must advance $\mathbf{V}_m$ under pure diffusion. Explicit time-stepping methods — from the workhorse Forward Euler to the high-accuracy RK4 — form a family of increasing sophistication. This chapter walks through each one, starting from intuition and building to implementation-ready equations.

10.1 Forward Euler and the CFL Condition

The simplest idea for advancing the diffusion equation is also the most intuitive: look at the current state, compute how fast things are changing, and take a step forward. This is *Forward Euler* — the computational equivalent of "if I'm heading north at 60 mph, in one hour I'll be 60 miles north." It works beautifully as long as you don't take steps that are too large. Take too big a step, and the solution oscillates wildly and blows up — this limit is called the *CFL condition*, and it is the fundamental price of simplicity.

Here is the core idea: at each timestep, we evaluate the right-hand side of the diffusion equation (the spatial Laplacian acting on $\mathbf{V}_m$), multiply by Δt , and add the result to the current state. We never look ahead — only at where we *are*. That is what makes it "explicit": the new state is computed entirely from the old state, with no system of equations to solve.

In the semi-discrete form from equation (8.2), the diffusion substep is the ODE system $\mathbf{M} d\mathbf{V}/dt = -\mathbf{K} \mathbf{V}$. Forward Euler approximates the time derivative by a forward difference:

$$\mathbf{V}^{n+1} = \mathbf{V}^n + \Delta t \mathbf{M}^{-1}(-\mathbf{K} \mathbf{V}^n) \quad (10.1)$$

When using FDM with lumped mass ($\mathbf{M} = \mathbf{I}$), this simplifies to $\mathbf{V}^{n+1} = \mathbf{V}^n - \Delta t \mathbf{K} \mathbf{V}^n$. Each grid point's update depends only on its neighbors at time level n — no global communication, no matrix solve. This is why Forward Euler is trivially parallelizable and ideal for GPU computation.

The price is the *CFL stability condition*. For a regular grid with spacing h and diffusivity d , Forward Euler is stable only when:

$$\Delta t \leq \frac{h^2}{2 n_{\text{dim}} d_{\text{max}}} \quad (10.2)$$

where n_{dim} is the spatial dimension and d_{max} is the largest eigenvalue of the conductivity tensor. For typical cardiac parameters ($h = 0.2$ mm, $d_{\text{max}} = 0.174$ mm²/ms, 2D), this gives $\Delta t \leq 0.057$ ms — tight, but workable for the $\Delta t = 0.01$ – 0.05 ms range that the ionic model already requires.

Intuition

Worked CFL example. Suppose we want to simulate a 2D cardiac sheet with $h = 0.1$ mm (a fine mesh) and longitudinal conductivity $d_L = 0.174$ mm²/ms. Then $\Delta t_{\text{max}} = (0.1)^2 / (2 \times 2 \times 0.174) = 0.01 / 0.696 = 0.0144$ ms. That is below the typical ionic timestep of 0.02 ms — so for fine meshes, the diffusion stability constraint becomes the bottleneck, not the ionic kinetics. This is exactly why Chapter 11's implicit methods exist.

△ Key Insight

The CFL condition scales as h^2 . Halve the mesh spacing and the maximum timestep drops by a factor of four. For fine 3D meshes ($h = 0.1$ mm), this can force Δt below what the ionic solver needs, making explicit methods the bottleneck. This is the primary motivation for implicit methods (Chapter 11).

10.2 Why Go Beyond Forward Euler?

Imagine driving from New York to Boston. Forward Euler checks your speedometer *once* at the start and assumes that speed holds for the entire trip. If the road curves, you end up in the ocean. A smarter driver checks the speedometer multiple times along the way — at the start, at the midpoint, near the end — and uses all those readings to estimate where they actually end up. That is what Runge–Kutta methods do: *sample the slope multiple times within each timestep* and combine the samples into a much better estimate of the true solution.

The key insight is that Forward Euler's error comes from assuming the derivative is constant across the entire step. In reality, the derivative changes as $\mathbf{V}_m$ changes. By evaluating the derivative at intermediate points within the step, we can capture this curvature. More evaluations = better curvature tracking = higher-order accuracy. The family of methods that does this systematically is called **Runge–Kutta (RK) methods**, named after Carl Runge and Martin Kutta who developed them in the early 1900s.

Every explicit RK method follows the same recipe: (1) evaluate the right-hand side at several carefully chosen points, (2) take a weighted average of those evaluations, (3) use that average as the effective slope for the timestep. The differences between RK1 (Forward Euler), RK2, and RK4 are simply *how many* sample points they use and *how* they weight them.

10.3 RK2 — Heun's Method (The Two-Sample Correction)

RK2 is like a rough draft followed by a revision. You start by making the same guess Forward Euler would (the "predictor") — but instead of stopping there, you evaluate the slope at that predicted point too, and then average the two slopes. The result is far more accurate because you have sampled the derivative at both ends of the interval instead of just one.

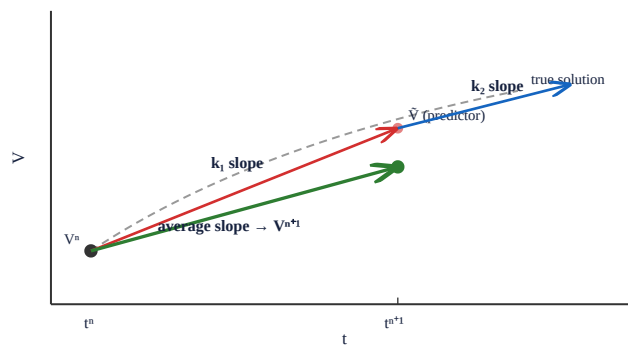


Figure 10.1 — RK2 (Heun's method) visualized. The red arrow is the Forward Euler predictor (k_1 slope). The blue arrow is the slope evaluated at the predicted point (k_2). The green arrow is the average of both slopes — which lands much closer to the true solution (dashed gray).

Formally, RK2 (Heun's method) advances from $\mathbf{V}^n$ to $\mathbf{V}^{n+1}$ in two stages. Let $\mathbf{f}(\mathbf{V}) = \mathbf{M}^{-1}(-\mathbf{K}\mathbf{V})$ denote the right-hand side of the diffusion ODE:

$$\begin{aligned}
\mathbf{k}_1 &= \Delta t \, \mathbf{f}(\mathbf{V}^n) && \text{(slope at start)} \\
\mathbf{k}_2 &= \Delta t \, \mathbf{f}(\mathbf{V}^n + \mathbf{k}_1) && \text{(slope at FE predictor)} \\
\mathbf{V}^{n+1} &= \mathbf{V}^n + \frac{1}{2}(\mathbf{k}_1 + \mathbf{k}_2) && \text{(average both slopes)}
\end{aligned}$$

(10.3)

That final line is the key: $\mathbf{V}^{n+1}$ is *not* the Forward Euler predictor $\mathbf{V}^n + \mathbf{k}_1$. Instead, we go back to $\mathbf{V}^n$ and use the average of the two slopes, $\frac{1}{2}(\mathbf{k}_1 + \mathbf{k}_2)$. This is where the name "predictor–corrector" comes from: $\mathbf{k}_1$ predicts, and the averaging with $\mathbf{k}_2$ corrects.

Expanding $\mathbf{f}(\mathbf{V}) = \mathcal{L}\mathbf{V}$ where $\mathcal{L}$ is the discrete diffusion operator (i.e., `apply_diffusion(V)` from Chapter 8), the matrix form computed at each timestep is:

$$\begin{aligned}
\mathbf{k}_1 &= \mathcal{L}\mathbf{V}^n \\
\mathbf{k}_2 &= \mathcal{L}(\mathbf{V}^n + \Delta t \mathbf{k}_1) \\
\mathbf{V}^{n+1} &= \mathbf{V}^n + \frac{\Delta t}{2}(\mathbf{k}_1 + \mathbf{k}_2)
\end{aligned}$$

(10.3b)

Note that each $\mathbf{k}_i$ here is a *bare* rate vector (units of mV/ms) — the factor Δt is applied only at the final update. This is the convention used by the solver code and is mathematically equivalent to the standard form (10.3) where $\mathbf{k}_i = \Delta t \cdot \mathbf{f}(\mathbf{V})$.

Mathematical Note

Why exactly $\frac{1}{2}$? Heun's method is designed so that the Taylor expansion of $\mathbf{V}^{n+1}$ matches the true solution through $O(\Delta t^2)$. Forward Euler matches only through $O(\Delta t)$. By evaluating $\mathbf{f}$ at the predicted point and averaging, we capture the second-order term $\frac{1}{2}\Delta t^2 \mathbf{f}'(\mathbf{V}^n)\mathbf{f}(\mathbf{V}^n)$ that Forward Euler misses. The coefficient $\frac{1}{2}$ is not arbitrary — it is the unique weight that achieves this cancellation for the two-stage Heun form.

Intuition

Worked example. Consider $dV/dt = -2V$ with $V^0 = 1.0$ and $\Delta t = 0.1$. The exact solution at $t = 0.1$ is $e^{-0.2} = 0.81873$.

Forward Euler: $V^1 = 1.0 + 0.1 \times (-2 \times 1.0) = 0.800$. Error = **0.01873**.

RK2 (Heun):

$$\mathbf{k}_1 = 0.1 \times (-2 \times 1.0) = -0.2$$

$$\tilde{V} = 1.0 + (-0.2) = 0.8$$

$$\mathbf{k}_2 = 0.1 \times (-2 \times 0.8) = -0.16$$

$$V^1 = 1.0 + \frac{1}{2}(-0.2 + (-0.16)) = 1.0 - 0.18 = 0.820. \text{ Error} = \mathbf{0.00127}.$$

RK2's error (**0.00127**) is **15× smaller** than Forward Euler's (**0.01873**) for the same timestep — and it only cost one extra function evaluation.

10.4 RK4 — The Classical Four-Stage Method

If RK2 is checking your speedometer twice, RK4 checks it *four times*: at the start, twice at the midpoint (using two different estimates), and once at the end. It then combines all four readings with a specific weighted average that is tuned to capture the curvature of the solution remarkably well. RK4 is so accurate that it has been the default ODE integrator in science and engineering since the 1900s — a century of trust.

The genius of RK4 is its *midpoint sampling*. Rather than just checking the endpoints (like RK2), it takes two independent samples at $t^n + \Delta t/2$. Why the midpoint? Because the midpoint rule for integration is second-order accurate all by itself, and by doing it twice (with refined estimates), the errors cancel to fourth order. The final weighted average — giving the midpoint samples twice the weight of the endpoints — is not a coincidence. It mirrors Simpson's rule from numerical integration, which achieves fourth-order accuracy by weighting the endpoints and midpoint as $\frac{1}{6}, \frac{4}{6}, \frac{1}{6}$.

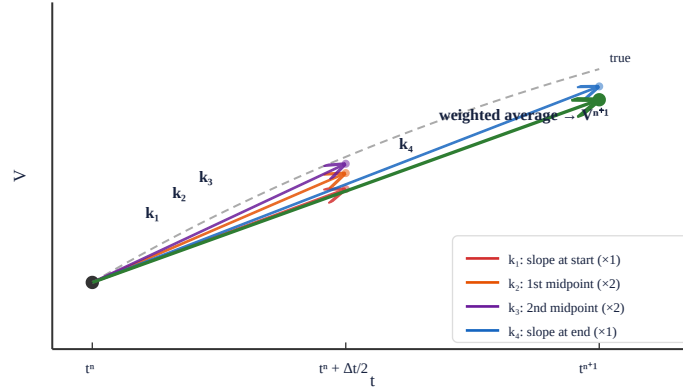


Figure 10.2 — The four stages of RK4. Each colored line shows a slope evaluation; the green result is the weighted average $\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$. The midpoint slopes (k_2, k_3) get double weight, mirroring Simpson's rule.

The classical RK4 method evaluates the right-hand side $f(V) = \mathbf{M}^{-1}(-\mathbf{K}V)$ at four carefully chosen points:

$$\begin{aligned}
 k_1 &= \Delta t f(V^n) && \text{(slope at } t^n) \\
 k_2 &= \Delta t f(V^n + \tfrac{1}{2} k_1) && \text{(1st midpoint, using } k_1 \text{ to get there)} \\
 k_3 &= \Delta t f(V^n + \tfrac{1}{2} k_2) && \text{(2nd midpoint, using } k_2 \text{ to get there)} \\
 k_4 &= \Delta t f(V^n + k_3) && \text{(slope at } t^{n+1}, \text{ using } k_3 \text{ to get there)} \\
 V^{n+1} &= V^n + \tfrac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) && \text{(weighted average)}
 \end{aligned} \tag{10.4}$$

Read the stages as a conversation: k_1 says "here's the slope at the start." k_2 says "okay, I used your slope to walk to the midpoint — here's the slope *there*." k_3 says "let me try the midpoint again using k_2 's improved estimate." k_4 says "and here's the slope at the far end, using k_3 ." The final weighted average gives the midpoint samples (k_2, k_3) double weight because the midpoint carries more information about the interval's curvature — exactly as in Simpson's 1-4-1 quadrature rule.

In discrete matrix form with $\mathcal{L}$ as the spatial operator:

$$\begin{aligned}
 \mathbf{k}_1 &= \mathcal{L} \mathbf{V}^n \\
 \mathbf{k}_2 &= \mathcal{L}(\mathbf{V}^n + \tfrac{\Delta t}{2} \mathbf{k}_1) \\
 \mathbf{k}_3 &= \mathcal{L}(\mathbf{V}^n + \tfrac{\Delta t}{2} \mathbf{k}_2) \\
 \mathbf{k}_4 &= \mathcal{L}(\mathbf{V}^n + \Delta t \mathbf{k}_3) \\
 \mathbf{V}^{n+1} &= \mathbf{V}^n + \tfrac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)
 \end{aligned} \tag{10.4b}$$

Again, each $\mathbf{k}_i$ is a bare rate (mV/ms), not pre-multiplied by Δt . The factor Δt appears only at the final update, which avoids four unnecessary multiplications per step.

Mathematical Note

Why these specific weights? The weights $\frac{1}{6}(1, 2, 2, 1)$ are not chosen by trial and error. They come from matching the Taylor expansion of $V(t^n + \Delta t)$ through the Δt^4 term. There are 8 "order conditions" that a 4th-order method must satisfy (corresponding to the 8 rooted trees with up to 4 vertices in Butcher's theory). The classical RK4 tableau — with its midpoint-endpoint structure — is the simplest 4-stage method satisfying all 8 conditions. No 3-stage explicit method can achieve 4th-order accuracy.

Intuition

Worked example. Same test: $dV/dt = -2V$, $V^0 = 1.0$, $\Delta t = 0.1$. Exact answer: $e^{-0.2} = 0.818731$.

$$k_1 = 0.1 \times (-2 \times 1.0) = -0.200$$

$$k_2 = 0.1 \times (-2 \times (1.0 - 0.100)) = 0.1 \times (-1.800) = -0.180$$

$$k_3 = 0.1 \times (-2 \times (1.0 - 0.090)) = 0.1 \times (-1.820) = -0.182$$

$$k_4 = 0.1 \times (-2 \times (1.0 - 0.182)) = 0.1 \times (-1.636) = -0.1636$$

$$\begin{aligned} V^1 &= 1.0 + \frac{1}{6}(-0.200 + 2(-0.180) + 2(-0.182) + (-0.1636)) \\ &= 1.0 + \frac{1}{6}(-1.0876) = 1.0 - 0.18127 = 0.81873 \end{aligned}$$

Error = 0.000001 — five orders of magnitude smaller than Forward Euler, for just 4× the cost. This is the power of fourth-order convergence.

10.5 Stability Regions: How Large Can Δt Be?

Every explicit method has a "safe zone" — a region in the complex plane where the timestep is small enough that the solution stays bounded. Step outside this zone, and the solution blows up exponentially. The size and shape of this safe zone determines the practical maximum timestep.

For the test equation $dV/dt = \lambda V$ (where λ is an eigenvalue of the diffusion operator), one timestep gives $V^{n+1} = R(z)V^n$ where $z = \lambda \Delta t$ and $R(z)$ is the *stability function*. The method is stable when $|R(z)| \leq 1$. For diffusion, λ is real and negative (eigenvalues of $-\mathbf{K}$), so we only need the intersection of the stability region with the negative real axis.

The stability functions for each method are:

Forward Euler:	$R(z) = 1 + z$	stable for $z \in [-2, 0]$	(10.5)
RK2 (Heun):	$R(z) = 1 + z + \frac{1}{2}z^2$	stable for $z \in [-2, 0]$	
RK4:	$R(z) = 1 + z + \frac{1}{2}z^2 + \frac{1}{6}z^3 + \frac{1}{24}z^4$	stable for $z \in [-2.785, 0]$	

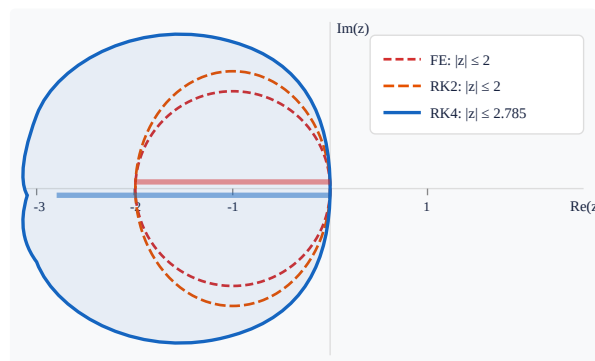


Figure 10.3 — Stability regions in the complex $z = \lambda \Delta t$ plane. The method is stable when z lies inside the boundary. Along the negative real axis (which is what matters for diffusion), RK4 allows $\sim 39\%$ larger timesteps than FE/RK2.

A critical observation: **RK2 and Forward Euler have the same stability limit along the real axis** ($|z| \leq 2$). So RK2 cannot take larger timesteps than Forward Euler for pure diffusion — its advantage is purely in *accuracy per step*, not stability. RK4, however, extends the real-axis limit to $|z| \leq 2.785$, allowing about 39% larger steps. In terms of the CFL condition:

$$\Delta t_{\text{RK4}} \leq \frac{2.785 h^2}{2 n_{\text{dim}} d_{\text{max}}} \approx 1.39 \Delta t_{\text{FE}}$$
(10.6)

10.6 Cost vs. Accuracy: Choosing Your Explicit Method

Each RK method trades cost for accuracy. The question is: for a given error tolerance, which method minimizes total work? The answer depends on whether you are *stability-limited* (the CFL condition forces small Δt) or *accuracy-limited* (you could take larger Δt but the error would be too large).

Method	f evals/step	Order	Real-axis stability	Δt_{max} (relative)	Best when...
Forward Euler	1	1	$[-2, 0]$	1.0×	CFL-limited (fine mesh, ionic $\Delta t \gg \text{CFL}$)
RK2 (Heun)	2	2	$[-2, 0]$	1.0×	Accuracy matters, not quite CFL-limited
RK4	4	4	$[-2.785, 0]$	1.39×	Smooth solution, accuracy-limited regime

The practical verdict for cardiac simulation: Most cardiac simulations are CFL-limited — the sharp wavefront during depolarization demands small Δt regardless of accuracy concerns. In this regime, Forward Euler is usually the best choice: the CFL condition forces a small enough Δt that first-order accuracy is adequate, and the cost per step is minimal. RK4 shines in accuracy-limited regimes (coarse mesh, smooth solutions) or when coupled with adaptive timestepping, which Chapter 10 does not cover but is an active research direction.

10.7 When Explicit Methods Shine

Despite the CFL constraint, explicit methods remain competitive in several important scenarios:

The mesh is coarse enough that the CFL limit does not bite. For $h = 0.2\text{--}0.5$ mm (common in 2D studies), the allowed Δt is comparable to or larger than what the ionic solver needs anyway.

GPU parallelism is critical. Explicit methods are embarrassingly parallel — each cell's update is independent. No linear system needs to be solved, no global communication is required. On modern GPUs with thousands of cores, this parallelism translates directly to speed.

Memory is limited. Explicit methods store only the current state and the Laplacian evaluation — no preconditioner, no Krylov vectors, no mass matrix factorization.

⚙ Engine V5.4 Connection

In Engine V5.4, all three explicit methods inherit from a `DiffusionSolver ABC` (`solver/diffusion_time_stepping/base.py`). Each calls `self._spatial.apply_diffusion(V)` — the $\mathcal{L}$ operator — which encapsulates FDM, FEM, or FVM (Chapter 8). Swapping between spatial methods is a one-line configuration change; the time integrator only sees the operator $\mathcal{L}$. Explicit solvers need no pre-built matrices; only the implicit solvers (Chapter 11) build `DiffusionOperators`.

⚠ Key Insight

Memory consideration for RK4. Each $\mathbf{k}_i$ is a full-length voltage vector (one float per mesh node). RK4 stores 4 of these simultaneously, plus the original $\mathbf{V}^n$. For a 3D mesh with 10^6 nodes at 8 bytes/float, that is $5 \times 8 \text{ MB} = 40 \text{ MB}$ — trivial on a CPU, but meaningful on a GPU where shared memory is limited. Forward Euler needs only one extra vector ($2 \times 8 \text{ MB}$). This is another reason why Forward Euler dominates GPU cardiac codes.

Implicit Diffusion Solvers

When meshes are fine or three-dimensional, the CFL constraint makes explicit methods impractical. Implicit methods trade the simplicity of a matrix-free update for unconditional stability — at the cost of solving a linear system at every timestep.

11.1 Why Implicit? The CFL Wall

Explicit methods (Chapter 10) are like steering a boat by looking only at where you *are*. If the river is calm, this works fine. But in rapids (fine meshes, fast diffusion), the boat oscillates wildly because you are always reacting to where you *were*, not where you are *going*. Implicit methods fix this by peeking ahead: they ask "where will I be at the end of this step?" and solve for that future state directly. This costs more per step (you must solve a system of equations), but the payoff is unconditional stability — you can take any step size without blowing up.

The fundamental issue is that the CFL condition (equation 10.2) scales as h^2 : halve the mesh spacing and the maximum timestep drops by a factor of four. For a 3D mesh with $h = 0.1$ mm ($d_{\max} = 0.174$ mm²/ms), the CFL limit is $\Delta t \leq 0.004$ ms — far below the 0.01–0.05 ms range the ionic solver prefers. Running 5–12 explicit diffusion substeps per ionic step wastes enormous effort. Implicit methods eliminate this constraint entirely: they are *unconditionally stable*, meaning Δt is limited only by accuracy, not by stability. In practice, this lets us match the diffusion timestep to the ionic timestep, halving or more the total simulation time.

The price is that at each timestep, the new voltage $\mathbf{V}^{n+1}$ appears on *both sides* of the equation — so we must solve a linear system $\mathbf{A} \mathbf{V}^{n+1} = \mathbf{b}$ to find it. This is where the machinery of Chapter 8 pays off: the stiffness matrix $\mathbf{K}$ and mass matrix $\mathbf{M}$ from the spatial discretization become the building blocks of $\mathbf{A}$ and $\mathbf{b}$.

11.2 Backward Euler (BDF1): The Simplest Implicit Method

If Forward Euler asks "where am I now, and where does the slope say I should go?", Backward Euler asks the reverse: "where do I end up such that the slope *there* points back to where I started?" This backward-looking logic makes it inherently stable — it cannot overshoot because it is defined by the destination, not the departure.

Formally, Backward Euler evaluates the diffusion operator at the *new* time level:

$$\mathbf{V}^{n+1} = \mathbf{V}^n + \Delta t \mathcal{D}(\mathbf{V}^{n+1}) \quad (11.1)$$

Since $\mathbf{V}^{n+1}$ appears on both sides, this is an *implicit* equation. To solve it, we substitute the semi-discrete diffusion operator from Chapter 8: $\mathbf{M} d\mathbf{V}/dt = -\mathbf{K} \mathbf{V}$. Replacing $d\mathbf{V}/dt$ with the backward difference $(\mathbf{V}^{n+1} - \mathbf{V}^n)/\Delta t$ and rearranging:

$$\underbrace{(\mathbf{M} + \Delta t \mathbf{K})}_{\mathbf{A}_{\text{lhs}}} \mathbf{V}^{n+1} = \underbrace{\mathbf{M}}_{\mathbf{B}_{\text{rhs}}} \mathbf{V}^n \quad (11.2)$$

This is the exact linear system the code solves. The left-hand side matrix $\mathbf{A}_{\text{lhs}} = \mathbf{M} + \Delta t \mathbf{K}$ is symmetric positive definite (SPD) for the standard monodomain problem with Neumann boundary conditions, so the

conjugate gradient method converges reliably. The right-hand side is simply $\mathbf{B}_{\text{rhs}} \cdot \mathbf{V}^n$ — a single sparse matrix-vector product.

Backward Euler is first-order accurate ($O(\Delta t)$) and *L-stable* — meaning it not only stays bounded but actively *damps* high-frequency modes. This aggressive damping is useful after discontinuous events (stimulus application, wavefront collision) where oscillations would otherwise pollute the solution. It is also the natural first step for multi-step methods like BDF2.

For FDM with lumped mass ($\mathbf{M} = \mathbf{I}$), the system simplifies to $(\mathbf{I} + \Delta t \mathbf{K}) \mathbf{V}^{n+1} = \mathbf{V}^n$.

11.3 Crank–Nicolson: Second-Order Implicit

Backward Euler looks only at the destination; Forward Euler looks only at the start. Crank–Nicolson *averages the two* — evaluating the diffusion operator at both the old and new time levels and taking the mid-point. This averaging is what gives it second-order accuracy: by sampling both ends of the interval, it captures the curvature that either method alone would miss.

The Crank–Nicolson method (also called the trapezoidal rule) applies the diffusion operator at the half-way point between old and new:

$$\mathbf{V}^{n+1} = \mathbf{V}^n + \frac{\Delta t}{2} [\mathcal{D}(\mathbf{V}^n) + \mathcal{D}(\mathbf{V}^{n+1})] \quad (11.3)$$

Substituting $\mathcal{D}(\mathbf{V}) = -\mathbf{M}^{-1} \mathbf{K} \mathbf{V}$, multiplying through by $\mathbf{M}$, and collecting terms:

$$\underbrace{\left(\mathbf{M} + \frac{\Delta t}{2} \mathbf{K} \right)}_{\mathbf{A}_{\text{lhs}}} \mathbf{V}^{n+1} = \underbrace{\left(\mathbf{M} - \frac{\Delta t}{2} \mathbf{K} \right)}_{\mathbf{B}_{\text{rhs}}} \mathbf{V}^n \quad (11.4)$$

Compare with Backward Euler (11.2): the only difference is the $\frac{1}{2}$ on the $\mathbf{K}$ terms and the appearance of $\mathbf{K}$ on the right-hand side. Both sides now involve the stiffness matrix, but the computation is the same: one sparse matrix-vector product for the RHS, then a linear solve with $\mathbf{A}_{\text{lhs}}$.

Crank–Nicolson is second-order accurate ($O(\Delta t^2)$) and A-stable — unconditionally stable, but *not* L-stable. The practical consequence: at very sharp wavefronts, CN can produce mild $2\Delta t$ oscillations (the Crank–Nicolson "ringing" artifact). These oscillations do not grow (A-stability) but they can be aesthetically unpleasant and may trigger spurious ionic model activation. For most cardiac simulations with $\Delta t \leq 0.05$ ms, the oscillations are negligible.

Intuition

Worked example. Consider a 5-node 1D cable (FDM, lumped mass, $\mathbf{M} = \mathbf{I}$) with stiffness matrix $\mathbf{K}$ from Chapter 8 and $\Delta t = 0.02$ ms. The CN matrices are:

$$\mathbf{A}_{\text{lhs}} = \mathbf{I} + 0.01 \mathbf{K}, \quad \mathbf{B}_{\text{rhs}} = \mathbf{I} - 0.01 \mathbf{K}$$

At each timestep: (1) compute $\mathbf{b} = \mathbf{B}_{\text{rhs}} \cdot \mathbf{V}^n$ (sparse matvec, $\sim 5N$ flops), then (2) solve $\mathbf{A}_{\text{lhs}} \cdot \mathbf{V}^{n+1} = \mathbf{b}$ (CG iteration, 10–30 iterations $\times \sim 5N$ flops per iteration). Compare with Forward Euler: one matvec per step, but requiring $\Delta t \leq h^2/(2d) \approx 0.003$ ms — meaning CN can take $\sim 7\times$ larger steps for the same total cost per unit simulation time.

△ Key Insight

Both $\mathbf{A}_{\text{lhs}}$ and $\mathbf{B}_{\text{rhs}}$ are precomputed once at the start of the simulation (or when Δt changes) and reused at every timestep. They are sparse matrices with the same sparsity pattern as $\mathbf{K}$. This one-time assembly cost is amortized over thousands of timesteps.

11.4 BDF2: Second-Order with Strong Damping

BDF2 is like Crank–Nicolson's tougher cousin. Both are second-order accurate, but BDF2 achieves this by using *two previous time levels* — $\mathbf{V}^n$ and $\mathbf{V}^{n-1}$ — rather than averaging the diffusion operator. The payoff is *L-stability*: BDF2 aggressively damps stiff transients that would cause CN to ring. For the steep cardiac wavefront (~ 0.3 mm wide, crossing a grid cell in ~ 0.01 ms), this damping is usually beneficial.

The BDF2 formula replaces the first-order backward difference with a second-order version using two history levels:

$$\frac{3\mathbf{V}^{n+1} - 4\mathbf{V}^n + \mathbf{V}^{n-1}}{2\Delta t} = \mathcal{D}(\mathbf{V}^{n+1}) \quad (11.5)$$

The left side is a second-order backward difference approximation to $d\mathbf{V}/dt$ at time t^{n+1} , using the three points $\{t^{n-1}, t^n, t^{n+1}\}$. Multiplying through by $2\Delta t\mathbf{M}$ and substituting $\mathcal{D} = -\mathbf{M}^{-1}\mathbf{K}$:

$$\underbrace{(3\mathbf{M} + 2\Delta t\mathbf{K})}_{\mathbf{A}_{\text{lhs}}} \mathbf{V}^{n+1} = \underbrace{4\mathbf{M}}_{\mathbf{B}_{\text{rhs}}} \mathbf{V}^n - \mathbf{M} \mathbf{V}^{n-1} \quad (11.6)$$

The right-hand side now involves *two* previous voltage vectors: $4\mathbf{M}\mathbf{V}^n - \mathbf{M}\mathbf{V}^{n-1}$. This is the price of being a two-step method: the solver must store the previous timestep's voltage $\mathbf{V}^{n-1}$ in addition to the current $\mathbf{V}^n$. Since $\mathbf{V}^{n-1}$ is not available at $t = 0$, the first step must use BDF1 (Backward Euler) as a bootstrap — a one-step method that requires only $\mathbf{V}^0$.

Intuition

Worked example. Take the same 5-node 1D cable as the Crank–Nicolson example above (FDM, lumped mass $\mathbf{M} = \mathbf{I}$, $\Delta t = 0.02$ ms) with the stiffness matrix $\mathbf{K} = 2.5 \mathbf{T}$ from Chapter 8 (equation 8.2), where $\mathbf{T} = \text{tridiag}(-1, 2, -1)$ carries a $\mathbf{1}$ on the first and last diagonal for the Neumann ends. BDF2's left-hand side (equation 11.6) is $\mathbf{A}_{\text{lhs}} = 3\mathbf{M} + 2\Delta t \mathbf{K} = 3\mathbf{I} + 0.04 \mathbf{K} = 3\mathbf{I} + 0.1 \mathbf{T}$:

$$\mathbf{A}_{\text{lhs}} = \begin{pmatrix} 3.1 & -0.1 & 0 & 0 & 0 \\ -0.1 & 3.2 & -0.1 & 0 & 0 \\ 0 & -0.1 & 3.2 & -0.1 & 0 \\ 0 & 0 & -0.1 & 3.2 & -0.1 \\ 0 & 0 & 0 & -0.1 & 3.1 \end{pmatrix}$$

The $\mathbf{3}$ on each diagonal and the $2\Delta t$ weight on $\mathbf{K}$ come straight from multiplying the BDF2 stencil (equation 11.5) through by $2\Delta t \mathbf{M}$ — there is no averaging of the old and new operators as in Crank–Nicolson. The right-hand side needs two history vectors, $\mathbf{b} = 4\mathbf{M} \mathbf{V}^n - \mathbf{M} \mathbf{V}^{n-1} = 4\mathbf{V}^n - \mathbf{V}^{n-1}$. Suppose a wavefront has just reached node 3: the previous step was fully at rest, $\mathbf{V}^{n-1} = (-85, -85, -85, -85, -85)$ mV, and the current step has node 3 depolarized, $\mathbf{V}^n = (-85, -85, -85, +20, -85)$ mV. Then the assembled RHS is

$$\mathbf{b} = 4 \begin{pmatrix} -85 \\ -85 \\ -85 \\ 20 \\ -85 \end{pmatrix} - \begin{pmatrix} -85 \\ -85 \\ -85 \\ -85 \\ -85 \end{pmatrix} = \begin{pmatrix} -340 \\ -340 \\ -340 \\ 80 \\ -340 \end{pmatrix} - \begin{pmatrix} -85 \\ -85 \\ -85 \\ -85 \\ -85 \end{pmatrix} = \begin{pmatrix} -255 \\ -255 \\ -255 \\ 165 \\ -255 \end{pmatrix}.$$

Solving $\mathbf{A}_{\text{lhs}} \mathbf{V}^{n+1} = \mathbf{b}$ (one CG solve) gives the updated voltages. Quick sanity check: every row of $\mathbf{A}_{\text{lhs}}$ sums to $\mathbf{3.0}$ (the rows of $\mathbf{T}$ sum to zero), so a cable resting everywhere would give $\mathbf{b} = 4(-85) - (-85) = -255$ at every node and reproduce $\mathbf{V}^{n+1} = -85$ mV — rest stays at rest. Only node 3 carries an excess ($\mathbf{165}$ against the resting $\mathbf{-255}$), and BDF2 spreads that bump into its neighbors while damping it hard in a single step.

Bootstrap. At $t = 0$ there is no $\mathbf{V}^{n-1}$ yet, so the very first step falls back to BDF1 (equation 11.2): $(\mathbf{M} + \Delta t \mathbf{K}) \mathbf{V}^1 = (\mathbf{I} + 0.05 \mathbf{T}) \mathbf{V}^1 = \mathbf{V}^0$. Once $\mathbf{V}^0$ and $\mathbf{V}^1$ are both in hand, the two-step recurrence above runs normally from step 2 onward.

Mathematical Note

BDF2 is both A-stable and L-stable: its amplification factor decays to zero as $z \rightarrow -\infty$ (both roots of its characteristic equation $(\frac{3}{2} - z)r^2 - 2r + \frac{1}{2} = 0$ tend to 0), so the stiffest high-frequency modes are annihilated in a single step. This is even stronger damping than Crank–Nicolson, whose amplification factor approaches $|\mathbf{R}(\infty)| = 1$ and therefore leaves a persistent "ringing" at sharp wavefronts that BDF2 suppresses.

Figure 11.1 makes this L-stability difference visible. Feed both methods the same sharp transient — a single stiff, high-frequency mode — and watch its amplitude once per timestep. BDF2 collapses the mode monotonically to zero within a few steps; Crank–Nicolson leaves a stubborn sign-flipping oscillation that fades only slowly.

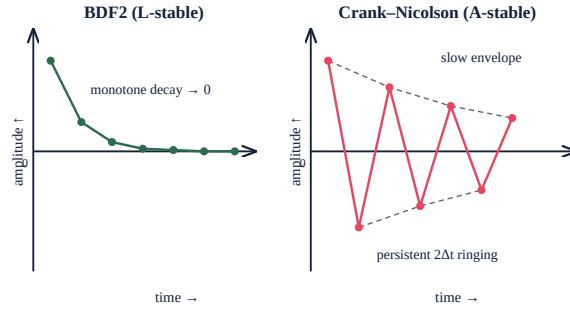


Figure 11.1. L-stable damping versus Crank–Nicolson ringing. Both panels track the amplitude of a single stiff, high-frequency mode after a sharp transient, sampled once per timestep. **Left (BDF2):** the amplification factor tends to **0** for stiff modes, so the mode decays monotonically to zero within a few steps. **Right (Crank–Nicolson):** the amplification factor approaches **−1**, producing a sign-flipping **2Δt** oscillation whose envelope (dashed) decays only slowly — the CN “ringing” artifact.

11.5 Comparing the Three Methods

Method	Order	$\mathbf{A}_{\text{lhs}}$	$\mathbf{B}_{\text{rhs}} \cdot \mathbf{V}^n$	Stability	Best when...
BDF1	1	$\mathbf{M} + \Delta t \mathbf{K}$	$\mathbf{M} \mathbf{V}^n$	L-stable	First step; after discontinuities
CN	2	$\mathbf{M} + \frac{\Delta t}{2} \mathbf{K}$	$(\mathbf{M} - \frac{\Delta t}{2} \mathbf{K}) \mathbf{V}^n$	A-stable	Default choice; smooth solutions
BDF2	2	$3\mathbf{M} + 2\Delta t \mathbf{K}$	$4\mathbf{M} \mathbf{V}^n - \mathbf{M} \mathbf{V}^{n-1}$	L-stable, strongly damping	Sharp wavefronts; large Δt

All three methods share the same computational skeleton: precompute $\mathbf{A}_{\text{lhs}}$ and $\mathbf{B}_{\text{rhs}}$ once, then at each timestep perform one sparse matrix-vector product (the RHS) followed by one linear solve (the LHS). The linear solve dominates the cost — typically 60–80% of wall-clock time in an implicit monodomain simulation.

11.6 Linear Solvers: PCG, Chebyshev, and FFT

The linear system $\mathbf{A}_{\text{lhs}} \mathbf{V}^{n+1} = \mathbf{b}$ must be solved efficiently at every timestep. Since $\mathbf{A}_{\text{lhs}}$ is SPD (for Neumann boundary conditions), the conjugate gradient method is the natural choice. Engine V5.4 offers three solver strategies, each suited to different hardware and problem configurations:

PCG (Preconditioned Conjugate Gradient) with Jacobi (diagonal) preconditioning is the default, and for good reason: it is the workhorse for sparse SPD systems. Plain conjugate gradient (see Appendix C.12) minimizes the error in the energy norm by walking along a sequence of $\mathbf{A}$ -conjugate search directions, converging in $O(\sqrt{\kappa})$ iterations, where κ is the condition number. For the monodomain problem $\kappa \propto h^{-2}$, so halving the mesh doubles the iteration count — which is exactly where preconditioning earns its keep. A preconditioner $\mathbf{P} \approx \mathbf{A}_{\text{lhs}}$ that is cheap to invert clusters the eigenvalues and shrinks the effective κ : Jacobi ($\mathbf{P} = \text{diag}(\mathbf{A}_{\text{lhs}})$) costs almost nothing and already helps, while an incomplete Cholesky factor helps more at a higher setup cost (see Appendix C.13). PCG is robust, needs no tuning, and works on *any* geometry or conductivity field — its weakness is that each iteration requires two global inner products (dot-product reductions), which become a synchronization bottleneck on the GPU. Use it as the safe default, especially on irregular meshes where the spectral solver does not apply; 10–30 iterations typically reach the tolerance cardiac simulation needs.

Chebyshev iteration trades those inner products away. It is a fixed-polynomial method: instead of computing optimal step lengths from inner products the way CG does, it applies a precomputed sequence of coefficients derived from the Chebyshev polynomials, which are provably optimal at damping error across a

known eigenvalue interval $[\lambda_{\min}, \lambda_{\max}]$. Because those coefficients are fixed ahead of time, every iteration is just one matvec plus a couple of AXPY updates with *zero global reductions* — no thread ever waits on a whole-grid sum. That is precisely what a GPU wants: each thread marches independently for a fixed iteration count, so Chebyshev often beats PCG in wall-clock time on the GPU despite doing more (mathematically less efficient) work per iteration. The price is that it needs a decent estimate of the eigenvalue bounds — too tight and it can diverge, too loose and it wastes iterations — but these are estimated once at setup (e.g. a few power-iteration steps for $\lambda_{\max}$). Reach for Chebyshev when the geometry is irregular (so the FFT is out) but you are on the GPU and reduction latency dominates (see Appendix C.14).

FFT/DCT solver is the fastest of the three — when it applies. On a regular rectangle with uniform spacing and constant conductivity, the discrete Laplacian is diagonalized by a Fourier basis: the discrete cosine transform (DCT) for Neumann boundaries, the discrete sine transform for Dirichlet. In that basis $\mathbf{A}_{\text{lhs}}$ becomes *diagonal*, so the entire implicit solve reduces to a direct, non-iterative three-step recipe — transform the RHS, divide by the known eigenvalues, inverse-transform — with no convergence loop and no tolerance to set. The cost is $O(N \log N)$ from the FFT and the result is exact to machine precision; on the GPU the transforms are heavily optimized, making this several times faster than the iterative solvers. The catch is the strict applicability: break the rectangle (real cardiac geometry), vary the conductivity in space (fiber structure, scar), or impose mixed boundaries, and the clean diagonalization is lost. Use it as the first choice for idealized benchmark slabs and cables, and fall back to PCG or Chebyshev the moment the domain or conductivity stops being trivial (see Appendix C.10).

△ Key Insight

The choice of linear solver often dominates the total simulation cost. In a typical implicit monodomain simulation, 60–80% of wall-clock time is spent in the linear solver. This is why the Chebyshev (GPU-friendly) and FFT (direct) solvers exist in V5.4 alongside PCG — each targets a different performance bottleneck.

⚙ Engine V5.4 Connection

In Engine V5.4, the implicit solvers (`CrankNicolsonSolver`, `BDF1Solver`, `BDF2Solver`) each inherit from `DiffusionSolver` and additionally own a `LinearSolver`. At construction, each calls `spatial.get_diffusion_operators(dt, scheme)` with `scheme = "CN"`, `"BDF1"`, or `"BDF2"` — this returns a `DiffusionOperators` object containing the precomputed sparse matrices $\mathbf{A}_{\text{lhs}}$ and $\mathbf{B}_{\text{rhs}}$. The `step()` method is then identical for all three: compute `rhs = B_rhs @ V`, solve `A_lhs @ V_new = rhs`. BDF2 additionally stores `V_prev` and subtracts `M @ V_prev` from the RHS, with an automatic fallback to BDF1 for the first timestep.

PART III

Bidomain Modeling

Two conducting domains, two coupled equations, one algorithm. From the physics of extracellular potentials to the five decoupled strategies that Engine V1 uses to solve the bidomain system.

The Bidomain Model

Before we write equations, we build a physical picture of why two "domains" are needed. Then we derive the bidomain equations step by step, from conservation laws to the coupled parabolic–elliptic system that Engine V1 will solve.

△ Key Insight

Two notations for the same physics. The bidomain literature uses two equivalent formulations:

- **Conductivity form (σ -form):** Uses conductivities σ_i, σ_e with explicit χC_m in the time derivative. Standard in journal papers and in the derivations below.
- **Diffusivity form (D -form):** Uses $D_i = \sigma_i/(\chi C_m)$, $D_e = \sigma_e/(\chi C_m)$, absorbing χC_m so the time derivative becomes simply $\partial V_m/\partial t$. Used in the Engine V1 code.

This chapter derives equations in σ -form (matching the literature). All discrete equations in Chapters 13–15 and the Engine connection boxes use D -form. To convert: replace $\chi C_m \partial V_m/\partial t$ with $\partial V_m/\partial t$, and replace σ with $D \cdot \chi C_m$.

12.1 Currents in a Living Heart

Cardiac tissue is not a simple conducting wire. It is a dense, three-dimensional mesh of individual cells (myocytes), each wrapped in a lipid bilayer membrane that acts as both a capacitor and a nonlinear switch. Between the cells lies a bath of salty fluid — the *extracellular space*. Inside each cell, the cytoplasm forms another conducting medium — the *intracellular space*. The two spaces are connected electrically by thousands of gap junctions that stitch neighboring cells together.

When a wave of electrical activation sweeps through the heart, current flows simultaneously in **both** spaces:

- **Intracellular current J_i** travels from cell to cell through gap junctions, following the fiber direction preferentially (anisotropy ratio $\approx 10:1$).
- **Extracellular current J_e** flows through the interstitial fluid, also anisotropic but with a milder ratio ($\approx 2.5:1$).
- **Transmembrane current I_m** crosses the cell membrane, carried by ion channels, pumps, and membrane capacitance.

The crucial observation is that *these anisotropy ratios are not equal*. Current takes different preferred paths in the two spaces. This mismatch is what the monodomain model cannot capture — and what the bidomain model was invented to describe.

12.2 Divergence and Conservation: The Language of Current Flow

To build the bidomain equations, we need one fundamental physical law: **conservation of charge**. In a region of tissue, charge cannot appear or disappear — it can only flow in or out. The mathematical way to say "what flows out must equal what is produced inside" is the *divergence*.

Intuition

What is divergence, intuitively? Imagine placing a tiny balloon at a point in a fluid. If the fluid is flowing outward from that point, the balloon inflates — the divergence is positive. If the fluid converges inward, the balloon deflates — the divergence is negative. For electric current, $\nabla \cdot \mathbf{J}$ measures how much current is "created" at each point (positive) or "absorbed" (negative).

Now apply this to our two domains. In the intracellular space, current $\mathbf{J}_i$ flows from cell to cell. At any point, some of that current may leak out of the intracellular space across the membrane into the extracellular space. That leaking current is the transmembrane current I_m . Conservation says:

$$\nabla \cdot \mathbf{J}_i = -\chi I_m \quad (12.1)$$

The negative sign tells us: if transmembrane current flows *outward* (from inside the cell to outside), it is a *sink* for intracellular current — current is leaving the intracellular space.

Similarly, in the extracellular space, that same transmembrane current is now a *source* — current arriving from inside:

$$\nabla \cdot \mathbf{J}_e = +\chi I_m \quad (12.2)$$

Here χ is the surface-to-volume ratio of the cells (how much membrane area per unit volume of tissue — typically 1000–1500 cm⁻¹). It converts a current per unit membrane area (I_m in $\mu\text{A}/\text{cm}^2$) into a current per unit volume.

Key Insight

The conservation identity. If you add equations (12.1) and (12.2), the transmembrane currents cancel exactly:

$$\nabla \cdot \mathbf{J}_i + \nabla \cdot \mathbf{J}_e = 0$$

This is simply the statement that *total current is conserved*: every charge that leaves the intracellular space enters the extracellular space, and vice versa. No charge is created or destroyed inside the tissue. This identity is the backbone of the entire bidomain framework.

12.3 From One Domain to Two

In the **monodomain** model, we make a simplifying assumption: the ratio of intracellular to extracellular conductivity is the same in every direction. Under this assumption, the extracellular potential ϕ_e becomes spatially uniform (or can be set to zero), and we need to track only one unknown — the transmembrane potential V_m . One PDE, one unknown. Simple and fast.

But real cardiac tissue violates this assumption. The intracellular space is strongly anisotropic (current flows about 10 times faster along fibers than across them), while the extracellular space is only mildly anisotropic ($\approx 2.5:1$). This **unequal anisotropy ratio** means the extracellular potential is *not* uniform — it develops complex spatial patterns of its own.

This matters in practice whenever we need to model:

- **Defibrillation** — an external electric field interacts with the tissue through the extracellular space.

- **Virtual electrode polarization** — field-induced regions of depolarization and hyperpolarization that cannot exist in the monodomain picture.
- **ECG generation** — body-surface potentials are driven by extracellular currents.
- **External stimulation** — electrode stimuli enter through ϕ_e .

To capture these phenomena, we must solve for **both** V_m and ϕ_e . We need the *bidomain* model: two coupled equations, two unknowns, one beautiful (and computationally expensive) system.

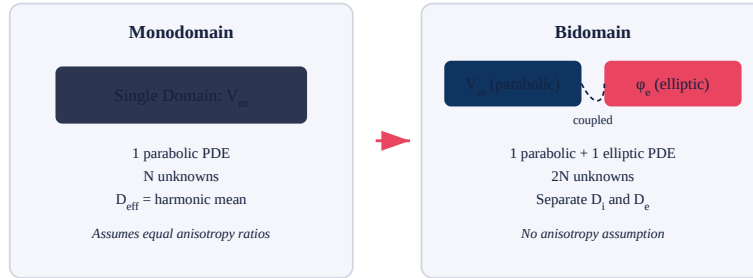


Figure 12.1. Comparison of monodomain and bidomain representations. The bidomain model introduces a second unknown (ϕ_e) and a second equation (elliptic), coupled through the intracellular conductivity tensor.

12.4 The Monodomain Equation (Review)

The monodomain equation (Part II) describes how the transmembrane potential V_m evolves in time:

$$\chi C_m \frac{\partial V_m}{\partial t} = -\chi I_{\text{ion}}(V_m, \mathbf{w}) + \nabla \cdot (\mathbf{D} \nabla V_m) + I_{\text{stim}} \quad (12.3)$$

Reading each term from left to right:

- **Left-hand side:** rate of change of membrane charge ($\chi C_m \partial V_m / \partial t$). The membrane is a capacitor.
- $-\chi I_{\text{ion}}$: ionic currents through membrane channels — these are nonlinear functions of V_m and gating variables $\mathbf{w}$.
- $\nabla \cdot (\mathbf{D} \nabla V_m)$: diffusion of voltage through the tissue. The tensor $\mathbf{D}$ encodes the *effective* conductivity — a blend of intracellular and extracellular contributions.
- I_{stim} : externally applied stimulus current.

This is a single **parabolic** PDE (it has a time derivative and a second-order spatial derivative). It is well-suited to standard diffusion solvers, LBM, or finite-element methods.

12.5 Building the Bidomain System

Let us now be precise. In each domain, Ohm's law gives us the current density:

$$\mathbf{J}_i = -\mathbf{D}_i \nabla \phi_i, \quad \mathbf{J}_e = -\mathbf{D}_e \nabla \phi_e$$

where ϕ_i and ϕ_e are the intracellular and extracellular potentials, and $\mathbf{D}_i$, $\mathbf{D}_e$ are their respective conductivity tensors. The transmembrane potential is the difference:

$$V_m = \phi_i - \phi_e$$

The transmembrane current per unit volume has two components — capacitive charging and ionic flow:

$$\chi I_m = \chi C_m \frac{\partial V_m}{\partial t} + \chi I_{\text{ion}}(V_m, \mathbf{w})$$

The Parabolic Equation (Membrane Dynamics)

Start with the intracellular conservation law (12.1): $\nabla \cdot \mathbf{J}_i = -\chi I_m$. Substitute Ohm's law $\mathbf{J}_i = -\mathbf{D}_i \nabla \phi_i$:

$$\nabla \cdot (\mathbf{D}_i \nabla \phi_i) = \chi C_m \frac{\partial V_m}{\partial t} + \chi I_{\text{ion}}$$

Now comes the key substitution. Since $\phi_i = V_m + \phi_e$, we have $\nabla \phi_i = \nabla V_m + \nabla \phi_e$. Substituting and using the linearity of divergence:

$$\nabla \cdot (\mathbf{D}_i (\nabla V_m + \nabla \phi_e)) = \nabla \cdot (\mathbf{D}_i \nabla V_m) + \nabla \cdot (\mathbf{D}_i \nabla \phi_e)$$

This is where the monodomain's single diffusion term splits into two distinct physical processes. Rearranging to isolate $\partial V_m / \partial t$:

$$\chi C_m \frac{\partial V_m}{\partial t} = -\chi I_{\text{ion}}(V_m, \mathbf{w}) + \underbrace{\nabla \cdot (\mathbf{D}_i \nabla V_m)}_{\text{propagation}} + \underbrace{\nabla \cdot (\mathbf{D}_i \nabla \phi_e)}_{\text{coupling}} + I_{\text{stim}}^i \quad (12.4)$$

Let us read each term as a physical current — what is it actually doing?

- $\nabla \cdot (\mathbf{D}_i \nabla V_m)$ — **the propagation term**. This is intracellular current driven by transmembrane potential gradients. Where V_m is high (depolarized tissue) and neighbors are low (resting tissue), intracellular current flows from depolarized to resting regions, spreading the action potential. This is exactly the same term as in the monodomain equation (7.1) — it is why the electrical wave propagates through the heart.
- $\nabla \cdot (\mathbf{D}_i \nabla \phi_e)$ — **the coupling term**. This is the *new* term that did not appear in the monodomain. It represents intracellular current driven by extracellular potential gradients. Why does extracellular potential affect intracellular current? Because $\phi_i = V_m + \phi_e$: if the extracellular potential varies in space (say, because of a defibrillation electrode), that shifts the intracellular potential everywhere, which changes the intracellular current flow. In the monodomain model, ϕ_e is assumed uniform, so $\nabla \phi_e = \mathbf{0}$ and this term vanishes. **This coupling term is the entire reason the bidomain model exists.**

Intuition

Comparing monodomain to bidomain. The monodomain equation (12.3) has a single diffusion term $\nabla \cdot (\mathbf{D} \nabla V_m)$ with an *effective* conductivity $\mathbf{D}$ that blends both domains. The bidomain parabolic equation (12.4) replaces this with two separate terms: propagation via $\mathbf{D}_i \nabla V_m$ and coupling via $\mathbf{D}_i \nabla \phi_e$. Both use the intracellular conductivity $\mathbf{D}_i$ — because this equation describes what happens inside the cells.

The Elliptic Equation (Current Conservation)

For the second equation, recall the conservation identity from Section 12.2: $\nabla \cdot \mathbf{J}_i + \nabla \cdot \mathbf{J}_e = 0$ — total current is conserved. Apply Ohm's law to both terms:

$$\nabla \cdot (\mathbf{D}_i \nabla \phi_i) + \nabla \cdot (\mathbf{D}_e \nabla \phi_e) = 0$$

Now substitute $\phi_i = V_m + \phi_e$ into the first term, using the same splitting as before:

$$\underbrace{\nabla \cdot (\mathbf{D}_i \nabla V_m)}_{\text{from } \phi_i = V_m + \phi_e} + \underbrace{\nabla \cdot (\mathbf{D}_i \nabla \phi_e)}_{\text{from } \phi_i = V_m + \phi_e} + \nabla \cdot (\mathbf{D}_e \nabla \phi_e) = 0$$

Collect the two terms involving $\nabla \phi_e$:

$$0 = \underbrace{\nabla \cdot (\mathbf{D}_i \nabla V_m)}_{\text{source}} + \underbrace{\nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e]}_{\text{response}} + I_{stim}^e \quad (12.5)$$

This is the **elliptic equation**. It has no time derivative — ϕ_e does not evolve; it adjusts instantaneously. Read the two terms as a source-response balance:

- $\nabla \cdot (\mathbf{D}_i \nabla V_m)$ — **the source**. The transmembrane potential V_m varies in space (especially at the wavefront, where it changes from -85 mV to $+30$ mV over a few tenths of a millimeter). This gradient drives intracellular current, which crosses the membrane and enters the extracellular space. The divergence of this current tells you how much charge is being "dumped" into the extracellular space at each point. At the wavefront, this dump is massive.
- $\nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e]$ — **the response**. The extracellular potential ϕ_e adjusts so that the combined current in both domains carries away all the charge that was dumped. The *combined* conductivity $(\mathbf{D}_i + \mathbf{D}_e)$ appears because current in both domains responds to extracellular potential gradients. This term acts like diffusion — it smooths ϕ_e across the tissue. The stronger the combined conductivity, the faster the smoothing, and the gentler the extracellular potential gradients.

Intuition

The steady-state heat analogy. If you have studied heat transfer, you know this structure: a steady-state temperature field adjusts so that heat production (source) equals heat diffusion (response) everywhere. The elliptic equation is exactly that — V_m is the "heat source" and ϕ_e is the "temperature." There is no thermal mass, no transient — just an instant equilibrium. The extracellular potential ϕ_e is a *slave variable*: it is completely determined by the current V_m at each instant, with no memory of the past.

Key Insight

Why is there no time derivative? The extracellular space has no capacitor — no mechanism to store charge over time. The intracellular space stores charge on the membrane capacitor (that is the $C_m \partial V_m / \partial t$ term). But the extracellular space simply conducts: charge enters at one point and leaves at another, with no accumulation. Mathematically, this means the extracellular conservation law produces an equation with spatial derivatives only — no $\partial \phi_e / \partial t$. This is what makes it elliptic rather than parabolic.

What the Numbers Look Like

To make these terms concrete, let us estimate their magnitudes at a cardiac wavefront using typical human ventricular parameters:

Parameter	Value	Source
$D_{i,L}$ (intracellular, along fibers)	0.0017 cm ² /ms	Clerc 1976
$D_{i,T}$ (intracellular, across fibers)	0.00019 cm ² /ms	Clerc 1976
$D_{e,L}$ (extracellular, along fibers)	0.0062 cm ² /ms	Clerc 1976
$D_{e,T}$ (extracellular, across fibers)	0.0024 cm ² /ms	Clerc 1976

At the wavefront, the action potential rises from -85 to $+30$ mV over roughly $\Delta x \approx 0.3$ mm along fibers. A crude estimate of the spatial curvature gives $\partial^2 V_m / \partial x^2 \approx -4000$ V/cm² (negative because V_m has a peak at the front). Along fibers:

- **Propagation term:** $D_{i,L} \cdot \partial^2 V_m / \partial x^2 \approx 0.0017 \times (-4000) = -6.8$ mV/ms
- **Coupling term:** $D_{i,L} \cdot \partial^2 \phi_e / \partial x^2$. Extracellular potential gradients are smaller — typically $|\phi_e|$ varies by ~ 1 – 5 mV at the wavefront, giving $\partial^2 \phi_e / \partial x^2 \approx 200$ V/cm². So the coupling contributes $\approx 0.0017 \times 200 = 0.34$ mV/ms — roughly 5% of the propagation term.
- **Elliptic balance:** The source term $\nabla \cdot (\mathbf{D}_i \nabla V_m)$ dumps charge at ~ 6.8 mV/ms, and the response $\nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e]$ must absorb all of it: $(0.0017 + 0.0062) \times \partial^2 \phi_e / \partial x^2 \approx 6.8$ mV/ms. This is why ϕ_e develops its characteristic spatial pattern — it *must* generate gradients large enough to carry away all the source current.

Intuition

The coupling is small but essential. The coupling term ($\sim 5\%$) has a modest direct effect on propagation speed. But it is responsible for *all* of the phenomena that distinguish bidomain from monodomain: virtual electrode polarization during defibrillation, the body-surface ECG, and the response to extracellular stimulation. A 5% correction in the PDE creates qualitatively different physics.

12.6 The Parabolic–Elliptic Couple: A New Kind of Problem

All of Part II dealt with a single parabolic PDE — the monodomain equation. "Parabolic" means the equation has a time derivative, and its solutions spread and smooth over time, like heat diffusing through a metal rod. We learned to discretize it (Chapter 8), split reaction from diffusion (Chapter 9), and march it forward in time one step at a time (Chapters 10–11).

Now we face something fundamentally different: *two coupled equations of different types*.

Equation	Type	Unknown	Character	Analogy
(12.4) — Membrane dynamics	Parabolic	V_m	Evolves in time; has $\partial V_m / \partial t$	A pot of water heating up
(12.5) — Current conservation	Elliptic	ϕ_e	No time derivative; instantaneous balance	A rubber sheet finding its shape

If you have seen coupled differential equations before — say, $dx/dt = f(x, y)$ and $dy/dt = g(x, y)$ — the bidomain is *harder*. In a standard coupled system, both unknowns evolve in time and you can march them both forward. Here, only V_m evolves. The equation for ϕ_e is not $d\phi_e/dt = \dots$ but $0 = \dots$ — a *constraint* that must be satisfied at every instant. You cannot "march" a constraint equation forward in time. You can only *solve* it.

The two equations are **bidirectionally coupled**: the parabolic equation (12.4) depends on ϕ_e through the coupling term $\nabla \cdot (\mathbf{D}_i \nabla \phi_e)$, and the elliptic equation (12.5) depends on $\mathbf{V}_m$ through the source term $\nabla \cdot (\mathbf{D}_i \nabla \mathbf{V}_m)$. Neither can be solved in isolation.

Intuition

Picture the coupling. Imagine a room with a heater (the action potential wavefront) and an air conditioning system (the extracellular space). The heater warms the air locally — this is the parabolic equation evolving $\mathbf{V}_m$ in time. The air conditioning system instantly adjusts the airflow pattern to maintain the right temperature everywhere else — this is the elliptic equation determining ϕ_e . But the heater's output depends on the ambient air temperature (from the AC), and the AC's required output depends on the heater. They influence each other continuously.

This coupling creates the central computational challenge of the bidomain model: **how do you advance a system where one variable evolves and the other is a constraint?**

There are three fundamental strategies, each with a different trade-off:

Strategy	Idea	Trade-off
Solve together (monolithic)	Combine both equations into one big $2N \times 2N$ system and solve simultaneously	Exact coupling, but the system is indefinite (not SPD) — needs specialized, expensive solvers
Advance then equilibrate (decoupled sequential)	Advance $\mathbf{V}_m$ one step using the old ϕ_e , then solve the elliptic equation for the new ϕ_e using the new $\mathbf{V}_m$	Two smaller $N \times N$ SPD solves (cheap!), but the one-step lag in ϕ_e introduces a small error
Advance explicitly, equilibrate implicitly (semi-implicit)	Advance $\mathbf{V}_m$ with an explicit step (no linear solve), then solve the elliptic for ϕ_e	Only one linear solve per step, but CFL-limited — must use small Δt

The key insight that makes decoupled strategies work is that **the elliptic equation has no memory**. Its solution at time t depends only on $\mathbf{V}_m(t)$, not on ϕ_e at any earlier time. So once we have $\mathbf{V}_m^{n+1}$ (or an approximation of it), we can solve for ϕ_e^{n+1} as a standalone problem — a fresh boundary-value problem with no history to track.

Chapter 14 develops these strategies in detail, starting with the simplest (decoupled sequential) and building toward more sophisticated variants. But first, we must turn these continuous equations into matrix equations — the subject of Chapter 13.

Additionally, the gating variables $\mathbf{w}$ evolve according to a system of local ODEs:

$$\frac{d\mathbf{w}}{dt} = \mathbf{h}(\mathbf{V}_m, \mathbf{w}) \quad (12.6)$$

These depend only on $\mathbf{V}_m$ and $\mathbf{w}$ at each point — no spatial derivatives. The ionic solver from Part II (Rush–Larsen exponential integrator, Section 9.4) can be reused without modification. This is the same operator-splitting approach as in the monodomain: handle the local ionic kinetics first, then tackle the spatial diffusion problem.

Intuition

Sidebar: When does the bidomain collapse to monodomain? If the two domains happen to have the same anisotropy ratio — meaning $\mathbf{D}_e = \lambda \mathbf{D}_i$ for some scalar λ — the elliptic equation degenerates. The combined conductivity becomes $(1 + \lambda) \mathbf{D}_i$, and the elliptic equation forces ϕ_e to be essentially constant. The parabolic equation then reduces to the monodomain form with an effective diffusivity $\mathbf{D}_{\text{eff}} = \mathbf{D}_i \mathbf{D}_e / (\mathbf{D}_i + \mathbf{D}_e)$. In real cardiac tissue, however, the intracellular anisotropy ratio is $\approx 10:1$ while the extracellular ratio is $\approx 2.5:1$ — so the equal-anisotropy assumption fails, and the full bidomain system is needed.

Mathematical Note

Sidebar: Fiber architecture and conductivity tensors. In each domain, the conductivity tensor is diagonal in local fiber coordinates: $\mathbf{D}_d^{\text{local}} = \text{diag}(\sigma_d^L, \sigma_d^T, \sigma_d^T)$ where L = along-fiber and T = cross-fiber. Because cardiac fibers rotate transmurally ($\approx 120^\circ$ from epicardium to endocardium), the local tensor must be rotated into global coordinates: $\mathbf{D}_d^{\text{global}}(\mathbf{x}) = \mathbf{R}(\theta) \mathbf{D}_d^{\text{local}} \mathbf{R}(\theta)^T$. This makes the conductivity a spatially varying tensor field — the same rotation machinery used for monodomain extends directly to bidomain by maintaining two tensor fields ($\mathbf{D}_i$ and $\mathbf{D}_e$) instead of one.

12.7 Boundary Conditions: Two Domains, Two Choices

In the monodomain model (Section 7.4), we needed one boundary condition for one unknown. The bidomain model has *two* unknowns — the intracellular potential ϕ_i and the extracellular potential ϕ_e — so we need *two* boundary conditions at every point on the tissue surface.

Think of a piece of cardiac tissue sitting in a glass dish. The intracellular space (inside the cells) is sealed at the tissue surface — cell membranes continue right up to the edge, and no intracellular current can escape. This is always Neumann. But the extracellular space (between the cells) is open — it may be in contact with a conductive bath (saline solution) or exposed to air. What happens at the boundary depends on *what is outside*.

There are three canonical physical scenarios:

Scenario	ϕ_i BC	ϕ_e BC	Null space?	Physical setting
Isolated tissue	Neumann	Neumann	Yes (constant ϕ_e)	Heart in air / pericardium
Grounded bath	Neumann	Dirichlet	No	Langendorff, tissue wedge
Current injection	Dirichlet (local)	Neumann	Yes (if ϕ_e has full Neumann)	Intracellular electrode stimulus

Intuition

The null space is physics, not a bug. When the entire boundary has Neumann conditions on ϕ_e , the extracellular potential is defined only relative to some reference point — just like altitude is defined relative to sea level. You can add any constant to ϕ_e without changing any current flow. In practice, we remove this ambiguity by *pinning* one node ($\phi_e = 0$ at a single point) or by zeroing the constant mode in a spectral solver. Both approaches are covered in Chapter 15 (Section 15.2).

From Equations to Matrices

Chapter 8 turned the monodomain PDE into a single matrix equation. Here we do the same for the bidomain — but because we have two coupled PDEs, we get a 2×2 block matrix system. The good news: the assembly tools from Chapter 8 carry over directly. We just call them twice.

13.1 From One Stiffness Matrix to Two

Recall from Chapter 8: spatial discretization converts a continuous PDE into a matrix equation. The monodomain model has one conductivity tensor $\mathbf{D}$, which produces one stiffness matrix (or Laplacian operator) $\mathbf{L}$. The bidomain model has two conducting domains, each with its own conductivity tensor. Applying the same discretization machinery to each domain separately produces two operators:

- **Intracellular Laplacian $\mathbf{L}_i$** : discretizes $\nabla \cdot (\mathbf{D}_i \nabla \cdot)$ using exactly the same stencil or assembly procedure as the monodomain $\mathbf{L}$.
- **Extracellular Laplacian $\mathbf{L}_e$** : discretizes $\nabla \cdot (\mathbf{D}_e \nabla \cdot)$, assembled identically but with $\mathbf{D}_e$.

The combined operator $\mathbf{L}_i + \mathbf{L}_e$ corresponds to the bulk conductivity $\mathbf{D}_i + \mathbf{D}_e$.

Intuition

Think of it this way. In the monodomain world, you built one road network (one $\mathbf{L}$) that carries all the traffic. In the bidomain world, there are two road networks overlaid on the same city — one for intracellular current ($\mathbf{L}_i$) and one for extracellular current ($\mathbf{L}_e$). Each network is built the same way, just with different road widths (conductivities). The two networks are coupled at every intersection: current can cross from one to the other through the cell membrane.

Engine V5.4 Connection

Engine V1 connection. The `BidomainSpatialDiscretization` class in `discretization/fdm.py` builds $\mathbf{L}_i$ and $\mathbf{L}_e$ by calling the same stencil-assembly routine twice — once with $\mathbf{D}_i$ and once with $\mathbf{D}_e$. The matrices are stored as sparse tensors and reused across all time steps.

13.2 The 2×2 Block System

Now let us put these two operators together. After discretization and time-stepping (details in Chapter 14), the bidomain system at each time step becomes a linear system with $2N$ unknowns — N values for $\mathbf{V}_m$ and N values for ϕ_e :

$$\underbrace{\begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}}_{\mathbf{A}} \begin{pmatrix} \mathbf{V}_m \\ \phi_e \end{pmatrix} = \begin{pmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{pmatrix} \quad (13.1)$$

The blocks have a simple structure:

Block	Definition	Role
$\mathbf{A}_{11}$	$\frac{1}{\Delta t} \mathbf{I} - \theta \mathbf{L}_i$	Parabolic: mass + intracellular diffusion
$\mathbf{A}_{12}$	$-\mathbf{L}_i$	Coupling: how ϕ_e affects V_m
$\mathbf{A}_{21}$	$-\mathbf{L}_i$	Coupling: how V_m drives ϕ_e
$\mathbf{A}_{22}$	$-(\mathbf{L}_i + \mathbf{L}_e)$	Elliptic: combined bulk conductivity

Here θ is the implicitness parameter ($\theta = 0.5$ for Crank–Nicolson, $\theta = 1$ for backward Euler), and $\mathbf{I}$ is the identity matrix (using lumped mass, standard for FDM/FVM). Notice: $\mathbf{L}_i$ appears in *every* block — it is the glue coupling the two unknowns. The extracellular operator $\mathbf{L}_e$ appears only in $\mathbf{A}_{22}$.

Intuition

Why does $\mathbf{L}_i$ appear everywhere? Go back to the physics. The intracellular domain is the *only* path connecting V_m and ϕ_e . Current crosses the membrane (changing V_m), flows through intracellular space (governed by $\mathbf{D}_i$), and must be balanced by extracellular return current. So $\mathbf{L}_i$ — the intracellular operator — naturally shows up in both the parabolic equation (how V_m evolves) and the elliptic equation (how ϕ_e responds).

$$\begin{bmatrix} \begin{array}{c} \mathbf{A}_{11} \\ 1/\Delta t \mathbf{I} - \theta \mathbf{L}_i \\ \text{(SPD)} \end{array} & \begin{array}{c} \mathbf{A}_{12} \\ -\mathbf{L}_i \\ \text{(coupling)} \end{array} \\ \begin{array}{c} \mathbf{A}_{21} \\ -\mathbf{L}_i \\ \text{(coupling)} \end{array} & \begin{array}{c} \mathbf{A}_{22} \\ -(\mathbf{L}_i + \mathbf{L}_e) \\ \text{(SPD after pinning)} \end{array} \end{bmatrix} \begin{bmatrix} V_m \\ \phi_e \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

Figure 13.1. The 2×2 block structure of the bidomain linear system. The diagonal blocks are SPD; the off-diagonal coupling blocks are the negated intracellular Laplacian $-\mathbf{L}_i$ (matching the sign of $\mathbf{A}_{22} = -(\mathbf{L}_i + \mathbf{L}_e)$, so the elliptic row reads $-(\mathbf{L}_i + \mathbf{L}_e)\phi_e = \mathbf{L}_i V_m$, consistent with eq (14.1)).

13.3 The FDM Face-Based Stencil

In Chapter 8, we built the monodomain Laplacian using the standard 5-point (2D) or 7-point (3D) FDM stencil. For Neumann boundary conditions, we used a *ghost-node mirror* approach (Section 8.7.1). This works perfectly for the monodomain, but the bidomain requires a subtle change.

The issue is with the **elliptic equation**. In the monodomain implicit system, the matrix is $\frac{1}{\Delta t} \mathbf{I} + \theta \mathbf{L}$ — the identity term dominates and guarantees symmetry and positive-definiteness regardless of how $\mathbf{L}$ handles boundaries. But the elliptic operator $-(\mathbf{L}_i + \mathbf{L}_e)$ has *no identity term*. If the ghost-node mirror produces an asymmetric boundary stencil (which it does — boundary rows have halved coefficients), the elliptic matrix is non-symmetric, non-SPD, and iterative solvers like PCG will fail.

Intuition

The fix: face-based stencils. Instead of mirroring ghost nodes, simply count the *faces* around each node. Interior nodes have 4 faces (in 2D); boundary nodes have 3 faces; corner nodes have 2. Each face contributes one neighbor term to the stencil. If a neighbor would be outside the domain, that face does not exist — skip it. The result is a symmetric Laplacian by construction: if node j contributes to node i 's stencil, then node i contributes equally to node j 's stencil.

This is a hard-won practical lesson. The ghost-node approach and the face-based approach produce the *same* results for the monodomain (the identity term papers over the asymmetry), but only the face-based approach works for the bidomain elliptic equation. The Engine V1 FDM discretization uses the face-based stencil for exactly this reason.

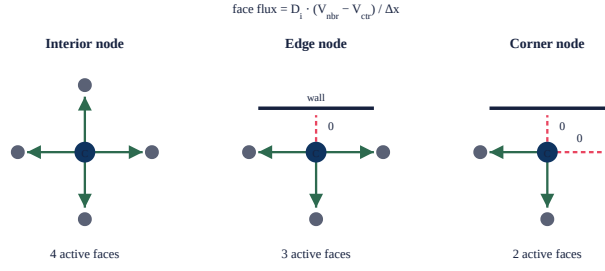


Figure 13.2. The face-based FDM stencil for the intracellular Laplacian $\mathbf{L}_i$. Each node accumulates one flux term per face; across a face separating the center node (C) from a neighbor, the flux is proportional to $D_i(V_{\text{nbr}} - V_{\text{ctr}})/\Delta x$. An interior node has 4 active faces (green arrows), an edge node 3, and a corner node 2. A face lying on the domain wall carries zero flux (no-flux Neumann BC, red dashed) and is simply skipped — never mirrored — which keeps $\mathbf{L}_i$ symmetric by construction (Section 13.3).

13.4 A Note on the Weak Form (FEM)

If you are using finite elements rather than finite differences, the bidomain weak form produces the same block structure as equation (13.1), but with mass matrices $\mathbf{M}$ replacing the identity $\mathbf{I}$ in the (1,1) block, and stiffness matrices $\mathbf{K}_i$, $\mathbf{K}_e$ replacing the Laplacians $\mathbf{L}_i$, $\mathbf{L}_e$. The FEM weak form and its derivation follow the same pattern as the monodomain weak form in Section 8.4 — multiply by test functions, integrate by parts, expand in the basis. The resulting 2×2 block structure is identical; only the matrix entries differ.

For the remainder of Part III, we work with the FDM operators $\mathbf{L}_i$, $\mathbf{L}_e$ and the identity mass matrix $\mathbf{I}$. Everything translates to FEM by replacing $\mathbf{I} \rightarrow \mathbf{M}$ and $\mathbf{L} \rightarrow \mathbf{K}$.

13.5 Worked Example: 5-Node Bidomain Cable

Let us build the complete bidomain system for a concrete case. Consider a 1D cable with 5 nodes, spacing $\Delta x = 0.025$ cm, and Formulation B diffusivities $D_i = 1.214 \times 10^{-3}$ cm²/ms, $D_e = 4.429 \times 10^{-3}$ cm²/ms (corresponding to Clerc 1976 conductivities with $\chi = 1400$ cm⁻¹, $C_m = 1.0$ μF/cm²).

Step 1: Build the Laplacians.

Using the face-based stencil, each $\mathbf{L}_d$ is a 5×5 tridiagonal matrix:

$$\mathbf{L}_d = \frac{D_d}{\Delta x^2} \begin{pmatrix} -1 & 1 & & & \\ 1 & -2 & 1 & & \\ & 1 & -2 & 1 & \\ & & 1 & -2 & 1 \\ & & & 1 & -1 \end{pmatrix}$$

For $\mathbf{L}_i$: the scale factor is $D_i/\Delta x^2 = 1.214 \times 10^{-3}/(6.25 \times 10^{-4}) = 1.943$ ms⁻¹. For $\mathbf{L}_e$: the scale factor is $D_e/\Delta x^2 = 4.429 \times 10^{-3}/(6.25 \times 10^{-4}) = 7.086$ ms⁻¹.

Step 2: Build the block system.

With $\Delta t = 0.02$ ms and $\theta = 0.5$ (Crank–Nicolson), the parabolic block is:

$$\mathbf{A}_{11} = \frac{1}{\Delta t} \mathbf{I} - \theta \mathbf{L}_i = 50 \mathbf{I} - 0.5 \mathbf{L}_i$$

The diagonal entries of $\mathbf{A}_{11}$ are $50 + 0.5 \times 1.943 \times 2 = 51.943$ for interior nodes and $50 + 0.5 \times 1.943 = 50.971$ for boundary nodes. The off-diagonal entries are $-0.5 \times 1.943 = -0.971$.

This is clearly SPD (diagonally dominant with positive diagonal).

The elliptic block is:

$$\mathbf{A}_{22} = -(\mathbf{L}_i + \mathbf{L}_e)$$

Combined scale factor: $(\mathbf{D}_i + \mathbf{D}_e)/\Delta x^2 = 9.029 \text{ ms}^{-1}$. Interior diagonal: $2 \times 9.029 = 18.058$. Off-diagonal: -9.029 . This matrix is symmetric positive *semi*-definite (every row sums to zero). After pinning $\phi_{e,0} = 0$ (replacing row/column 0 with the identity), it becomes SPD.

Step 3: Solve via decoupled Gauss-Seidel.

Rather than solving the full 10×10 system, the decoupled approach solves two separate 5×5 systems sequentially. This is exactly what the code does — and it is the subject of the next chapter.

Solving the Coupled System

We now have two coupled PDEs and a 2×2 block matrix. How do we actually solve this at each time step? This is the heart of the bidomain implementation. The key insight — which makes everything tractable — is that we do not solve the full $2N \times 2N$ system. Instead, we split it into two separate $N \times N$ solves. This chapter presents the complete algorithm and five concrete strategies for executing it.

14.1 Why Fully Explicit Methods Fail

Before building the solution, let us understand the constraint we are working under. A natural first instinct is: "Can we just march everything forward explicitly, like we did with RK4 for the monodomain?" The answer is **no**, for a fundamental structural reason.

The parabolic equation (for $\mathbf{V}_m$) has a $\partial \mathbf{V}_m / \partial t$ term, so in principle it can be marched forward in time. But the elliptic equation — $\mathbf{0} = \nabla \cdot ((\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e) + \nabla \cdot (\mathbf{D}_i \nabla \mathbf{V}_m)$ — is a *constraint*. There is no time derivative to march. At every instant, ϕ_e must satisfy this equation exactly. That means solving a linear system:

$$-(\mathbf{L}_i + \mathbf{L}_e) \phi_e = \mathbf{L}_i \mathbf{V}_m \quad (14.1)$$

This is the single most important distinction between bidomain and monodomain: **at least one implicit linear solve per time step is unavoidable**.

Intuition

Think of it like a puppet on strings. $\mathbf{V}_m$ is the hand — it moves freely (parabolic, has momentum). ϕ_e is the puppet — it has no momentum of its own, and must instantly adjust to wherever the hand moves. You cannot "predict" where the puppet will be next; you must *solve* for its position given the current hand position. This instant adjustment is the elliptic solve.

Even if we accept the mandatory elliptic solve, can the *parabolic* part be treated explicitly? Yes — but at a severe cost. The CFL condition for forward Euler on the bidomain parabolic equation gives:

$$\Delta t \leq \frac{\Delta x^2}{2d D_{\max}} \quad (14.2)$$

For typical cardiac parameters with $\Delta x = 0.1$ mm in 2D, this gives $\Delta t_{\text{explicit}} \lesssim 0.5 \mu\text{s}$. Semi-implicit methods allow $\Delta t \approx 50 \mu\text{s}$ — a factor of **100× larger**.

14.2 The Decoupled Approach: Divide and Conquer

The 2×2 block system (13.1) has $2N$ unknowns. Solving it monolithically would require a specialized solver for symmetric indefinite systems — MINRES, GMRES, or similar (see the sidebar in Section 14.5). But there is a much simpler approach: **solve the two equations one at a time**.

The idea is exactly like Gauss-Seidel iteration for a system of equations. First, update $\mathbf{V}_m$ using the current (or previous) ϕ_e . Then, update ϕ_e using the new $\mathbf{V}_m$. Each sub-problem is $N \times N$ and SPD — solvable by

the familiar tools from Part II (PCG, Chebyshev, spectral methods).

Intuition

Imagine untangling two knotted ropes. Trying to straighten both simultaneously (monolithic solve) is hard — you have to coordinate every move. But if you straighten one rope first, then use the straightened rope as a guide to straighten the other, the problem becomes two easy steps instead of one hard one. That is the decoupled approach.

The complete bidomain time step, including the ionic update from operator splitting (Chapter 9), is shown in Algorithm 14.1.

Engine V5.4 Connection

Algorithm 14.1 — Decoupled Gauss-Seidel Bidomain Step

Given: $\mathbf{V}_m^n, \phi_e^n$, ionic states at time t^n

Stage 1 — Ionic update (same as monodomain, Section 9.4):

$$\mathbf{V}_m^* \leftarrow \text{Rush-Larsen}(\mathbf{V}_m^n, \text{ionic_states}, \Delta t)$$

Stage 2 — Parabolic solve (update $\mathbf{V}_m$, using lagged ϕ_e^n):

$$\underbrace{\left(\frac{1}{\Delta t} \mathbf{I} - \theta \mathbf{L}_i \right)}_{\mathbf{A}_{\text{para}}} \mathbf{V}_m^{n+1} = \underbrace{\left(\frac{1}{\Delta t} \mathbf{I} - (1 - \theta) \mathbf{L}_i \right)}_{\mathbf{B}_{\text{para}}} \mathbf{V}_m^* + \mathbf{L}_i \phi_e^n$$

$[N \times N, \text{SPD}]$ — solve with PCG, Chebyshev, or spectral method

Stage 3 — Elliptic solve (update ϕ_e , using new $\mathbf{V}_m^{n+1}$):

$$\underbrace{-(\mathbf{L}_i + \mathbf{L}_e)}_{\mathbf{A}_{\text{ellip}}} \phi_e^{n+1} = \mathbf{L}_i \mathbf{V}_m^{n+1}$$

$[N \times N, \text{SPD after pinning}]$ — solve with spectral or PCG

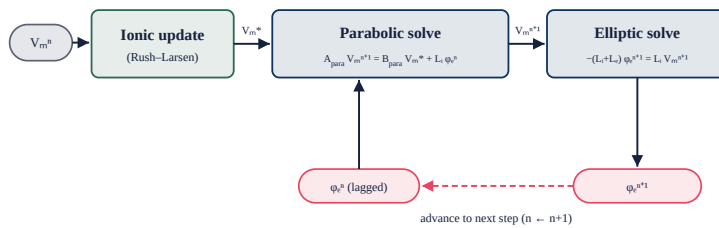


Figure 14.1. Data-flow of one decoupled Gauss–Seidel bidomain time step (Algorithm 14.1). The lagged extracellular potential ϕ_e^n feeds the parabolic right-hand side; the parabolic solve produces $\mathbf{V}_m^{n+1}$, which becomes the right-hand side of the elliptic solve $-(\mathbf{L}_i + \mathbf{L}_e)\phi_e^{n+1} = \mathbf{L}_i \mathbf{V}_m^{n+1}$. The new ϕ_e^{n+1} loops back as the lagged input for the next step — this sequential reuse of the freshest value is exactly the Gauss–Seidel dependency.

Three key observations about this algorithm:

1. **Stage 1 is identical to monodomain.** The ionic model does not know or care about ϕ_e . It sees only V_m and the gating/concentration states.
2. **Both linear systems are $N \times N$ and SPD.** We never form or solve the full $2N \times 2N$ system. Every solver tool from Part II (PCG, Chebyshev, DCT) applies directly.
3. **The coupling is sequential:** ϕ_e^n feeds into the parabolic RHS, and the resulting V_m^{n+1} feeds into the elliptic RHS. This is why it is called "Gauss-Seidel" — each unknown is updated using the most recent value of the other.

The parameter θ controls the time-stepping scheme. $\theta = 0.5$ gives Crank–Nicolson (second-order, A-stable). $\theta = 1.0$ gives backward Euler (first-order, L-stable — damps spurious oscillations near steep wavefronts). Both are unconditionally stable, meaning there is no CFL restriction on Δt .

14.3 Four More Strategies

Algorithm 14.1 is the baseline — simple, robust, and the most common choice in the literature. But there are good reasons to consider alternatives: GPU parallelism, higher temporal accuracy, or avoiding the parabolic linear solve entirely. The Engine V1 codebase implements five strategies, all sharing the same structure — ionic step, then parabolic step, then elliptic step — but differing in *how* each step is executed.

Strategy 2: Semi-Implicit (Forward Euler + Implicit Elliptic)

What if we skip the parabolic linear solve entirely? Replace Stage 2 with a single explicit forward Euler step:

$$V_m^{n+1} = V_m^* + \Delta t L_i (V_m^* + \phi_e^n)$$

This is just one sparse matrix-vector product — no linear solve. Stage 3 (elliptic) remains implicit, because it must (Section 14.1). The cost drops from two linear solves to one SpMV + one linear solve.

The catch: this is **CFL-constrained**. The stability condition is $\Delta t < \Delta x^2 / (4 D_i)$. For $\Delta x = 0.025$ cm and $D_i = 1.214 \times 10^{-3}$: $\Delta t_{\max} \approx 0.126$ ms. This is workable — and on GPUs, the cheap per-step cost can more than compensate for the smaller time step.

Strategy 3: Jacobi (Fully Parallel)

The Gauss-Seidel strategy has a sequential dependency: Stage 3 needs V_m^{n+1} from Stage 2. The **Jacobi** variant breaks this dependency by using *old* values for both:

- Stage 2 (parabolic): same as GS — uses ϕ_e^n
- Stage 3 (elliptic): uses V_m^n instead of V_m^{n+1}

Now both solves depend only on time- n data. They can run **simultaneously on two GPU streams**. The price: slightly weaker coupling (the elliptic solve lags the parabolic by one step). Fernandez & Zemzemi (2010) proved this variant is still energy-stable.

Strategy 4: IMEX-SBDF2 (Second-Order, L-Stable)

Both GS and Jacobi with $\theta = 0.5$ (Crank–Nicolson) are second-order in time, but CN can produce spurious oscillations near steep wavefronts. The **SBDF2** (semi-implicit BDF2) method achieves second-order accuracy with **L-stability** — it *damps* high-frequency errors rather than preserving them.

SBDF2 uses values from two previous time steps. The parabolic stage becomes:

$$\left(\frac{3}{2\Delta t}\mathbf{I} - \mathbf{L}_i\right)\mathbf{V}_m^{n+1} = \frac{2}{\Delta t}\mathbf{V}_m^n - \frac{1}{2\Delta t}\mathbf{V}_m^{n-1} + \mathbf{L}_i\phi_e^n \quad (14.3)$$

Because SBDF2 requires two previous solutions, it cannot self-start. The first step uses backward Euler ($\theta = 1.0$), then all subsequent steps use the BDF2 formula. The cost is identical to GS (two linear solves per step), but the temporal accuracy is higher and the L-stability prevents oscillation artifacts.

Strategy 5: Explicit RKC (Chebyshev-Stabilized)

Runge–Kutta–Chebyshev (RKC) is the most creative approach. Like the semi-implicit strategy, it avoids the parabolic linear solve. But instead of forward Euler (1 stage, tiny stability region), it uses s stages of a Chebyshev polynomial recurrence, extending the stability region **quadratically**:

$$\text{Stability limit} \approx 0.65 s^2 \times \Delta t_{\text{FE}}$$

With $s = 20$ stages, the stability limit is $\approx 260\times$ that of forward Euler. Each stage requires only a sparse matrix-vector product (cheap, parallelizable). The total cost is s SpMV + 1 elliptic solve.

The Chebyshev stages use a **3-term recurrence**: $W_0 = V_m^*$, $W_1 = W_0 + \omega_1 \Delta t \mathbf{L}_i(W_0 + \phi_e^n)$, and for $j = 2, \dots, s$: $W_j = 2\omega_0 W_{j-1} - W_{j-2} + 2\omega_1 \Delta t \mathbf{L}_i(W_{j-1} + \phi_e^n)$, where ω_0, ω_1 are derived from Chebyshev polynomial properties. The final $V_m^{n+1} = W_s$ (after normalization). The elliptic solve is identical to all other strategies.

Intuition

The spectrum of explicitness. These five strategies span a continuum. At one end, GS with $\theta = 1.0$ treats everything implicitly (two linear solves, no CFL). At the other end, semi-implicit uses forward Euler for the parabolic part (one SpMV, CFL-constrained). RKC sits in between — explicit stages with an extended stability region. SBDF2 is fully implicit but optimized for accuracy. Jacobi is identical to GS in cost, but parallel.

14.4 Strategy Comparison

Strategy	Parabolic step	Elliptic step	Cost per step	Temporal order	Stability	Best for
Gauss-Seidel	CN or BDF1 solve	Uses V_m^{n+1}	2 linear solves	1st or 2nd	Unconditional	Default, robust
Semi-implicit	Forward Euler SpMV	Uses V_m^{n+1}	1 SpMV + 1 solve	1st	CFL: $\Delta t < \frac{\Delta x^2}{4D_i}$	GPU, cheap per step
Jacobi	CN or BDF1 solve	Uses V_m^n	2 linear solves	1st or 2nd	Energy-stable	GPU parallel streams
IMEX-SBDF2	BDF2 solve	Uses V_m^{n+1}	2 linear solves	2nd (L-stable)	Unconditional	Accuracy-critical
Explicit RKC	s -stage Chebyshev	Uses V_m^{n+1}	s SpMV + 1 solve	1st	$\Delta t < 0.65 s^2 \Delta t_{\text{FE}}$	No parabolic solve

⚙ Engine V5.4 Connection

Engine V1 connection. All five strategies are implemented in the `solver/diffusion_stepping/` directory. The factory function in `bidomain.py` accepts a string to select the strategy: `'decoupled'` (or `'gauss_seidel'`), `'semi_implicit'`, `'jacobi'`, `'imex_sbd2'`, or `'explicit_rkc'`. The default is `'decoupled'` with $\theta = 0.5$.

14.5 Worked Example: One Decoupled Step on a 5-Node Cable

Let us continue the 5-node cable from Section 13.5 and advance one time step using the decoupled GS strategy with $\theta = 0.5$, $\Delta t = 0.02$ ms.

Initial conditions: $V_m^0 = [-85, -85, -20, -85, -85]^T$ mV (a single node depolarized), $\phi_e^0 = [0, 0, 0, 0, 0]^T$ mV.

Stage 1 — Ionic update. Rush–Larsen produces $V_m^* \approx [-85, -85, -19.5, -85, -85]^T$ (the depolarized node changes slightly due to ionic currents; exact value depends on the model).

Stage 2 — Parabolic solve.

Build the RHS: $\mathbf{b}_{\text{para}} = \mathbf{B}_{\text{para}} V_m^* + \mathbf{L}_i \phi_e^0$. Since $\phi_e^0 = \mathbf{0}$, the coupling term vanishes on the first step. The parabolic solve produces V_m^1 — the depolarized region begins to spread to neighbors due to intracellular diffusion: $V_m^1 \approx [-84.75, -83.74, -20.52, -83.74, -84.75]^T$ mV.

Stage 3 — Elliptic solve.

RHS: $\mathbf{b}_{\text{ellip}} = \mathbf{L}_i V_m^1$. The Laplacian $\mathbf{L}_i$ applied to V_m^1 produces a vector that is nonzero primarily near the depolarized region (where gradients are large). After solving $-(\mathbf{L}_i + \mathbf{L}_e) \phi_e^1 = \mathbf{b}_{\text{ellip}}$ with pinning ($\phi_{e,0} = 0$), we get $\phi_e^1 \approx [0, -0.18, 1.36, -0.18, 0]^T$ mV. The extracellular potential peaks near the depolarized cell — current is leaving the intracellular space and returning through the extracellular space, creating a dipole-like pattern.

🌿 Intuition

The extracellular signal is the "shadow" of the wavefront. Wherever V_m has a steep gradient, intracellular current is flowing out of cells and returning through the extracellular space, creating the ϕ_e signal. This is exactly the signal picked up by an ECG electrode. The elliptic solve computes this shadow at every time step.

14.6 Operator Splitting: The Outer Loop

The strategies above handle the diffusion sub-step of operator splitting. The full bidomain simulation uses the same Godunov or Strang splitting from Chapter 9:

- **Godunov:** ionic step (full Δt) $\rightarrow$ diffusion step (full Δt). First-order splitting error.
- **Strang:** ionic step ($\Delta t/2$) $\rightarrow$ diffusion step (Δt) $\rightarrow$ ionic step ($\Delta t/2$). Second-order splitting error.

The splitting error comes from separating reaction from diffusion, *not* from the parabolic–elliptic coupling. The elliptic equation has no time derivative and no memory (Section 12.6): its solution depends only on the current V_m , so it "comes along for the ride" within the diffusion sub-step without introducing additional splitting error.

Mathematical Note

Monodomain splitting vs. bidomain splitting. In both cases, operator splitting separates reaction (local ionic ODEs) from diffusion (global PDE). The fact that the bidomain diffusion sub-step involves two coupled equations does not introduce extra splitting error — both are solved together within the diffusion sub-step.

Key Insight

Sidebar: The monolithic alternative. In the research literature, you may encounter solvers that assemble the full $2N \times 2N$ block system and solve it with GMRES or MINRES using block preconditioners (block LDU, Schur complement approximations, AMG sub-solves). This monolithic approach has theoretical advantages (tighter coupling, potential for fewer iterations) but is much more complex to implement and requires specialized indefinite-system solvers. The decoupled approach described above is simpler, produces $N \times N$ SPD sub-problems solvable by standard tools, and is the dominant approach in modern production codes. Engine V1 uses decoupled strategies exclusively.

Linear Solvers and Implementation

Every strategy from Chapter 14 ends with one or two linear solves. The parabolic solve is identical to monodomain (Chapter 11). The elliptic solve is new — and it is the computational bottleneck of the entire bidomain simulation. This chapter covers the solvers for both sub-problems, the null-space handling required for isolated-tissue BCs, and the three-tier automatic solver selection that Engine V1 uses to pick the fastest option.

15.1 The Parabolic Sub-Problem: Familiar Territory

The parabolic matrix $\mathbf{A}_{\text{para}} = \frac{1}{\Delta t} \mathbf{I} - \theta \mathbf{L}_i$ looks exactly like the monodomain implicit system from Chapter 11. It is $N \times N$, symmetric, and positive definite (the $1/\Delta t$ identity dominates). All three solver families from Part II apply directly:

- **PCG** (Section 11.3): Preconditioned conjugate gradient. Works on any mesh. With warm-starting (using $\mathbf{V}_m^n$ as initial guess), converges in 3–8 iterations.
- **Chebyshev**: Fixed iteration count, zero synchronization points. Ideal for GPU. Needs eigenvalue bounds (computed via Gershgorin, cached).
- **Spectral (DCT/DST)** (Section 15.3): Direct solve via transform. $O(N \log N)$, exact, no iteration. Requires structured Cartesian grid with uniform BCs per axis.

Because this sub-problem is identical to monodomain, we do not repeat the solver details here. See Chapter 11 for implicit methods, and Appendix C.13 for PCG theory.

15.2 The Elliptic Sub-Problem: The New Challenge

The elliptic matrix $\mathbf{A}_{\text{ellip}} = -(\mathbf{L}_i + \mathbf{L}_e)$ is the combined bulk Laplacian — a pure diffusion operator with no mass/identity term. It has different properties from the parabolic matrix:

Property	Parabolic $\mathbf{A}_{\text{para}}$	Elliptic $\mathbf{A}_{\text{ellip}}$
Size	$N \times N$	$N \times N$
Symmetric?	Yes	Yes (with face-based stencil)
Positive definite?	Yes (always)	Semi-definite with Neumann BCs
Null space	None	Constant vector $\mathbf{1}$ (Neumann case)
Condition number	$O(1/\Delta t)$	$O(N)$ (grid-dependent)

The semi-definiteness means the matrix is singular when all boundaries are Neumann (the isolated-tissue case from Section 12.7). We must handle this null space before solving.

Null-Space Handling

The null space exists because ϕ_e is defined only up to an additive constant — adding the same voltage everywhere does not change any gradient or current flow. The engine uses two strategies to remove this ambiguity:

1. **Pin one node**: Set $\phi_{e,k} = 0$ for some chosen node k . This replaces row k and column k of $\mathbf{A}_{\text{ellip}}$ with the identity (1 on diagonal, 0 elsewhere), and sets $b_k = 0$. The matrix becomes non-singular and SPD. This is the default in Engine V1.
2. **Spectral zero-mode removal**: In the DCT solver (Section 15.3), the $(0, 0)$ transform coefficient corresponds exactly to the constant mode. Setting $\hat{u}_{0,0} = 0$ after spectral division pins the spatial mean

to zero — no matrix modification needed.

Other approaches exist in the literature — Lagrange multipliers (enforce $\int \phi_e dx = 0$) and Krylov deflation (project the constant mode out at each iteration) — but they add complexity without practical benefit for the cardiac setting. Engine V1 uses pinning for iterative solvers and zero-mode removal for spectral solvers.

Intuition

Pinning = choosing sea level. Just as altitude is meaningless without a reference point, ϕ_e is meaningless without a reference voltage. Pinning one node to zero is like declaring "this is sea level." All other ϕ_e values are then measured relative to this reference. The choice of which node to pin does not affect any physical quantity — only ϕ_e differences matter, not absolute values.

Worked Example: A 3-Node Elliptic Solve

Section 15.1 hands the parabolic solve to Chapter 11, but the elliptic solve is new — so let us do one by hand. Take the smallest cable that still has an interior node: **3 nodes**, reusing the diffusivities from Section 13.5 ($D_i = 1.214 \times 10^{-3}$, $D_e = 4.429 \times 10^{-3}$ cm²/ms, $\Delta x = 0.025$ cm). The combined scale factor is $(D_i + D_e)/\Delta x^2 = 9.029$ ms⁻¹ and the intracellular scale is $D_i/\Delta x^2 = 1.943$ ms⁻¹ (both carried over from Section 13.5).

With the face-based stencil, the elliptic matrix and the intracellular Laplacian are the same tridiagonals we built before:

$$\mathbf{A}_{\text{ellip}} = -(\mathbf{L}_i + \mathbf{L}_e) = 9.029 \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}, \quad \mathbf{L}_i = 1.943 \begin{pmatrix} -1 & 1 & 0 \\ 1 & -2 & 1 \\ 0 & 1 & -1 \end{pmatrix}.$$

Suppose the parabolic solve just handed us a partial upstroke — a wavefront ramp $\mathbf{V}_m = [-85, -60, -35]^T$ mV. The elliptic right-hand side is $\mathbf{b} = \mathbf{L}_i \mathbf{V}_m$. Multiplying row by row: $b_0 = 1.943(85 - 60) = 48.6$, $b_1 = 1.943(-85 + 120 - 35) = 0$, and $b_2 = 1.943(-60 + 35) = -48.6$, so $\mathbf{b} = [48.6, 0, -48.6]^T$. Notice the entries sum to zero — a necessary condition for the singular system to have a solution.

Mathematical Note

Pinning and the solve, step by step.

$\mathbf{A}_{\text{ellip}}$ is singular: every row sums to zero, so the constant vector $\mathbf{1}$ is a null vector and ϕ_e is fixed only up to an additive constant. Pin node 0 — set $\phi_{e,0} = 0$, replace row 0 and column 0 with the identity, and zero the first RHS entry. The pinned system is

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 18.058 & -9.029 \\ 0 & -9.029 & 9.029 \end{pmatrix} \begin{pmatrix} \phi_{e,0} \\ \phi_{e,1} \\ \phi_{e,2} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ -48.6 \end{pmatrix}.$$

Row 0 fixes $\phi_{e,0} = 0$, and the remaining 2×2 block is now nonsingular (SPD). From row 2: $-9.029 \phi_{e,1} + 9.029 \phi_{e,2} = -48.6$, i.e. $\phi_{e,2} - \phi_{e,1} = -5.383$. Substituting into row 1, $18.058 \phi_{e,1} - 9.029 \phi_{e,2} = 0$, gives $18.058 \phi_{e,1} - 9.029(\phi_{e,1} - 5.383) = 0 \Rightarrow 9.029 \phi_{e,1} = -48.6 \Rightarrow \phi_{e,1} = -5.383$, hence $\phi_{e,2} = -10.766$. The solution is

$$\phi_e = [0, -5.383, -10.766]^T \text{ mV}.$$

Check. The equation for node 0 that we discarded should still hold: $9.029(\phi_{e,0} - \phi_{e,1}) = 9.029(5.383) = 48.6 = b_0$. It does — the pinned row is automatically satisfied precisely because $\mathbf{b}$ summed to zero.

The extracellular potential falls monotonically across the wavefront — the electrical "shadow" of the V_m ramp, and exactly the kind of signal an ECG electrode records (Section 14.5). Pinning cost us nothing physical: only ϕ_e differences matter, and those are identical no matter which node we pin.

15.3 Three-Tier Automatic Solver Selection

Not all bidomain problems are equally hard. A simple isotropic simulation on a Cartesian grid can be solved exactly with a spectral method in $O(N \log N)$. An anisotropic simulation with mixed boundary conditions requires iterative solvers. Engine V1 automatically selects the fastest solver based on the problem's properties:

Tier	Solver	When used	Cost
Tier 1	Direct spectral (DCT/DST/FFT)	Isotropic + structured Cartesian + uniform BCs per axis	$O(N \log N)$, exact
Tier 2	PCG with spectral preconditioner	Anisotropic + structured Cartesian + uniform BCs per axis	$O(k \cdot N \log N)$, $k \approx 5\text{--}15$
Tier 3	PCG (Jacobi preconditioner)	Unstructured mesh, mixed BCs, or general geometry	$O(k \cdot N)$, $k \approx 20\text{--}50$

The selection logic inspects two properties of the problem:

1. **Is the grid Cartesian with uniform BCs per axis?** If yes, spectral transforms (DCT for Neumann, DST for Dirichlet, FFT for periodic) can diagonalize the Laplacian. This enables Tier 1 or Tier 2.
2. **Is the conductivity isotropic?** If yes, a single scalar $D = D_i + D_e$ characterizes the elliptic operator, and the spectral solver gives an exact answer (Tier 1). If the conductivity is anisotropic (different in $\mathbf{x}$ and $\mathbf{y}$), the spectral solver can still serve as a *preconditioner* for PCG (Tier 2) — it captures the dominant structure, and PCG handles the anisotropic correction in a few iterations.

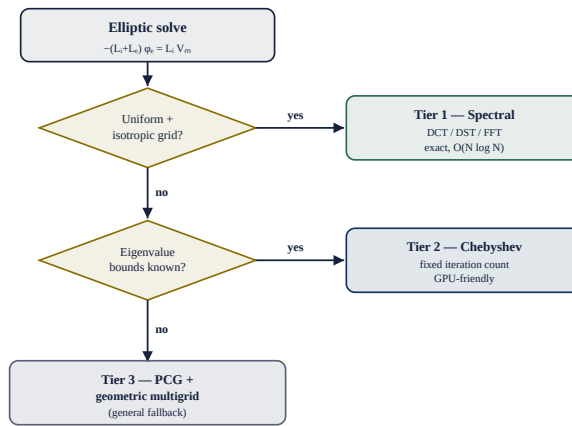


Figure 15.1. The solver-selection decision tree for the elliptic sub-problem. A uniform, isotropic Cartesian grid is diagonalized exactly by a spectral transform (Tier 1). Otherwise, if eigenvalue bounds are available the fixed-iteration Chebyshev solver applies (Tier 2); when neither holds — unstructured meshes, mixed BCs, general geometry — the engine falls back to preconditioned conjugate gradient with geometric multigrid (Tier 3). Each step down the tree trades exactness and speed for generality.

Intuition

Think of it like choosing a tool for a job. If the problem is perfectly regular (isotropic, Cartesian), use a precision instrument (spectral — exact, fast). If it is slightly irregular (anisotropic but still Cartesian), use the precision instrument as a starting point and refine with a general-purpose tool (spectral-preconditioned PCG). If it is fully irregular (unstructured mesh, complex geometry), fall back to the general-purpose tool alone (PCG).

Engine V5.4 Connection

Engine V1 connection. The auto-selection logic lives in `BidomainSimulation._auto_select_elliptic_solver()` in `bidomain.py`. It inspects the `BoundarySpec` protocol to determine if BCs are spectral-eligible, and the conductivity to determine if the problem is isotropic. The spectral solver (`SpectralSolver` in `linear_solver/spectral.py`) supports per-axis BC selection: `bc_x='neumann'`, `bc_y='dirichlet'`, etc.

15.4 Engine V1 Architecture

Let us map the mathematics to the actual code. Engine V1 lives in `Bidomain/Engine_V1/` and follows the same architectural principles as the monodomain Engine V5.4:

Concept	Class / Module	Key Methods
Spatial discretization	discretization/fdm.py	<code>get_L_i()</code> , <code>get_L_e()</code> , <code>get_parabolic_operators(dt, theta)</code> , <code>get_elliptic_operator()</code>
Diffusion strategies	<code>solver/diffusion_stepping/*.py</code>	<code>step(state, dt)</code> — updates $\mathbf{V}_m$ and ϕ_e in-place
Linear solvers	<code>solver/linear_solver/*.py</code>	<code>solve(A, b)</code> — returns solution vector
Operator splitting	<code>solver/splitting/</code>	Godunov or Strang (reused from monodomain)
Simulation orchestrator	<code>bidomain.py</code>	<code>BidomainSimulation(diffusion_solver=..., ...)</code>

The factory function in `bidomain.py` wires everything together:

Factory string	Strategy class
'decoupled' or 'gauss_seidel'	DecoupledBidomainDiffusionSolver
'semi_implicit'	SemiImplicitSolver
'jacobi'	DecoupledJacobiSolver
'imex_sbdf2'	IMEXSBDF2Solver
'explicit_rkc'	ExplicitRKCSolver

Linear solver selection uses a separate factory: 'pcg', 'chebyshev', 'spectral' (Tier 1), 'pcg_spectral' (Tier 2), or the automatic three-tier selection described above.

⚙️ **Engine V5.4 Connection**

Key design principle: zero allocation per step. All diffusion solvers pre-allocate workspace tensors ($\mathbf{A}_{\text{para}}$, $\mathbf{B}_{\text{para}}$, temporary vectors) at construction time. The `step()` method performs only matrix-vector products and linear solves — no memory allocation, no Python object creation. This is critical for GPU performance.

End of Parts I–III. The full research documentation, including detailed code examples and implementation guides, is available in the `Research/Bidomain/` folder of the Engine V1 repository.

PART IV

Lattice-Boltzmann Methods

A radically different paradigm for solving diffusion equations — no matrices, no linear solves, just fictitious particles streaming and colliding on a lattice. From the heat equation to cardiac monodomain and bidomain, LBM trades algebraic machinery for embarrassingly parallel data movement.

The Lattice-Boltzmann Method: From Kinetic Theory to Computation

In Chapters 13–16, we discretized partial differential equations. We started with physics at the macroscopic scale — fluid flow, heat diffusion, electrical signal propagation — and asked: how do we approximate the derivatives on a computer? Those methods (FDM, FEM, FVM) work brilliantly and are the industrial standard. But there is another path, one that begins not with a PDE, but with something more fundamental: the motion of individual particles, governed by kinetic theory. This chapter introduces the Lattice-Boltzmann Method (LBM). Unlike classical discretizations that approximate a PDE, LBM simulates a discrete analogue of the Boltzmann equation — a statement about how particles move and collide. The miracle is that when you correctly simulate the microscopic collisions, the macroscopic behavior (flow, diffusion, electrical conduction) emerges automatically. For cardiac problems, especially when spatial resolution is tight or parallelization is critical, LBM offers a compelling alternative to classical methods. We will build this from first principles: the Boltzmann equation, its simplification to the BGK collision operator, the discretization of velocity space onto a lattice, and the final algorithm. By chapter's end, you will understand not just what LBM computes, but why the method works — and how to tune it for cardiac electrophysiology.

Intuition

The whole method in 30 seconds. Before the theory, here is the destination. One lattice-Boltzmann timestep is just six operations, repeated at every grid node:

1. **Initialize** the distributions by splitting the voltage across the Q lattice directions: $f_i = w_i V$.
2. **Collide** — relax each f_i a fraction of the way toward its local equilibrium f_i^{eq} .
3. **React** — inject the ionic reaction source into the distributions.
4. **Stream** — shift each f_i to its neighbouring node in direction i .
5. **Bounce back** — reflect the distributions that meet a tissue boundary.
6. **Recover** the voltage by summing, $V = \sum_i f_i$ — then repeat next timestep.

Here is the whole method in six steps; the rest of the chapter explains *why* each one works.

18.1 The Thermodynamic Foundation: Phase Space and the Equilibrium State

Intuition

Core Insight: Every numerical method we have studied so far — FDM, FEM, FVM — starts from a macroscopic PDE and discretizes it. LBM takes a radically different route. It starts from a much deeper level: the statistical mechanics of particles. If you correctly model how particles distribute themselves in velocity space and how that distribution evolves, the macroscopic behavior — diffusion, fluid flow, heat conduction — emerges automatically. To understand LBM, we must first understand the space it lives in and the equilibrium it relaxes toward.

Position Space: What You Already Know

In every method we have encountered so far, the unknown is a scalar field: $V(\mathbf{x}, t)$ for voltage, $T(\mathbf{x}, t)$ for temperature, $\phi(\mathbf{x}, t)$ for a generic concentration. At each point $\mathbf{x}$ in the domain, we store **one number** — the local value of the field. This is position space: a map from location to field value. Position space is

intuitive. You can picture a heat map of the heart where each pixel has a color representing voltage. FDM puts a number on each grid point. FEM interpolates between nodes. FVM averages over cells. In all cases, the fundamental object is simple: one value per spatial location. But this simplicity comes at a cost. The macroscopic PDE — the diffusion equation, the monodomain equation — is a coarse-grained description. It tells you how the *average* behavior evolves, but it has already thrown away information about what individual particles are doing. To build LBM, we need to recover some of that microscopic information. That means working in a richer space.

Velocity Space: What Are the Particles Doing?

Imagine standing at a single point in cardiac tissue and watching every ion that passes through. Some ions are moving fast to the right. Some are drifting slowly upward. Some are nearly stationary, vibrating in place. If you could catalog every ion and record its velocity, you would not get a single number — you would get a **distribution**. This distribution is the velocity-space view at a single point. It answers the question: "Of all the particles at position $\mathbf{x}$, how many have velocity $\mathbf{c}$?" In a hot gas, the distribution is wide — particles are moving at all speeds in all directions. In a cold gas, the distribution is narrow — most particles are clustered near zero velocity. In a gas with bulk flow, the distribution is shifted — its peak is no longer at zero but at the flow velocity $\mathbf{u}$. The crucial point: **at every single location in physical space, there is an entire distribution of velocities**. Not one velocity, not an average velocity — a full spread. The shape of this spread encodes the thermodynamic state at that point: temperature (width of the spread), bulk flow (shift of the peak), pressure (area under the curve).

Phase Space: Position and Velocity Together

Phase space is the combined space where you track both where a particle is AND how fast it is going. In two spatial dimensions, a particle's full state is four numbers: (x, y, c_x, c_y) — two for position, two for velocity. In three dimensions, it is six numbers: (x, y, z, c_x, c_y, c_z) . The **distribution function** $f(\mathbf{x}, \mathbf{c}, t)$ lives in this combined space. It is a function of position, velocity, and time simultaneously. Its value tells you: at position $\mathbf{x}$, at time t , the density of particles with velocity $\mathbf{c}$.

$$f : (\mathbf{x}, \mathbf{c}, t) \longrightarrow \text{particle density at } (\mathbf{x}, \mathbf{c}) \text{ at time } t$$

(18.1)

This is the fundamental object of kinetic theory. Compare it to the macroscopic field:

	Macroscopic (FDM/FEM)	Kinetic (LBM)
Unknown	$\phi(\mathbf{x}, t)$ — one number per point	$f(\mathbf{x}, \mathbf{c}, t)$ — a distribution per point
Lives in	Position space (D dimensions)	Phase space ($2D$ dimensions)
Information	Average field value	Full velocity distribution
Storage per node	1 value	Q values (one per velocity direction)

The extra information in the velocity distribution is not wasted. It is exactly what allows LBM to avoid assembling and solving matrix systems — the transport physics is encoded in the distribution itself, and the collision-streaming algorithm updates it locally.

The Equilibrium State: Where Does the Distribution Settle?

Here is the central question of this section — and arguably the most important idea in all of LBM: **When a system of particles reaches thermal equilibrium, what shape does the velocity distribution take?** The answer has been known since the 19th century, and it is one of the great results of statistical mechanics.

James Clerk Maxwell and Ludwig Boltzmann proved that the equilibrium velocity distribution of a gas is a **Gaussian** (a bell curve) in velocity space:

$$f^{\text{eq}}(\mathbf{c}) = \frac{\rho}{(2\pi c_s^2)^{D/2}} \exp\left(-\frac{|\mathbf{c} - \mathbf{u}|^2}{2c_s^2}\right) \quad (18.2)$$

This is the **Maxwell-Boltzmann distribution**. Every symbol has a physical meaning:

- ρ — the total density of particles at this location. This is the macroscopic quantity we are ultimately tracking (voltage, temperature, concentration).
- $\mathbf{u}$ — the bulk (average) velocity of the particles. For diffusion problems, $\mathbf{u} = \mathbf{0}$ — there is no net flow, and the bell curve is centered at the origin.
- c_s — a parameter that controls the **width** of the bell curve. Physically, it is related to temperature: a higher c_s means a wider spread (hotter gas, faster particles on average). In LBM, c_s will become the **lattice speed of sound**, connecting the microscopic model to macroscopic transport properties.
- D — the spatial dimension (2 for 2D, 3 for 3D). The normalization $(2\pi c_s^2)^{D/2}$ ensures the total probability integrates to ρ .

What the Gaussian Tells Us: Three Physical Claims

The Maxwell-Boltzmann distribution (18.2) is not just a formula — it makes three specific physical claims about what equilibrium looks like. Each claim corresponds to a different "order" of the distribution, and each will turn out to be critically important when we build the lattice method.

Zeroth Order — Conservation

Integrate the distribution over all velocities:

$$\int f^{\text{eq}}(\mathbf{c}) d\mathbf{c} = \rho \quad (18.3)$$

The total amount of "stuff" is ρ . This is conservation of mass (or charge, or energy, depending on what ρ represents). The equilibrium does not create or destroy anything — it only redistributes velocities. No matter how the particles rearrange their speeds and directions, the total count stays the same.

First Order — Symmetry of Flux

Compute the average velocity by weighting with $\mathbf{c}$:

$$\int \mathbf{c} f^{\text{eq}}(\mathbf{c}) d\mathbf{c} = \rho \mathbf{u} \quad (18.4)$$

For a gas at rest ($\mathbf{u} = \mathbf{0}$), the mean velocity is zero. The bell curve is symmetric — as many particles move left as right, up as down. There is no net flux. For diffusion problems in cardiac EP, this is exactly the equilibrium condition: at rest, the distribution is centered and symmetric.

Second Order — Isotropic Variance

Compute the velocity covariance (the "spread" of the bell curve):

$$\int (c_\alpha - u_\alpha)(c_\beta - u_\beta) f^{\text{eq}}(\mathbf{c}) d\mathbf{c} = \rho c_s^2 \delta_{\alpha\beta} \quad (18.5)$$

The variance is **isotropic** — the bell curve has the same width in every direction. The $\delta_{\alpha\beta}$ (Kronecker delta: 1 if $\alpha = \beta$, 0 otherwise) means there is no preferred axis. The parameter c_s^2 is the variance of the Gaussian, and it is the same along $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$. This isotropy is a physical claim: at thermodynamic equilibrium, no direction is special. The random thermal motion of particles is equally vigorous in all directions. This will later impose constraints on the lattice weights and will be the reason why the **standard LBM lattice produces isotropic diffusion by default**.

Intuition

Three orders, three physics: The Gaussian equilibrium encodes conservation (zeroth order), zero net flux (first order), and isotropic variance (second order). These are not abstract mathematical properties — they are the defining characteristics of thermodynamic equilibrium. Every lattice-Boltzmann scheme must respect all three, because they are the physical laws that the scheme must recover at the macroscopic level. In Section 18.4, we will see that these three conditions are exactly the constraints that determine the lattice weights w_i and velocities $\mathbf{c}_i$.

Two Cases: Rest and Flow

The Maxwell-Boltzmann distribution (18.2) handles two physically distinct situations: **Case 1: System at rest ($\mathbf{u} = \mathbf{0}$)**. The bell curve is centered at the origin of velocity space. No bulk motion — particles jiggle randomly in all directions with equal probability. This is the relevant case for **diffusion**: cardiac tissue conducts electrical signals by ion diffusion, not bulk fluid flow. The equilibrium distribution simplifies to:

$$f^{\text{eq}}(\mathbf{c}) = \frac{\rho}{(2\pi c_s^2)^{D/2}} \exp\left(-\frac{|\mathbf{c}|^2}{2c_s^2}\right) \quad (18.6)$$

a symmetric Gaussian centered at the origin. This is the equilibrium that our cardiac LBM will use. **Case 2: System with bulk flow ($\mathbf{u} \neq \mathbf{0}$)**. The bell curve is shifted — its peak is at $\mathbf{u}$ instead of at the origin. More particles are moving in the flow direction than against it. This is the relevant case for **fluid dynamics** (Navier-Stokes). The distribution is still a Gaussian, but the center has moved. The shift breaks the first-order symmetry: $\int \mathbf{c} f^{\text{eq}} d\mathbf{c} = \rho \mathbf{u} \neq \mathbf{0}$.

Mathematical Note

Why c_s is the bridge between micro and macro: The parameter c_s plays a dual role. In the continuous Maxwell-Boltzmann distribution, it is the **thermal velocity** — the standard deviation of the velocity Gaussian, set by the temperature of the gas ($c_s^2 = k_B T / m$). When we discretize onto a lattice, c_s becomes the **lattice speed of sound** ($c_s^2 = c^2 / 3$ where $\mathbf{c} = \Delta \mathbf{x} / \Delta t$). This is not a coincidence: the Chapman-Enskog analysis (Section 18.2) shows that the lattice speed of sound must equal the Gaussian's variance parameter for the discrete scheme to recover the correct macroscopic transport equations. The connection $c_s^2 = c^2 / 3$ is a fixed consequence of the lattice geometry and the quadrature exactness conditions (Section 18.4).

Why the Equilibrium Is the Heart of LBM

In FDM, FEM, and FVM, the physics enters through the PDE — you discretize the diffusion equation or the monodomain equation directly. In LBM, **the physics enters through the equilibrium distribution**. The entire method can be summarized in one sentence: *Relax the velocity distribution toward the Maxwell-Boltzmann equilibrium, and the correct macroscopic PDE emerges automatically*. This is why we have

spent this section building the equilibrium from physical principles rather than just writing down a formula. The Gaussian is not an arbitrary choice — it is the unique distribution that satisfies the constraints of thermodynamic equilibrium (maximum entropy subject to conservation of mass, momentum, and energy). Every property of LBM — the lattice weights, the velocity sets, the collision operator, the transport coefficients — traces back to this Gaussian. In the sections that follow, we will:

- **Section 18.2:** Write down the equation that governs how f evolves toward this equilibrium (the Boltzmann equation), and simplify it for practical computation (the BGK collision operator).
- **Section 18.3:** Discretize velocity space onto a lattice, turning the continuous $f(\mathbf{x}, \mathbf{c}, t)$ into a finite set of distributions $f_i(\mathbf{x}, t)$, and write the lattice-Boltzmann equation.
- **Section 18.4:** Return to the Gaussian equilibrium and ask: what constraints do equations (18.3)–(18.5) impose on the discrete lattice? This will determine the weights w_i , velocity vectors $\mathbf{c}_i$, and the discrete equilibrium f_i^{eq} .
- **Section 18.5:** Introduce moment space — a representation that reveals which physical laws the collision operator is actually enforcing, and how to control them independently.

18.2 The Boltzmann Equation and Kinetic Theory

The Boltzmann Equation: Fundamental Form

The Boltzmann equation describes how the distribution function $f(\mathbf{x}, \mathbf{c}, t)$ — introduced in Section 18.1 — evolves in time. In the **Eulerian frame** (you stand at a fixed point in phase space and watch the distribution change around you), it reads:

$$\frac{\partial f}{\partial t} + \mathbf{c} \cdot \nabla f + \mathbf{F} \cdot \nabla_{\mathbf{c}} f = \Omega(f) \quad (18.7)$$

Let us parse each term on the left-hand side:

- $\frac{\partial f}{\partial t}$: How the distribution changes in time at a fixed point in phase space.
- $\mathbf{c} \cdot \nabla f$: Particles streaming through physical space at velocity $\mathbf{c}$. This is advection: a particle at $\mathbf{x}$ with velocity $\mathbf{c}$ will be at $\mathbf{x} + \mathbf{c}\Delta t$ after time Δt .
- $\mathbf{F} \cdot \nabla_{\mathbf{c}} f$: External forces accelerating particles, changing their velocities. The $\nabla_{\mathbf{c}}$ means the derivative is in *velocity* space, not physical space. This term accounts for gravity, electric fields, or any body force that shifts the velocity distribution.

The first two terms together form the **material (total) derivative** familiar from fluid mechanics — the rate of change following a particle that streams at velocity $\mathbf{c}$. The third term is separate: it represents forces that *change* the particle's velocity, pushing it through velocity space. If we define the streaming derivative as

$$\frac{Df}{Dt} \equiv \frac{\partial f}{\partial t} + \mathbf{c} \cdot \nabla f$$

then equation (18.7) can be written compactly as

$$\frac{Df}{Dt} + \mathbf{F} \cdot \nabla_{\mathbf{c}} f = \Omega(f) \quad (18.7a)$$

This **Lagrangian perspective** is often more intuitive: ride along with a particle that streams at constant velocity $\mathbf{c}$. From its frame, the distribution f changes for only two reasons — external forces push particles to different velocities, and collisions reshuffle the velocity population. If there were no forces and no

collisions, $\mathbf{f}$ would be carried unchanged along the streaming trajectory, like a ball on a frictionless surface. For LBM applications (diffusion, monodomain), we will drop the external force term entirely (no body forces accelerate particles through velocity space). This makes the Eulerian form (18.7) the natural starting point for deriving the lattice-Boltzmann equation. **The right-hand side is the collision operator:** The symbol $\Omega(\mathbf{f})$ represents collisions. When particles collide, their velocities change. The full collision integral is:

$$\Omega(\mathbf{f}) = \int \int (\mathbf{f}' \mathbf{f}'_1 - \mathbf{f} \mathbf{f}_1) g B(g, \theta) d\Omega d\mathbf{c}_1 \quad (18.8)$$

The notation is intimidating. Here is what it means:

- $\mathbf{f}$ and $\mathbf{f}_1$: the distribution functions of two particles BEFORE collision.
- $\mathbf{f}'$ and $\mathbf{f}'_1$: the distributions of those same two particles AFTER collision.
- $g = |\mathbf{c} - \mathbf{c}_1|$: the relative speed of the two particles.
- $B(g, \theta)$: the scattering cross-section, which depends on the relative velocity and scattering angle θ .
- $d\Omega$: integration over all possible scattering angles.
- $d\mathbf{c}_1$: integration over all velocities of the second particle.

The expression $(\mathbf{f}' \mathbf{f}'_1 - \mathbf{f} \mathbf{f}_1)$ is a balance: the gain term $\mathbf{f}' \mathbf{f}'_1$ counts particles scattered INTO the state $(\mathbf{x}, \mathbf{c})$ by collisions, while the loss term $\mathbf{f} \mathbf{f}_1$ counts particles scattered OUT.

The BGK Simplification

The collision integral (18.8) is a 5-dimensional integral — computationally intractable. For a coarse 3D grid with 100^3 points and even a modest 10^3 velocities per point, evaluating it requires $\sim 10^{15}$ operations per timestep. For most of the 20th century, the Boltzmann equation remained a theoretical tool, not a practical one. The breakthrough came from Bhatnagar, Gross, and Krook in 1954. They asked: **Can we replace the complex collision integral with something simple that still produces the correct macroscopic physics?** Their answer was to replace the collision integral with a single relaxation toward the equilibrium distribution we met in Section 18.1:

$$\Omega(\mathbf{f}) \approx -\frac{1}{\tau} (\mathbf{f} - \mathbf{f}^{\text{eq}}) \quad (18.9)$$

where τ is a single relaxation time and $\mathbf{f}^{\text{eq}}$ is the Maxwell-Boltzmann equilibrium (equation 18.2). The idea is elegantly simple: the actual distribution $\mathbf{f}$ is nudged toward its local equilibrium at a rate determined by τ . No detailed calculation of scattering angles or cross-sections is needed. This is the **BGK approximation**. Notice the direct connection to Section 18.1: the $\mathbf{f}^{\text{eq}}$ that appears here is exactly the Gaussian equilibrium (18.2) — the bell curve in velocity space that encodes conservation, zero flux, and isotropic variance. The BGK operator says: whatever the current distribution looks like, push it toward that Gaussian. The three physical claims we established in Section 18.1 — equations (18.3), (18.4), and (18.5) — are the properties that $\mathbf{f}^{\text{eq}}$ carries into the collision step. Substituting (18.9) into (18.7), we get the **BGK Boltzmann equation**:

$$\frac{\partial \mathbf{f}}{\partial t} + \mathbf{c} \cdot \nabla \mathbf{f} = -\frac{1}{\tau} (\mathbf{f} - \mathbf{f}^{\text{eq}}) \quad (18.10)$$

where we have dropped the external force term $\mathbf{F} \cdot \nabla_{\mathbf{c}} \mathbf{f}$ (in LBM for diffusion, there are no external forces accelerating particles through velocity space).

Why BGK Works: Chapman-Enskog Analysis

Why should such a drastic simplification — replacing a 5-dimensional integral with a simple relaxation — give the right macroscopic physics? The answer comes from **Chapman-Enskog analysis**, one of the deepest results in statistical mechanics. The idea: assume the distribution function is close to equilibrium, with small corrections:

$$f = f^{\text{eq}} + \epsilon f^{(1)} + \epsilon^2 f^{(2)} + \dots$$

where ϵ is a small parameter measuring how far the system is from equilibrium (technically, the Knudsen number $\text{Kn} = \lambda/L$, the ratio of mean free path to system size). Then expand the Boltzmann equation order by order. What emerges at each order is a specific piece of the macroscopic equations: **Zeroth order** (ϵ^0): the distribution equals the equilibrium $f = f^{\text{eq}}$. Taking the zeroth and first moments of f^{eq} (exactly the integrals from equations 18.3 and 18.4) gives the **conservation laws**:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \quad (\text{mass conservation})$$

These are the Euler equations — ideal fluid dynamics with no dissipation. **First order** (ϵ^1): the correction $f^{(1)}$ introduces gradients of velocity and temperature. Taking moments of $f^{(1)}$ produces the **dissipative terms** — this is where viscosity and diffusivity enter:

$$\rho \frac{D\mathbf{u}}{Dt} = -\nabla p + \underbrace{\eta \nabla^2 \mathbf{u}}_{\text{viscous stress}} + \mathbf{f} \quad (18.11)$$

for fluid flow (Navier-Stokes), and

$$\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T = \underbrace{D \nabla^2 T}_{\text{diffusion}} \quad (18.12)$$

for heat or mass diffusion. The underbraced terms are the ones that the first-order correction adds — they are absent in the Euler (zeroth-order) equations. The transport coefficients (viscosity η , diffusivity D) are determined by the relaxation time τ . Large τ (slow relaxation toward equilibrium) gives high viscosity or diffusivity; small τ gives low viscosity or diffusivity. The relationship is precise and will be derived explicitly in Section 18.4 as the τ - D relationship.

Intuition

The key insight: The macroscopic PDEs we have been solving throughout this textbook — the diffusion equation, the monodomain equation, even Navier-Stokes — are not fundamental. They are **emergent**. They arise as the zeroth- and first-order limits of the Boltzmann equation in the collision-dominated regime. The zeroth-order (equilibrium) contribution gives conservation laws. The first-order (near-equilibrium) correction gives the dissipative transport terms. The equilibrium f^{eq} carries the conservation physics (18.3)–(18.5); the relaxation rate $1/\tau$ sets the transport coefficients. Together, they recover the full macroscopic PDE.

Mathematical Note

Why Chapman-Enskog Works: The expansion assumes the distribution is locally near equilibrium, with small non-hydrodynamic corrections. The collision operator acts to restore equilibrium. By expanding around this equilibrium and using the fact that the collision operator has specific zero eigenvalues (corresponding to conserved quantities like mass and momentum), one can systematically extract the macroscopic equations order by order in the Knudsen number.

18.3 The Lattice-Boltzmann Equation: Single Relaxation Time (SRT/BGK)

The Core Strategy of LBM

1. **Start from the kinetic level:** the Boltzmann equation governs how $f(\mathbf{x}, \mathbf{c}, t)$ evolves in phase space (Section 18.2).
2. **Simplify the collision:** replace the intractable collision integral with BGK relaxation toward f^{eq} — the Maxwell-Boltzmann Gaussian from Section 18.1 (Section 18.2).
3. **Discretize velocity space:** replace the continuous $\mathbf{c}$ with a small finite set of lattice velocities $\mathbf{c}_i$ and write the discrete update rule (this section).
4. **Derive the lattice from the Gaussian:** the weights w_i , velocity sets $\mathbf{c}_i$, and discrete equilibrium f_i^{eq} all follow from quadrature on the Maxwell-Boltzmann distribution (Section 18.4).
5. **Trust Chapman-Enskog:** this analysis guarantees that the simplified, discretized scheme recovers the correct macroscopic PDE in the hydrodynamic limit.

The equilibrium f^{eq} is the thread that connects everything: it defines the target of relaxation (BGK), it carries the conservation laws (Section 18.1), and it determines which macroscopic equation the scheme recovers (Chapman-Enskog).

From Continuous to Discrete: Three Deliberate Choices

In Section 18.2 we arrived at the BGK Boltzmann equation (18.10):

$$\frac{\partial f}{\partial t} + \mathbf{c} \cdot \nabla f = -\frac{1}{\tau}(f - f^{\text{eq}})$$

This is still a continuous equation — f lives on a continuous position space, a continuous velocity space, and continuous time. To turn it into something a computer can execute, we make three deliberate discretization choices:

1. **Discretize velocity space.** Replace the continuous velocity $\mathbf{c}$ with a small finite set of lattice velocities $\{\mathbf{c}_0, \mathbf{c}_1, \dots, \mathbf{c}_{Q-1}\}$. The continuous distribution $f(\mathbf{x}, \mathbf{c}, t)$ becomes a finite collection of distributions $f_i(\mathbf{x}, t)$, one for each lattice direction i . This is the step where velocity space — the infinite-dimensional space from Section 18.1 — collapses to a handful of directions. The choice of which velocities to keep (and the associated weights) is governed by the Gaussian quadrature we will derive in Section 18.4.
2. **Discretize time.** Replace $\partial f / \partial t$ with a finite difference over a timestep Δt . This turns the differential equation into an update rule.
3. **Lock the grid to the velocities.** Choose the spatial grid spacing Δx and timestep Δt so that each lattice velocity $\mathbf{c}_i$ moves a particle from one grid node to an adjacent grid node in exactly one timestep: $\mathbf{c}_i \Delta t = \mathbf{c}_i \Delta x / |\mathbf{c}_i|$. This eliminates interpolation entirely — streaming is exact.

Notice one term that is conspicuously absent: the external force $\mathbf{F} \cdot \nabla_{\mathbf{c}} f$ from equation (18.7). Recall that this term describes forces that accelerate particles through velocity space — gravity pulling molecules

downward, electric fields pushing ions. For diffusion and the monodomain equation, there are no such body forces: particles stream freely and collide, but nothing pushes them to change velocity mid-flight. We therefore drop it. (For applications that do require forcing — buoyancy-driven flows, electrokinetics — the force re-enters as a source term added to the right-hand side, but that is beyond our scope.) With these three choices, the material derivative Df/Dt becomes a simple finite difference along the lattice direction i , and the BGK equation becomes:

The Discrete Lattice-Boltzmann Equation

$$f_i(\mathbf{x}, t + \Delta t) - f_i(\mathbf{x}, t) = -\frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)] \quad (18.13)$$

The left-hand side is the change in the distribution f_i at node $\mathbf{x}$ over one timestep. The right-hand side is the collision: a relaxation nudge toward the local equilibrium f_i^{eq} , scaled by $\Delta t/\tau$. This is the entire algorithm at a single node before streaming. After computing the post-collision state, we stream: each f_i is moved to the neighboring node in the direction $\mathbf{c}_i$. The combined collision-then-stream update gives the full lattice-Boltzmann equation in its standard form:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)]$$

This single line says: compute the collision at $\mathbf{x}$, then place the result at the downstream neighbor $\mathbf{x} + \mathbf{c}_i \Delta t$. The equation is fully explicit — no matrix inversions, no implicit coupling between nodes, no iterative solves. You compute it one node at a time, and the entire simulation marches forward. In practice, we split this into two conceptual steps, which we now describe.

Step 1: Collision (Local, Pure Relaxation)

At each node, we compute:

$$f_i^*(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)] \quad (18.14)$$

This is the collision step. The superscript $*$ on f_i^* denotes the **post-collision** distribution — the state of the distribution function AFTER the collision relaxation has been applied but BEFORE streaming moves data to neighbors. You can read f_i^* as "f-i-star" or "f-i post-collision." It is an intermediate quantity: f_i is the pre-collision state, f_i^* is the post-collision state, and after streaming, f_i^* becomes the new f_i at the destination node. The collision step happens at a single node, purely local. The distributions f_i at node $\mathbf{x}$ are nudged toward their equilibrium values f_i^{eq} . The speed of this nudge is controlled by $\Delta t/\tau$:

- If τ is large, the nudge is small — the distribution relaxes slowly.
- If τ is small (but must remain $> \Delta t/2$ for stability), the nudge is large — the distribution relaxes quickly.

The parameter τ directly controls the transport properties (diffusivity, viscosity). This is the heart of the method: all the physics (how fast heat diffuses, how viscous the fluid is) is encoded in τ .

Step 2: Streaming (Pure Data Movement)

After collision, we shift the post-collision distributions to their neighboring nodes:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) = f_i^*(\mathbf{x}, t) \quad (18.15)$$

This is the streaming step. Each distribution f_i^* at node $\mathbf{x}$ slides to the neighboring node in direction i . The destination is $\mathbf{x} + \mathbf{c}_i \Delta t$. If the lattice is designed so that $\mathbf{c}_i \Delta t = \Delta \mathbf{x}$ (one lattice spacing), then streaming is a simple data copy — a value at the center moves to a neighbor. Critically, streaming is not a collision. There is no physics in streaming — it is pure kinematics. A particle moving with velocity $\mathbf{c}_i$ will naturally travel a distance $\mathbf{c}_i \Delta t$ in time Δt . Streaming enforces this.

The Combined Form

Substituting the collision step (18.14) into the streaming step (18.15) — replacing f_i^* with its definition — gives the single combined equation:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)] \quad (18.16)$$

This is the **lattice-Boltzmann equation in streaming form**. It is the same physics as equation (18.13), but written from the destination's perspective: the value arriving at node $\mathbf{x} + \mathbf{c}_i \Delta t$ at the next timestep equals the current value at node $\mathbf{x}$, corrected by the collision. You will encounter this form throughout the LBM literature — it is the most compact way to write the full collide-and-stream update.

Intuition

Locality and Parallelization: Both collision and streaming are local operations. Collision affects only one node at a time. Streaming is a data copy — the value $f_i^*(\mathbf{x}, t)$ moves to a neighbor, but it does not interact with neighbors' data. After collision, all the f_i at all nodes are independent. Streaming then redistributes them. This extreme locality means LBM is trivially parallelizable: each node (or even each lattice direction at each node) can be processed on a separate processor with zero communication overhead during the step. Only a halo exchange is needed between processors after streaming. This is one reason LBM is popular on GPUs.

Variable Definitions and Component Form

To implement LBM, you must understand every variable. Here is a complete reference:

Symbol	Meaning	Units / Constraints
$f_i(\mathbf{x}, t)$	Distribution function for direction i at node $\mathbf{x}$, time t	Density (or concentration if diffusion)
$f_i^*(\mathbf{x}, t)$	Post-collision distribution for direction i (after relaxation, before streaming)	Same units as f_i
$f_i^{\text{eq}}(\mathbf{x}, t)$	Equilibrium distribution for direction i	Density (or concentration)
τ	Relaxation time	Must satisfy $\tau > \Delta t/2$
Δt	Timestep	In lattice units, $\Delta t = 1$
$\mathbf{c}_i$	Discrete velocity vector for direction i	$\mathbf{c}_0 = \mathbf{0}$ (rest); $ \mathbf{c}_i = c = \Delta \mathbf{x} / \Delta t$ (moving)
w_i	Weight for direction i	$\sum_i w_i = 1$
$\phi(\mathbf{x}, t) = \sum_i f_i$	Macroscopic variable (temperature, voltage, density)	Physical units (K, mV, kg/m ³)
Q	Number of lattice directions (velocities)	i ranges from 0 to $Q - 1$

In detail: $f_i(\mathbf{x}, t)$ is the distribution function for direction i . Think of it as "the portion of the macroscopic quantity assigned to direction i at this node at this time." There are Q values of f_i stored at each node — one per lattice direction. The sum $\sum_i f_i = \phi$ recovers the total macroscopic quantity. $f_i^{\text{eq}}(\mathbf{x}, t)$ is the equilibrium distribution for direction i — the target that the collision step relaxes toward. Its exact form depends on the weights w_i and the macroscopic field, and will be derived in Section 18.4 from the Gaussian equilibrium we built in Section 18.1. τ is the relaxation time. It controls how quickly the distribution approaches equilibrium. The relationship between τ and the physical diffusivity D comes from Chapman-Enskog analysis and will be derived in Section 18.4. Δt is the timestep. In lattice units, we often set $\Delta t = 1$, but it is useful to keep Δt explicit when connecting lattice variables to physical ones. $\mathbf{c}_i$ is the discrete velocity vector for direction i . Each $\mathbf{c}_i$ points from a node to a neighbor (or stays put if $i = 0$). The specific velocity sets and their associated weights are derived in Section 18.4 from the Gaussian quadrature framework. w_i is the weight for direction i . Like $\mathbf{c}_i$, the weights are uniquely determined by the quadrature exactness conditions and will be derived in Section 18.4. $\phi(\mathbf{x}, t) = \sum_i f_i(\mathbf{x}, t)$ is the recovered macroscopic quantity. For diffusion of temperature, ϕ is temperature. For cardiac electrophysiology, ϕ is the voltage. The sum over all lattice directions gives the total.

18.4 From the Gaussian to the Lattice: Discretizing f^{eq} , w_i , and $\mathbf{c}_i$

From Integrals to Sums

In Section 18.1 we wrote down the Maxwell-Boltzmann equilibrium (equation 18.2) and showed that it satisfies three integral identities — conservation of mass (18.3), momentum (18.4), and isotropic variance (18.5). Each of these is an integral over velocity space: you integrate $f^{\text{eq}}(\mathbf{c})$ — or f^{eq} multiplied by some power of $\mathbf{c}$ — over *all directions and all magnitudes*. LBM replaces "all directions and all magnitudes" with **a few lattice vectors $\mathbf{e}_i$ and weights w_i** . The lattice vectors are dimensionless integer vectors — their components are just 0 and ± 1 . For example, in D2Q5:

$$\mathbf{e}_0 = (0, 0), \quad \mathbf{e}_1 = (1, 0), \quad \mathbf{e}_2 = (-1, 0), \quad \mathbf{e}_3 = (0, 1), \quad \mathbf{e}_4 = (0, -1)$$

Each lattice vector comes with a weight w_i that says "this sample point represents this much of the Gaussian bell curve." Every continuous integral becomes a finite sum:

$$\int p(\mathbf{c}) \underbrace{\frac{1}{(2\pi c_s^2)^{D/2}} e^{-|\mathbf{c}|^2/2c_s^2}}_{\text{Gaussian kernel}} d\mathbf{c} \approx \sum_{i=0}^{Q-1} w_i p(\mathbf{e}_i) \quad (18.17)$$

The left-hand side integrates over the full continuous velocity space $\mathbf{c}$. The right-hand side evaluates at the lattice nodes $\mathbf{e}_i$ in dimensionless space — this is where the quadrature lives. Here p is a placeholder — it stands for whatever function of velocity you want to integrate against the Gaussian. Plug in $p = 1$ and you compute total mass. Plug in $p = e_{i\alpha}$ (a single component of $\mathbf{e}_i$) and you compute momentum. Plug in $p = e_{i\alpha}e_{i\beta}$ and you compute the stress tensor. The nodes $\mathbf{e}_i$ and their weights w_i are chosen so that these sums are *exact* for all polynomials up to the degree you need. **This is where w_i and $\mathbf{e}_i$ come from.** They are not free parameters that we tune. They are the nodes and weights of a quadrature rule applied to the Gaussian — a married pair, born together from the bell curve's shape. Once you choose a lattice geometry (how many sample points, and where), the weights are uniquely determined.

Intuition

How Can Five Points Replace an Entire Plane? In the continuous integral, $\mathbf{c}$ ranges over all directions and all magnitudes — an infinite velocity plane. On the lattice, we replace this with just 5 (or 9) sample points whose components are 0 and ± 1 . This seems like a drastic reduction. But remember what we are integrating: a Gaussian bell curve that decays rapidly away from the origin. Almost all of the integral's value is concentrated within a few lattice spacings of the center. The quadrature exploits this — it places its sample points precisely where the Gaussian has significant weight, and the weights w_i encode how much of the bell curve each point represents. Five well-chosen points on a steep hill capture more information than a thousand points in the flat plains beyond it.

Physical Velocities

The $\mathbf{e}_i$ are dimensionless integers — they tell you which neighbor to hop to on the grid. But the Boltzmann equation and the physics it describes involve velocities with real units. To recover a **physical velocity $\mathbf{c}_i$** (length/time), scale by the lattice speed $\mathbf{c} = \Delta\mathbf{x}/\Delta t$:

$$\mathbf{c}_i = \mathbf{e}_i \frac{\Delta\mathbf{x}}{\Delta t}$$

So $\mathbf{e}_1 = (1, 0)$ is a one-node hop on the grid, and $\mathbf{c}_1 = (\Delta\mathbf{x}/\Delta t, 0)$ is the physical speed of that hop. Throughout this chapter, we use $\mathbf{e}_i$ when writing velocity tables, streaming rules, and the quadrature formula — anything that lives in lattice space. We use $\mathbf{c}_i$ when connecting to physical quantities: the speed of sound $\mathbf{c}_s$, the τ - D diffusion relationship, and the equilibrium distribution in fluid-flow problems. A natural worry: if $\mathbf{c}_i$ changes when you refine your grid (because $\Delta\mathbf{x}/\Delta t$ changes), do the weights w_i change too? They do not. The weights are married to $\mathbf{e}_i$, not to $\mathbf{c}_i$, and $\mathbf{e}_i$ is a fixed set of integer vectors that never changes. When you halve $\Delta\mathbf{x}$, every physical velocity $\mathbf{c}_i$ shrinks — but the lattice speed of sound $\mathbf{c}_s$ shrinks by the same factor, so the dimensionless ratio $\mathbf{c}_i/\mathbf{c}_s = \mathbf{e}_i/\sqrt{3}$ is unchanged. The Gaussian in lattice units always has the same shape, the sample points sit in the same places relative to its width, and the weights stay fixed. Grid refinement changes the physics that $\mathbf{c}_s$ maps to, not the quadrature itself.

The Discrete Equilibrium

For **pure diffusion** ($\mathbf{u} = 0$), the Gaussian is centered at the origin. When you evaluate it at the quadrature nodes and fold in the weights, all the velocity-dependent structure vanishes. The discrete equilibrium is simply:

$$f_i^{\text{eq}} = w_i \phi$$

(18.18)

where $\phi = \sum_j f_j$ is the local macroscopic quantity (temperature, voltage, concentration). Each direction carries a fraction w_i of the total field — the rest direction carries the most (it sits at the peak of the bell curve), and the moving directions carry the remainder. There is no preferred direction — the equilibrium is symmetric, just as the centered Gaussian is symmetric. This is the equilibrium we use for cardiac electrophysiology, since the monodomain and bidomain equations are diffusion equations with no bulk flow.

Gauss-Hermite Quadrature: Why These Specific Nodes and Weights

The quadrature rule in equation (18.17) has a classical name: **Gauss-Hermite quadrature**. It is the standard method for integrating polynomials multiplied by a Gaussian kernel, and it has been studied since the 19th century. The key result is this: given a Gaussian in D dimensions, there exist specific sets of nodes and weights such that the finite sum is *exact* (not approximate) for all polynomial integrands up to a certain degree.

Intuition

Nodes and Weights Are a Married Pair: In Gauss-Hermite quadrature, you do not get to choose the nodes and the weights independently. Once you pick a set of sample points on the bell curve, the weights are uniquely determined (and vice versa). This is why every lattice specification — D2Q5, D2Q9, D3Q7 — always lists both $\mathbf{e}_i$ AND w_i together. They are a package deal, born from the same Gaussian.

For **diffusion** (no bulk flow), the equilibrium is the Gaussian centered at zero. The macroscopic equations involve integrals of polynomials up to second degree in $\mathbf{c}$, so exactness through second order is sufficient. D2Q5 and D3Q7 satisfy this. For **fluid flow** (Navier-Stokes), the equilibrium is a shifted Gaussian, and higher-order polynomial terms appear. The quadrature must also be exact for fourth-degree polynomials, which requires more lattice directions (D2Q9 instead of D2Q5).

Discrete vs. Continuous: Lattice Naming Convention

Lattices are named using the notation $\mathbf{D}_d\mathbf{Q}_q$, where:

- $\mathbf{D}$ = spatial dimension (1, 2, or 3)
- $\mathbf{Q}$ = number of lattice directions (discrete velocities)

Common examples:

- **D1Q3:** 1D, 3 velocities (left, right, rest)
- **D2Q5:** 2D, 5 velocities (rest + 4 cardinal neighbors)
- **D2Q9:** 2D, 9 velocities (rest + 4 cardinal + 4 diagonal)
- **D3Q7:** 3D, 7 velocities (rest + 6 face-centered neighbors)
- **D3Q19:** 3D, 19 velocities (face-centered + edge-adjacent)
- **D3Q27:** 3D, 27 velocities (complete cubic stencil)

The choice of lattice is determined by the physics:

- **For diffusion** (no bulk flow), D2Q5 or D3Q7 suffice. These have only cardinal directions.
- **For fluid flow** (Navier-Stokes), diagonal directions are needed to represent the full stress tensor. D2Q9 or D3Q19 are required.

Since cardiac electrophysiology primarily solves diffusion problems (the monodomain and bidomain equations), **D2Q5 and D3Q7 are the natural choices**.

△ Key Insight

Memory vs. Physics: The number of distributions stored per node equals Q . A D2Q9 lattice requires storing 9 values per node; a D2Q5 requires only 5. For a $500 \times 500 \times 500$ 3D mesh, a D2Q5 saves a factor of $(7 \text{ directions}) / (27 \text{ directions}) \approx 4 \times$ memory compared to D3Q27, even though physically, D3Q7 produces the same diffusion results. This is one reason LBM for cardiac problems uses minimal lattices.

The Discrete Velocity Set

With the quadrature framework in mind, here are the standard lattices. Each is a specific Gauss-Hermite quadrature rule on a regular grid — the nodes are placed at grid points so that streaming is a simple data shift. **D1Q3 (One-Dimensional):**

$$e_0 = 0, \quad e_1 = 1, \quad e_2 = -1$$

(18.19)

Three quadrature nodes in one dimension: one at the center of the bell curve (rest), one hop to the right, one hop to the left. The physical velocities are $c_i = e_i \cdot \Delta x / \Delta t$. **D2Q5 (Two-Dimensional, Cardinal Directions):**

i	Direction	e_i
0	Rest	$(0, 0)$
1	East	$(1, 0)$
2	West	$(-1, 0)$
3	North	$(0, 1)$
4	South	$(0, -1)$

Five quadrature nodes: one at the peak of the 2D bell curve (rest), four at one lattice spacing along each cardinal direction. These five sample points are sufficient to exactly integrate second-degree polynomials against the 2D Gaussian.

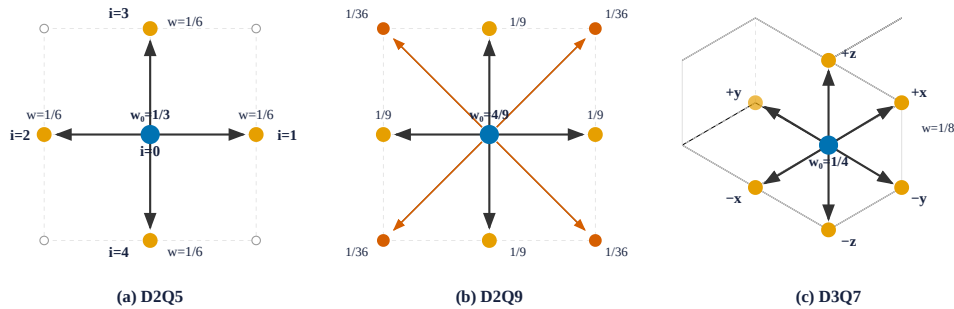


Figure 18.1: Common lattice stencils for LBM. (a) D2Q5: five velocities (rest + four cardinal) — sufficient for isotropic diffusion. (b) D2Q9: nine velocities (rest + cardinal + diagonal) — required for fluid flow to capture cross-velocity terms. (c) D3Q7: seven velocities in 3D (rest + six face-centered) — the minimal 3D stencil for diffusion. Blue center node = rest velocity (peak of the Gaussian); orange = cardinal neighbors; vermilion = diagonal neighbors. Arrows indicate streaming directions; weights w_i annotated at each node.

D2Q9 (Two-Dimensional, Cardinal + Diagonal): For fluid flow, the polynomial expansion of the shifted Gaussian contains terms like $e_{ix}e_{iy}$ (cross-products of velocity components). To integrate these exactly, we need diagonal quadrature nodes. D2Q9 adds four diagonal directions to D2Q5:

i	Direction	$\mathbf{e}_i$
0	Rest	$(0, 0)$
1	East	$(1, 0)$
2	West	$(-1, 0)$
3	North	$(0, 1)$
4	South	$(0, -1)$
5	NE	$(1, 1)$
6	NW	$(-1, 1)$
7	SE	$(1, -1)$
8	SW	$(-1, -1)$

Directions 1–4 are the same cardinal nodes as D2Q5. Directions 5–8 are the diagonal nodes. In lattice units, they sit at distance $\sqrt{2}$ from the center (vs. distance 1 for the cardinal nodes). They are further out on the bell curve, sampling the Gaussian's tails in diagonal directions. **D3Q7 (Three-Dimensional, Face-Centered)**: The 3D analogue of D2Q5. One quadrature node at the peak, six at one lattice spacing along each coordinate axis:

i	Direction	$\mathbf{e}_i$
0	Rest	$(0, 0, 0)$
1	$+x$	$(1, 0, 0)$
2	$-x$	$(-1, 0, 0)$
3	$+y$	$(0, 1, 0)$
4	$-y$	$(0, -1, 0)$
5	$+z$	$(0, 0, 1)$
6	$-z$	$(0, 0, -1)$

All six moving nodes sit at the same distance (one lattice spacing) from the center — they sample the Gaussian at the same height on the bell curve, which is why they all receive the same weight.

The Weight Lattice

The weights paired with each velocity set are determined by Gauss-Hermite quadrature — equation (18.17). Each weight w_i represents how much of the Gaussian bell curve the quadrature node $\mathbf{e}_i$ is responsible for. Nodes near the peak of the Gaussian (where the bell curve is tallest) represent more of the total area and receive larger weights. Nodes further out (where the bell curve has decayed) represent less area and receive smaller weights.

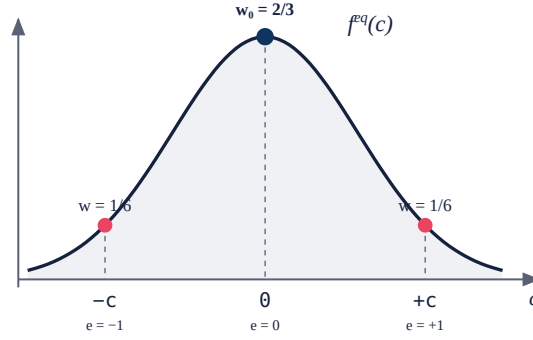


Figure 18.2. The Maxwell–Boltzmann equilibrium drawn as a bell curve over velocity c . The smooth Gaussian is the continuous equilibrium $f^{\text{eq}}(c)$ of §18.1 (equation 18.6); the lattice never stores it directly. Instead it samples the curve at a few points — here the three D1Q3 velocities $c_i \in \{-c, 0, +c\}$, which are the Gauss–Hermite quadrature nodes of §18.4. The rest node sits at the peak and carries the largest weight ($w_0 = 2/3$); the two moving nodes sit lower on the curve, where the Gaussian has decayed, and carry less ($w_1 = w_2 = 1/6$). Weighting these samples reproduces the integrals of the Gaussian exactly (equation 18.17), which is why a handful of points can stand in for the entire continuous distribution.

D1Q3 weights:

$$w_0 = \frac{2}{3}, \quad w_1 = w_2 = \frac{1}{6} \quad (18.20)$$

The rest node sits at the peak of the 1D bell curve — it captures two-thirds of the total area. The two moving nodes, one lattice spacing away from the peak, each capture one-sixth. Most of the Gaussian's area is concentrated near the center, which is why the rest weight dominates. **D2Q5 weights:**

$$w_0 = \frac{1}{3}, \quad w_1 = w_2 = w_3 = w_4 = \frac{1}{6} \quad (18.21)$$

The rest node captures one-third of the 2D bell curve. Each cardinal node captures one-sixth. The total is $1/3 + 4 \times 1/6 = 1$. All four cardinal nodes are at the same distance from the peak, so they receive the same weight — the bell curve is symmetric. **D2Q9 weights:**

$$w_0 = \frac{4}{9}, \quad w_{1,2,3,4} = \frac{1}{9} \quad (\text{cardinal}), \quad w_{5,6,7,8} = \frac{1}{36} \quad (\text{diagonal}) \quad (18.22)$$

Three distinct weight classes, directly reflecting the Gaussian's shape. The rest node at the peak captures $4/9$ of the area — nearly half. The cardinal nodes at distance 1 (in lattice units) each capture $1/9$. The diagonal nodes at distance $\sqrt{2}$ each capture only $1/36$ — four times less than a cardinal node. This factor of four is the Gaussian at work: moving from distance c to $\sqrt{2}c$ from the peak, the bell curve has decayed, so those nodes represent less area. Check: $4/9 + 4 \times 1/9 + 4 \times 1/36 = 4/9 + 4/9 + 1/9 = 1 \checkmark$.

Intuition

Why Diagonal Weights Are Smaller: Picture a 2D bell curve — a smooth hill centered at the origin. The cardinal quadrature nodes sit at distance 1 from the peak (along the axes). The diagonal nodes sit at distance $\sqrt{2}$ (along the diagonals). Because the Gaussian decays exponentially with distance from the center, the bell curve is lower at the diagonal nodes than at the cardinal nodes. Lower height means less area to represent, which means a smaller quadrature weight. This is not a choice — it is a consequence of the Gaussian's shape. The ratio $w_{\text{diagonal}}/w_{\text{cardinal}} = (1/36)/(1/9) = 1/4$ is determined by the exactness conditions, but the physical reason is the Gaussian decay.

D3Q7 weights:

$$w_0 = \frac{1}{4}, \quad w_1 = \dots = w_6 = \frac{1}{8} \quad (18.23)$$

The rest node captures one-quarter of the 3D bell curve. Each of the six face-centered nodes captures one-eighth. All moving nodes are at the same distance c from the peak (same height on the bell curve), so they receive equal weight. Check: $1/4 + 6 \times 1/8 = 1/4 + 3/4 = 1 \checkmark$.

Mathematical Note

Deriving D2Q5 Weights from Quadrature: For D2Q5, we have two unknowns: the rest weight w_0 and the cardinal weight w_c (shared by directions 1–4 by symmetry). Two quadrature exactness conditions pin them down. Plugging $\mathbf{p} = \mathbf{1}$ into (18.17): $w_0 + 4w_c = 1$ (the weights must sum to one). Plugging $\mathbf{p} = \mathbf{e}_{ix}^2$ into (18.17): $\sum_i w_i e_{ix}^2 = 1/3$. The rest node ($\mathbf{e}_{0x} = \mathbf{0}$) contributes zero. Directions 1 and 2 ($\mathbf{e}_{ix} = \pm 1$) each contribute $w_c \cdot 1$. Directions 3 and 4 ($\mathbf{e}_{ix} = \mathbf{0}$) contribute nothing. This gives $2w_c = 1/3$, yielding $w_c = 1/6$. Substituting back: $w_0 = 1 - 4(1/6) = 1/3$. The weights are not chosen — they are uniquely determined by Gauss-Hermite quadrature on this lattice geometry.

The Navier-Stokes Equilibrium

For **fluid flow** (Navier-Stokes), the Gaussian is centered at the bulk velocity $\mathbf{u}$ rather than at zero. Expanding the exponential in (18.2) as a Taylor series in $\mathbf{u}/c_s$ (valid when the flow speed is much smaller than the speed of sound — the low Mach number regime), the discrete equilibrium becomes:

$$f_i^{\text{eq}} = w_i \rho \left(1 + \frac{\mathbf{u} \cdot \mathbf{c}_i}{c_s^2} + \frac{(\mathbf{u} \cdot \mathbf{c}_i)^2}{2c_s^4} - \frac{u^2}{2c_s^2} \right) \quad (18.24)$$

The terms beyond $w_i \rho$ are the polynomial corrections that arise from shifting the bell curve's center from zero to $\mathbf{u}$. These terms break the directional symmetry — they bias the distribution toward the direction of flow. Because these corrections involve products like $\mathbf{c}_{ix} \mathbf{c}_{iy}$, the quadrature must exactly integrate fourth-degree polynomials, which is why fluid flow requires more quadrature nodes (D2Q9 instead of D2Q5).

Lattice Requirements for Different Physics

The choice of lattice is determined by how many terms in the polynomial expansion of the Gaussian need to be captured exactly. For diffusion with the isotropic equilibrium (18.18), only terms up to second degree matter — the Gauss-Hermite quadrature (18.17) need only be exact through second order. D2Q5 (and D3Q7 in 3D) satisfy these, making them the most efficient choices for cardiac problems. For fluid flow with the velocity-dependent equilibrium (18.24), terms up to fourth degree appear. The quadrature must be exact for fourth-order products like $\sum_i w_i \mathbf{e}_{i\alpha} \mathbf{e}_{i\beta} \mathbf{e}_{i\gamma} \mathbf{e}_{i\delta}$. D2Q5 does not have enough nodes, but D2Q9 does. In cardiac electrophysiology, we solve diffusion (not Navier-Stokes), so D2Q5 and D3Q7 are

sufficient. Using a minimal lattice saves both memory (Q distributions per node) and computation (fewer collision and streaming operations per timestep).

Lattice Speed of Sound and the τ - D Relationship

For each lattice, there is a characteristic speed c_s called the **lattice speed of sound**. From the Gauss-Hermite exactness conditions and the lattice geometry, we have:

$$c_s^2 = \frac{c^2}{3} \quad (18.25)$$

The parameter c_s plays a dual role. In the continuous Gaussian of Section 18.1, c_s^2 is the **variance** — it controls the width of the bell curve. On the lattice, this same c_s becomes the **lattice speed of sound**, with the specific value $c_s^2 = c^2/3$ where $c = \Delta x / \Delta t$ is the lattice speed. This is not a coincidence: the lattice speed of sound is inherited directly from the variance of the Maxwellian that the lattice discretizes. Every time you see c_s^2 in an LBM formula — in the weights, in the equilibrium, in the τ - D relationship — it traces back to this Gaussian width. The speed of sound c_s is just a lattice constant — it emerges from the geometry and isotropy conditions. However, it plays a crucial role in relating the relaxation time τ to the physical diffusivity D . Through Chapman-Enskog analysis, one can show that for the BGK collision operator applied to a diffusion problem, the relationship is:

$$D = c_s^2 \left(\tau - \frac{\Delta t}{2} \right) \quad (18.26)$$

or, rearranging:

$$\tau = \frac{\Delta t}{2} + \frac{D}{c_s^2} \quad (18.27)$$

Substituting $c_s^2 = c^2/3 = \Delta x^2 / (3\Delta t^2)$:

$$\tau = \frac{\Delta t}{2} + \frac{3D\Delta t}{\Delta x^2} \quad (18.28)$$

This is the fundamental bridge between the discrete lattice parameter τ and the physical diffusivity D .

Practical meaning: To solve a problem with a given physical diffusivity D , you:

1. Choose your spatial and temporal resolution: Δx and Δt .
2. Compute τ from equation (18.28).
3. Use that τ in the collision step (equation 18.14).

Note the stability constraint: $\tau > \Delta t/2$ is required for stability. This places an upper bound on D for a given Δx and Δt . If you need very high diffusivity, you must either reduce Δx or accept a smaller Δt , which increases the total number of timesteps.

△ Key Insight

Stability Constraint: The requirement $\tau > \Delta t/2$ comes from the local truncation error analysis of the BGK collision operator. If $\tau \leq \Delta t/2$, the relaxation becomes too aggressive and unphysical oscillations grow. This is similar to the CFL condition in hyperbolic PDEs — it is a fundamental constraint on the simulation. For a fixed spatial grid, high diffusivity requires a small Δt , which can make the simulation expensive.

18.5 Moment Space: The Physical Decomposition of the Collision Operator

The Problem: What Does the Collision Operator Actually Conserve?

Look again at the BGK collision equation we derived in Section 18.3:

$$f_i^*(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)] \quad (18.29)$$

This equation relaxes each distribution f_i toward its equilibrium value f_i^{eq} , all at the same rate $1/\tau$. It works — we showed in Section 18.4 that the equilibrium was carefully designed to encode the right physics. But ask yourself: what *guarantees* that this relaxation conserves mass? What guarantees it produces the correct diffusion coefficient? What guarantees the stress tensor has the right symmetry? The answers are hiding inside f^{eq} , but equation (18.29) doesn't show them. Each f_i is a single number that mixes together information about density, flux, stress, and energy — all tangled up. When BGK relaxes f_1 toward f_1^{eq} , it is simultaneously relaxing *every* physical quantity that f_1 carries, at the same rate. You can't see which conservation laws are being enforced, and you have no way to control them independently. Is there a way to **see** what physical laws the collision operator is actually enforcing — and to **verify** them directly?

The Macroscopic PDE Dictates Which Moments Matter

The answer starts not with the lattice, but with the PDE you are trying to solve. Consider the diffusion equation:

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (D \nabla \phi) \quad (18.30)$$

This PDE makes three specific physical demands, and each one maps to a weighted sum of the distributions: **Demand 1 — Conservation of mass (or charge, or heat):** The total amount of ϕ at a node does not change during collision. Only streaming moves ϕ between nodes. This means:

$$\sum_{i=0}^{Q-1} f_i^* = \sum_{i=0}^{Q-1} f_i \quad (18.31)$$

The sum $\sum_i f_i$ is a weighted sum of the distributions with weight 1 for every direction. This is the **zeroth moment** — the simplest possible summary of the distributions. You take it because *mass conservation demands it*. **Demand 2 — Fick's law (diffusive flux):** The macroscopic flux must satisfy $\mathbf{J} = -D \nabla \phi$. In the lattice-Boltzmann framework, the flux emerges from:

$$J_\alpha = \sum_{i=0}^{Q-1} c_{i\alpha} f_i \quad (18.32)$$

These sums weight each distribution by its velocity component $c_{i\alpha}$. They are the **first moments**. You take them because *Fick's law demands them* — the diffusion coefficient is encoded in how fast these flux quantities relax toward equilibrium. **Demand 3 — Correct tensor structure:** For the diffusion to be isotropic (or anisotropic in exactly the right way), the stress-like quantity:

$$\Pi_{\alpha\beta} = \sum_{i=0}^{Q-1} c_{i\alpha} c_{i\beta} f_i \quad (18.33)$$

must have the correct symmetry. These are the **second moments**. You take them because *the diffusion tensor structure demands them*. For isotropic diffusion, $\Pi_{\alpha\beta}$ must be proportional to $\delta_{\alpha\beta}$ at equilibrium. For anisotropic diffusion, its structure must match the physical diffusion tensor.

Intuition

The Rule: Each conservation law or transport property in your target PDE maps to a specific weighted sum of the f_i . These weighted sums *are* the moments. You don't choose them for mathematical convenience — the physics tells you which ones matter. If you are solving a diffusion equation, you need the zeroth moment (conservation), the first moments (flux), and the second moments (tensor structure). If you were solving Navier-Stokes, the same zeroth and first moments would enforce mass and momentum conservation, but the second moments would need to capture the viscous stress tensor, and you might need third- and fourth-order moments for the energy equation.

What Are Moments?

The pattern is now clear. A **moment** is a weighted sum of the distribution functions, where the weights are chosen to extract a specific macroscopic quantity. In the most general form:

$$m_k = \sum_{i=0}^{Q-1} M_{ki} f_i \quad (18.34)$$

where M_{ki} is the weight that the k -th moment assigns to the i -th distribution. This is not unique to LBM — it is the same concept that appears across all of engineering:

- **Zeroth moment** ($M_{0i} = 1$ for all i): total density, total charge, total energy. The conserved quantity in any transport equation.
- **First moments** ($M_{ki} = c_{i\alpha}$): momentum flux in fluids, diffusive flux in electrophysiology, heat flux in conduction. The direction and rate at which "stuff" moves.
- **Second moments** ($M_{ki} = c_{i\alpha} c_{i\beta}$): the viscous stress tensor in fluid mechanics, the diffusion tensor in EP, the energy density in heat transfer. These control how transport varies with direction.
- **Higher moments** (M_{ki} involves cubic or higher products of $c_{i\alpha}$): in most diffusion and low-Mach flow problems, no conservation law requires them. They exist because the lattice has Q degrees of freedom but only a few physical constraints. These are the **ghost moments** — mathematically necessary to fill out the Q -dimensional space, but carrying no macroscopic physical content.

Building **M**: Each Row Is a Conservation Law

We can collect all the moment definitions into a single matrix equation:

$$\mathbf{m} = \mathbf{M} \mathbf{f} \quad (18.35)$$

where $\mathbf{m} = (m_0, m_1, \dots, m_{Q-1})^T$ is the vector of moments and $\mathbf{f} = (f_0, f_1, \dots, f_{Q-1})^T$ is the vector of distributions. The matrix $\mathbf{M}$ is the **transformation matrix** — each row is the recipe for extracting one physical quantity from the distributions. For **D2Q5**, we have $Q = 5$ distributions and therefore need a 5×5 matrix. The rows are constructed directly from the physics:

Row	Moment	Physical meaning	Why we need it
0	$\sum_i f_i$	Density (conserved scalar ϕ)	Mass/charge conservation
1	$\sum_i c_{ix} f_i$	x -flux	Fick's law in x
2	$\sum_i c_{iy} f_i$	y -flux	Fick's law in y
3	$\sum_i c_i ^2 f_i$	Energy	Ghost — no PDE requires it
4	$\sum_i (c_{ix}^2 - c_{iy}^2) f_i$	Energy difference (stress trace)	Ghost — no PDE requires it

The explicit 5×5 matrix, using the D2Q5 velocity vectors $(c_{ix}, c_{iy}) = (0, 0), (1, 0), (-1, 0), (0, 1), (0, -1)$:

$$\mathbf{M}_{\text{D2Q5}} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & -1 & -1 \end{pmatrix} \quad (18.36)$$

Read it row by row. Row 0: every entry is 1, so this row sums all f_i — that's the density. Row 1: the entries are the x -components of the velocity vectors, so this row computes the x -flux. Row 2: the y -components — the y -flux. Rows 3 and 4: combinations of c_{ix}^2 and c_{iy}^2 that complete the basis. The first three rows are *demanded by the physics*. The last two rows are chosen to make $\mathbf{M}$ invertible (so we can transform back), but any linearly independent choice would work. For **D3Q7**, the 7×7 transformation matrix from the Rapaka et al. LBM-EP paper follows the same logic. With velocity ordering $(0, 0, 0), (1, 0, 0), (-1, 0, 0), (0, 1, 0), (0, -1, 0), (0, 0, 1), (0, 0, -1)$:

$$\mathbf{M}_{\text{D3Q7}} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & -1 \\ -6 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & -2 & -2 \end{pmatrix} \quad (18.37)$$

Rows 0–3: density and the three flux components (x, y, z) — required by mass conservation and Fick's law in 3D. Rows 4–6: ghost moments with no macroscopic counterpart, chosen to complete the 7×7 basis.

Mathematical Note

Why the ghost rows look arbitrary: The first rows of $\mathbf{M}$ are dictated by physics — row 0 must be all ones (density), rows 1– D must use the velocity components (flux). The remaining $Q - 1 - D$ rows just need to be linearly independent from the physical rows. Different textbooks choose different ghost rows, and *all valid choices produce the same physical solution*. The ghost rows affect only numerical stability, not the macroscopic PDE that the scheme recovers.

The Round Trip: Rewriting the Collision Operator in Moment Space

Now we have the tool to see inside the collision operator. The key idea is simple: instead of relaxing the $\mathbf{f}_i$ directly (where every physical quantity is tangled together), we **decode** the distributions into moments, **relax** each moment separately, then **re-encode** back to distributions for streaming. **Step 1 — Decode:** Transform the distributions into moments:

$$\mathbf{m} = \mathbf{M} \mathbf{f} \quad (18.38)$$

Now $\mathbf{m}_0$ is the density, $\mathbf{m}_1$ is the x -flux, $\mathbf{m}_2$ is the y -flux, and so on. Each component has exactly one physical meaning. **Step 2 — Decode the equilibrium too:**

$$\mathbf{m}^{\text{eq}} = \mathbf{M} \mathbf{f}^{\text{eq}} \quad (18.39)$$

The equilibrium moments tell you the target state for each physical quantity. For the diffusion equilibrium $\mathbf{f}_i^{\text{eq}} = w_i \phi$, the equilibrium density is $\mathbf{m}_0^{\text{eq}} = \phi$ (all the field), the equilibrium fluxes are $\mathbf{m}_1^{\text{eq}} = \mathbf{m}_2^{\text{eq}} = 0$ (no net flow at equilibrium), and the ghost moments take whatever values w_i and the lattice geometry dictate. **Step 3 — Relax in moment space:**

$$\mathbf{m}^* = \mathbf{m} - \mathbf{S} (\mathbf{m} - \mathbf{m}^{\text{eq}}) \quad (18.40)$$

where $\mathbf{S} = \text{diag}(s_0, s_1, \dots, s_{Q-1})$ is the **relaxation matrix**. Each diagonal entry s_k controls how fast moment k relaxes toward its equilibrium value. This is the step where the physics is enforced:

- For the conserved moment (density): $s_0 = 0$. The density is never changed by collision — it is a conserved quantity. Setting $s_0 = 0$ means $\mathbf{m}_0^* = \mathbf{m}_0$ exactly, regardless of what happens to the other moments.
- For the flux moments: s_1 and s_2 control how fast the flux relaxes, which determines the diffusion coefficient. In the simplest case (isotropic BGK), $s_1 = s_2 = \Delta t / \tau$.
- For the ghost moments: $s_3, s_4, \dots$ can be chosen freely — they do not affect the macroscopic PDE, only the numerical behavior of the scheme.

Step 4 — Re-encode: Transform back to distribution space:

$$\mathbf{f}^* = \mathbf{M}^{-1} \mathbf{m}^* \quad (18.41)$$

Now $\mathbf{f}^*$ is ready for streaming. The distributions have been updated in a way that *explicitly* conserves density, *explicitly* relaxes flux at a controlled rate, and handles the ghost moments separately.

Intuition

The destination vs. the journey: It is worth pausing to see how $\mathbf{f}^{\text{eq}}$ and moment space play complementary roles. The equilibrium $\mathbf{f}_i^{\text{eq}}$ defines the **destination** — the state the distributions are relaxing toward. It encodes *what* the physics looks like at equilibrium (isotropic for diffusion, velocity-biased for fluid flow). Moment space governs the **journey** — *how* the distributions get to that destination and *what is preserved along the way*. By decomposing the non-equilibrium deviation $\mathbf{f} - \mathbf{f}^{\text{eq}}$ into physically meaningful components, you can verify that conservation laws are respected, that transport coefficients are correct, and that numerical artifacts are controlled.

BGK Is a Special Case: All Dials Locked Together

Here is the crucial connection. Set $\mathbf{S} = \frac{\Delta t}{\tau} \mathbf{I}$ (the identity matrix times a single rate). Then the moment-space collision (18.40) becomes:

$$\mathbf{m}^* = \mathbf{m} - \frac{\Delta t}{\tau} (\mathbf{m} - \mathbf{m}^{\text{eq}})$$

and the full round trip is:

$$\mathbf{f}^* = \mathbf{M}^{-1} \left[\mathbf{M} \mathbf{f} - \frac{\Delta t}{\tau} \mathbf{M} (\mathbf{f} - \mathbf{f}^{\text{eq}}) \right] = \mathbf{f} - \frac{\Delta t}{\tau} (\mathbf{f} - \mathbf{f}^{\text{eq}})$$

The $\mathbf{M}^{-1}$ and $\mathbf{M}$ cancel. We recover exactly the BGK equation (18.29). **BGK was always operating in moment space** — it just didn't need to make the round trip explicit because every moment was being relaxed at the same rate. The single τ locks all the dials together: conservation, transport, and ghost moments all receive the same treatment. This is why BGK works for isotropic diffusion: when $\mathbf{s}_1 = \mathbf{s}_2$, the $\mathbf{x}$ - and $\mathbf{y}$ -flux moments relax at the same rate, producing equal diffusion in all directions. And this is why BGK *cannot* produce anisotropic diffusion: there is no way to set $\mathbf{s}_1 \neq \mathbf{s}_2$ when $\mathbf{S}$ is constrained to be a scalar multiple of the identity.

Key Insight

Conservation in BGK — a hidden accident: In BGK, the conserved moment (density) formally receives the same relaxation rate $\mathbf{s}_0 = \Delta t / \tau$ as every other moment. But conservation is not violated — because $\mathbf{f}^{\text{eq}}$ is designed so that $\mathbf{m}_0^{\text{eq}} = \mathbf{m}_0$ (the equilibrium density equals the actual density), the relaxation step computes $\mathbf{m}_0^* = \mathbf{m}_0 - \frac{\Delta t}{\tau} (\mathbf{m}_0 - \mathbf{m}_0) = \mathbf{m}_0$. Conservation holds, but only because the equilibrium was carefully constructed to make it so. In the moment-space view, you enforce conservation *explicitly* by setting $\mathbf{s}_0 = \mathbf{0}$ rather than relying on this cancellation.

What Each Moment Sees During Collision

To make this concrete, consider a D2Q5 node mid-simulation. The distributions $\mathbf{f}_i$ are some non-equilibrium state (a diffusion front is passing through). When we decode:

- $\mathbf{m}_0 = \sum_i \mathbf{f}_i = \phi$ — the local voltage. After collision with $\mathbf{s}_0 = \mathbf{0}$: $\mathbf{m}_0^* = \phi$. Unchanged. Mass conservation enforced.
- $\mathbf{m}_1 = \sum_i c_{ix} \mathbf{f}_i = J_x$ — the $\mathbf{x}$ -flux. Non-zero because a diffusion front creates a gradient. After collision: $\mathbf{m}_1^* = J_x - \mathbf{s}_1 (J_x - 0) = (1 - \mathbf{s}_1) J_x$. The flux decays toward zero (equilibrium has no net flux). The rate $\mathbf{s}_1$ determines how fast — which sets the diffusion coefficient $\mathbf{D}$.
- $\mathbf{m}_2 = \sum_i c_{iy} \mathbf{f}_i = J_y$ — same story for $\mathbf{y}$.
- $\mathbf{m}_3, \mathbf{m}_4$ — the ghost moments. They relax too, but toward values dictated by the lattice weights, not by any conservation law. Their rates $\mathbf{s}_3, \mathbf{s}_4$ are free parameters.

This decomposition reveals the internal structure that equation (18.29) hides. You can now **verify** conservation (is $\mathbf{m}_0$ really unchanged?), **measure** the flux relaxation rate (does it match your desired $\mathbf{D}$?), and **tune** the ghost moments independently of the physics.

Worked Example: One D2Q5 Collision in Both Spaces

Let us make the round trip concrete with numbers. Consider a D2Q5 node with $\phi = 1.0$, $\Delta x = 0.025$ cm, $\Delta t = 0.01$ ms, and isotropic diffusivity $D = 0.001$ cm²/ms. From equation (18.28):

$$\tau = \frac{\Delta t}{2} + \frac{3D\Delta t}{\Delta x^2} = 0.005 + \frac{3(0.001)(0.01)}{0.000625} = 0.005 + 0.048 = 0.053 \text{ ms}$$

so $\omega = \Delta t/\tau = 0.01/0.053 \approx 0.189$. Suppose the current distributions are (a diffusion front has created a slight asymmetry):

$$\mathbf{f} = (0.340, 0.175, 0.155, 0.168, 0.162)^T$$

Check: $\sum f_i = 1.000 = \phi$. Good. The equilibrium is

$f_i^{\text{eq}} = w_i \phi = (1/3, 1/6, 1/6, 1/6, 1/6) \approx (0.333, 0.167, 0.167, 0.167, 0.167)$. BGK (distribution space):

$$f_i^* = f_i - 0.189(f_i - f_i^{\text{eq}})$$

Computing each component:

i	f_i	f_i^{eq}	$f_i - f_i^{\text{eq}}$	f_i^*
0	0.340	0.333	+0.007	0.3387
1	0.175	0.167	+0.008	0.1735
2	0.155	0.167	-0.012	0.1573
3	0.168	0.167	+0.001	0.1678
4	0.162	0.167	-0.005	0.1629

Check: $\sum f_i^* = 1.0002 \approx 1.000$. Conservation holds (the small deviation is rounding). **Moment space** (same collision, decoded): Apply $\mathbf{m} = \mathbf{M}\mathbf{f}$:

k	Moment	m_k	m_k^{eq}	$m_k^* (s_k = \omega)$	Physical role
0	ρ	1.000	1.000	1.000	Conserved — unchanged
1	J_x	+0.020	0	+0.0162	Flux decays $\rightarrow$ sets D_x
2	J_y	+0.006	0	+0.0049	Flux decays $\rightarrow$ sets D_y
3	e	0.660	0.667	0.6613	Ghost — no PDE role
4	ϵ	-0.000	0	-0.000	Ghost — no PDE role

Now you can see the physics. The density ($m_0 = 1.000$) is untouched — conservation is enforced. The x -flux ($m_1 = 0.020$) relaxes toward zero at rate ω — this sets the diffusion coefficient. The ghost moments relax too, but they don't affect the macroscopic solution. In BGK, all moments happen to use the same ω . In the moment-space view, you can verify each constraint independently. Applying $\mathbf{f}^* = \mathbf{M}^{-1}\mathbf{m}^*$ gives back the same $\mathbf{f}^*$ as the BGK calculation above — the round trip is exact.

Intuition

What we have gained: The moment-space formulation does not change the physics — for BGK with a single τ , the result is identical. What it changes is **visibility**. You can now see which quantity is conserved, which quantities control transport, and which quantities are free parameters with no physical role. This visibility is the foundation for everything that follows: anisotropic diffusion (relaxing x - and y -flux at different rates), stability tuning (adjusting ghost-moment rates without affecting the solution), and the full MRT collision operator.

What Comes Next

We now have the complete theoretical framework for the lattice-Boltzmann method: the thermodynamic foundation (Section 18.1), the Boltzmann equation and BGK simplification (Section 18.2), the discrete lattice-Boltzmann equation (Section 18.3), the equilibrium distribution and its constraints on weights and velocities (Section 18.4), and the moment-space decomposition that reveals the physical content of the collision operator (Section 18.5). In the next chapter we put this machinery to work on the cardiac monodomain equation: the $\mathbf{f}_i$ become our fundamental variables, and — building on the moment-space view of Section 18.5 — we unlock the individual moment relaxation rates to achieve anisotropic diffusion (the Multiple-Relaxation-Time collision operator). Chapter 20 then extends the method to the bidomain equations.

Intuition

Complete Reference Tables: Appendix A collects the velocity vectors $\mathbf{c}_i$ and weights w_i for all standard lattices (D1Q3, D2Q5, D2Q9, D3Q7, D3Q19, D3Q27) in a single reference table. These are derived from Gauss-Hermite quadrature (18.17) applied to the lattice geometries described in this chapter.

Lattice-Boltzmann Methods for Monodomain

Chapter 18 built the LBM machinery from first principles: distribution functions, lattice geometry, collision and streaming, the equilibrium distribution born from Gauss-Hermite quadrature, and the moment-space decomposition that reveals the physical content of the collision operator. Now we put that machinery to work on the cardiac monodomain equation. The key additions are surprisingly mechanical: an ionic source term injected into the collision step, operator splitting to separate reaction from diffusion, and bounce-back boundary conditions to handle tissue edges. For anisotropic tissue, we unlock the moment-space relaxation rates derived in Section 18.5 — but now we must connect them quantitatively to the cardiac conductivity tensor. This chapter derives that connection, presents the complete algorithm, and discusses the implementation choices that make LBM fast on GPUs.

19.1 The Monodomain Equation as a Target PDE

Our target is equation (7.1), the monodomain PDE that governs wave propagation in cardiac tissue:

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi I_{\text{ion}}(V_m, \mathbf{w}) + I_{\text{stim}}$$

The left-hand side is the rate of change of the transmembrane potential V_m , scaled by the surface-to-volume ratio χ and membrane capacitance C_m . On the right, three terms compete: diffusion ($\nabla \cdot (\mathbf{D} \nabla V_m)$) spreads the voltage through tissue, ionic currents (I_{ion}) from the cell membrane shape the action potential, and stimulus current (I_{stim}) initiates activation. Chapters 5 and 6 detailed the ionic models (TTP06 and ORd) that compute I_{ion} from V_m and the gating variables $\mathbf{w}$. Chapters 9 and 10 solved the diffusion side with explicit and implicit finite-difference methods. Now we solve it with LBM.

The diffusion tensor $\mathbf{D}$ encodes the anisotropy of cardiac tissue. In a 2D slab with fiber direction $\mathbf{a}_l = (\cos \theta, \sin \theta)$, the tensor takes the form:

$$\mathbf{D} = D_l \mathbf{a}_l \mathbf{a}_l^T + D_t (\mathbf{I} - \mathbf{a}_l \mathbf{a}_l^T) \quad (19.1)$$

where D_l is the along-fiber diffusivity and D_t is the cross-fiber diffusivity. Typical values are $D_l \approx 0.001 \text{ cm}^2/\text{ms}$ and $D_t \approx 0.00025 \text{ cm}^2/\text{ms}$ (a 4:1 ratio). In component form:

$$D_{xx} = D_l \cos^2 \theta + D_t \sin^2 \theta, \quad D_{yy} = D_l \sin^2 \theta + D_t \cos^2 \theta, \quad D_{xy} = (D_l - D_t) \sin \theta \cos \theta$$

For isotropic tissue ($D_l = D_t = D$), the tensor reduces to $D\mathbf{I}$ and the off-diagonal term vanishes. For aligned fibers ($\theta = 0$), we get $D_{xx} = D_l$, $D_{yy} = D_t$, $D_{xy} = 0$. The general case ($\theta \neq 0$, $D_l \neq D_t$) produces a full tensor with off-diagonal coupling.

Intuition

The two-part structure. The monodomain PDE has a clear two-part structure: *diffusion* (the divergence term, which spreads voltage through tissue) and *reaction* (the ionic and stimulus terms, which generate and shape the action potential locally). These two parts operate on vastly different scales — diffusion is a spatial coupling between neighboring nodes, while reaction is purely local (each node's ionic currents depend only on that node's voltage and gating variables). This separation is exactly what operator splitting exploits, and it maps perfectly onto the LBM framework: collision handles diffusion, and the ionic ODE handles reaction.

19.2 Operator Splitting: Reaction Then Diffusion

Following the Godunov splitting framework of Chapter 8, we solve the monodomain equation in two sequential substeps per timestep. The reaction substep is purely local — it updates V_m and $\mathbf{w}$ at each node independently. The diffusion substep couples neighbors through LBM collision and streaming.

Substep 1 — Reaction (ionic ODE): At each node, hold the diffusion term frozen and solve:

$$\chi C_m \frac{dV_m}{dt} = -\chi I_{\text{ion}}(V_m, \mathbf{w}) + I_{\text{stim}}, \quad \frac{d\mathbf{w}}{dt} = \mathbf{g}(V_m, \mathbf{w})$$

This is an ODE system at each node — identical to what we solve in Chapters 5 and 6. The Rush-Larsen method (equation 8.1) handles the gating variables $\mathbf{w}$, exploiting their exponential relaxation structure for accuracy and stability. After this substep, we have updated gating variables $\mathbf{w}^{n+1}$ and an intermediate voltage V_m^* that includes the ionic contribution but not yet the diffusion.

Substep 2 — Diffusion (LBM): Now solve the diffusion part $\partial V_m / \partial t = \nabla \cdot (\mathbf{D} \nabla V_m) / (\chi C_m)$ using the LBM collision-streaming cycle. The collision operator splits cleanly into two pieces that mirror the two-part physics of the monodomain equation:

- Ω^{NR} (**non-reactive collision**) — drives distributions toward equilibrium, producing diffusive flux. This is the LBM mechanism for the divergence term $\nabla \cdot (\mathbf{D} \nabla V_m)$.
- Ω^R (**reactive collision**) — injects the ionic source into the distribution. This is how the reaction substep's output couples back into the lattice populations.

The reactive collision needs a **source term** $\mathcal{S}$ that encapsulates the net effect of ionic currents and external stimulus:

$$\mathcal{S} = - \frac{I_{\text{ion}}(V_m^n, \mathbf{w}^n) + I_{\text{stim}}}{\chi C_m} \quad (19.2)$$

This source has units of voltage per unit time (mV/ms). The sign convention follows equation (7.1): inward current (depolarizing) makes $\mathcal{S} > 0$, outward current (repolarizing) makes $\mathcal{S} < 0$. The precise definitions of Ω^{NR} and Ω^R — and the physical reasons they take the forms they do — are derived in Section 19.3 (BGK) and Section 19.4 (MRT).

△ Key Insight

Splitting order matters. Godunov splitting (reaction first, then diffusion) is first-order accurate in time. Strang splitting (half-reaction, full diffusion, half-reaction) is second-order but requires two ionic evaluations per timestep. For cardiac LBM, first-order Godunov splitting is standard because the timestep Δt is already small (typically 0.005–0.02 ms) due to the lattice resolution requirements, and the splitting error is dominated by the spatial discretization error. See Chapter 8 for the full analysis.

19.3 BGK Collision with Source Term

The diffusion substep is where LBM does its work. Recall from Chapter 18 that each lattice node carries Q distribution functions f_i (five for D2Q5, seven for D3Q7). The macroscopic voltage is their sum: $V_m = \sum_i f_i$. The equilibrium distribution for a pure diffusion problem, derived from Gauss-Hermite quadrature in Section 18.4, is:

$$f_i^{\text{eq}} = w_i V_m$$

This is equation (18.18) with the scalar field ϕ replaced by the transmembrane potential V_m , and no advection ($\mathbf{u} = \mathbf{0}$). The weights w_i are the Gauss-Hermite quadrature weights from Section 18.4: for D2Q5, $w_0 = 1/3$ and $w_{1,2,3,4} = 1/6$.

The BGK collision with ionic source combines two physically distinct operations. We define each as a **rate** (per unit time), consistent with the continuous collision operator $\Omega(f)$ from equation (18.9). The **non-reactive collision rate** Ω_i^{NR} is the BGK relaxation toward equilibrium:

$$\Omega_i^{NR} = -\frac{1}{\tau} (f_i - w_i V_m)$$

This is a direct transcription of equation (18.9) with $f^{eq} = w_i V_m$. Physically, Ω_i^{NR} drives each distribution toward the local equilibrium at rate $1/\tau$. When $f_i > w_i V_m$ (more particles than equilibrium in direction i), the rate is negative — particles are removed. When $f_i < w_i V_m$, particles are added. This redistribution of particles among directions is diffusion: non-equilibrium structure in the distribution function corresponds to flux, and relaxing that structure spreads voltage through tissue. The τ - D relationship (equation 18.28) quantifies exactly how the relaxation rate maps to the macroscopic diffusion coefficient.

The **reactive collision rate** Ω_i^R injects the ionic source into the distribution:

$$\Omega_i^R = w_i S$$

This is a pure source injection: the reaction term S (equation 19.2) — which encapsulates all ionic currents and external stimulus — is distributed among lattice directions with weights w_i . Since $\sum_i w_i = 1$, the total injected rate is $\sum_i \Omega_i^R = S$, preserving the correct total source. The weights ensure isotropy: the source does not preferentially inject particles in any direction, because ionic currents have no spatial directionality — they flow across the cell membrane, not along any lattice vector.

The full collision update discretizes these rates with timestep Δt :

$$f_i^* = f_i + \Delta t (\Omega_i^{NR} + \Omega_i^R) = f_i - \frac{\Delta t}{\tau} (f_i - w_i V_m) + \Delta t w_i S \quad (19.3)$$

The separation is physically exact, not merely algebraic convenience. Ω^{NR} handles **transport** — the spatial coupling between neighboring nodes that produces the $\nabla \cdot (D \nabla V_m)$ term in the monodomain PDE. Ω^R handles **reaction** — the purely local ionic and stimulus currents that produce the $-\chi I_{ion} + I_{stim}$ terms. This mirrors the operator-splitting structure of Section 19.2, but now at the level of the collision operator itself. The decomposition carries directly into the MRT formulation (Section 19.4), where Ω^{NR} gains direction-dependent relaxation rates to model anisotropic diffusion, while Ω^R remains unchanged — because the ionic physics is isotropic.

The relaxation time τ is set by the τ - D relationship (equation 18.28):

$$\tau = \frac{\Delta t}{2} + \frac{3D \Delta t}{\Delta x^2}$$

For isotropic tissue with $D = 0.001$ cm²/ms, $\Delta x = 0.025$ cm, $\Delta t = 0.01$ ms: $\tau = 0.005 + 0.048 = 0.053$ ms. The collision rate is $\omega = \Delta t/\tau \approx 0.189$. Stability requires $\tau > \Delta t/2$, which holds for any positive diffusivity.

After collision, streaming (equation 18.15) shifts each post-collision distribution one lattice spacing in its designated direction:

$$f_i(\mathbf{x} + \mathbf{e}_i \Delta x, t + \Delta t) = f_i^*(\mathbf{x}, t)$$

No new mathematical machinery is required beyond what Chapter 18 established. The cardiac monodomain is solved by the same collide-and-stream cycle that solves any diffusion equation — the ionic model simply provides the source term.

Intuition

Physical interpretation. Think of the source $\mathcal{S}$ as a faucet (or drain) at each lattice node. When a stimulus depolarizes a cell ($\mathcal{S} > 0$), it adds voltage to the local particle population — distributed equally among all directions via the weights $\mathbf{w}_i$. The collision then relaxes the distribution toward equilibrium (spreading the perturbation), and streaming carries the information to neighboring nodes. This is how a depolarization wave propagates through the lattice: source injection $\rightarrow$ collision $\rightarrow$ streaming $\rightarrow$ recovery, repeated every timestep.

19.4 MRT for Anisotropic Tissue

Why BGK Fails for Anisotropy

Cardiac tissue conducts three to four times faster along fibers than across them. This anisotropy is encoded in the diffusion tensor $\mathbf{D}$ (equation 19.1), which has different eigenvalues along and perpendicular to the fiber direction. Can BGK handle this?

No. Recall from Section 18.5 that BGK uses a single relaxation rate $\omega = \Delta t / \tau$ for *all* moments. When we decoded the collision into moment space, we saw that the $\mathbf{x}$ -flux and $\mathbf{y}$ -flux moments both relax at the same rate $\mathbf{s}_1 = \mathbf{s}_2 = \omega$. Since the diffusion coefficient in each direction is determined by the corresponding flux relaxation rate, BGK forces $\mathbf{D}_x = \mathbf{D}_y$. It can only produce isotropic diffusion. For anisotropic cardiac tissue, we need to unlock those rates — relax the $\mathbf{x}$ -flux and $\mathbf{y}$ -flux at different speeds. This is exactly what MRT (Multiple Relaxation Time) does.

From the Conductivity Tensor to Relaxation Rates

Section 18.5 showed that MRT relaxes each moment independently: the conserved moment (density) at rate $\mathbf{s}_0 = 0$, the flux moments at rates $\mathbf{s}_1, \mathbf{s}_2$, and the ghost moments at freely chosen rates. The question is: what values of $\mathbf{s}_1$ and $\mathbf{s}_2$ reproduce a given anisotropic diffusion tensor?

Start from the scalar τ - $\mathbf{D}$ relationship (equation 18.26), which Chapman-Enskog analysis derives for the BGK operator:

$$\mathbf{D} = c_s^2 \left(\tau - \frac{\Delta t}{2} \right)$$

This relates a single diffusivity $\mathbf{D}$ to a single relaxation time τ . For anisotropic diffusion, we need a **tensor** generalization. The Chapman-Enskog expansion applied to the MRT collision operator (equation 18.40) shows that each component of the diffusion tensor maps to a corresponding relaxation parameter:

$$D_{\alpha\beta} = c_s^2 \left(\tau_{\alpha\beta} - \frac{\Delta t}{2} \delta_{\alpha\beta} \right) \quad (19.4)$$

where $\alpha, \beta \in \{x, y\}$ (or $\{x, y, z\}$ in 3D). This is the tensor τ - $\mathbf{D}$ relationship — the natural extension of equation (18.26) from a scalar to a matrix. Rearranging for the relaxation parameters:

$$\tau_{\alpha\beta} = \frac{\Delta t}{2} \delta_{\alpha\beta} + \frac{D_{\alpha\beta}}{c_s^2} \quad (19.5)$$

Substituting $c_s^2 = \Delta x^2 / (3\Delta t^2)$:

$$\tau_{\alpha\beta} = \frac{\Delta t}{2} \delta_{\alpha\beta} + \frac{3 D_{\alpha\beta} \Delta t}{\Delta x^2} \quad (19.6)$$

This is the tensor generalization of equation (18.28). Each entry of the τ matrix is determined by the corresponding entry of the diffusion tensor. The Kronecker delta $\delta_{\alpha\beta}$ ensures that the $\Delta t/2$ offset only appears on the diagonal.

For D2Q5 with axis-aligned fibers ($\theta = 0$), the diffusion tensor is diagonal ($D_{xx} = D_l$, $D_{yy} = D_t$, $D_{xy} = 0$), and the relaxation parameters simplify to:

$$\tau_{xx} = \frac{\Delta t}{2} + \frac{3D_l\Delta t}{\Delta x^2}, \quad \tau_{yy} = \frac{\Delta t}{2} + \frac{3D_t\Delta t}{\Delta x^2}, \quad \tau_{xy} = 0$$

These become the flux relaxation rates: $s_1 = \Delta t/\tau_{xx}$ controls the x -flux decay (and hence D_{xx}), while $s_2 = \Delta t/\tau_{yy}$ controls the y -flux decay (and hence D_{yy}). Since $D_l > D_t$, we have $\tau_{xx} > \tau_{yy}$, so the x -flux relaxes more slowly — it takes longer to reach equilibrium, which means more information persists in the x -direction, producing faster diffusion along the fiber.

Intuition

Why slower relaxation means faster diffusion. This seems backwards at first. A larger τ means *less* relaxation per timestep — the distribution retains more of its non-equilibrium structure. But non-equilibrium structure *is* the flux. A large departure from equilibrium means a large gradient, which drives faster diffusion. If τ were very small (rapid relaxation), the distribution would snap to equilibrium immediately — zero flux, zero diffusion. This is the lattice-Boltzmann version of "a stiff spring returns to rest quickly but doesn't transmit waves far."

The Off-Diagonal Problem: Rotated Fibers

When fibers are not aligned with the lattice axes ($\theta \neq 0$), the diffusion tensor has off-diagonal entries ($D_{xy} \neq 0$). Equation (19.6) then gives $\tau_{xy} \neq 0$, which means the relaxation matrix $\mathbf{S}$ in equation (18.40) is no longer diagonal in the Cartesian moment basis. There are two standard approaches:

Approach 1 — Rotate the moment basis. Construct $\mathbf{M}$ using the fiber-aligned coordinate system instead of the lattice axes. The flux moments then align with the fiber and cross-fiber directions, where $\mathbf{D}$ is diagonal, and $\mathbf{S}$ stays diagonal. This requires a different $\mathbf{M}$ at each node (since fiber angles vary spatially), which adds computational cost.

Approach 2 — Modify the equilibrium distribution. Keep the standard $\mathbf{M}$ (equation 18.36) but replace the isotropic equilibrium $f_i^{\text{eq}} = w_i V_m$ with an anisotropic equilibrium that encodes the full diffusion tensor:

$$f_i^{\text{eq}} = w_i V_m \left(1 + \frac{e_{i\alpha} e_{i\beta} D_{\alpha\beta}}{2 c_s^2 D_0} - \frac{1}{2} \frac{\text{tr}(\mathbf{D})}{D_0 \cdot d} \right) \quad (19.7)$$

where D_0 is a reference diffusivity and summation over α, β is implied. This approach absorbs the anisotropy into the equilibrium target rather than the relaxation rates, allowing a single scalar τ to be used. The implementation is simpler (no per-node $\mathbf{M}$ matrices) but the equilibrium is more complex.

For the remainder of this chapter, we use **Approach 1** (the MRT formulation with diagonal $\mathbf{S}$ in the principal-axis frame), which makes the connection to Chapter 18's moment-space machinery most transparent.

The Full MRT Collision with Source

Assembling the pieces from Section 18.5 (equations 18.38–18.41), the MRT collision with ionic source uses the same $\Omega^{NR} + \Omega^R$ decomposition from Section 19.3, but with the non-reactive part now operating

in moment space. Defining the rate-based collision operators:

$$\mathbf{\Omega}^{NR} = -\frac{1}{\Delta t} \mathbf{M}^{-1} \mathbf{S} (\mathbf{M}\mathbf{f} - \mathbf{M}\mathbf{f}^{\text{eq}}), \quad \mathbf{\Omega}^R = \mathbf{w} S$$

where $\mathbf{S} = \Delta t \cdot \mathbf{\Lambda}$ is the dimensionless relaxation matrix (with entries $s_k = \Delta t / \tau_k$), the factor $1/\Delta t$ converts the finite increment back to a rate. The full MRT update is then:

$$\mathbf{f}^* = \mathbf{f} + \Delta t \left(\underbrace{\mathbf{\Omega}^{NR}}_{\text{diffusion}} + \underbrace{\mathbf{\Omega}^R}_{\text{reaction}} \right) = \mathbf{f} - \mathbf{M}^{-1} \mathbf{S} (\mathbf{M}\mathbf{f} - \mathbf{M}\mathbf{f}^{\text{eq}}) + \Delta t \mathbf{w} S \quad (19.8)$$

The expanded right-hand side is identical to the standard MRT form — the $1/\Delta t$ in $\mathbf{\Omega}^{NR}$ cancels with Δt in the update. But the conceptual decomposition is preserved: $\mathbf{\Omega}^{NR}$ is the rate at which moment-space relaxation produces diffusive flux (anisotropic, direction-dependent), while $\mathbf{\Omega}^R$ is the rate at which ionic source is injected (isotropic, identical to the BGK case).

Here $\mathbf{M}$ is the transformation matrix (equation 18.36 for D2Q5, equation 18.37 for D3Q7), $\mathbf{S}$ is the diagonal relaxation matrix, $\mathbf{w} = (w_0, w_1, \dots, w_{Q-1})^T$ is the weight vector, and S is the ionic source (equation 19.2). The relaxation matrix for D2Q5 is:

$$\mathbf{S}_{\text{D2Q5}} = \text{diag} \left(0, \frac{\Delta t}{\tau_{xx}}, \frac{\Delta t}{\tau_{yy}}, s_3, s_4 \right) \quad (19.9)$$

The first entry is zero (density is conserved — never relaxed). Entries 1 and 2 are the flux relaxation rates that control D_{xx} and D_{yy} through equation (19.6). Entries 3 and 4 are the **ghost-moment rates** — the free parameters discussed in Section 18.5.

Ghost-Moment Rates and the Gram-Schmidt Construction

Section 18.5 stated that the ghost rows of $\mathbf{M}$ "just need to be linearly independent" from the physical rows to make the matrix invertible. But *how* do you construct them in practice?

The standard procedure is **Gram-Schmidt orthogonalization**. Start with the physical rows — the density row $(1, 1, 1, 1, 1)$ and the flux rows $(0, 1, -1, 0, 0)$, $(0, 0, 0, 1, -1)$ — which are dictated by conservation laws and Fick's law. Then construct the remaining rows by taking higher-order velocity products ($|\mathbf{e}_i|^2$, $e_{ix}^2 - e_{iy}^2$, etc.) and orthogonalizing them against the physical rows using the Gram-Schmidt process. This yields a well-conditioned $\mathbf{M}$ whose inverse is numerically stable.

For D2Q5, the two ghost rows in equation (18.36) correspond to the "energy" moment $\sum_i |\mathbf{e}_i|^2 f_i$ and the "energy difference" moment $\sum_i (e_{ix}^2 - e_{iy}^2) f_i$. These are second-order velocity products that are linearly independent of the zeroth- and first-order physical rows. The Gram-Schmidt process ensures they are also orthogonal, which makes $\mathbf{M}$ well-conditioned. For D3Q7 (equation 18.37), the same procedure generates four ghost rows from second-order products in three dimensions.

What about the ghost relaxation rates s_3 and s_4 ? Since no macroscopic conservation law constrains them, they are **free parameters**. The Chapman-Enskog analysis shows that they do not appear in the recovered PDE at leading order — they affect only the truncation error and numerical stability. In practice, the standard choice is:

$$s_3 = s_4 = 1.0 \quad (\text{full relaxation to equilibrium each timestep})$$

Setting a ghost rate to 1.0 means the corresponding moment is driven entirely to its equilibrium value in a single collision step — maximum damping of non-physical modes. This is the most stable choice. Some authors optimize ghost rates for specific accuracy targets (for example, setting

$s_3 = s_4 = 8(2 - s_1)/(8 - s_1)$ to cancel certain third-order error terms), but for cardiac simulations where stability matters more than squeezing out extra order of accuracy, $s_3 = s_4 = 1.0$ is the standard.

Mathematical Note

Why ghost rates affect stability but not the PDE. The Chapman-Enskog expansion recovers the macroscopic PDE from the lattice-Boltzmann equation order by order in the Knudsen number $\epsilon = \Delta x/L$ (where L is a macroscopic length scale). The zeroth-order recovery gives mass conservation. The first-order recovery gives the diffusion equation, with D determined by the flux relaxation rates s_1, s_2 . The ghost rates s_3, s_4 first appear at *second* order, in the truncation error. They modify the coefficient of the $O(\Delta x^2)$ error term but leave the leading-order PDE untouched. This is why different textbooks use different ghost rows and different ghost rates yet recover the same diffusion equation — they differ only in their numerical error properties.

Worked Example: MRT Collision with Anisotropic Tissue

Consider D2Q5 with $D_l = 0.001 \text{ cm}^2/\text{ms}$, $D_t = 0.00025 \text{ cm}^2/\text{ms}$, fiber angle $\theta = 0$ (aligned with x), $\Delta x = 0.025 \text{ cm}$, $\Delta t = 0.01 \text{ ms}$.

From equation (19.6):

$$\begin{aligned}\tau_{xx} &= 0.005 + \frac{3 \times 0.001 \times 0.01}{0.000625} = 0.005 + 0.048 = 0.053 \text{ ms} \\ \tau_{yy} &= 0.005 + \frac{3 \times 0.00025 \times 0.01}{0.000625} = 0.005 + 0.012 = 0.017 \text{ ms}\end{aligned}$$

The relaxation rates:

$$s_1 = \Delta t / \tau_{xx} = 0.01 / 0.053 = 0.189, \quad s_2 = \Delta t / \tau_{yy} = 0.01 / 0.017 = 0.588$$

As expected, the cross-fiber direction (y) relaxes faster ($s_2 > s_1$), producing slower diffusion. The full relaxation matrix is:

$$\mathbf{S} = \text{diag}(0, 0.189, 0.588, 1.0, 1.0)$$

Now suppose $V_m = -85 \text{ mV}$, with the same non-equilibrium distributions from the Chapter 18 worked example: $\mathbf{f} = (0.340, 0.175, 0.155, 0.168, 0.162)^T \times (-85)$ (scaled to voltage), and a stimulus source $S = 0.0371 \text{ mV/ms}$. The MRT collision proceeds as:

1. Decode: $\mathbf{m} = \mathbf{M}\mathbf{f}$ — extract density, x -flux, y -flux, and ghost moments.
2. Relax: $m_0^* = m_0$ (conserved), $m_1^* = m_1 - 0.189(m_1 - 0)$, $m_2^* = m_2 - 0.588(m_2 - 0)$, $m_3^* = 0$ (full relaxation), $m_4^* = 0$ (full relaxation).
3. Re-encode: $\mathbf{f}^* = \mathbf{M}^{-1}\mathbf{m}^*$.
4. Add source: $f_i^* \leftarrow f_i^* + \Delta t w_i S$.

The x -flux decays by 18.9% (slow — high diffusivity along fibers), while the y -flux decays by 58.8% (fast — low diffusivity across fibers). The ghost moments snap entirely to equilibrium. This is the same collision from the Chapter 18 worked example, but now with the x and y dials set to different values.

19.5 Boundary Conditions for LBM Electrophysiology

In classical methods (FEM, FDM), boundary conditions are applied by modifying the global stiffness matrix or by constraining node values. In LBM, boundaries are handled at the *distribution level* — you modify individual $\mathbf{f}_i$ at boundary nodes, one direction at a time. This local, direction-by-direction approach is one of LBM's most elegant features: no global matrix is ever touched, and complex geometries require no special mesh generation.

For cardiac electrophysiology, the dominant boundary condition is **no-flux Neumann**: no transmembrane current leaves the tissue domain. Physically, the heart is electrically insulated — ionic currents flow within

the myocardium but cannot escape through the epicardial surface. Mathematically, this is $\nabla V_m \cdot \hat{n} = 0$ at every tissue boundary, where $\hat{n}$ is the outward normal.

19.5.1 Full-Way Bounce-Back

Imagine a ball thrown at a wall: it bounces straight back. **Full-way bounce-back** works the same way. When a distribution f_i at a boundary node would stream into a solid (non-tissue) region, it reverses direction — it becomes $f_{\bar{i}}$, where $\bar{i}$ denotes the opposite direction ($\mathbf{e}_{\bar{i}} = -\mathbf{e}_i$).

Concretely, at a boundary node $\mathbf{x}_b$, after the collision step produces $f_i^*(\mathbf{x}_b)$, instead of streaming f_i^* to the neighbor $\mathbf{x}_b + \mathbf{e}_i \Delta x$ (which lies outside the domain), we apply:

$$f_{\bar{i}}(\mathbf{x}_b, t + \Delta t) = f_i^*(\mathbf{x}_b, t) \quad (19.10)$$

The post-collision distribution that *would have* left the domain in direction i is redirected to arrive at the same node in the opposite direction $\bar{i}$. Every unit of “flux” heading outward is perfectly reflected inward, so the net flux at the boundary is zero — exactly the no-flux Neumann condition.

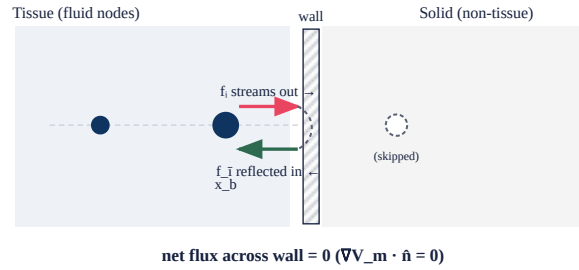


Figure 19.1. Half-way bounce-back enforces the no-flux boundary. The post-collision population f_i at the last fluid node $\mathbf{x}_b$ streams outward toward the wall (red); the wall reflects it back to the same node as the opposite population $f_{\bar{i}}$ (green), where $\mathbf{e}_{\bar{i}} = -\mathbf{e}_i$. Because every unit leaving in direction i returns in direction $\bar{i}$, the net flux across the wall is zero — exactly $\nabla V_m \cdot \hat{n} = 0$. The effective wall sits half a lattice spacing beyond $\mathbf{x}_b$, and the first solid node is skipped during streaming.

For the D2Q5 lattice used in this chapter, the opposite pairs are:

Type	Direction i	Velocity $\mathbf{e}_i$	Opposite $\bar{i}$	Velocity $\mathbf{e}_{\bar{i}}$
Axial	1	$(+1, 0)$	3	$(-1, 0)$
	2	$(0, +1)$	4	$(0, -1)$
Rest	0	$(0, 0)$	No partner — never streamed	

Intuition

Corner nodes. At a corner where two boundaries meet (say, the bottom-left corner of a rectangular domain), bounce-back applies to *both* boundary directions independently. If $\mathbf{e}_3 = (-1, 0)$ and $\mathbf{e}_4 = (0, -1)$ would stream out of the domain, both are reflected: $f_1 \leftarrow f_3^*$ and $f_2 \leftarrow f_4^*$. No special corner treatment is needed — bounce-back handles it automatically, direction by direction.

19.5.2 Half-Way Bounce-Back

Full-way bounce-back places the effective wall *at* the boundary node itself. **Half-way bounce-back** places the wall halfway between the last fluid node and the first solid node. This seemingly small shift has an im-

portant consequence: the boundary condition becomes **second-order accurate** in space, matching the second-order accuracy of the interior LBM scheme.

The rule is almost identical to full-way bounce-back, but the timing changes. Instead of reflecting at the same timestep, the reflected distribution arrives one timestep later:

$$f_i^*(\mathbf{x}_b, t + \Delta t) = f_i^*(\mathbf{x}_b, t) \quad (19.10b)$$

The formula *looks* the same as full-way bounce-back (eq. 19.10), but the interpretation differs: the wall is now located at $\mathbf{x}_b + \frac{1}{2}\mathbf{e}_i\Delta\mathbf{x}$, not at $\mathbf{x}_b$. On a uniform Cartesian grid, half-way bounce-back is the standard choice because the wall sits exactly at the cell face — which is where finite-difference methods also place the boundary.

Mathematical Note

Why second-order? The key insight is that the streaming step in LBM is *exact* — distributions travel exactly one lattice spacing per timestep. When the wall is at the half-way point, the round-trip distance (fluid node → wall → fluid node) is exactly $\Delta\mathbf{x}$, which takes exactly Δt . This geometric coincidence eliminates the first-order error that arises when the wall does not sit at a half-integer position. For regular Cartesian grids (as used in cardiac LBM), this condition is automatically satisfied.

19.5.3 Identifying Boundary Nodes

Before applying bounce-back, we must know *which* nodes are boundary nodes and *which directions* need reflection. In a cardiac simulation on a regular grid, the tissue geometry is typically defined by a binary mask: each grid point is either “tissue” (active) or “non-tissue” (inactive).

A fluid node $\mathbf{x}$ is a **boundary node** if at least one of its lattice neighbors $\mathbf{x} + \mathbf{e}_i\Delta\mathbf{x}$ is non-tissue. The specific directions i that point into non-tissue are the **boundary links** — these are the directions where bounce-back must be applied. In practice, this is precomputed once at initialization:

```
for each fluid node x:
  for each direction i = 1, ..., Q-1:
    neighbor = x + e_i * dx
    if neighbor is outside domain or non-tissue:
      mark (x, i) as a boundary link
```

At runtime, the streaming step skips boundary links (no data is sent to non-tissue nodes), and the bounce-back rule (eq. 19.10) is applied instead. This keeps the boundary treatment local and branchless — ideal for GPU execution.

19.5.4 Handling Irregular Tissue Geometries

Real cardiac tissue is not rectangular. The left ventricle is roughly ellipsoidal, the atria have complex curved surfaces, and scar tissue creates irregular internal boundaries. On a regular Cartesian grid, these curved boundaries are approximated by a **staircase** — the tissue mask follows the grid lines, creating jagged edges where the true boundary is smooth.

For cardiac electrophysiology, staircase boundaries are generally acceptable for two reasons:

1. **Diffusion dominates geometry.** The monodomain equation is a reaction-diffusion PDE, not a fluid-flow problem. Small geometric errors at the boundary (on the order of $\Delta\mathbf{x}$) produce errors in the diffusion flux that are $O(\Delta\mathbf{x})$ — the same order as the spatial discretization error. Refining the grid improves both simultaneously.

2. **No pressure or velocity fields.** In fluid LBM, staircase boundaries create spurious pressure oscillations and inaccurate drag forces. These problems do not arise in diffusion-only LBM because there is no momentum equation — only a scalar field V_m that diffuses smoothly.

Key Insight

When staircases fail. If you need sub-grid accuracy at the boundary (e.g., modeling the exact endocardial surface for defibrillation electrode placement), interpolated bounce-back methods exist. The simplest is **linear interpolation bounce-back**, which uses the fractional distance q from the fluid node to the actual wall: $f_i^*(\mathbf{x}_b, t + \Delta t) = 2q f_i^*(\mathbf{x}_b, t) + (1 - 2q) f_i^*(\mathbf{x}_b - \mathbf{e}_i \Delta \mathbf{x}, t)$ for $q < 1/2$, with a complementary formula for $q \geq 1/2$. This restores second-order accuracy on curved boundaries but adds implementation complexity. For most cardiac EP applications, standard bounce-back on a sufficiently fine grid ($\Delta \mathbf{x} \leq 0.2$ mm) is adequate.

19.5.5 Comparison with Classical BC Methods

Aspect	LBM (Bounce-Back)	FDM (Ghost Nodes)	FEM (Natural BC)
No-flux Neumann	Reflect f_i^*	Mirror V_m across boundary	Omit boundary integral (automatic)
Implementation	Local, per-link	Ghost node layer	Modify stiffness matrix
Curved geometry	Staircase or interpolated BB	Immersed boundary	Unstructured mesh (exact)
Accuracy	1st order (full-way) / 2nd order (half-way)	2nd order	Arbitrary (mesh-dependent)
GPU friendliness	Excellent (local, branchless)	Good	Moderate (sparse matrix ops)

Engine V5.4 Connection

Engine V5.4 implementation. The LBM solver uses half-way bounce-back on a Cartesian grid. Boundary links are precomputed during initialization into a compressed list [(node_idx, direction)]. During the streaming step, boundary links are processed separately: instead of writing to the neighbor, the post-collision value is written back to the opposite direction at the same node. The tissue mask is read from the geometry input (same binary mask used by the FDM solver) and converted to boundary links automatically.

19.6 The Complete LBM-EP Algorithm

Assembling all the pieces — operator splitting (Section 19.2), BGK or MRT collision with source (Sections 19.3–19.4), streaming (Chapter 18), and bounce-back boundaries (Section 19.5) — the complete algorithm for one timestep is:

Complete LBM-EP step (BGK or MRT):

(19.11)

1. **Ionic:** $I_{\text{ion}} = \text{IonicModel.compute_Iion}(V_m, \mathbf{w});$ update $\mathbf{w}$ via Rush-Larsen
2. **Source:** $S = -\frac{I_{\text{ion}} + I_{\text{stim}}}{\chi C_m}$
3. **Collide:** $f_i^* = f_i + \Delta t (\Omega_i^{NR} + \Omega_i^R)$ (BGK eq. 19.3, or MRT eq. 19.8)
4. **Stream:** $f_i(\mathbf{x} + \mathbf{e}_i \Delta x) \leftarrow f_i^*(\mathbf{x}) \quad \forall i$
5. **Bounce-back:** $f_i(\mathbf{x}_b) \leftarrow f_i^*(\mathbf{x}_b)$ at boundary nodes
6. **Recover:** $V_m^{n+1} = \sum_i f_i$

Steps 3–6 constitute the diffusion solver. Note the complete absence of matrix assembly, matrix-vector products, or iterative convergence checks. Every operation is either purely local (collision, source injection, recovery) or a nearest-neighbor shift (streaming). This is why LBM achieves its speed advantage on GPUs.

Intuition

Worked example. For cardiac tissue with $D = 0.001 \text{ cm}^2/\text{ms}$, $\Delta x = 0.025 \text{ cm}$, $\Delta t = 0.01 \text{ ms}$; $\tau = 0.053 \text{ ms}$, so $\omega = \Delta t/\tau = 0.189$. Consider a node with $V_m = -85 \text{ mV}$, all distributions at equilibrium, and a stimulus $I_{\text{stim}} = -52 \mu\text{A}/\mu\text{F}$ (with $I_{\text{ion}} = 0$, $\chi = 1400 \text{ cm}^{-1}$, $C_m = 1 \mu\text{F}/\text{cm}^2$). Then $S = 52/1400 = 0.0371 \text{ mV/ms}$.

After collision: $f_0^* = f_0 + 0.01 \times \frac{1}{3} \times 0.0371 \approx f_0 + 1.24 \times 10^{-4}$.

After streaming and recovery: $V_m^{n+1} = \sum f_i \approx -85 + 0.01 \times 0.0371 = -84.9996 \text{ mV}$. The stimulus produces a tiny depolarization in one 0.01 ms step, as expected.

19.7 Implementation: Memory Layout and GPU Acceleration

LBM is a textbook example of SIMD parallelism: every node performs the same collision and streaming operations in lockstep. There are no global data dependencies, no reduce operations, no convergence criteria — each GPU thread is completely independent. On modern GPUs, this translates to near-peak memory bandwidth utilization and minimal synchronization overhead.

The Swap-Streaming Optimization

The naive implementation of Algorithm (19.11) uses two arrays: a pre-collision array $\mathbf{f}$ and a post-collision array $\mathbf{f}^*$. After streaming populates $\mathbf{f}^*$, the arrays are swapped. This doubles memory usage. The **swap-streaming** (or "AA-pattern") optimization fuses collision and streaming into a single pass:

On even timesteps, read $f_i(\mathbf{x})$ from the source node and write f_i^* directly to the destination node $\mathbf{x} + \mathbf{e}_i \Delta x$ (scatter pattern). On odd timesteps, reverse: read f_i from the neighbor $\mathbf{x} - \mathbf{e}_i \Delta x$ and write the post-collision value to $\mathbf{x}$ (gather pattern). By alternating read/write patterns every other timestep, a single array suffices — halving memory and improving cache performance.

Memory Layout: Structure of Arrays

For GPU kernels, the distributions are stored in **Structure of Arrays** (SoA) format: one contiguous array per direction i , each of size $N_x \times N_y$ (or $N_x \times N_y \times N_z$ in 3D). The node at position $(\mathbf{x}, \mathbf{y})$ has its f_i stored at index $i \times N_x N_y + \mathbf{y} \times N_x + \mathbf{x}$. This layout ensures that when a GPU warp processes adjacent nodes, it accesses contiguous memory locations within each direction — maximizing memory coalescing.

Sparse/Indirect Addressing for Irregular Geometries

Cardiac tissue is not rectangular. A ventricular slab, a cross-section of the septum, or a full biventricular geometry all have irregular boundaries. Storing distributions on a full Cartesian grid wastes memory on

nodes outside the tissue. The solution is **indirect addressing**: an adjacency array maps each active node to its neighbors. Instead of computing neighbor positions from $(\mathbf{x} \pm \mathbf{1}, \mathbf{y} \pm \mathbf{1})$, the streaming step looks up the neighbor index from a precomputed table. This adds one integer load per direction per node but eliminates wasted storage on inactive nodes.

⚙️ **Engine V5.4 Connection**

In Engine V5.4, the LBM monodomain solver is in the `cardiac_sim.simulation.lbm` package. The lattice definitions (`D2Q5`, `D3Q7`) are frozen dataclasses holding weights, velocity vectors, and the τ - D conversion formula. Collision operators (`BGKCollision`, `MRTCollision`) inherit from `CollisionOperator(ABC)` and implement the update step (equation 19.3 or 19.8). The `LBMState` container holds the distribution tensor $\mathbf{f}$ (shape $Q \times N_x \times N_y$ for 2D), voltage V_m , ionic states $\mathbf{w}$, and the domain mask. The `LBMSimulation` class orchestrates the six-step loop (19.11): call `collision.collide(f, V, source, dt)`, apply `state.stream()` (one `torch.roll` per direction), `state.apply_bounce_back()`, and finally `state.recover_voltage()` to compute $V_m = \sum \mathbf{f}_i$. The ionic substep is identical to the classical path — only the diffusion step changes. Factory functions `create_isotropic_bgk(D, dx, dt)` and `create_anisotropic_mrt(D_fiber, D_cross, fiber_angle, dx, dt)` handle initialization.

19.8 LBM vs. Classical Methods: When to Choose What

LBM, explicit methods (Chapter 9), and implicit methods (Chapter 10) each have distinct trade-offs:

Aspect	Explicit (Ch 9)	Implicit (Ch 10)	LBM-BGK/MRT
Linear solve?	No	Yes (PCG/FFT)	No
Stability limit?	Yes ($\Delta t \leq h^2/2dD$)	No (unconditional)	$\tau > \Delta t/2$ (always satisfied)
Anisotropy?	Via diffusion tensor	Via diffusion tensor	BGK = isotropic only; MRT = full tensor
Unstructured mesh?	Yes (FEM)	Yes (FEM)	No (Cartesian only)
GPU suitability	Good (SpMV)	Fair (global reductions)	Excellent (zero global sync)
Memory per node	1 float (V_m)	1 float + sparse $\mathbf{A}$	Q floats ($\mathbf{f}_i$): 5 (D2Q5) or 7 (D3Q7)
Typical use	Small problems, rapid prototyping	Production FEM, large Δt	GPU throughput, parameter sweeps

When to use LBM: GPU hardware, Cartesian grids, and when throughput (simulations per unit wall-clock time) matters more than memory. The total absence of global synchronization makes LBM ideal for multi-GPU scaling and massive parameter sweeps. **When to use classical methods:** unstructured meshes (patient-specific geometries from MRI), problems where memory is the bottleneck (very large 3D domains), or when existing FEM infrastructure is already validated and optimized.

With the monodomain LBM now complete, we turn to its greatest challenge: the bidomain system. The parabolic intracellular equation can be solved by LBM, but the elliptic extracellular equation has no time dependence — it is a constraint that must be satisfied at every instant. Chapter 20 develops three strategies to overcome this fundamental obstacle: pseudo-time relaxation, hybrid LBM–classical solvers, and the dual-lattice architecture.

Lattice-Boltzmann Methods for Bidomain

Engine V5.4's LBM solver achieves 10–45× speedups over FEM for the monodomain equation. Can this advantage be preserved for bidomain? Yes — but the elliptic equation requires creative treatment. This chapter develops the core strategies: pseudo-time relaxation, hybrid LBM–classical solvers, and the dual-lattice architecture that couples two independent distribution systems. We show how to extend the computational fiction of particle distributions to solve problems with fundamentally different mathematical structure.

20.1 The Bidomain Challenge: One Parabolic + One Elliptic

Imagine two fish tanks connected by pipes: an intracellular tank (inside the cells) and an extracellular tank (outside the cells). Water flows from one tank to the other through countless tiny pipes — the ion channels. The voltage across the cell membrane is the water-level difference. Now here is the key: the intracellular voltage evolves over time (it changes as currents flow), so it satisfies a time-dependent equation. But the extracellular voltage is constrained to always satisfy a balance — at every instant, whatever current flows out of the intracellular space must flow through the external medium. This constraint is instantaneous, with no time dependence. One tank's level evolves; the other's is a slave variable determined by the constraint.

The bidomain equations (from Chapter 12) are:

$$\chi C_m \frac{\partial V_m}{\partial t} + I_{\text{ion}} = \nabla \cdot (\mathbf{D}_i \nabla V_m) + \nabla \cdot (\mathbf{D}_e \nabla \phi_e)$$

$$0 = \nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e] + \nabla \cdot (\mathbf{D}_i \nabla V_m) + I_{\text{stim}}$$

The first equation is *parabolic* — it has a time derivative, so it describes an evolution. The second is *elliptic* — no time derivative at all. It is a constraint that must be satisfied at every instant. LBM, by its nature, is a time-marching method: it evolves distributions forward in time. This is perfect for the parabolic equation but fundamentally at odds with the elliptic constraint.

The core mismatch is this: LBM asks "How do the distributions evolve?" But the elliptic equation says "The distributions must be such that this constraint is satisfied right now." These are two different questions. Resolving the tension between them — evolution versus constraint — is the central challenge of bidomain LBM.

Intuition

Think of the monodomain equation (Chapter 19) as a single fish tank with a changing water level. LBM marches the level forward in time with each step. The bidomain is two tanks. The intracellular tank (V_m) still evolves. But the extracellular tank (ϕ_e) is not free to evolve — its level is pinned down by the requirement that water conservation is always satisfied. This is why ϕ_e is an elliptic equation: it is an instantaneous constraint, not a time-evolution equation.

20.2 Strategy 1: Pseudo-Time Relaxation

If you heat a cold metal plate in a furnace with a fixed pattern of heat sources, the temperature rises over time. Eventually it reaches a steady state where the heat input exactly balances the heat loss through the

edges. The temperature at that steady state is determined by the heat sources alone — it is independent of the initial condition or the "time" you waited. In mathematics, finding this steady state is the same as solving the elliptic equation that governs heat diffusion in equilibrium.

Pseudo-time relaxation exploits this observation. We cannot use LBM directly on an elliptic equation because there is no time-derivative to march. But we can invent a fictitious time — call it τ (not to be confused with the relaxation time parameter) — and add an artificial time derivative to the elliptic equation:

$$\frac{\partial \phi_e}{\partial \tau} = \nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e] + \nabla \cdot (\mathbf{D}_i \nabla V_m) + I_{\text{stim}} \quad (20.1)$$

This is now a parabolic equation in the pseudo-time τ . We march it with LBM in pseudo-time for many sub-iterations until the solution settles to a steady state, meaning $\partial \phi_e / \partial \tau$ becomes negligibly small. At that point, the solution is the same as what the elliptic equation demands — the constraint is satisfied.

The beauty of this approach: LBM works just as well on the pseudo-time equation as on the physical parabolic equation. The lattice relaxation matrix $\tau_{e,\alpha\beta}$ is set by the same tensor Chapman–Enskog relationship (equation 18.28):

$$\tau_{e,\alpha\beta} = \frac{\Delta \tau}{2} \delta_{\alpha\beta} + \frac{3(\mathbf{D}_i + \mathbf{D}_e)_{\alpha\beta} \Delta \tau}{\Delta x^2} \quad (20.2)$$

where $\Delta \tau$ is the pseudo-time step size, $\delta_{\alpha\beta}$ is the Kronecker delta (so the $\frac{\Delta \tau}{2}$ term adds only to the diagonal, exactly as in the monodomain tensor law of §19.5), and the conductivity is the tensor $\mathbf{D}_i + \mathbf{D}_e$ (the sum of intracellular and extracellular tensors, as required by the elliptic equation).

Intuition

The convergence speed depends on τ_e (which determines the collision rate) and on the geometry of the domain. A larger τ_e means gentler relaxation and slower convergence. Typical practice is to use a moderately large τ_e (around 0.8–1.5) to balance stability with speed. The number of pseudo-time sub-iterations per physical time step is usually 10–100, depending on the desired residual reduction. With multi-grid acceleration (not detailed here), this can be reduced to 5–10 iterations.

20.3 Strategy 2: Hybrid LBM–Classical Solver

You would not use a hammer to write on paper — you use a pen. And you would not use a pen to drive nails — you use a hammer. The principle is simple: use the right tool for the job. LBM is superb at time-marching parabolic equations on Cartesian grids with GPUs. Classical solvers (conjugate gradient with multigrid preconditioning) are superb at solving elliptic systems exactly (or nearly exactly) without iteration over a fictitious time.

The hybrid approach splits the labor:

- **V_m equation (parabolic):** Use LBM with MRT collision (as in monodomain). Advance it by one physical time step.
- **ϕ_e equation (elliptic):** Use a classical solver — Conjugate Gradient (CG) preconditioned by Algebraic Multigrid (AMG). Solve it directly to a tight tolerance.

At each physical time step:

1. Compute ionic currents and update gates (Rush–Larsen).
2. March V_m forward one LBM step using the V_m distribution lattice.

3. Assemble the right-hand side for the elliptic ϕ_e equation, which includes the $\mathbf{V}_m$ gradient (now known from step 2) and the stimulus term.
4. Solve the elliptic system $\nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e] = \dots$ using CG+AMG, yielding ϕ_e .
5. Feed ϕ_e back as a source term for the next step's $\mathbf{V}_m$ equation.

The key advantage: avoiding the pseudo-time convergence loop. The classical solver reaches machine precision in 10–30 iterations (typically much faster than 100 pseudo-time iterations). The tradeoff: data must be transferred between the LBM lattice (which lives on a Cartesian GPU grid) and the classical solver's matrix assembly (which may involve FEM or FDM operators).

Mathematical Note

The hybrid approach requires careful bookkeeping of the gradient $\nabla \mathbf{V}_m$ on the LBM lattice and its transfer to the classical solver. One way to do this: compute $\nabla \mathbf{V}_m$ on the Cartesian grid using finite differences (natural in LBM since we have nearest-neighbor data), then pass this to the classical solver's RHS assembly. The solver can run on the same grid (FDM) or on a finer/coarser grid (via interpolation operators) to trade memory for accuracy.

20.4 The Dual-Lattice Architecture

Instead of mixing tools, use two of the same tool — but in different modes. Imagine having two bookkeepers, each tracking inventory for different tanks. Tank 1's bookkeeper (Lattice 1) evolves the intracellular voltage. Tank 2's bookkeeper (Lattice 2) repeatedly adjusts the extracellular potential until the water-balance constraint is met. They hand notes to each other at the end of each physical time step, telling each other what changed.

The dual-lattice architecture maintains two independent LBM lattices, each solving its own equation:

- **Lattice 1:** Distribution functions $f_i^{(m)}(\mathbf{x}, t)$ solving the parabolic $\mathbf{V}_m$ equation. Relaxation time τ_m encodes the intracellular diffusivity $\mathbf{D}_i$.
- **Lattice 2:** Distribution functions $f_i^{(e)}(\mathbf{x}, t)$ solving the pseudo-time-augmented ϕ_e equation. Relaxation time τ_e encodes the sum $\mathbf{D}_i + \mathbf{D}_e$.

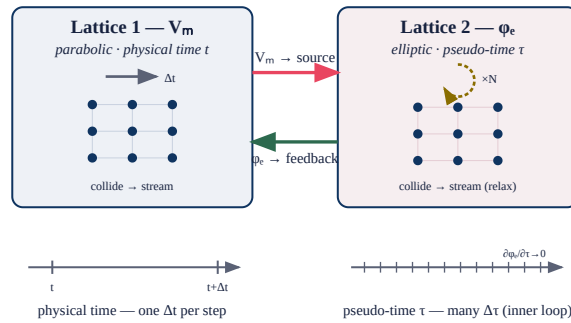


Figure 20.1. The dual-lattice bidomain architecture. Lattice 1 marches $\mathbf{V}_m$ (parabolic) forward one physical step Δt ; Lattice 2 relaxes ϕ_e (elliptic) through many pseudo-time sub-steps $\Delta \tau$ until $\partial \phi_e / \partial \tau \rightarrow 0$. The red link carries the $\mathbf{V}_m$ -derived source $\nabla \cdot (\mathbf{D}_i \nabla \mathbf{V}_m)$ that drives ϕ_e ; the green link feeds the converged ϕ_e back into the next $\mathbf{V}_m$ update. Physical time advances once per timestep (left axis); pseudo-time iterates many times within each timestep (right axis).

The two lattices are coupled through source terms. The source for Lattice 1 includes the gradient of ϕ_e (the extracellular potential) plus the ionic current. The source for Lattice 2 includes the gradient of $\mathbf{V}_m$ plus the stimulus:

$$f_i^{(m)*} = f_i^{(m)} - \omega_m (f_i^{(m)} - f_i^{(m)\text{eq}}) + \Delta t w_i [\text{ionic term} + \text{coupling to } \phi_e] \quad (20.3)$$

$$f_i^{(e)*} = f_i^{(e)} - \omega_e (f_i^{(e)} - f_i^{(e)\text{eq}}) + \Delta \tau w_i [\text{coupling to } V_m + \text{stimulus}] \quad (20.4)$$

The algorithm at each physical time step is:

Dual-Lattice Bidomain LBM Timestep:

- | | |
|--|---|
| 1. Reaction: | Ionic currents and state updates (Rush–Larsen) |
| 2. V_m update: | One LBM step on Lattice 1 (physical time step Δt) |
| 3. ϕ_e convergence: | Iterate Lattice 2 until $\frac{\partial \phi_e}{\partial \tau}$ is negligible |
| 4. Source exchange: | Compute coupling terms for the next physical step |

Worked example — one pseudo-time sub-iteration of the ϕ_e relaxation. Let us make Strategy 1 (§20.2) concrete on a tiny slice of Lattice 2. Take five nodes of the ϕ_e lattice in 1D, indices **0–4**, spacing Δx . During this physical timestep V_m has just advanced on Lattice 1, so the elliptic source $s_j = \nabla \cdot (\mathbf{D}_i \nabla V_m) + I_{\text{stim}}$ is now *fixed and known* at every node. Our only job here is to relax ϕ_e in pseudo-time τ until it stops changing.

Given.

- $D_i = 0.001$, $D_e = 0.0005$ cm²/ms, so the elliptic diffusivity is $\sigma = D_i + D_e = 0.0015$ cm²/ms.
- $\Delta x = 0.025$ cm, so $\Delta x^2 = 6.25 \times 10^{-4}$ cm² and $\sigma/\Delta x^2 = 2.4$ ms⁻¹.
- Pseudo-time step $\Delta \tau = 0.05$ ms (comfortably inside the stability limit $\Delta \tau < \Delta x^2/2\sigma = 0.208$ ms).
- No-flux ends: the bounce-back of §19.5 mirrors the edge, giving $\phi_0 = \phi_1$ and $\phi_4 = \phi_3$.
- Source from the just-updated V_m (interior nodes only): $(s_1, s_2, s_3) = (+0.8, -1.6, +0.8)$ mV/ms. It sums to zero — the compatibility condition a purely no-flux elliptic problem must satisfy, or ϕ_e would drift without bound.
- Start from the previous steady state, which was flat: $\phi_e^0 = (0, 0, 0, 0, 0)$.

At the macroscopic (Chapman–Enskog) level, one collide-and-stream sweep of the pseudo-time Lattice 2 is one forward step of

$$\phi_j^{k+1} = \phi_j^k + \Delta \tau r_j^k, \quad r_j^k = \frac{\sigma}{\Delta x^2} (\phi_{j+1} - 2\phi_j + \phi_{j-1}) + s_j, \quad (20.5)$$

where r_j is the **elliptic residual**. Reaching steady state means $r_j \rightarrow 0$ at every node — that is exactly the discrete elliptic constraint $\sigma \phi_e'' + s = 0$.

Sweep 0 → 1. With ϕ^0 flat, every discrete Laplacian is zero, so the residual is just the source: $r_j^0 = s_j$, giving $\mathbf{r}^0 = (0.8, -1.6, 0.8)$ with $\max |\mathbf{r}| = 1.6$. The update $\phi_j^1 = 0 + 0.05 r_j^0$ gives

$$\phi_1^1 = 0.05(0.8) = 0.04, \quad \phi_2^1 = 0.05(-1.6) = -0.08, \quad \phi_3^1 = 0.05(0.8) = 0.04 \text{ mV},$$

and the mirror ends copy inward: $\phi_0 = \phi_1 = 0.04$, $\phi_4 = \phi_3 = 0.04$ mV.

Recompute the residual after the sweep. Now the Laplacian is no longer zero. Using $\sigma/\Delta x^2 = 2.4$:

$$\text{Node 1: } 2.4(0.04 - 2(0.04) + (-0.08)) = 2.4(-0.12) = -0.288; \quad r_1^1 = -0.288 + 0.8 = 0.512.$$

$$\text{Node 2: } 2.4(0.04 - 2(-0.08) + 0.04) = 2.4(0.24) = 0.576; \quad r_2^1 = 0.576 - 1.6 = -1.024.$$

Node 3: by symmetry, $r_3^1 = 0.512$.

The residual has shrunk: $\max |r|$ dropped from **1.600** to **1.024**. One more sweep, $\phi^2 = \phi^1 + 0.05 \mathbf{r}^1 = (0.0656, -0.1312, 0.0656)$ mV, gives residual **(0.328, -0.655, 0.328)**:

sweep k	r_1	r_2	r_3	$\max r $
0	0.800	-1.600	0.800	1.600
1	0.512	-1.024	0.512	1.024
2	0.328	-0.655	0.328	0.655

Every sweep multiplies the residual by exactly **0.64**. Why *exactly*? The source excites the mode **(1, -2, 1)**, which is an eigenvector of the discrete no-flux Laplacian with eigenvalue $\lambda = -7.2 \text{ ms}^{-1}$ (check node 2: $2.4(1 - 2(-2) + 1) = 2.4(6) = 14.4 = -7.2 \times (-2)$). The update in equation (20.5) therefore multiplies the residual by $(1 + \Delta\tau\lambda) = 1 + 0.05(-7.2) = 0.64$. So the residual decays geometrically, $1.600 \rightarrow 1.024 \rightarrow 0.655 \rightarrow \dots$, driving $\sigma\phi_e'' + s \rightarrow 0$. After about **16** sweeps $\max |r|$ has fallen below 10^{-3} of its start — that is what “iterate Lattice 2 until $\partial\phi_e/\partial\tau$ is negligible” (step 3 of the algorithm above) means in actual numbers.

⚙ Engine V5.4 Connection

On Lattice 2 proper, this same update is realized by one collide-stream cycle (equation 20.4): the injected source $\Delta\tau w_i s_j$ supplies s_j , the collision-plus-streaming of the populations reproduces the $\sigma/\Delta x^2$ Laplacian coupling, and $\sum_i f_i^{(e)} = \phi_e$ recovers the field. The bounce-back of §19.5 supplies the $\phi_0 = \phi_1$ mirror automatically. Once the residual is negligible, the converged ϕ_e is handed back to Lattice 1 as the coupling source for the next physical Δt (the green link in Figure 20.1).

How do the three strategies compare?

Architecture	V_m Solver	ϕ_e Solver	Pros	Cons
A: Dual-Lattice	LBM	LBM (pseudo-time)	Pure LBM; maximal GPU parallelism; unified code path	Pseudo-time convergence overhead (typically 20–100 sub-iterations)
B: Hybrid	LBM	CG+AMG	Optimal solver for each PDE type; fewer iterations for ϕ_e	Data transfer overhead between LBM and classical grids
C: Full Classical	FEM/FDM	FEM/FDM	Mature solver ecosystem; handles unstructured meshes	Slower than LBM for parabolic; matrix assembly overhead

Architecture A (dual-lattice) is the most elegant and maximizes GPU utilization. It is the natural choice for research and exploration. Architecture B (hybrid) trades elegance for robustness: it leverages decades of optimization in multigrid solvers and avoids the pseudo-time convergence cost. In practice, Architecture A is recommended as the starting point for Engine V5.4, with Architecture B as a fallback.

⚙ Engine V5.4 Connection

Engine V5.4 already has the infrastructure for Lattice 1: the D3Q7 lattice definitions, the MRT collision operator, and the torch-based streaming and recovery. For Lattice 2, you need: (1) a second set of distribution arrays $f_i^{(e)}$, (2) a pseudo-time loop that iterates until convergence (checking $\|\partial\phi_e/\partial\tau\|$ after each step), (3) source term construction that couples V_m gradients from Lattice 1 into Lattice 2's collision. The factory functions can be extended: `create_dual_lattice_bidomain(D_i, D_e, dx, dt)` would initialize both lattices and return a `DualLatticeBidomainSolver` object with methods `step_reaction()`, `step_vm()`, `converge_phi_e(residual_tol)`.

20.5 Performance and Implementation Outlook

How much faster is bidomain LBM than classical bidomain solvers? That depends on how much time the classical solver spends on the elliptic solve. If the elliptic solve is fast (because the domain is simple or multigrid works well), then the speedup is modest. If the elliptic solve dominates, then LBM's advantage is large.

Based on the literature for monodomain LBM (which achieves 10–45× speedup over FEM) and accounting for the added cost of the elliptic ϕ_e equation, estimated performance ranges for bidomain LBM are:

- **Dual-Lattice (Architecture A):** 5–20× speedup over classical FEM bidomain, depending on the pseudo-time convergence rate and domain geometry. The lower end occurs when pseudo-time takes many iterations; the upper end occurs on simple geometries or with multigrid acceleration of the pseudo-time loop.
- **Hybrid (Architecture B):** 3–10× speedup. The elliptic solve using AMG is already near-optimal, so gains come mainly from the LBM parabolic solve and from avoiding the global reductions (inner products) required by iterative classical solvers.

Memory overhead: Each architecture stores two voltage fields (V_m and ϕ_e) plus either one or two lattices. The dual-lattice uses $2Q$ floats per node (two sets of Q distributions); the hybrid uses Q floats plus the classical solver's matrix storage. For a 3D grid with D3Q7 ($Q = 7$), the dual-lattice costs 14 floats per node — manageable on modern GPUs with 24–80 GB.

The recommended implementation path for Engine V5.4:

1. **Phase 1:** Extend the existing LBM infrastructure to support dual lattices. Test on a simple 2D domain with uniform conductivity.
2. **Phase 2:** Implement pseudo-time convergence checks and convergence acceleration (e.g., line-by-line SOR, incomplete factorization).
3. **Phase 3:** Add the hybrid solver as an alternative path: call external AMG libraries (AMGX on GPU, Hypr on CPU).
4. **Phase 4:** Benchmark against classical FEM bidomain solvers on realistic geometries and compare accuracy and wall-clock time.
5. **Phase 5:** Extend to anisotropic conductivity (different D_f and D_e for intracellular and extracellular spaces) using MRT on both lattices.

⚙ Engine V5.4 Connection

Recommended starting point for Engine V5.4: Implement Architecture A (dual-lattice) first. The pseudocode:

```
class DualLatticeBidomainLBMSolver(ABC):
    def __init__(self, D_i, D_e, dx, dt, ionic_model):
        self.lattice_vm = LBMLattice(D_i, dx, dt)
        self.lattice_phi_e = LBMLattice(D_i + D_e, dx, dt_pseudo)
        self.ionic = ionic_model

    def step(self, V_m, phi_e, I_stim, residual_tol=1e-6):
        # Step 1: Ionic
        I_ion, gates = self.ionic.step(V_m)

        # Step 2: V_m LBM
        source_vm = -(I_ion + I_stim) / (chi * Cm)
        source_vm += gradient_phi_e # coupling term
        V_m = self.lattice_vm.step(source_vm)

        # Step 3: phi_e convergence (pseudo-time loop)
        source_phi_e = -gradient(D_i @ gradient(V_m)) + I_stim_e
        phi_e = self.lattice_phi_e.converge(source_phi_e, tol=residual_tol)

        return V_m, phi_e, gates
```

Extend `LBMLattice` (already in the codebase for monodomain) to support pseudo-time iteration with convergence checks. Reuse the collision operator; only the time step size and convergence criterion change.

With the completion of Part IV (Chapters 18–20), we have the full mathematical and algorithmic foundation for bidomain electrophysiology. From the ionic model (Part I) through the monodomain and bidomain frameworks (Parts II and III), every piece is in place for a production cardiac simulator. The final part of this textbook turns to validation, performance optimization, and the art of choosing the right solver for your problem — the practical craft of turning equations into fast, accurate simulations.

APPENDICES

Reference Material

Key references, a visual guide to differential equations, linear algebra for cardiac simulation, numerical analysis, and an introduction to PyTorch for scientific computing.

Key References

Core Bidomain Theory

1. Tung, L. (1978). *A bidomain model for describing ischemic myocardial D-C potentials*. PhD thesis, MIT.
2. Henriquez, C.S. (1993). Simulating the electrical behavior of cardiac tissue using the bidomain model. *Crit. Rev. Biomed. Eng.*, 21(1):1–77.
3. Sundnes, J., Lines, G.T., Cai, X., et al. (2006). *Computing the Electrical Activity in the Heart*. Springer.

Numerical Methods and Solvers

4. Vigmond, E.J., Weber dos Santos, R., Prassl, A.J., et al. (2008). Solvers for the cardiac bidomain equations. *Prog. Biophys. Mol. Biol.*, 96(1-3):3–18.
5. Pennacchio, M., Simoncini, V. (2002). Efficient algebraic solution of reaction–diffusion systems for the cardiac excitation process. *J. Comput. Appl. Math.*, 145(1):49–70.
6. Plank, G., Liebmman, M., Weber dos Santos, R., et al. (2007). Algebraic multigrid preconditioner for the cardiac bidomain model. *IEEE Trans. Biomed. Eng.*, 54(4):585–596.

Lattice-Boltzmann Methods for Cardiac EP

7. Rapaka, S., et al. (2012). LBM-EP: Lattice-Boltzmann method for fast cardiac electrophysiology simulation. *MICCAI 2012*.
8. Belmiloudi, A., et al. (2015). Coupled lattice Boltzmann modeling of bidomain type models in cardiac electrophysiology. *Springer*.
9. Belmiloudi, A., et al. (2019). Coupled lattice Boltzmann simulation method for bidomain type models with multiple time-delays. *Math. Model. Nat. Phenom.*

Software Frameworks

10. Africa, P.C., et al. (2023). lifex-ep: A flexible framework for cardiac electrophysiology. *BMC Bioinformatics*, 24:389.
11. Plank, G., et al. (2021). The openCARP simulation environment. *Comput. Methods Programs Biomed.*

Cellular Electrophysiology Models

12. ten Tusscher, K.H.W.J., Panfilov, A.V. (2006). Alternans and spiral breakup in a human ventricular tissue model. *Am. J. Physiol. Heart Circ. Physiol.*, 291(3):H1088–H1100.
13. O'Hara, T., Virág, L., Varró, A., Rudy, Y. (2011). Simulation of the undiseased human cardiac ventricular action potential: model formulation and experimental validation. *PLoS Comput. Biol.*, 7(5):e1002061.

Numerical Analysis and Linear Algebra

14. LeVeque, R.J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM.
15. Trefethen, L.N., Bau, D. (1997). *Numerical Linear Algebra*. SIAM.
16. Gutknecht, M.H., Röllin, S. (2002). The Chebyshev iteration revisited. *Parallel Comput.*, 28(2):263–283.

17. Keener, J., Sneyd, J. (2009). *Mathematical Physiology* (2nd ed.). Springer.

Visual and Educational Resources

18. Sanderson, G. (3Blue1Brown) (2016). *Essence of Linear Algebra* (video series). Available at youtube.com/3blue1brown.

This document synthesizes research from the four comprehensive research documents in the `Research/Bidomain/` folder: Discretization, Solver Methods, Linear Solvers, and LBM Bidomain. Code examples and implementation guides accompany this textbook in the `Code_Examples/` subfolder.

A Visual Guide to Differential Equations

This appendix steps back from the bidomain model to build a deep, visual intuition for the three fundamental types of partial differential equations: parabolic, elliptic, and hyperbolic. We begin with the heat equation — the simplest and most intuitive PDE — and use it as a lens to understand all three families. The style here follows the philosophy of 3Blue1Brown: start with a concrete picture, build understanding layer by layer, and let the math emerge naturally from the intuition.

A.1 The Heat Equation: Where It All Begins

Imagine you have a thin metal rod. You heat one spot in the middle with a blowtorch, creating a sharp temperature peak, and then you turn off the blowtorch. What happens?

You already know the answer intuitively: the hot spot cools down, and the warmth spreads outward into the cooler parts of the rod. Over time, the temperature profile gets smoother and flatter, until eventually the whole rod reaches a uniform temperature. This simple physical process — heat spreading from hot regions to cold regions — is described by one of the most important equations in all of mathematics:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} \quad (A.1)$$

This is the **heat equation** in one dimension. Here $u(x, t)$ is the temperature at position x and time t , and $\alpha > 0$ is the *thermal diffusivity* — a material property that tells you how quickly heat spreads (copper has high α ; wood has low α).

But what does this equation actually say? Let us read it like a sentence.

Intuition

Reading the heat equation as English: "The rate at which the temperature changes at a point ($\partial u / \partial t$) is proportional to how much that point's temperature differs from the average of its neighbors ($\partial^2 u / \partial x^2$)." If a point is hotter than its surroundings, it cools down. If it is cooler, it warms up. The equation says nothing more — and nothing less — than this.

This is the fundamental insight, and it is worth dwelling on. The second derivative $\partial^2 u / \partial x^2$ is not just a mathematical abstraction — it measures *concavity*. At a temperature peak (concave down), the second derivative is negative, so $\partial u / \partial t < 0$: the temperature decreases. At a temperature valley (concave up), the second derivative is positive, so $\partial u / \partial t > 0$: the temperature increases. The heat equation is a machine that relentlessly destroys peaks and fills valleys.

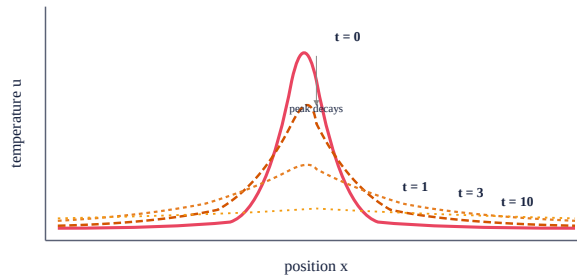


Figure A.1. A sharp temperature peak (red) diffuses over time, becoming broader and flatter. The heat equation smooths all discontinuities — information about the initial sharp shape is irreversibly lost.

Notice three properties that are visible even from the picture:

1. **Smoothing.** The solution gets smoother over time. A sharp corner at $t = 0$ becomes an infinitely smooth curve at any $t > 0$. The heat equation is a "smoothing machine."
2. **Irreversibility.** You cannot run time backwards and recover the sharp peak from the flat profile. Information is destroyed. (Mathematically: the backwards heat equation is ill-posed.)
3. **Maximum principle.** The hottest point at any later time is never hotter than the hottest point at $t = 0$. Diffusion can only reduce extremes, never amplify them.

These three properties — smoothing, irreversibility, and a maximum principle — are the *defining fingerprint* of **parabolic** equations. Every parabolic PDE shares them.

A.2 What the Laplacian Really Means

Before we classify PDEs, we need to build intuition for the single most important differential operator in physics: the **Laplacian** $\nabla^2 u$ (also written Δu).

In one dimension, $\nabla^2 u = \partial^2 u / \partial x^2$ — just the second derivative. In two or three dimensions:

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}$$

But what does it *mean*? Here is the deepest and most useful interpretation:

△ Key Insight

The Laplacian measures the difference between a point's value and the average of its neighbors.

Specifically, for a function u that is smooth enough:

$$\nabla^2 u(\mathbf{x}) = \lim_{r \rightarrow 0} \frac{2d}{r^2} [\bar{u}_r(\mathbf{x}) - u(\mathbf{x})]$$

where $\bar{u}_r(\mathbf{x})$ is the average of u on a sphere of radius r centered at $\mathbf{x}$, and d is the spatial dimension. If $u(\mathbf{x})$ is less than its neighborhood average, then $\nabla^2 u > 0$. If more, then $\nabla^2 u < 0$.

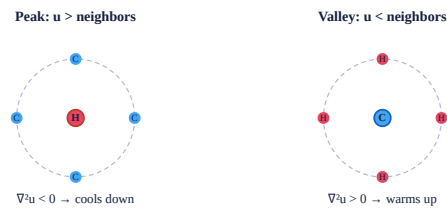


Figure A.2. The Laplacian as a "disagreement detector." Left: a hot point (H) surrounded by cool neighbors (C) has $\nabla^2 u < 0$. Right: a cool point surrounded by hot neighbors has $\nabla^2 u > 0$. The heat equation says: resolve the disagreement.

This is why the Laplacian appears in so many equations across physics. Any time nature wants to smooth out differences — heat, chemical concentration, electrical potential — the Laplacian is doing the work. The heat equation $\partial u / \partial t = \alpha \nabla^2 u$ simply says: *evolve toward the local average*.

Think About It

A discrete analogy. On a grid, approximate $\nabla^2 u$ at node i as $\nabla^2 u_i \approx \frac{1}{h^2} (\bar{u}_{\text{neighbors}} - u_i)$ where $\bar{u}_{\text{neighbors}}$ is the average of the neighboring values. The heat equation on a grid becomes: each node moves a small fraction toward the average of its neighbors at each time step. This is literally a smoothing filter — the same operation used in image processing.

A.3 The Three Families of PDEs

Now we are ready for the big classification. Consider the most general second-order linear PDE in two variables:

$$A \frac{\partial^2 u}{\partial x^2} + 2B \frac{\partial^2 u}{\partial x \partial y} + C \frac{\partial^2 u}{\partial y^2} + (\text{lower-order terms}) = 0 \quad (\text{A.2})$$

The character of this equation — its fundamental behavior, the types of solutions it admits, the numerical methods that work — is determined by a single number: the **discriminant**

$$\Delta = B^2 - AC$$

This is exactly the same discriminant from the quadratic formula, and the analogy to conic sections is deep. Just as $B^2 - 4AC$ classifies conics into ellipses, parabolas, and hyperbolas, this discriminant classifies PDEs into three families:

Parabolic ($\Delta = 0$)

Prototype: Heat equation

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u$$

Diffusion, smoothing, irreversible

Elliptic ($\Delta < 0$)

Prototype: Laplace equation

$$\nabla^2 u = 0$$

Equilibrium, steady-state, no time

Hyperbolic ($\Delta > 0$)

Prototype: Wave equation

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u$$

Propagation, waves, reversible

Think About It

Why does the name "parabolic" come from the degenerate case $\Delta = 0$? Think of the conic $Ax^2 + 2Bxy + Cy^2 = 1$. When $\Delta = 0$, this degenerates from a closed curve (ellipse) to an open, unbounded shape (parabola). Similarly, the PDE loses one "degree of freedom" — information propagates in one direction only (forward in time), just as a parabola is open in one direction.

Let us now build deep intuition for each family.

A.4 Parabolic Equations: Diffusion and Smoothing

The heat equation is the prototype, but parabolic equations describe far more than heat. Any process where "stuff" spreads out from high-concentration regions to low-concentration regions is parabolic: chemical diffusion (Fick's law), financial option pricing (Black–Scholes), probability spreading (Fokker–Planck), and — most relevant to us — the evolution of the transmembrane potential V_m in the monodomain equation.

The unifying feature is the structure: **one time derivative balanced against two spatial derivatives**. This mismatch (∂_t vs. ∂_{xx}) dictates everything:

$$\underbrace{\frac{\partial u}{\partial t}}_{\text{1st order in } t} = \alpha \underbrace{\frac{\partial^2 u}{\partial x^2}}_{\text{2nd order in } x}$$

Because there is only a *first* derivative in time, you need only *one* initial condition: $u(x, 0) = u_0(x)$. Time marches in one direction only. There is no "bouncing back." This is why parabolic equations model irreversible processes.

The smoothing property, precisely stated. If you start with any initial condition — even one with sharp corners or discontinuities — the solution $u(x, t)$ is infinitely smooth for all $t > 0$. This is remarkable. A square-wave initial condition instantly (at any positive time, no matter how small) becomes a C^∞ function. Intuitively: diffusion smears out all sharp features immediately.

Infinite propagation speed. Here is a subtle and often confusing point. Heat propagates at *infinite speed* in the mathematical heat equation — if you heat one end of an infinitely long rod, the other end feels it instantly (though the effect is exponentially small). This is physically unrealistic but mathematically convenient. The parabolic monodomain equation inherits this property: a stimulus at one point technically affects the entire domain immediately, though the practical effect dies off exponentially fast.

Key Insight

The parabolic CFL condition. When you discretize a parabolic equation with an explicit time stepper (like forward Euler), stability demands $\Delta t \leq C \Delta x^2$. The *quadratic* dependence on Δx is severe: halving the mesh spacing requires quartering the time step. This is why implicit methods (Crank–Nicolson, BDF, SDIRK) are essential for parabolic problems — they are unconditionally stable, allowing Δt dictated by accuracy rather than stability.

A.5 Elliptic Equations: Equilibrium and Constraints

Now remove time from the picture entirely. Ask: *what does the temperature look like after the system has fully equilibrated?* At equilibrium, $\partial u / \partial t = 0$, and the heat equation becomes:

$$\nabla^2 u = 0 \quad (\text{Laplace's equation})$$

(A.3)

This is the prototypical **elliptic** equation. It says: at every point, the value of u equals the average of its neighbors. There is no time. There is no evolution. It is a pure *constraint*.

Intuition

The rubber sheet analogy. Imagine a flat rubber membrane stretched over a wire frame. The frame is bent into some shape (the boundary condition). Where does the membrane settle? At every interior point, the height of the membrane equals the average height of the surrounding points — because the elastic forces balance in all directions. The equilibrium shape satisfies Laplace's equation.

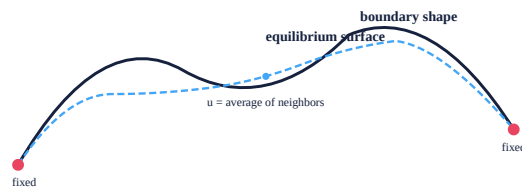


Figure A.3. The rubber sheet analogy for Laplace's equation. The boundary conditions (wire frame) determine the shape; the interior settles into an equilibrium where each point's value equals the average of its neighbors.

The key properties of elliptic equations mirror the equilibrium intuition:

1. **Mean value property.** The value at any interior point equals the average over any surrounding sphere. This is the deepest characterization of harmonic functions (solutions to $\nabla^2 u = 0$).
2. **Maximum principle.** The maximum and minimum of u occur on the *boundary*, never in the interior. The rubber sheet cannot bulge above or below the wire frame.
3. **Global coupling.** Changing the boundary condition at one point affects the solution *everywhere* in the interior. Information propagates in all directions simultaneously. There is no concept of "marching" from one end to the other.
4. **No time, no initial conditions.** You need only boundary conditions. The equation determines the interior values uniquely from the boundary.

Think About It

Why you cannot "march" an elliptic equation. In a parabolic equation, you know u at $t = 0$ and march forward to $t = \Delta t$, then to $t = 2\Delta t$, and so on. In an elliptic equation, there is no "forward." Changing one boundary value ripples through the *entire* domain simultaneously. This is why elliptic equations always require solving a **global linear system** — you must find all unknowns at once.

If there is a source term, Laplace's equation becomes **Poisson's equation**: $\nabla^2 u = f$. This is still elliptic. It says: the difference between a point's value and its neighborhood average is proportional to the local source strength f . This is exactly the form of the bidomain elliptic constraint.

A.6 Hyperbolic Equations: Waves and Propagation

The third family is fundamentally different from the first two. Consider the **wave equation**:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2} \quad (\text{A.4})$$

Notice the crucial structural difference: the *second* derivative in time. Where the heat equation had ∂_t (first order), the wave equation has ∂_{tt} (second order). This single change transforms the physics completely.

With a first-order time derivative, you need one initial condition (u at $t = 0$). With a second-order time derivative, you need *two*: the initial shape $u(x, 0)$ and the initial velocity $\partial u / \partial t|_{t=0}$. This is like specifying both the position and velocity of a particle — and it means the equation is *reversible*. You can run time forward or backward and get valid solutions.

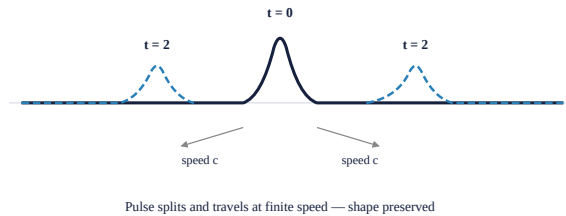


Figure A.4. The wave equation splits an initial pulse into two copies traveling in opposite directions at speed c . Unlike diffusion, the pulse shape is preserved — no smoothing occurs.

The contrast with the heat equation could not be sharper:

Property	Parabolic (heat)	Hyperbolic (wave)
Time derivative order	1st (∂_t)	2nd (∂_{tt})
Initial conditions needed	1 (value)	2 (value + velocity)
Reversible?	No	Yes
Propagation speed	Infinite (exponential decay)	Finite (c)
Smoothing?	Yes — destroys sharp features	No — preserves sharp features
Energy	Dissipated over time	Conserved

The finite propagation speed is the hallmark of hyperbolic equations. A disturbance at point x_0 at time t_0 can only be felt at point x after time $(x - x_0)/c$. This region of influence is called the **light cone** (even for non-relativistic waves), and the boundary of this cone consists of **characteristics** — curves in space-time along which information travels.

Intuition

Why cardiac EP is NOT hyperbolic. At first glance, the action potential "wavefront" sweeping through the heart looks like a traveling wave — so you might expect a wave equation. But it is not a wave in the hyperbolic sense. The action potential front is driven by *diffusion* (current spreading from depolarized cells to resting cells) plus a *nonlinear reaction* (ion channels opening in a cascade). There is no second time derivative. There is no energy conservation. And the wavefront *does* smooth out over time. The monodomain and bidomain equations are parabolic, not hyperbolic — the traveling wavefront emerges from the interplay of diffusion and reaction, not from wave mechanics.

A.7 Why the Bidomain Model Has Two Types

We can now understand the bidomain model's structure at a much deeper level. The bidomain system is not a single PDE — it is a *coupled pair*, and each equation belongs to a different family:

$$\underbrace{\chi C_m \frac{\partial V_m}{\partial t} = -\chi I_{\text{ion}} + \nabla \cdot (\mathbf{D}_i \nabla V_m) + \nabla \cdot (\mathbf{D}_i \nabla \phi_e)}_{\text{Parabolic: has } \partial V_m / \partial t} \quad (\text{A.5})$$

$$\underbrace{0 = \nabla \cdot (\mathbf{D}_i \nabla V_m) + \nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e]}_{\text{Elliptic: no time derivative at all}} \quad (\text{A.6})$$

This mixed parabolic–elliptic structure is the source of both the model's physical richness and its computational difficulty:

1. **The parabolic equation** governs the evolution of the transmembrane potential V_m . It is a diffusion equation — with all the properties we discussed: smoothing, irreversibility, CFL restrictions for explicit methods. The "stuff" being diffused is electrical charge, and the diffusion coefficient is determined by the intracellular conductivity tensor $\mathbf{D}_i$. It can in principle be marched forward in time.
2. **The elliptic equation** constrains the extracellular potential ϕ_e . It has no time derivative — it is an *equilibrium condition* that must hold at every instant, like the rubber sheet analogy. At every moment, the extracellular potential adjusts *instantly* so that the total current in and out of every point balances. It cannot be marched — it must be solved as a global linear system.

△ Key Insight

The fundamental computational consequence. The parabolic equation can (in theory) be treated explicitly. The elliptic equation *cannot* — ever. This is why every bidomain solver requires at least one implicit linear solve per time step: the elliptic constraint demands it. This is not a choice of method; it is a consequence of the equation's type. No clever algorithm can avoid it, because the elliptic equation couples all spatial points simultaneously.

The monodomain equation, by contrast, is purely parabolic — a single equation with a single time derivative. This is why it is so much cheaper: you never need to solve an elliptic constraint. The LBM methods in Part IV (Chapters 18–20) exploit this: a lattice-Boltzmann scheme can handle a parabolic PDE naturally (the collision-streaming cycle is essentially an explicit time stepper), but it has no mechanism for enforcing a global elliptic constraint.

Understanding the type of each equation also tells you *which numerical methods to reach for*:

Equation type	Natural solvers	Avoid
Parabolic (V_m)	Implicit time steppers (CN, BDF, SDIRK), operator splitting, IMEX, LBM	Fully explicit on fine meshes (CFL: $\Delta t \propto \Delta x^2$)
Elliptic (ϕ_e)	Direct solvers, CG/MINRES with AMG preconditioner, multigrid	Explicit time stepping (impossible — no time derivative)

The classification of PDEs into parabolic, elliptic, and hyperbolic is not merely academic taxonomy. It determines which initial/boundary conditions are needed, which numerical methods will work, what stability restrictions apply, and whether information flows forward only or in all directions simultaneously. For the

bidomain model, this classification is the key to understanding why the computational challenge is fundamentally harder than monodomain — and why the solver strategies of Chapter 15 are not optional refinements but structural necessities.

The numerical tools for solving these equations — transforms, iterative methods, and convergence analysis — are developed in Appendix C.

The treatment in this appendix follows the pedagogical philosophy of Grant Sanderson's *3Blue1Brown* series on differential equations. For animated visualizations of these concepts, the reader is encouraged to watch the "Differential Equations" playlist on the 3Blue1Brown YouTube channel.

Linear Algebra

This appendix builds the linear algebra you need from the ground up, following the philosophy of 3Blue1Brown: start with pictures, develop geometric intuition, and let the formalism emerge naturally. Everything here is abstract — we work with vectors, matrices, and transformations as mathematical objects, independent of any particular application. The connection to differential equations and cardiac simulation is the subject of Appendix C.

B.1 Vectors and Spaces

A **vector** is an ordered list of numbers — and simultaneously an arrow pointing from the origin to a location in space. In low dimensions, think of the arrow (the 3Blue1Brown viewpoint). In high dimensions ($n = 90,000$), the arrow is unvisualizable but the list of numbers works just fine. Both pictures are correct; they emphasize different things:

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix} \tag{B.1}$$

The number of entries is the **dimension**. A 2-vector lives in $\mathbb{R}^2$ (the plane), a 3-vector in $\mathbb{R}^3$ (3D space), and an n -vector in $\mathbb{R}^n$. When $n = 90,000$, you cannot visualize the space, but the algebra works identically to $\mathbb{R}^2$.

Two operations define the algebra of vectors:

- **Addition:** Add corresponding entries. $\mathbf{u} + \mathbf{v}$ has entries $u_i + v_i$.
- **Scalar multiplication:** Multiply every entry by a number. $c\mathbf{v}$ has entries $c \cdot v_i$.

Any combination $c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \dots + c_k\mathbf{v}_k$ is called a **linear combination**. The set of all linear combinations of a collection of vectors is their **span** — the "space" they can reach.

Intuition

Why "linear"? These two operations — addition and scaling — are the only ones allowed. No squaring entries, no taking square roots, no multiplying entries together. This restriction seems limiting, but it is what makes the entire theory tractable. Linearity means superposition: if $\mathbf{u}$ and $\mathbf{v}$ are solutions, so is $c_1\mathbf{u} + c_2\mathbf{v}$. This single property drives everything in this appendix.

The **inner product** (dot product) of two vectors measures how aligned they are:

$$\mathbf{u}^T \mathbf{v} = u_1 v_1 + u_2 v_2 + \dots + u_n v_n$$

The **norm** (length) of a vector is $\|\mathbf{v}\| = \sqrt{\mathbf{v}^T \mathbf{v}}$. A vector with $\|\mathbf{v}\| = 1$ is called a **unit vector**.

B.2 Matrices as Transformations

A matrix is not just a grid of numbers — it is a *machine*. You feed it a vector, and it produces a new vector. In the language of 3Blue1Brown: a matrix is a transformation that takes every point in space and moves it to a new location, in a way that keeps straight lines straight and the origin fixed.

Consider a 2×2 matrix acting on vectors in the plane:

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$$

This matrix stretches every vector by a factor of 2 in the $\mathbf{x}$ -direction and leaves the $\mathbf{y}$ -direction unchanged. The unit square becomes a rectangle twice as wide. The matrix *is* the stretch — it encodes the transformation completely.

Now consider a rotation by 90° :

$$R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

Every vector gets rotated counterclockwise by a quarter turn. The transformation is encoded in the columns of R : the first column $[0, 1]^T$ tells you where the $\mathbf{x}$ -axis unit vector lands, and the second column $[-1, 0]^T$ tells you where the $\mathbf{y}$ -axis unit vector lands.

Intuition

Columns are destinations. The j -th column of a matrix tells you where the j -th basis vector goes under the transformation. Once you know where the basis vectors land, you know where *every* vector lands, because every vector is a combination of basis vectors. This is the single most important insight in all of linear algebra.

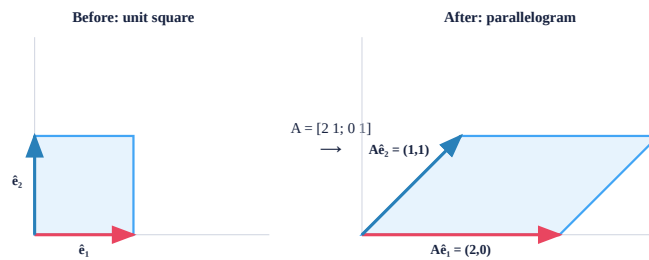


Figure B.1. A matrix transforms the unit square (left) into a parallelogram (right). The columns of A are the destinations of the basis vectors: $\hat{\mathbf{e}}_1 \rightarrow (2, 0)$ and $\hat{\mathbf{e}}_2 \rightarrow (1, 1)$. Every other point follows by linearity.

B.3 Matrix-Vector Multiplication: What It Actually Does

If A is an $m \times n$ matrix and $\mathbf{x}$ is an n -vector, the product $\mathbf{y} = A\mathbf{x}$ is an m -vector whose i -th entry is:

$$y_i = \sum_{j=1}^n A_{ij} x_j \quad (\text{B.2})$$

Each entry of $\mathbf{y}$ is a *weighted sum* of the entries of $\mathbf{x}$, with weights given by the corresponding row of A . There are two complementary ways to think about this.

Row picture (dot products). Each row of A dots with $\mathbf{x}$ to produce one entry of $\mathbf{y}$. Row i defines a "recipe" for computing y_i : which entries of $\mathbf{x}$ to combine and with what weights. If most entries in a row are zero, the recipe uses only a few ingredients — this is the basis of *sparse* computation.

Column picture (linear combination). The product $A\mathbf{x}$ is a linear combination of the columns of A :

$$A\mathbf{x} = x_1 \mathbf{a}_1 + x_2 \mathbf{a}_2 + \cdots + x_n \mathbf{a}_n$$

Each column $\mathbf{a}_j$ gets scaled by the corresponding entry x_j , and the results are summed. This is the "columns as destinations" viewpoint from Section B.2.

Computational Cost

Multiplying a *dense* $N \times N$ matrix by an N -vector costs $O(N^2)$ operations. But if the matrix is *sparse* — with only k nonzeros per row — the cost drops to $O(kN)$. For $k = 5$ (a common case), that is $O(5N) = O(N)$: linear in the problem size. This is why **sparse matrix-vector multiplication (SpMV)** is the workhorse operation of large-scale linear algebra.

B.4 Special Matrix Structures

Not all matrices are created equal. Recognizing structure is the key to choosing efficient algorithms.

Diagonal Matrices

A diagonal matrix has nonzero entries only on the main diagonal. Multiplying by it simply scales each entry: $(D\mathbf{x})_i = d_i x_i$. Cost: $O(N)$. Inverting: $(D^{-1})_{ii} = 1/d_i$. Diagonal matrices are the easiest possible case.

Tridiagonal Matrices

A tridiagonal matrix has nonzero entries only on the main diagonal and the two adjacent diagonals:

$$T = \begin{pmatrix} b_1 & c_1 & & \\ a_2 & b_2 & c_2 & \\ & a_3 & b_3 & c_3 \\ & & a_4 & b_4 \end{pmatrix}$$

Each row connects an entry to its two neighbors. Tridiagonal systems $T\mathbf{x} = \mathbf{b}$ can be solved in $O(N)$ time by the *Thomas algorithm* — a simplified form of Gaussian elimination.

Banded and Sparse Matrices

A **banded** matrix has nonzeros within a fixed number of diagonals of the main diagonal. A **sparse** matrix has the vast majority of entries equal to zero — typically a fixed number of nonzeros per row, regardless of N . For example, a matrix with 5 nonzeros per row and $N = 90,000$ columns is 99.99% zeros.

Sparse matrices are stored in compressed formats (CSR, COO) that record only the nonzero entries, reducing memory from $O(N^2)$ to $O(\text{nnz})$.

Intuition

Sparsity reflects locality. A sparse matrix means each equation involves only a few variables — those that are "nearby" in some sense. When a physical system has local interactions (each node couples only to its neighbors), the resulting matrix is sparse. Sparsity is what makes large-scale computation possible.

B.5 The Identity Matrix and Inverses

The **identity matrix** I is the "do nothing" transformation: $I\mathbf{x} = \mathbf{x}$ for every vector $\mathbf{x}$. It has ones on the diagonal and zeros everywhere else.

The **inverse** of a matrix A , written A^{-1} , is the matrix that "undoes" the transformation: $A^{-1}A = I$. If A stretches by 2, then A^{-1} compresses by $\frac{1}{2}$. Not every matrix has an inverse — a matrix that collapses a dimension cannot be undone.

In principle, solving $A\mathbf{x} = \mathbf{b}$ means computing $\mathbf{x} = A^{-1}\mathbf{b}$. In practice, we **never compute A^{-1} explicitly**. For an $N \times N$ matrix, forming A^{-1} costs $O(N^3)$ and produces a dense matrix even when A is sparse. Instead, we use solvers that compute $A^{-1}\mathbf{b}$ without forming A^{-1} itself. When you see $A^{-1}\mathbf{b}$ in this textbook, read it as "the vector $\mathbf{x}$ that solves $A\mathbf{x} = \mathbf{b}$."

B.6 Linear Systems: $A\mathbf{x} = \mathbf{b}$

The equation $A\mathbf{x} = \mathbf{b}$ is the central problem of computational linear algebra. Given a matrix A (the "rules") and a vector $\mathbf{b}$ (the "data"), find the vector $\mathbf{x}$ (the "answer"). In one sentence: *which input $\mathbf{x}$, when transformed by A , produces the desired output $\mathbf{b}$?*

Three Possible Outcomes

1. **Exactly one solution** — when A is invertible (non-singular). The generic, well-behaved case.
2. **No solution** — when $\mathbf{b}$ is not in the range (column space) of A .
3. **Infinitely many solutions** — when A is singular (has a null space). Multiple inputs map to the same output.

Intuition

The geometry of $A\mathbf{x} = \mathbf{b}$. Think of A as a transformation that maps "input space" to "output space." The solution $\mathbf{x}$ is the point in input space that maps to $\mathbf{b}$. If A collapses some directions (its null space), multiple inputs map to the same output. If $\mathbf{b}$ points in a direction that A cannot reach (outside its column space), there is no solution.

B.7 Eigenvalues and Eigenvectors: The DNA of a Matrix

Most vectors change direction when you multiply them by a matrix. But some special vectors only get *scaled* — they point in the same direction (or flip to the opposite direction) after the transformation. These are **eigenvectors**, and the scaling factors are **eigenvalues**:

$$A\mathbf{v} = \lambda \mathbf{v}$$

(B.3)

The matrix A acts on $\mathbf{v}$ by simply multiplying it by λ . No rotation, no shearing — just scaling. If $\lambda > 1$: stretched. If $0 < \lambda < 1$: compressed. If $\lambda < 0$: flipped and scaled. If $\lambda = 0$: sent to zero (the vector is in the null space).

Intuition

Eigenvectors are the natural directions of a matrix. They are the directions along which the matrix acts most simply — as pure scaling. Understanding a matrix's eigenvectors and eigenvalues is like finding the "natural coordinate system" in which the matrix becomes diagonal.

Finding Eigenvalues

For a general $N \times N$ matrix, eigenvalues are the roots of the **characteristic polynomial** $\det(A - \lambda I) = 0$, a polynomial of degree N . There are exactly N eigenvalues (counting multiplicities, possibly complex). For $N > 4$, there is no closed-form formula — eigenvalues must be computed numerically.

Worked Example: A 2×2 Matrix

Let $A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$. The characteristic polynomial is:

$$\det(A - \lambda I) = (4 - \lambda)(3 - \lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2) = 0$$

So $\lambda_1 = 5$ and $\lambda_2 = 2$. For $\lambda_1 = 5$: $(A - 5I)\mathbf{v} = \mathbf{0}$ gives $-v_1 + v_2 = 0$, so $\mathbf{v}_1 = [1, 1]^T$. For $\lambda_2 = 2$: $2v_1 + v_2 = 0$, so $\mathbf{v}_2 = [1, -2]^T$. Along $\mathbf{v}_1$, A stretches by 5; along $\mathbf{v}_2$, by 2. These are the natural axes of A .

Diagonalization

If a matrix A has N linearly independent eigenvectors, we can write $A = P\Lambda P^{-1}$, where P is the matrix of eigenvectors and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_N)$. In this "eigenbasis," A acts as pure scaling along each axis. This is the key to spectral methods: change to the eigenbasis, do simple arithmetic, change back.

B.8 Symmetric and Positive Definite Matrices

A matrix A is **symmetric** if $A = A^T$: $A_{ij} = A_{ji}$ for all i, j . Symmetric matrices have two remarkable properties:

1. **All eigenvalues are real.** No complex values — just real scaling.
2. **Eigenvectors can be chosen orthogonal.** The matrix diagonalizes as $A = Q\Lambda Q^T$, where Q is orthogonal ($Q^{-1} = Q^T$) and Λ is real diagonal.

A symmetric matrix is **positive definite (PD)** if all eigenvalues are strictly positive: $\lambda_i > 0$ for all i . Equivalently:

$$\mathbf{x}^T A \mathbf{x} > 0 \quad \text{for all } \mathbf{x} \neq \mathbf{0} \quad (\text{B.4})$$

This quadratic form measures a kind of "energy." Positive definiteness means the energy is always positive — the matrix defines a bowl in N -dimensional space with a unique minimum at the origin. The linear system $A\mathbf{x} = \mathbf{b}$ is equivalent to finding the bottom of this bowl.

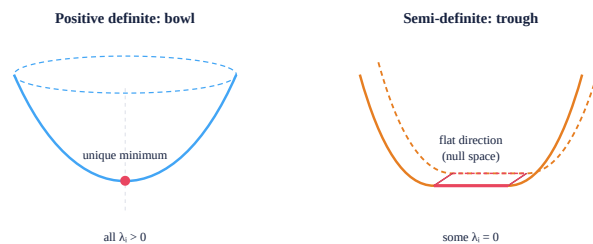


Figure B.2. Left: a positive-definite matrix defines a bowl — every direction curves upward, and the minimum is unique. Right: a positive semi-definite matrix defines a trough — flat along the null-space direction, so infinitely many minimizers exist along the valley floor.

Intuition

SPD = the gold standard. Symmetric positive-definite (SPD) matrices have real positive eigenvalues, are always invertible, and admit the fastest solvers (Cholesky factorization, conjugate gradient). Every desirable property you could want, SPD matrices have.

Semi-definite means allowing $\lambda_i \geq 0$ (some eigenvalues may be zero). A symmetric positive *semi*-definite (SPSD) matrix satisfies $\mathbf{x}^T A \mathbf{x} \geq 0$, with equality for vectors in the null space. SPD matrices are singular but "barely so" — only the null-space directions are collapsed.

B.9 The Condition Number: When Matrices Go Bad

Not all SPD systems are equally easy to solve. Consider two bowls: one is nearly round (a soup bowl), and the other is long and narrow (a bathtub). Both have a unique minimum, but finding the minimum of the bathtub is much harder — gradient descent zigzags across the narrow valley.

The **condition number** quantifies this shape:

$$\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}} \quad (B.5)$$

A condition number near 1 means a round bowl (easy). A condition number of 10^6 means an extremely elongated bowl (hard). Iterative solvers need roughly $O(\sqrt{\kappa})$ iterations.

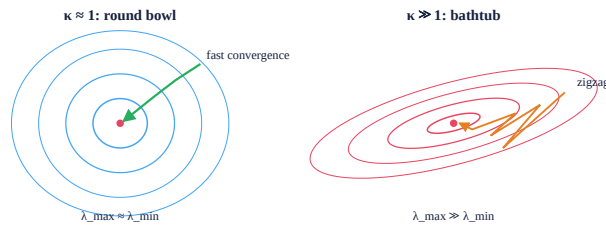


Figure B.3. Contour plots of $E(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x}$. Left: well-conditioned ($\kappa \approx 1$) — nearly circular contours, gradient descent converges quickly. Right: ill-conditioned ($\kappa \gg 1$) — elongated contours, gradient descent zigzags across the narrow valley.

Intuition

Condition number measures amplification of error. If you solve $A\mathbf{x} = \mathbf{b}$ with a small perturbation in $\mathbf{b}$, the perturbation in $\mathbf{x}$ is amplified by at most $\kappa(A)$. For $\kappa = 10^6$ and double-precision arithmetic (16 digits), you may lose 6 digits of accuracy. **Preconditioning** — transforming the system to reduce κ — is the standard remedy.

B.10 Null Spaces and Singular Systems

The **null space** of a matrix A is the set of all vectors that A sends to zero:

$$\text{null}(A) = \{\mathbf{v} : A\mathbf{v} = \mathbf{0}\} \quad (B.6)$$

If the null space contains only $\mathbf{0}$, A is invertible. If it contains nonzero vectors, A is **singular**: it collapses some directions to zero.

Solving Singular Systems

If A is singular, $A\mathbf{x} = \mathbf{b}$ has solutions only when $\mathbf{b}$ is orthogonal to the null space of A^T (for symmetric A , this is the null space of A itself). This is a **compatibility condition**. When it is satisfied, there are infinitely many solutions: if $\mathbf{x}^*$ is one, then $\mathbf{x}^* + \mathbf{v}$ is another for any $\mathbf{v} \in \text{null}(A)$. To select a unique solution, we impose an extra constraint — typically **pinning** (fixing one component) or **projecting out** the null-space component.

B.11 Block Matrices

A **block matrix** is a matrix partitioned into rectangular sub-matrices (blocks):

$$\begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix} \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{pmatrix} = \begin{pmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{pmatrix} \quad (\text{B.7})$$

Block structure arises naturally when a system couples multiple variables. The diagonal blocks $\mathbf{A}_{11}, \mathbf{A}_{22}$ describe how each variable interacts with itself; the off-diagonal blocks $\mathbf{A}_{12}, \mathbf{A}_{21}$ describe the coupling.

Block Elimination and the Schur Complement

From the first block row: $\mathbf{x}_1 = \mathbf{A}_{11}^{-1}(\mathbf{b}_1 - \mathbf{A}_{12}\mathbf{x}_2)$. Substituting into the second row gives:

$$(\mathbf{A}_{22} - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12})\mathbf{x}_2 = \mathbf{b}_2 - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{b}_1$$

The matrix $\mathbf{S} = \mathbf{A}_{22} - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12}$ is the **Schur complement**. It reduces a $2N \times 2N$ system to an $N \times N$ system in $\mathbf{x}_2$ alone. This is a powerful technique: solve for $\mathbf{x}_2$ first, then back-substitute to find $\mathbf{x}_1$.

Worked Example

Consider the 4×4 block system with 2×2 blocks:

$$\begin{pmatrix} 4 & 1 & 1 & 0 \\ 1 & 3 & 0 & 1 \\ 1 & 0 & 5 & 2 \\ 0 & 1 & 2 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 8 \\ 7 \\ 14 \\ 11 \end{pmatrix}$$

Here $\mathbf{A}_{11} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{A}_{12} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \mathbf{I}$, $\mathbf{A}_{21} = \mathbf{I}$, $\mathbf{A}_{22} = \begin{pmatrix} 5 & 2 \\ 2 & 4 \end{pmatrix}$. Since $\mathbf{A}_{12} = \mathbf{A}_{21} = \mathbf{I}$, the Schur complement simplifies to $\mathbf{S} = \mathbf{A}_{22} - \mathbf{A}_{11}^{-1}$. This reduces a 4×4 problem to a 2×2 problem: solve $\mathbf{S}\mathbf{x}_2 = \mathbf{b}_2 - \mathbf{A}_{11}^{-1}\mathbf{b}_1$ for $\mathbf{x}_2$, then back-substitute for $\mathbf{x}_1$.

If the full block system is symmetric indefinite (not SPD), the individual blocks or the Schur complement may still be SPD — allowing the use of fast solvers on the sub-problems even when the full system resists them.

B.12 Orthogonality and Projections

Two vectors $\mathbf{u}$ and $\mathbf{v}$ are **orthogonal** if $\mathbf{u}^T\mathbf{v} = 0$. Geometrically, they are perpendicular. Orthogonality is the most powerful simplifying concept in linear algebra: orthogonal directions do not interfere with each other.

A **projection** decomposes a vector into a component along a given direction and a component perpendicular to it:

$$\text{proj}_{\mathbf{u}} \mathbf{v} = \frac{\mathbf{u}^T\mathbf{v}}{\mathbf{u}^T\mathbf{u}} \mathbf{u} \quad (\text{B.8})$$

An **orthogonal basis** is a set of mutually orthogonal vectors that spans the space. In an orthogonal basis, every vector decomposes uniquely into components along each basis direction, and the components are just projections. This is the mathematical foundation of spectral transforms (Appendix C, Section C.10) and the conjugate gradient method (Appendix C, Section C.12).

Intuition

$\mathbf{A}$ -orthogonality. Given an SPD matrix $\mathbf{A}$, two vectors $\mathbf{u}$ and $\mathbf{v}$ are called $\mathbf{A}$ -orthogonal (or conjugate) if $\mathbf{u}^T \mathbf{A} \mathbf{v} = 0$. This is orthogonality with respect to the "geometry" defined by $\mathbf{A}$ — the bowl shape from Section B.8. The conjugate gradient method (Appendix C) exploits $\mathbf{A}$ -orthogonality to converge in at most N steps.

The geometric approach in this appendix follows Grant Sanderson's *Essence of Linear Algebra* series (3Blue1Brown). For animated visualizations of matrix transformations, eigenvalues, and change of basis, the reader is encouraged to watch the complete series on YouTube.

Numerical Analysis: The Bridge

Appendix A introduced differential equations — the continuous language of physics. Appendix B introduced linear algebra — the language of computation. This appendix connects them. The central question: how do you take a differential equation that lives in the continuous world and turn it into a matrix problem that a computer can solve? We will build one running example — a 2D grid with a hot spot — and solve it with every method from the simplest (Forward Euler) to the most sophisticated (Chebyshev iteration). By the end, every concept from Appendices A and B will have a job to do.

C.1 The Discretization Leap

In calculus, $u(\mathbf{x}, \mathbf{y})$ is a smooth surface — a rule that assigns a number to every point in a region. A computer cannot store infinitely many values. So we make a choice: lay down a grid, sample the function at each grid point, and store only those values.

Intuition

From film to frames. A movie camera captures a continuous scene by taking 24 frames per second. Each frame is a finite approximation of a continuous reality. Discretization is the same idea: a smooth function becomes a finite list of samples, and the spacing between samples determines what features you can resolve.

1D Warmup: The Three-Point Stencil

On a 1D rod with equally spaced points $\mathbf{x}_j = j \Delta \mathbf{x}$, the second derivative at point j can be approximated by:

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_{\mathbf{x}_j} \approx \frac{u_{j-1} - 2u_j + u_{j+1}}{\Delta x^2} \quad (C.1)$$

This is a *weighted sum* of three values — weights $[1, -2, 1]/\Delta x^2$. That weighted sum is one row of a matrix-vector product (Section B.3). Stacking all rows gives the **discrete Laplacian L** : a tridiagonal matrix where each row encodes the stencil at one grid point.

The Running Example: A 3×3 Grid

Now the real thing. Consider a square domain discretized into a 3×3 grid of points, with spacing $\Delta x = \Delta y = 1$. We number the 9 nodes using row-major ordering: $\mathbf{k} = \mathbf{i} + \mathbf{j} \cdot 3$ where $\mathbf{i}$ is the column index and $\mathbf{j}$ is the row index.

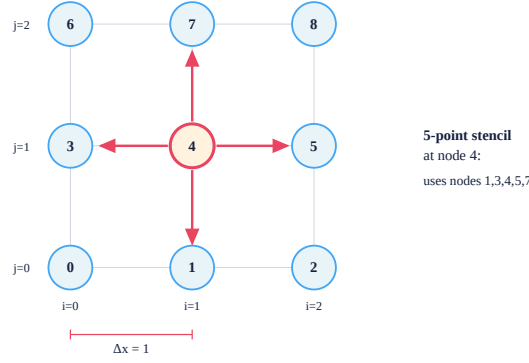


Figure C.1. The 3×3 running example grid. Node numbering is row-major: $k = i + 3j$. The 5-point stencil at node 4 (center) connects to its four direct neighbors. Corner and edge nodes have fewer neighbors.

Building the 2D Laplacian Matrix

At the center node (node 4), the 5-point stencil approximates $\nabla^2 u$ using four neighbors:

$$\nabla^2 u|_4 \approx \frac{u_1 + u_3 + u_5 + u_7 - 4u_4}{\Delta x^2}$$

This is row 4 of the matrix L : coefficients $[0, 1, 0, 1, -4, 1, 0, 1, 0]$.

At boundary nodes, the stencil has *fewer* neighbors. With Neumann (zero-flux) boundary conditions, we simply skip the missing neighbor — no ghost nodes needed. Node 0 (bottom-left corner) has only two neighbors (1 and 3):

$$\nabla^2 u|_0 \approx \frac{u_1 + u_3 - 2u_0}{\Delta x^2}$$

Row 0 of L : $[-2, 1, 0, 1, 0, 0, 0, 0, 0]$. Edge node 1 (bottom edge) has three neighbors:

$$\nabla^2 u|_1 \approx \frac{u_0 + u_2 + u_4 - 3u_1}{\Delta x^2}$$

Assembling all 9 rows (with $\Delta x = 1$) gives the complete 9×9 Laplacian:

$$L = \begin{pmatrix} -2 & 1 & . & 1 & . & . & . & . & . \\ 1 & -3 & 1 & . & 1 & . & . & . & . \\ . & 1 & -2 & . & . & 1 & . & . & . \\ 1 & . & . & -3 & 1 & . & 1 & . & . \\ . & 1 & . & 1 & -4 & 1 & . & 1 & . \\ . & . & 1 & . & 1 & -3 & . & . & 1 \\ . & . & . & 1 & . & . & -2 & 1 & . \\ . & . & . & . & 1 & . & 1 & -3 & 1 \\ . & . & . & . & . & 1 & . & 1 & -2 \end{pmatrix}$$

Dots represent zeros. Check: every row sums to zero (Neumann BCs conserve total heat). The matrix is sparse (at most 5 nonzeros per row), symmetric (Section B.8), and negative semi-definite.

Intuition

This is the moment Appendix A meets Appendix B. The differential operator ∇^2 from calculus has become the matrix L from linear algebra. The continuous heat equation $\partial u / \partial t = D \nabla^2 u$ becomes the matrix ODE:

$$\frac{d\mathbf{u}}{dt} = D L \mathbf{u}$$

A partial differential equation is now an ordinary differential equation whose right-hand side is a sparse matrix-vector product. Everything from here is about solving this system efficiently.

△ Key Insight

Mass matrices. The finite-difference derivation above gives $d\mathbf{u}/dt = D\mathbf{L}\mathbf{u}$ directly. For finite-element methods (Chapter 8), the discretization produces $M d\mathbf{u}/dt = K\mathbf{u}$, where M is a *mass matrix* (not the identity). This does not change the conceptual story but adds one more matrix to handle. Chapter 8 covers this in detail.

C.2 Truncation Error: The Price of Approximation

The three-point stencil (C.1) is an approximation. How good is it? Taylor-expand $u(x \pm \Delta x)$ around x and add them:

$$\frac{u(x + \Delta x) - 2u(x) + u(x - \Delta x)}{\Delta x^2} = u''(x) + \frac{\Delta x^2}{12} u^{(4)}(x) + \dots$$

The error is $O(\Delta x^2)$ — **second order**. Halving Δx reduces the error by a factor of 4. This **truncation error** sets the spatial accuracy.

Order p	Halving h reduces error by	Examples
1	2×	Forward Euler (time), upwind FD (space)
2	4×	Crank-Nicolson, RK2, central FD, standard FVM
4	16×	RK4 (time)

To verify order in practice, run at two resolutions and compute:

$$p = \frac{\log(E_1/E_2)}{\log(h_1/h_2)} \quad (C.2)$$

C.3 Forward Euler: The Simplest Method

We have the matrix ODE $d\mathbf{u}/dt = D\mathbf{L}\mathbf{u}$. The simplest time-stepping approach: approximate the time derivative as a forward difference and evaluate the right-hand side at the *current* time:

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t D\mathbf{L}\mathbf{u}^n = \underbrace{(I + \Delta t D\mathbf{L})}_G \mathbf{u}^n \quad (C.3)$$

This is just a **sparse matrix-vector multiply** (SpMV). No linear system to solve — given $\mathbf{u}^n$, one SpMV produces $\mathbf{u}^{n+1}$. Cost: $O(N)$ per time step. It does not get simpler than this.

Worked Example: Hot Spot on the 3×3 Grid

Set $D = 1$, $\Delta t = 0.05$. Place all the heat at the center node:

$$\mathbf{u}^0 = [0, 0, 0, 9, 0, 0, 0, 0]^T$$

First, compute $\mathbf{L}\mathbf{u}^0$. Most rows give zero because the only nonzero entry in $\mathbf{u}^0$ is at node 4. The rows that have a 1 in column 4 (nodes 1, 3, 5, 7) pick up that value:

$$\mathbf{L}\mathbf{u}^0 = [0, 9, 0, 9, -36, 9, 0, 9]^T$$

Now apply Forward Euler:

$$\mathbf{u}^1 = \mathbf{u}^0 + 0.05 \times \mathbf{L} \mathbf{u}^0 = [0, 0.45, 0, 0.45, 7.20, 0.45, 0, 0.45, 0]^T$$

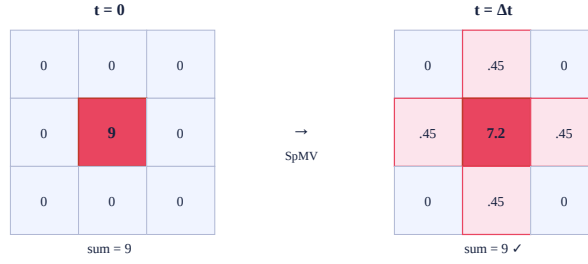


Figure C.2. One Forward Euler step. Heat flows from center ($9 \rightarrow 7.2$) to the four edge-adjacent neighbors ($0 \rightarrow 0.45$). Corners stay at zero — the 5-point stencil does not connect diagonally. Total heat is conserved.

Two things to notice. First, heat only spreads to the four *edge-adjacent* neighbors, not the four corners — this is because the 5-point stencil has no diagonal connections. Second, the total heat is conserved: $\sum \mathbf{u}_j = 9$ before and after. This is guaranteed by the row-sum-zero property of $\mathbf{L}$ under Neumann BCs.

C.4 Stability: When Forward Euler Blows Up

Forward Euler worked beautifully with $\Delta t = 0.05$. What happens if we try $\Delta t = 0.5$?

$$\mathbf{u}^1 = \mathbf{u}^0 + 0.5 \times \mathbf{L} \mathbf{u}^0 = [0, 4.5, 0, 4.5, -9, 4.5, 0, 4.5, 0]^T$$

The center value is -9 — **negative temperature**. This is physically impossible and numerically catastrophic. The next step will amplify the error further, producing wilder oscillations until the solution explodes to infinity. This is **numerical instability**.

Why It Blows Up: Eigenvalue Analysis

Apply Forward Euler to a single eigenmode. If $\mathbf{v}$ is an eigenvector of $\mathbf{L}$ with eigenvalue λ (so $\mathbf{L}\mathbf{v} = \lambda\mathbf{v}$), then:

$$\mathbf{G}\mathbf{v} = (\mathbf{I} + \Delta t \mathbf{D} \mathbf{L}) \mathbf{v} = (1 + \Delta t \mathbf{D} \lambda) \mathbf{v}$$

The **amplification factor** is $g = 1 + \Delta t \mathbf{D} \lambda$. For stability, we need $|g| \leq 1$ for every eigenvalue. Since $\lambda < 0$ for the Laplacian:

$$\Delta t \leq \frac{2}{D |\lambda_{\max}|} \quad (\text{C.4})$$

This is the **CFL condition**. For the 2D discrete Laplacian with Neumann BCs, the eigenvalues are:

$$\lambda_{p,q} = \frac{2}{\Delta x^2} \left(\cos \frac{\pi p}{N_x} - 1 \right) + \frac{2}{\Delta y^2} \left(\cos \frac{\pi q}{N_y} - 1 \right) \quad (\text{C.5})$$

For our 3×3 grid ($N_x = N_y = 3$, $\Delta x = \Delta y = 1$), the most negative eigenvalue is $\lambda_{2,2} = -6$, giving:

$$\Delta t \leq \frac{2}{1 \times 6} = \frac{1}{3} \approx 0.333$$

Our stable step ($\Delta t = 0.05$) is well within the limit. The unstable step ($\Delta t = 0.5$) violates it. In general, the CFL condition for Forward Euler on a 2D grid is:

$$\Delta t \leq \frac{\Delta x^2}{4D} \quad (\text{2D, square grid})$$

The *quadratic* dependence on Δx is brutal: halving the grid spacing requires quartering the time step.

Stability Regions

For a general time-stepper, the **stability region** is the set of complex values $z = \Delta t \lambda$ for which $|g(z)| \leq 1$:

- **Forward Euler:** A disk of radius 1 centered at $z = -1$. Limited.
- **RK4:** A larger region extending further along the negative real axis. Allows $\sim 2.8\times$ larger Δt .
- **Backward Euler:** Everything outside a disk of radius 1 centered at $z = +1$ — includes the *entire left half-plane*. Unconditionally stable.
- **Crank-Nicolson:** The entire left half-plane, but with $|g| = 1$ on the imaginary axis.

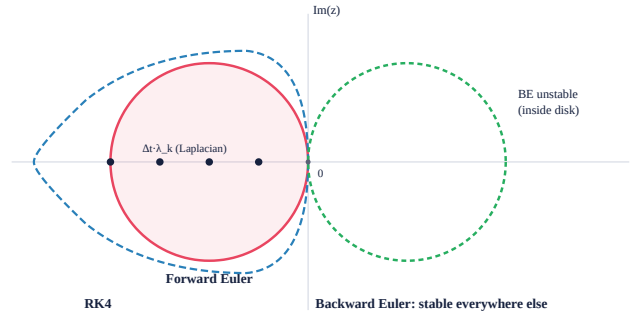


Figure C.3. Stability regions in the complex $z = \Delta t \lambda$ plane. All eigenvalues of the Laplacian (black dots) must fall inside the stability region. Backward Euler contains the entire left half-plane, so it is unconditionally stable.

Intuition

Explicit methods put a speed limit on Δt ; implicit methods remove it. But implicit methods pay a price: instead of a matrix-vector multiply, they require solving a linear system $A\mathbf{x} = \mathbf{b}$ at each step. This is the central engineering trade-off of time-stepping.

C.5 Backward Euler: Unconditional Stability

Forward Euler evaluates the right-hand side at the *current* time. What if we evaluate it at the *future* time instead?

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t D L \mathbf{u}^{n+1} \quad (C.6)$$

The unknown $\mathbf{u}^{n+1}$ appears on both sides. Rearranging:

$$\underbrace{(I - \Delta t D L)}_A \mathbf{u}^{n+1} = \mathbf{u}^n$$

This is $A\mathbf{x} = \mathbf{b}$ — a **linear system**. The unknown is $\mathbf{x} = \mathbf{u}^{n+1}$, the right-hand side is $\mathbf{b} = \mathbf{u}^n$. We must *solve* this system at every time step.

Intuition

This is the climactic moment. The PDE from Appendix A has become a linear system from Appendix B. The matrix $A = I - \Delta t D L$ inherits structure from the Laplacian: it is sparse (Section B.4), symmetric (Section B.8), and positive definite (because L is negative semi-definite and I pushes all eigenvalues above zero). It is SPD — the gold standard from Section B.8.

Worked Example: Same Grid, Same Hot Spot

With $D = 1$, $\Delta t = 0.05$, the matrix $A = I - 0.05 L$ for our 3×3 grid is:

$$A = \begin{pmatrix} 1.1 & -0.05 & . & -0.05 & . & . & . & . & . \\ -0.05 & 1.15 & -0.05 & . & -0.05 & . & . & . & . \\ . & -0.05 & 1.1 & . & . & -0.05 & . & . & . \\ -0.05 & . & . & 1.15 & -0.05 & . & -0.05 & . & . \\ . & -0.05 & . & -0.05 & 1.2 & -0.05 & . & -0.05 & . \\ . & . & -0.05 & . & -0.05 & 1.15 & . & . & -0.05 \\ . & . & . & -0.05 & . & . & 1.1 & -0.05 & . \\ . & . & . & . & -0.05 & . & -0.05 & 1.15 & -0.05 \\ . & . & . & . & . & -0.05 & . & -0.05 & 1.1 \end{pmatrix}$$

Solving $A \mathbf{u}^1 = \mathbf{u}^0$ (with $\mathbf{u}^0 = [0, 0, 0, 0, 9, 0, 0, 0, 0]^T$) gives:

$$\mathbf{u}^1 \approx [0.016, 0.405, 0.016, 0.405, 7.316, 0.405, 0.016, 0.405, 0.016]^T$$

Compare with Forward Euler: the center value is 7.32 vs. 7.20, and the corners now receive a tiny amount of heat (0.016) through the implicit coupling. The result is similar but computed fundamentally differently — by solving a system rather than multiplying a matrix. The payoff: Backward Euler is **unconditionally stable**. We could use $\Delta t = 0.5$ or $\Delta t = 100$ without blowup.

C.6 Crank-Nicolson: Second-Order Accuracy

Forward Euler (first-order, explicit) and Backward Euler (first-order, implicit) sit at opposite ends of a tradeoff. Can we get the best of both? **Crank-Nicolson** averages them:

$$(I - \frac{\Delta t}{2} D L) \mathbf{u}^{n+1} = (I + \frac{\Delta t}{2} D L) \mathbf{u}^n \quad (C.7)$$

Still $A \mathbf{x} = \mathbf{b}$, but now both sides involve matrix operations. The left-hand matrix is SPD; the right-hand side requires one additional SpMV. The reward: **second-order accuracy in time** while remaining unconditionally stable.

Comparison on the 3×3 Grid

All three methods, same problem, $\Delta t = 0.05$:

Node	Forward Euler	Backward Euler	Crank-Nicolson
Center (4)	7.200	7.316	7.257
Edge (1,3,5,7)	0.450	0.405	0.428
Corner (0,2,6,8)	0.000	0.016	0.007
Accuracy	$O(\Delta t)$	$O(\Delta t)$	$O(\Delta t^2)$
Stability	CFL restricted	Unconditional	Unconditional
Cost/step	1 SpMV	solve $A \mathbf{x} = \mathbf{b}$	1 SpMV + solve $A \mathbf{x} = \mathbf{b}$

Crank-Nicolson sits between FE and BE at each node — this is exactly what "averaging" means geometrically. The same pattern extends to **BDF2** and other implicit methods: each produces a system $(aI - bDL) \mathbf{u}^{n+1} = \mathbf{known}$, differing only in the constants a, b . Chapters 10 and 11 develop all these methods for cardiac simulation.

C.7 The Lax Equivalence Theorem

We have now seen three methods. Before adding more, a powerful theorem ties the theory together. Every numerical scheme must have three properties:

- **Consistency:** As $\Delta x, \Delta t \rightarrow 0$, the discrete scheme approaches the continuous PDE. (Truncation error vanishes.)
- **Stability:** Errors do not grow unboundedly. (Amplification factor controlled.)
- **Convergence:** The numerical solution approaches the true solution.

Mathematical Note

Lax Equivalence Theorem (1956): For a consistent linear scheme, *stability is equivalent to convergence*.

Verify consistency (check truncation error, Section C.2) and stability (check CFL, Section C.4), and convergence is guaranteed. This is the theoretical foundation for Chapters 10 and 11.

C.8 Stiffness and Operator Splitting

A system is **stiff** when it contains widely separated time scales — some components change very fast, others very slowly. Cardiac electrophysiology is inherently stiff. During the upstroke, sodium channels open in ~ 1 ms, changing voltage by 100 mV. During the plateau, voltage changes by a few mV over 200 ms.

Intuition

Stiffness is "rush hour traffic." A highway where some cars travel at 200 km/h and others at 2 km/h. A simulation using one Δt for both is like setting the speed limit for the fastest car — the slow ones barely move per step. **Operator splitting** is separate lanes: treat the fast dynamics (ionic currents) with a small step, and the slow dynamics (diffusion) with a larger step.

The Idea: Divide and Conquer

Instead of solving $\partial u / \partial t = \mathcal{D}(u) + \mathcal{R}(u)$ (diffusion + reaction) all at once, split each time step into sub-problems:

Godunov (Lie) splitting — first order:

1. **Reaction:** Solve $\partial u / \partial t = \mathcal{R}(u)$ for one full step $\Delta t \rightarrow$ gives u^*
2. **Diffusion:** Solve $\partial u / \partial t = \mathcal{D}(u)$ for one full step Δt , starting from $u^* \rightarrow$ gives u^{n+1}

Strang splitting — second order:

1. **Half diffusion:** Solve $\partial u / \partial t = \mathcal{D}(u)$ for $\Delta t/2 \rightarrow$ gives u^*
2. **Full reaction:** Solve $\partial u / \partial t = \mathcal{R}(u)$ for Δt , starting from $u^* \rightarrow$ gives u^{**}
3. **Half diffusion:** Solve $\partial u / \partial t = \mathcal{D}(u)$ for $\Delta t/2$, starting from $u^{**} \rightarrow$ gives u^{n+1}

Why This Is Powerful

The **reaction sub-problem** $\mathcal{R}(u)$ is a system of ODEs at each grid point independently — no spatial coupling, embarrassingly parallel. On our 3×3 grid, this means solving 9 independent ODE systems (one per node), each handling its own ion channel dynamics. Use Rush-Larsen or explicit methods (Chapter 9).

The **diffusion sub-problem** $\mathcal{D}(u)$ is a linear PDE with no reaction stiffness. It produces exactly the $A\mathbf{x} = \mathbf{b}$ systems from Sections C.5–C.6. The matrix A is SPD, sparse, and the same at every step — we can precompute and reuse.

Each sub-problem is much easier than the combined system. Chapter 9 develops operator splitting in full detail for cardiac simulation.

C.9 Solving $A\mathbf{x} = \mathbf{b}$: Three Families

Sections C.5–C.6 showed that implicit time-stepping requires solving a linear system at every step. How do we actually solve it? There are three families of methods, each with distinct strengths:

	Direct	Spectral	Iterative
Idea	Factor $A = LL^T$	Change to eigenbasis	Improve a guess
Cost	$O(N^{3/2})$ for 2D sparse	$O(N \log N)$	$O(kN)$, $k \ll N$
Memory	Large (fill-in)	$O(N)$	$O(N)$
Requirements	None	Structured grid + uniform BCs	SPD for CG
GPU-friendly	No (sequential)	Yes (FFT)	Yes (SpMV)

For the large sparse systems of cardiac simulation ($N = 10^4$ to 10^6), iterative and spectral methods dominate. We do not need the exact solution — only one accurate enough that time-stepping error dominates. Sections C.10–C.14 develop each approach.

C.10 Transform Methods: Spectral Solvers

Imagine listening to an orchestra — violins, cellos, flutes all at once. Your ear receives a single pressure wave, but your brain separates it into individual pitches. A **transform** does the same to a vector: re-expresses it in a new basis where the problem becomes simpler.

Intuition

The discrete Laplacian has known eigenvectors — sines and cosines. In the eigenvector basis, the matrix becomes diagonal (Section B.7). The system $A\mathbf{x} = \mathbf{b}$ becomes N independent scalar divisions. No iteration, no factorization — just divide.

Choosing the Right Basis

Cosines → **Neumann BCs**. The functions $\cos(k\pi x/L)$ have zero derivative at both endpoints — they automatically satisfy the zero-flux boundary condition. The **Discrete Cosine Transform (DCT)** decomposes a vector into these modes.

Sines → **Dirichlet BCs**. The functions $\sin(k\pi x/L)$ are zero at both endpoints — they automatically satisfy fixed-value boundary conditions. The **Discrete Sine Transform (DST)** decomposes a vector into these modes.

Intuition

The basis function is the boundary condition. Cosines have flat endpoints (zero slope) → Neumann. Sines pass through zero → Dirichlet. No ghost nodes, no special boundary treatment.

Eigenvalues

The 1D Neumann eigenvalues come from equation (C.5) with one dimension. For Dirichlet:

$$\lambda_k^{\text{DST}} = \frac{2}{\Delta x^2} \left(\cos \frac{\pi k}{N+1} - 1 \right), \quad k = 1, \dots, N \quad (\text{C.8})$$

The Neumann eigenvalues include $\lambda_0 = 0$ (the null space — a constant is unchanged by the Laplacian). The Dirichlet eigenvalues have no zero eigenvalue — the boundary condition pins the solution. In 2D, eigenvalues are sums of 1D eigenvalues:

$$\lambda_{p,q} = \lambda_p^{(x)} + \lambda_q^{(y)} \quad (\text{C.9})$$

The Algorithm

To solve $(I - \Delta t D L) \mathbf{u}^{n+1} = \mathbf{u}^n$:

1. **Forward transform:** Decompose $\mathbf{u}^n$ into spectral coefficients $\hat{\mathbf{u}}_k$ via DCT or DST.
2. **Divide:** $\hat{\mathbf{u}}_k^{n+1} = \hat{\mathbf{u}}_k^n / (1 - \Delta t D \lambda_k)$.
3. **Inverse transform:** Recover $\mathbf{u}^{n+1}$ from $\hat{\mathbf{u}}_k^{n+1}$.

The transforms cost $O(N \log N)$ via FFT. The division is $O(N)$. Total: $O(N \log N)$ per time step — faster than any iterative solver. In 2D, apply the 1D transform along each axis separately (the transforms are separable).

Worked Example: 1D Neumann, $N = 4$

Rod with $N = 4$ points, $\Delta x = 1$, $D = 1$, Neumann BCs. Initial: $\mathbf{u} = [4, 2, 1, 3]^T$. One Backward Euler step with $\Delta t = 0.1$.

Eigenvalues from (C.5): $\lambda_0 = 0$, $\lambda_1 \approx -0.586$, $\lambda_2 = -2$, $\lambda_3 \approx -3.414$.

k	$\hat{\mathbf{u}}_k$ (DCT)	$1 - 0.1\lambda_k$	$\hat{\mathbf{u}}_k^{n+1}$ (divide)
0	5.000	1.000	5.000
1	0.924	1.059	0.873
2	2.000	1.200	1.667
3	-0.383	1.341	-0.285

Inverse DCT gives $\mathbf{u}^{n+1}$. Higher modes are damped more — exactly what diffusion does. The mean ($k = 0$) is preserved because Neumann BCs conserve total heat.

Key Insight

When spectral methods fail. They require a **structured grid** and **uniform BCs per axis**. On irregular geometries, unstructured meshes, or mixed BCs, the Laplacian does not have known eigenvectors. Fall back to iterative methods. Chapter 15 describes the three-tier automatic selection used in Engine V1.

C.11 Introduction to Iterative Methods

When spectral methods cannot be used — irregular geometry, anisotropic conductivity, mixed boundary conditions — we need a general-purpose approach. **Iterative methods** start from an initial guess $\mathbf{x}_0$ and

improve it step by step:

The iterative loop:

1. Compute residual: $\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$
2. If $\|\mathbf{r}_k\|/\|\mathbf{b}\| < \varepsilon$: stop (converged)
3. Compute an update direction $\mathbf{p}_k$ and step size α_k
4. $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$
5. Go to 1

Each iteration costs $O(N)$ — one SpMV to compute $\mathbf{A}\mathbf{p}$. The methods differ in *how they choose the direction* $\mathbf{p}_k$. A good choice converges in few iterations; a bad one zigzags forever.

 Intuition

Warm-starting is free lunch. In time-stepping, we solve $\mathbf{A}\mathbf{x} = \mathbf{b}$ at every step, and the solution changes only slightly between steps. Using the previous step's solution as $\mathbf{x}_0$ starts us close to the answer — convergence in 3–8 iterations instead of 20–50.

The three iterative methods below form a progression: CG is the foundation, PCG adds preconditioning for faster convergence, and Chebyshev removes global communication for GPU scalability.

The Running Example for Iterative Methods

For Sections C.12–C.14, we switch to a **4-point 1D rod** ($\Delta x = 1$, $D = 1$, Neumann BCs) so we can trace every number by hand. The Backward Euler system with $\Delta t = 0.1$ is:

$$\mathbf{A} = \begin{pmatrix} 1.1 & -0.1 & 0 & 0 \\ -0.1 & 1.2 & -0.1 & 0 \\ 0 & -0.1 & 1.2 & -0.1 \\ 0 & 0 & -0.1 & 1.1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 4 \\ 2 \\ 1 \\ 3 \end{pmatrix}$$

$\mathbf{A}$ is SPD and tridiagonal. The exact solution is $\mathbf{x}^* \approx [3.82, 2.08, 1.28, 2.82]^T$.

C.12 Conjugate Gradient (CG)

Imagine you are blindfolded in a hilly landscape, trying to find the lowest valley. You can feel the slope beneath your feet. The obvious strategy: always walk downhill. This is **gradient descent**. But in a long, narrow valley (an ill-conditioned problem — Section B.9), gradient descent zigzags back and forth, making painfully slow progress.

The **conjugate gradient** method fixes this with a geometric idea from Section B.12: choose search directions that are ***A-conjugate*** (orthogonal with respect to the bowl shape defined by $\mathbf{A}$). Each conjugate direction eliminates error along one dimension permanently, guaranteeing convergence in at most N steps.

The Energy Minimization Viewpoint

When $\mathbf{A}$ is SPD, solving $\mathbf{A}\mathbf{x} = \mathbf{b}$ is equivalent to minimizing:

$$E(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x} \tag{C.10}$$

The gradient is $\nabla E = \mathbf{A}\mathbf{x} - \mathbf{b} = -\mathbf{r}$, where $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$ is the **residual**. CG builds conjugate search directions cheaply — each new direction needs only the current residual and the previous direction.

The picture in Figure C.4 makes the payoff visible. The energy $E(\mathbf{x})$ is a bowl; its level sets are nested ellipses, stretched long and thin when $\mathbf{A}$ is ill-conditioned. Steepest descent always steps straight downhill, so it hits each contour at a right angle and ricochets across the narrow valley — dozens of short steps for an elongated bowl. Conjugate gradient corrects each direction against the bowl's shape, so it reaches the exact minimum in at most N steps (here, two).

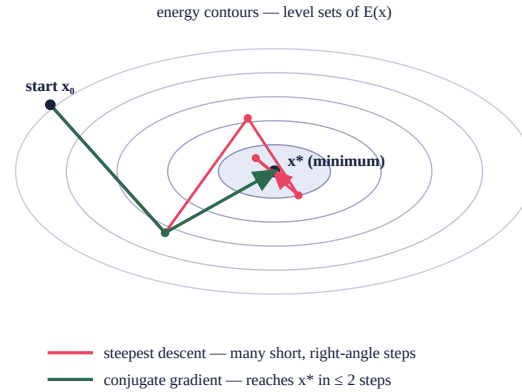


Figure C.4. Steepest descent versus conjugate gradient minimizing the same quadratic energy $E(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x}$. The contours are level sets of E ; the long, thin ellipses signal an ill-conditioned $\mathbf{A}$ (large κ). Steepest descent (red) steps straight downhill and meets each contour at a right angle, zig-zagging across the narrow valley in many small steps. Conjugate gradient (green) takes the same first step, then uses an $\mathbf{A}$ -conjugate direction to land on the minimum $\mathbf{x}^*$ — for this 2D problem, in $N = 2$ steps.

The CG Algorithm

Algorithm C.1: Conjugate Gradient

Input: SPD matrix $\mathbf{A}$, RHS $\mathbf{b}$, initial guess $\mathbf{x}_0$, tolerance ϵ

1. $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$, $\mathbf{p}_0 = \mathbf{r}_0$
2. **for** $k = 0, 1, 2, \dots$
3. $\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}$ (step size)
4. $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$ (update solution)
5. $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k$ (update residual)
6. **if** $\|\mathbf{r}_{k+1}\| / \|\mathbf{b}\| < \epsilon$: **stop**
7. $\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}$ (conjugacy weight)
8. $\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$ (new search direction)
9. **end for**

Each iteration: one SpMV ($\mathbf{A}\mathbf{p}_k$), two inner products ($\mathbf{r}^T \mathbf{r}$ and $\mathbf{p}^T \mathbf{A} \mathbf{p}$), and a few vector updates. Cost per iteration: $O(N)$.

Worked Example: CG on the 4-Point Rod

Start with $\mathbf{x}_0 = [0, 0, 0, 0]^T$ (cold start).

Iteration 1. $\mathbf{r}_0 = \mathbf{b} = [4, 2, 1, 3]^T$, $\mathbf{p}_0 = \mathbf{r}_0$.

$$\mathbf{A}\mathbf{p}_0 = [4.2, 1.9, 0.7, 3.2]^T$$

$$\alpha_0 = \frac{\mathbf{r}_0^T \mathbf{r}_0}{\mathbf{p}_0^T \mathbf{A} \mathbf{p}_0} = \frac{30}{30.9} = 0.971$$

$$\mathbf{x}_1 = 0.971 \times [4, 2, 1, 3]^T = [3.88, 1.94, 0.97, 2.91]^T$$

$$\mathbf{r}_1 = \mathbf{r}_0 - 0.971 \times \mathbf{A} \mathbf{p}_0 = [-0.08, 0.16, 0.32, -0.11]^T$$

Iteration 2. $\beta_0 = \|\mathbf{r}_1\|^2 / \|\mathbf{r}_0\|^2 = 0.144/30 = 0.0048$.

$$\mathbf{p}_1 = \mathbf{r}_1 + 0.0048 \mathbf{p}_0 = [-0.06, 0.16, 0.32, -0.09]^T$$

$$\alpha_1 = 0.84, \quad \mathbf{x}_2 = [3.83, 2.08, 1.24, 2.84]^T$$

Convergence summary:

Iteration	$\ \mathbf{r}\ /\ \mathbf{b}\ $
0	100%
1	6.9%
2	0.3%
3	< 0.01%

Three iterations to reach machine-level residual on a 4-point system. CG is guaranteed to converge in at most N steps (in exact arithmetic).

Convergence Rate

The error after k iterations satisfies:

$$\|\mathbf{x}_k - \mathbf{x}^*\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|\mathbf{x}_0 - \mathbf{x}^*\|_A \quad (C.11)$$

where $\kappa = \lambda_{\max}/\lambda_{\min}$ is the condition number (Section B.9). CG needs $O(\sqrt{\kappa})$ iterations. For the Laplacian, $\kappa \propto N$, so unpreconditioned CG needs $O(\sqrt{N})$ iterations — fine for small problems, too many for $N = 90,000$.

Key Insight

CG requires $\mathbf{A}$ to be SPD. If $\mathbf{A}$ is symmetric but indefinite (both positive and negative eigenvalues), CG breaks down. The full bidomain block system (Chapter 13) is symmetric indefinite — use MINRES or GMRES. But the decoupled sub-problems (Chapter 14) are each SPD, so CG works on them individually.

C.13 Preconditioned Conjugate Gradient (PCG)

CG converges in $O(\sqrt{\kappa})$ iterations. For a fine cardiac mesh with $N = 90,000$ nodes, $\kappa \propto N$, so CG needs ~ 300 iterations — expensive. The solution: **preconditioning**.

The Idea

A **preconditioner** $\mathbf{M}$ is a matrix that approximates $\mathbf{A}$ but is cheap to invert. Instead of solving $\mathbf{A}\mathbf{x} = \mathbf{b}$, we solve $\mathbf{M}^{-1}\mathbf{A}\mathbf{x} = \mathbf{M}^{-1}\mathbf{b}$. If $\mathbf{M} \approx \mathbf{A}$, then $\mathbf{M}^{-1}\mathbf{A} \approx \mathbf{I}$, so $\kappa(\mathbf{M}^{-1}\mathbf{A}) \approx 1$ — a round bowl instead of a bathtub. CG on the preconditioned system converges in far fewer iterations.

Intuition

Preconditioning reshapes the bowl. Recall from Section B.9: an ill-conditioned system looks like an elongated bathtub — gradient descent zigzags across the narrow valley. The preconditioner stretches the bathtub back toward a round bowl, making CG converge in a handful of iterations instead of hundreds.

Jacobi Preconditioning

The simplest preconditioner: $\mathbf{M} = \text{diag}(\mathbf{A})$. One line of code, $O(N)$ to apply, and it is often enough to halve the iteration count. For our 4-point rod:

$$\mathbf{M}^{-1} = \text{diag}\left(\frac{1}{1.1}, \frac{1}{1.2}, \frac{1}{1.2}, \frac{1}{1.1}\right) = \text{diag}(0.909, 0.833, 0.833, 0.909)$$

The PCG Algorithm

Algorithm C.2: Preconditioned Conjugate Gradient

Input: SPD matrix $\mathbf{A}$, RHS $\mathbf{b}$, initial guess $\mathbf{x}_0$, preconditioner $\mathbf{M}$, tolerance ϵ

1. $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$, $\mathbf{z}_0 = \mathbf{M}^{-1}\mathbf{r}_0$, $\mathbf{p}_0 = \mathbf{z}_0$
2. **for** $k = 0, 1, 2, \dots$
3. $\alpha_k = \frac{\mathbf{r}_k^T \mathbf{z}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}$
4. $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$
5. $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k$
6. **if** $\|\mathbf{r}_{k+1}\| / \|\mathbf{b}\| < \epsilon$: **stop**
7. $\mathbf{z}_{k+1} = \mathbf{M}^{-1} \mathbf{r}_{k+1}$ (apply preconditioner)
8. $\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{z}_{k+1}}{\mathbf{r}_k^T \mathbf{z}_k}$
9. $\mathbf{p}_{k+1} = \mathbf{z}_{k+1} + \beta_k \mathbf{p}_k$
10. **end for**

The only difference from CG (Algorithm C.1) is: (a) the preconditioner solve $\mathbf{z} = \mathbf{M}^{-1}\mathbf{r}$ at line 7, and (b) the inner products use $\mathbf{z}$ instead of $\mathbf{r}$ for the direction computation. When $\mathbf{M} = \mathbf{I}$, PCG reduces to CG exactly.

Cost vs. Benefit

Each PCG iteration costs slightly more than CG (one extra $\mathbf{M}^{-1}$ application). But the iteration count drops:

	CG	PCG (Jacobi)
Iterations on 4×4 rod	3	3 (system too small to benefit)
Iterations on 300×300 mesh	~ 300	~ 50
Cost per iteration	1 SpMV + 2 dots	1 SpMV + 2 dots + 1 diagonal scale

For our tiny 4-point example, preconditioning adds no benefit — the system is already well-conditioned. The payoff appears at scale: on a 90,000-node cardiac mesh, Jacobi preconditioning typically cuts iterations from $O(\sqrt{N})$ to $O(\sqrt{N}/c)$ with $c \approx 3\text{--}5$. More sophisticated preconditioners (incomplete Cholesky, AMG) reduce further.

C.14 Chebyshev Iteration

CG and PCG have a hidden cost: two **inner products** per iteration (steps 3 and 7/8 in Algorithms C.1/C.2). An inner product $\mathbf{r}^T \mathbf{z}$ requires summing N products across all nodes — a **global synchronization** where every processor must wait for every other processor before continuing. On a GPU with thousands of cores, these synchronization points become the bottleneck.

Intuition

The GPU bottleneck. A SpMV is embarrassingly parallel — each row of $\mathbf{A}$ can be processed independently. But an inner product requires collecting results from all rows into a single number. On a GPU, this "reduce" operation stalls thousands of cores waiting for one sum. Two reduces per iteration $\times$ thousands of time steps = significant overhead.

Chebyshev's Deal

Chebyshev iteration offers a trade: *give me the eigenvalue bounds $\lambda_{\min}$ and $\lambda_{\max}$ of $\mathbf{A}$, and I will converge at the same rate as CG with **zero inner products**.* The update uses only local operations — SpMV, vector additions, and scalar multiplications:

$$\mathbf{x}_{k+1} = \omega_{k+1} \left(\frac{2}{\gamma} M^{-1} \mathbf{r}_k + \mathbf{x}_k - \mathbf{x}_{k-1} \right) + \mathbf{x}_{k-1} \quad (\text{C.12})$$

where $\gamma = (\lambda_{\max} + \lambda_{\min})/2$ is the spectral center and ω_k are Chebyshev-derived weights computed from a simple recurrence:

$$\omega_1 = 1, \quad \omega_2 = \frac{2}{2 - \rho^2}, \quad \omega_{k+1} = \frac{1}{1 - \rho^2 \omega_k / 4}$$

with $\rho = (\lambda_{\max} - \lambda_{\min})/(\lambda_{\max} + \lambda_{\min})$.

The Three-Term Recurrence

Chebyshev iteration is a **three-term recurrence**: each update uses $\mathbf{x}_k$ and $\mathbf{x}_{k-1}$ (the current and previous iterates). This is what enables zero inner products — the algorithm looks ahead using its knowledge of the eigenvalue bounds rather than adapting to the residual at each step.

Algorithm C.3: Preconditioned Chebyshev Iteration

Input: SPD matrix $\mathbf{A}$, RHS $\mathbf{b}$, initial guess $\mathbf{x}_0$, preconditioner $\mathbf{M}$, eigenvalue bounds $\lambda_{\min}, \lambda_{\max}$, iterations K

Setup: $\gamma = (\lambda_{\max} + \lambda_{\min})/2$, $\delta = (\lambda_{\max} - \lambda_{\min})/2$

1. $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$, $\mathbf{x}_1 = \mathbf{x}_0 + (1/\gamma) M^{-1} \mathbf{r}_0$, $\omega_1 = 1$
2. **for** $k = 1, 2, \dots, K - 1$
3. $\omega_{k+1} = 1 / (1 - \delta^2 \omega_k / (4\gamma^2))$
4. $\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$
5. $\mathbf{x}_{k+1} = \omega_{k+1} \left(\frac{2}{\gamma} M^{-1} \mathbf{r}_k + \mathbf{x}_k - \mathbf{x}_{k-1} \right) + \mathbf{x}_{k-1}$
6. **end for**

Every operation is local — no inner products, no global synchronization. This makes Chebyshev iteration **ideal for GPU execution**.

The Practical Workflow: CG Then Chebyshev

Chebyshev needs eigenvalue bounds, but CG discovers them automatically. The standard approach:

- 1. **First few time steps:** Use PCG. Track the extreme eigenvalue estimates from the CG recurrence coefficients (the Lanczos connection).
- 2. **Remaining time steps:** Switch to Chebyshev using the estimated bounds. Since the matrix **A** is the same at every step (it depends only on **Δt**, **D**, and the mesh), the bounds computed once are valid forever.

Comparison: CG vs. PCG vs. Chebyshev

	CG	PCG	Chebyshev
Inner products/iter	2	2	0
SpMV/iter	1	1	1
Adapts to spectrum?	Yes	Yes	No (fixed bounds)
Needs eigenvalue bounds?	No	No	Yes
GPU scalability	Good	Good	Excellent
Typical use	Small problems	General purpose	GPU time-stepping

 Intuition

The hierarchy in practice. Start with PCG (robust, adaptive). If GPU performance matters and the same **A** is used for many solves (as in time-stepping), switch to Chebyshev after a few PCG steps calibrate the eigenvalue bounds. Chapter 15 describes how Engine V1 implements this three-tier strategy: spectral (if possible) → Chebyshev (if eigenvalue bounds known) → PCG (fallback).

The numerical analysis in this appendix draws on LeVeque's *Finite Difference Methods for ODEs and PDEs* (2007) and Trefethen & Bau's *Numerical Linear Algebra* (1997). The Chebyshev iteration follows Gutknecht & Röllin (2002). The pedagogical approach follows 3Blue1Brown.

Introduction to PyTorch

This appendix is a self-contained tutorial on PyTorch, written for scientists who know Python and perhaps some NumPy but have never touched PyTorch. We start at the very beginning — creating a single number — and build up through arithmetic, matrices, linear algebra, GPU computing, automatic differentiation, neural networks, and the intermediate patterns that bridge them to production simulation code. By the final section, every PyTorch concept you have learned will map directly onto a piece of Engine V5.4's source code, and you will realize you already understand the entire computational backend of this textbook's cardiac simulation engine.

D.1 Installation and Your First Tensor

Install PyTorch with a single command:

```
pip install torch
```

Now open a Python shell and create your first tensor:

```
import torch

x = torch.tensor(3.14)
print(x)           # tensor(3.1400)
print(x.shape)     # torch.Size([])      ← a scalar has an empty shape
print(x.dtype)     # torch.float32      ← default floating-point type
print(x.item())    # 3.14               ← extract the plain Python number
```

A **tensor** is just a number — or a collection of numbers — that PyTorch knows how to do math on. A scalar (single number) is a 0-dimensional tensor. A list of numbers is a 1-D tensor (a vector). A grid of numbers is a 2-D tensor (a matrix). And it keeps going: 3-D, 4-D, any number of dimensions. But at the end of the day, it is just numbers in a box with a shape.

```
scalar = torch.tensor(42.0)           # shape: []      (0-D)
vector = torch.tensor([1.0, 2.0, 3.0]) # shape: [3]    (1-D)
matrix = torch.tensor([[1, 2], [3, 4], [5, 6]]) # shape: [3, 2] (2-D)
cube = torch.zeros(2, 3, 4)           # shape: [2, 3, 4] (3-D)

print(scalar.ndim, vector.ndim, matrix.ndim, cube.ndim) # 0 1 2 3
```

D.2 Arithmetic: Scalars, Vectors, and Matrices

PyTorch arithmetic works the same way as Python arithmetic, but it operates on entire tensors at once — no loops required.

Scalar arithmetic.

```
a = torch.tensor(3.0)
b = torch.tensor(2.0)

print(a + b) # tensor(5.)
print(a - b) # tensor(1.)
print(a * b) # tensor(6.)
print(a / b) # tensor(1.5000)
print(a ** b) # tensor(9.)      ← 3^2
```

Vector arithmetic. Every operation applies element-by-element:


```

u = torch.tensor([1.0, 2.0, 3.0])
v = torch.tensor([4.0, 5.0, 6.0])

print(u + v)          # tensor([5., 7., 9.])
print(u * v)          # tensor([ 4., 10., 18.]) ← element-wise, NOT dot product
print(2.0 * u)         # tensor([2., 4., 6.]) ← scalar × vector

```

The **dot product** (inner product) of two vectors is a separate operation:

```
print(torch.dot(u, v)) # tensor(32.) ← 1·4 + 2·5 + 3·6 = 32
```

Matrix arithmetic follows the same pattern. Element-wise operations use `+`, `*`, etc. Matrix multiplication uses the `@` operator (or `torch.matmul`):

```

A = torch.tensor([[1., 2.], [3., 4.]])
B = torch.tensor([[5., 6.], [7., 8.]])

print(A * B)      # element-wise: tensor([[ 5., 12.], [21., 32.]])
print(A @ B)      # matrix multiply: tensor([[19., 22.], [43., 50.]])

```

If you have used NumPy before, this should feel identical — because it is. PyTorch deliberately mirrors the NumPy API. The only visible differences appear when we start using GPUs (D.8) and gradients (D.9).

D.3 Tensors and Shapes

The word “tensor” sounds intimidating, but the idea is simple: a tensor is a multi-dimensional array. You already know the special cases:

Dimensions	Name	Shape example	What it holds
0	Scalar	[]	A single number
1	Vector	[3]	A list of numbers
2	Matrix	[3, 4]	A table (rows × columns)
3	3-D tensor	[2, 3, 4]	A stack of matrices (e.g., RGB image)
4	4-D tensor	[8, 3, 256, 256]	A batch of images

Reshaping. You can rearrange the elements of a tensor without copying the data:

```

x = torch.arange(12)          # tensor([ 0,  1,  2, ..., 11])  shape [12]
y = x.view(3, 4)              # shape [3, 4] – same data, different layout
z = x.view(2, 2, 3)           # shape [2, 2, 3]

print(y)
# tensor([[ 0,  1,  2,  3],
#         [ 4,  5,  6,  7],
#         [ 8,  9, 10, 11]])

```

Slicing and indexing works like Python lists and NumPy arrays:

```

A = torch.tensor([[10, 20, 30],
                  [40, 50, 60]])

print(A[0, :])      # tensor([10, 20, 30]) ← first row
print(A[:, 1])      # tensor([20, 50]) ← second column
print(A[1, 2])      # tensor(60) ← single element

```

Broadcasting. When two tensors have different shapes, PyTorch automatically expands the smaller one to match — the same rules as NumPy. The key rule: dimensions are compared from the right, and a dimen-

sion of size 1 can stretch to match any size.

```
V = torch.tensor([[1.0], [2.0], [3.0]]) # shape [3, 1]
w = torch.tensor([10.0, 20.0, 30.0]) # shape [3]

# V + w broadcasts: [3,1] + [3] → [3,1] + [1,3] → [3,3]
print(V + w)
# tensor([[11., 21., 31.],
#         [12., 22., 32.],
#         [13., 23., 33.]])
```

D.4 Data Types and Precision

Every tensor has a **dtype** that controls how much memory each number uses and how precisely it is stored:

dtype	Bits	Significant digits	Typical use
<code>torch.float32</code>	32	~7	Machine learning (default)
<code>torch.float64</code>	64	~16	Scientific computing, PDE solvers
<code>torch.int64</code>	64	exact (integers)	Indices, counts
<code>torch.bool</code>	8	True / False	Masks, conditions

Machine learning uses `float32` because neural networks are tolerant of rounding — a gradient that is off by 0.001% still points in the right direction. PDE solvers are not tolerant. Consider a single forward-Euler step of the monodomain equation: $V^{n+1} = V^n + \Delta t \cdot K \cdot V^n$. The stiffness matrix K has entries of order $D/\Delta x^2 \sim 10^4 \text{ s}^{-1}$, and $\Delta t \sim 10^{-5} \text{ s}$. The product $\Delta t \cdot K \cdot V$ requires at least 10 significant digits to avoid catastrophic cancellation when subtracted from V^n . Seven digits (`float32`) is not enough.

```
# The default is float32:
x = torch.randn(5)
print(x.dtype) # torch.float32

# For scientific computing, use float64:
V = torch.zeros(1000, dtype=torch.float64)

# Or change the global default:
torch.set_default_dtype(torch.float64)
V = torch.zeros(1000) # now float64 by default
```

⚠ Key Insight

Rule of thumb. If your code involves physical units, differential equations, or any computation where small differences between large numbers matter, use `float64`. If you are training a neural network, `float32` (or even `float16`) is fine.

D.5 Common Operations Cheat Sheet

This section is a quick reference. Skim it now; come back when you need a specific operation.

Creating tensors.

```
torch.zeros(3, 4) # 3x4 matrix of zeros
torch.ones(5) # vector of ones
torch.arange(0, 10, 2) # tensor([0, 2, 4, 6, 8])
torch.linspace(0, 1, 11) # 11 points from 0 to 1 inclusive
torch.randn(3, 3) # 3x3 matrix of standard-normal random numbers
torch.eye(4) # 4x4 identity matrix
```

Reductions.

```
x = torch.tensor([3.0, 1.0, 4.0, 1.0, 5.0])

x.sum()          # tensor(14.)
x.mean()         # tensor(2.8000)
x.max()          # tensor(5.)
x.min()          # tensor(1.)
x.argmax()       # tensor(4)      ← index of the maximum
x.std()          # tensor(1.7889) ← standard deviation
```

Conditional and clamping.

```
V = torch.tensor([-120.0, -40.0, 20.0, 80.0])

torch.clamp(V, min=-85.0, max=40.0) # tensor([-85., -40., 20., 40.])
torch.where(V > 0, V, torch.zeros_like(V)) # ReLU: tensor([0., 0., 20., 80.])
```

Stacking and concatenating.

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

torch.cat([a, b])      # tensor([1, 2, 3, 4, 5, 6]) ← concatenate
torch.stack([a, b])    # tensor([[1, 2, 3], [4, 5, 6]]) ← stack into new dimension
```

In-place operations. Any operation with an underscore suffix modifies the tensor directly, saving memory:

```
x = torch.zeros(5)
x.add_(1.0)      # x is now tensor([1., 1., 1., 1., 1.]) – no copy made
x.mul_(3.0)      # x is now tensor([3., 3., 3., 3., 3.])
```

Math functions.

```
x = torch.tensor([0.0, 1.0, 2.0])

torch.exp(x)      # tensor([1.0000, 2.7183, 7.3891])
torch.log(x + 1)  # tensor([0.0000, 0.6931, 1.0986])
torch.sin(x)      # tensor([0.0000, 0.8415, 0.9093])
torch.abs(x - 1)  # tensor([1., 0., 1.])
```

D.6 Linear Algebra

PyTorch includes a full linear algebra library in `torch.linalg`. If you have used `numpy.linalg`, the API is nearly identical.

Solving $A\mathbf{x} = \mathbf{b}$.

```
A = torch.tensor([[2., 1., -1.],
                  [-3., -1., 2.],
                  [-2., 1., 2.]], dtype=torch.float64)
b = torch.tensor([8., -11., -3.], dtype=torch.float64)

x = torch.linalg.solve(A, b)
print(x)  # tensor([ 2.,  3., -1.]) ← the solution

# Verify: A @ x should equal b
print(A @ x)  # tensor([ 8., -11., -3.])
```

Norms, determinants, eigenvalues.

```

M = torch.tensor([[4., 2.], [1., 3.]], dtype=torch.float64)

torch.linalg.norm(M)          # Frobenius norm: tensor(5.4772)
torch.linalg.det(M)           # determinant: tensor(10.)
torch.linalg.inv(M)           # inverse: tensor([[ 0.3, -0.2], [-0.1,  0.4]])

eigvals, eigvecs = torch.linalg.eig(M)
print(eigvals)               # tensor([5.+0.j, 2.+0.j]) ← eigenvalues (complex)

```

Intuition

When to use `linalg.solve` vs. `linalg.inv`. Never compute $\mathbf{A}^{-1}\mathbf{b}$ by inverting the matrix first. `linalg.solve(A, b)` is faster, more numerically stable, and uses less memory. The explicit inverse is only useful if you need $\mathbf{A}^{-1}$ for a different reason.

D.7 Sparse Matrices

Most matrices in scientific computing are **sparse**: the vast majority of their entries are zero. Think of the stiffness matrix for the monodomain equation on a 512×512 grid. It has 262,144 rows and 262,144 columns — over 68 billion entries — but each row contains at most five nonzeros (the node itself and its four neighbours). Storing the full matrix would need 512 GB of memory. Storing only the nonzeros needs about 30 MB. Every PDE solver in this textbook lives or dies on that ratio.

PyTorch stores sparse matrices in **COO (Coordinate) format**: each nonzero entry is described by three numbers — its row index, column index, and value. You build a COO tensor by providing the indices as a `[2, nnz]` integer tensor and the values as a `[nnz]` float tensor:

```

import torch

# A 4x4 matrix with 6 nonzero entries
rows = [0, 0, 1, 1, 2, 3]
cols = [0, 1, 0, 1, 2, 3]
vals = [4.0, -1.0, -1.0, 4.0, 4.0, 4.0]

indices = torch.tensor([rows, cols], dtype=torch.long) # shape [2, 6]
values = torch.tensor(vals, dtype=torch.float64)      # shape [6]
S = torch.sparse_coo_tensor(indices, values, size=(4, 4))

print(S.to_dense())
# tensor([[ 4., -1.,  0.,  0.],
#         [-1.,  4.,  0.,  0.],
#         [ 0.,  0.,  4.,  0.],
#         [ 0.,  0.,  0.,  4.]])

```

Coalescing. A COO tensor can contain duplicate index pairs. For example, during finite-element assembly, two neighbouring elements may both contribute to entry (i, j) . Calling `.coalesce()` sums duplicates and sorts the indices — most sparse operations require a coalesced tensor:

```

# Two entries at (0, 0): 2.0 and 3.0
idx = torch.tensor([[0, 0], [0, 0]], dtype=torch.long)
val = torch.tensor([2.0, 3.0], dtype=torch.float64)
A = torch.sparse_coo_tensor(idx, val, size=(2, 2))

print(A.is_coalesced()) # False
A = A.coalesce()
print(A.values())        # tensor([5.]) ← 2.0 + 3.0 merged
print(A.is_coalesced()) # True

```

Inspecting a sparse tensor. Once coalesced, you can read back the internal storage:

```

print(S.indices()) # tensor([[0, 0, 1, 1, 2, 3],
                        #      [0, 1, 0, 1, 2, 3]])      ← [2, nnz]
print(S.values()) # tensor([ 4., -1., -1.,  4.,  4.,  4.])
print(S.shape)    # torch.Size([4, 4])
print(S.is_sparse) # True
print(S._nnz())   # 6

```

Operations on sparse tensors

The operations you need for PDE solvers form a small but essential set. Here is the complete catalogue:

1. Sparse $\times$ dense vector — the workhorse of every time-stepping loop. This computes $\mathbf{y} = \mathbf{A}\mathbf{x}$ where $\mathbf{A}$ is sparse and $\mathbf{x}$ is a dense column vector:

```

x = torch.randn(4, 1, dtype=torch.float64) # column vector [4, 1]
y = torch.sparse.mm(S, x)                   # sparse matvec → dense [4, 1]

# For a 1-D vector (no column dimension), reshape first:
x_flat = torch.randn(4, dtype=torch.float64)
y_flat = torch.sparse.mm(S, x_flat.unsqueeze(1)).squeeze(1)

```

2. Scalar $\times$ sparse — scales every nonzero value. The sparsity pattern is unchanged:

```

dt = 0.01
K_scaled = dt * S          # every value multiplied by dt
K_scaled = S * 0.5         # also works: sparse * scalar

```

3. Sparse + sparse — adds two sparse matrices with (possibly different) sparsity patterns. The result has the union of both patterns:

```

# Create a sparse identity
def speye(n, dtype=torch.float64):
    idx = torch.arange(n, dtype=torch.long)
    return torch.sparse_coo_tensor(
        torch.stack([idx, idx]),
        torch.ones(n, dtype=dtype),
        size=(n, n)
    ).coalesce()

I = speye(4)
A = (2.0 * I + 0.5 * S).coalesce() # sparse + sparse → sparse (coalesce after!)

```

4. Sparse – sparse — works the same way. This is how you build the implicit time-stepping matrix $\mathbf{A} = \chi \mathbf{C}_m \mathbf{I} - \frac{1}{2} \Delta t \mathbf{K}$:

```

chi_Cm = 1.0
dt = 0.01
A_implicit = (chi_Cm * I - 0.5 * dt * S).coalesce()

```

5. Convert to/from dense — useful for debugging small matrices. Never do this for production-sized matrices:

```

dense = S.to_dense() # sparse → dense (full matrix in memory)
sparse_again = dense.to_sparse() # dense → COO sparse

```

△ Key Insight

Always coalesce after arithmetic. Operations like `A + B` on sparse tensors concatenate the index arrays without merging duplicates. If you skip `.coalesce()`, the next sparse operation may silently produce wrong results or raise an error. Rule of thumb: coalesce immediately after every `+`, `-`, or any operation that combines two sparse tensors.

Worked example: the 5-point Laplacian

The discrete Laplacian on a uniform grid approximates $\nabla^2 V$ by comparing each node's voltage to its four neighbours:

$$\nabla^2 V_{i,j} \approx \frac{V_{i-1,j} + V_{i+1,j} + V_{i,j-1} + V_{i,j+1} - 4V_{i,j}}{\Delta x^2}$$

Written as a matrix-vector product $\mathbf{y} = \mathbf{K}\mathbf{V}$, each row of $\mathbf{K}$ has at most five nonzero entries: $+1/\Delta x^2$ for each neighbour that exists, and $-n_{\text{nbrs}}/\Delta x^2$ on the diagonal (where n_{nbrs} is 2, 3, or 4 depending on whether the node is a corner, edge, or interior point). Here is the assembly:

```
def build_laplacian(N, dx, dtype=torch.float64):
    """Assemble the 5-point Laplacian on an N×N grid as a sparse COO tensor."""
    rows, cols, vals = [], [], []
    inv_dx2 = 1.0 / dx**2

    for j in range(N):
        for i in range(N):
            node = j * N + i
            nbrs = 0
            for di, dj in [(1,0), (-1,0), (0,1), (0,-1)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < N and 0 <= nj < N:
                    rows.append(node)
                    cols.append(nj * N + ni)
                    vals.append(inv_dx2)
                    nbrs += 1
            # Diagonal: negative sum of off-diagonals
            rows.append(node)
            cols.append(node)
            vals.append(-nbrs * inv_dx2)

    indices = torch.tensor([rows, cols], dtype=torch.long)
    values = torch.tensor(vals, dtype=dtype)
    n = N * N
    return torch.sparse_coo_tensor(indices, values, size=(n, n)).coalesce()

K = build_laplacian(N=4, dx=0.025)
print(f"Matrix size: {K.shape[0]}×{K.shape[1]},  nonzeros: {K._nnz()}")
# Matrix size: 16×16,  nonzeros: 56
```

A single sparse matrix-vector product now computes the discrete Laplacian of any voltage vector in one call:

```
V = torch.randn(16, 1, dtype=torch.float64)
lap_V = torch.sparse.mm(K, V)      # ∇²V for all 16 nodes simultaneously
```

⚙ Engine V5.4 Connection

Engine V5.4 assembly. The `build_laplacian` loop above is the pedagogical version. Engine V5.4's `FDMDiscretization` does the same job but vectorized — it never loops over nodes in Python. The FEM assembler goes further, using `torch.einsum` (Section D.9) to compute all element stiffness matrices in a single batched operation, then uses duplicate COO indices with `.coalesce()` to merge overlapping contributions into the global matrix.

D.8 GPU Acceleration

Everything so far has run on your CPU. Moving to a GPU requires exactly three changes: (1) choose a device, (2) place your tensors there, (3) ... there is no step 3. The math is the same.

```
# Step 1: choose a device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)    # 'cuda' if you have an NVIDIA GPU, else 'cpu'

# Step 2: create tensors on that device (or move existing ones)
V = torch.zeros(1000, dtype=torch.float64, device=device)
K = build_laplacian(N=32, device=device)    # your sparse matrix, on GPU

# All operations now run on the GPU automatically:
KV = torch.sparse.mm(K, V.unsqueeze(1)).squeeze(1)
V_new = V + dt * KV
```

The algorithm reads identically whether `device` is `'cpu'` or `'cuda'`. This is PyTorch's key selling point for scientific computing: **write once, run anywhere**.

To move data between CPU and GPU:

```
x_gpu = x_cpu.to('cuda')    # CPU → GPU (copies data across PCIe bus)
x_cpu = x_gpu.cpu()         # GPU → CPU (copies data back)
x_gpu = x_gpu.to('cuda')    # already on GPU – no-op, no copy
```

▲ Key Insight

Minimise transfers. The PCIe bus between CPU and GPU is slow compared to GPU computation. The best practice is to move your data to the GPU once at the start, do all computation there, and move results back only when you need to save or plot. Avoid calling `.cpu()` inside inner loops.

D.9 Advanced Operations for Scientific Computing

Sections B.1–B.8 gave you the fundamentals: tensors, arithmetic, sparse matrices, and GPUs. That is enough to *understand* a PDE solver. But to *write* one — to build something like Engine V5.4 from scratch — you need a handful of intermediate-level operations that bridge the gap between “PyTorch as a NumPy replacement” and “PyTorch as a simulation backend.” This section collects them in one place.

Piecewise functions with `torch.where`

Ionic models are full of piecewise-defined functions. The sodium gate inactivation variable h in TTP06, for instance, has different rate expressions above and below $V = -40$ mV. In plain Python you would write an `if/else` — but your voltage array holds 262,144 values, each needing a different branch. `torch.where` evaluates both branches for the entire array and selects element-by-element:

```
V = torch.linspace(-100, 60, 1000, dtype=torch.float64)

# αh: one formula below -40 mV, zero above
alpha_h = torch.where(
    V < -40.0,
    0.057 * torch.exp(-(V + 80.0) / 6.8),    # branch A: V < -40
    torch.zeros_like(V)                    # branch B: V ≥ -40
)

# βh: different formula in each branch
beta_h = torch.where(
    V < -40.0,
    2.7 * torch.exp(0.079 * V) + 3.1e5 * torch.exp(0.3485 * V),
```

```
0.77 / (0.13 * (1.0 + torch.exp(-(V + 10.66) / 11.1)))
)
```

This is the most common pattern in cardiac ionic model code. Engine V5.4’s TTP06 and ORd implementations use `torch.where` more than thirty times each — every gating variable with voltage-dependent kinetics has at least one call.

△ Key Insight

Both branches are always evaluated. Unlike an `if` statement, `torch.where` computes both the true and false tensors before selecting. If a branch contains a division or logarithm that could produce `inf` or `NaN` for out-of-range inputs, those bad values will exist briefly in memory. The selection discards them, but if you need to avoid the computation entirely (e.g., for performance or to prevent floating-point exceptions), use masking with separate indexed assignments instead.

Circular shifts with `torch.roll`

The lattice-Boltzmann streaming step (Chapter 18) moves distribution functions along lattice velocities. On a periodic grid, moving every element one position to the right is a circular shift — the last element wraps around to the first position. `torch.roll` does exactly this:

```
x = torch.tensor([10., 20., 30., 40., 50.])

print(torch.roll(x, shifts=1))      # tensor([50., 10., 20., 30., 40.]) ← shift right
print(torch.roll(x, shifts=-1))     # tensor([20., 30., 40., 50., 10.]) ← shift left
```

On a 2-D grid, you specify which axis to shift along. The D2Q5 streaming step is five `torch.roll` calls — one per lattice velocity:

```
# f has shape [5, Nx, Ny] – 5 velocities on an NxNy grid
# After collision, stream each population to its neighbour:
f_new = torch.zeros_like(f)
f_new[0] = f[0]                                # rest: no movement
f_new[1] = torch.roll(f[1], shifts=-1, dims=0)  # +x direction
f_new[2] = torch.roll(f[2], shifts=+1, dims=0)  # -x direction
f_new[3] = torch.roll(f[3], shifts=-1, dims=1)  # +y direction
f_new[4] = torch.roll(f[4], shifts=+1, dims=1)  # -y direction
```

The entire streaming step — moving a quarter-million distribution functions to their new homes — takes five lines and runs on the GPU without a single Python loop.

Batched contractions with `torch.einsum`

Einstein summation notation is a compact way to describe tensor contractions. In the FEM assembler, each triangular element produces a 3×3 local stiffness matrix from two gradient vectors **b** and **c** (the spatial derivatives of the basis functions). With 100,000 elements, you need 100,000 outer products in parallel. `torch.einsum` expresses this in a single line:

```
# b, c have shape [num_elements, 3] – gradient components per element
# We want K_local[e, i, j] = b[e,i]*b[e,j] + c[e,i]*c[e,j] for all e

K_local = (
    torch.einsum('ei,ej->eij', b, b) +  # b_i * b_j for each element
    torch.einsum('ei,ej->eij', c, c)     # c_i * c_j for each element
)
# K_local has shape [num_elements, 3, 3] – all local matrices at once
```

The string `'ei,ej->eij'` reads: “for each element `e`, take the outer product of index `i` with index `j`.” The `e` dimension is the batch axis — every element is processed independently and in parallel.

Common `einsum` patterns you will encounter:

```
# Matrix multiply:      C[i,j] = Σ_k A[i,k] * B[k,j]
C = torch.einsum('ik,kj->ij', A, B)

# Batched dot product:  d[e] = Σ_i a[e,i] * b[e,i]
d = torch.einsum('ei,ei->e', a, b)

# Trace of a matrix:    t = Σ_i A[i,i]
t = torch.einsum('ii->', A)

# Batched outer product:  O[e,i,j] = a[e,i] * b[e,j]
O = torch.einsum('ei,ej->eij', a, b)
```

Accumulation with `scatter_add_`

When you assemble a global vector from local contributions — for example, summing element-level forces into a global force vector — you need to add multiple values to the same index. `scatter_add_` does this in one vectorized call:

```
# 4 elements, each contributing to 3 global nodes
element_nodes = torch.tensor([0, 1, 2,      # element 0 → nodes 0,1,2
                              1, 2, 3,      # element 1 → nodes 1,2,3
                              0, 2, 3,      # element 2 → nodes 0,2,3
                              1, 3, 4])     # element 3 → nodes 1,3,4
local_values = torch.tensor([1., 2., 3.,
                              4., 5., 6.,
                              7., 8., 9.,
                              10., 11., 12.], dtype=torch.float64)

# Accumulate into global vector of size 5
global_vec = torch.zeros(5, dtype=torch.float64)
global_vec.scatter_add_(0, element_nodes, local_values)
print(global_vec)  # tensor([ 8., 16., 16., 27., 12.])
# Node 0: 1+7=8, Node 1: 2+4+10=16, Node 2: 3+5+8=16, ...
```

The Chebyshev solver in Engine V5.4 uses `scatter_add_` to compute Gershgorin circle radii — extracting the diagonal and off-diagonal row sums of a sparse matrix to bound its eigenvalues, without converting to dense format.

In-place operations and solver workspaces

A PDE simulation runs for millions of timesteps. If every linear-solver iteration allocates fresh tensors for the residual, search direction, and matrix-vector product, the garbage collector will spend more time freeing memory than the GPU spends doing math.

The solution is a **pre-allocated workspace**: create the buffers once, then reuse them every iteration with in-place operations. Here is the pattern used in Engine V5.4's PCG solver, stripped to its essentials:

```
class SimplePCG:
    def __init__(self, n, device, dtype):
        # Pre-allocate workspace – done ONCE
        self.r = torch.zeros(n, device=device, dtype=dtype)  # residual
        self.z = torch.zeros(n, device=device, dtype=dtype)  # preconditioned residual
        self.p = torch.zeros(n, device=device, dtype=dtype)  # search direction
        self.Ap = torch.zeros(n, device=device, dtype=dtype) # matrix-vector product

    def solve(self, A, b, x):
        r, z, p, Ap = self.r, self.z, self.p, self.Ap

        # r = b - A*x (in-place)
        r.copy_(b)  # r ← b
        r.sub_(torch.sparse.mm(A, x.unsqueeze(1)).squeeze(1))  # r ← r - Ax
```

```

z.copy_(r)          #  $z \leftarrow M^{-1}r$  (no preconditioner here)
p.copy_(z)          #  $p \leftarrow z$ 

for k in range(100):
    Ap.copy_(torch.sparse.mm(A, p.unsqueeze(1)).squeeze(1)) #  $Ap \leftarrow A \cdot p$ 
    alpha = torch.dot(r, z) / torch.dot(p, Ap)

    x.add_(p, alpha=alpha)      #  $x \leftarrow x + \alpha \cdot p$  (in-place)
    r.sub_(Ap, alpha=alpha)     #  $r \leftarrow r - \alpha \cdot Ap$  (in-place)

    if torch.norm(r) < 1e-10:
        break

    z.copy_(r)          #  $z \leftarrow M^{-1}r$ 
    beta = torch.dot(r, z) / torch.dot(r, z)
    p.mul_(beta).add_(z)     #  $p \leftarrow \beta \cdot p + z$  (chain: mul then add)

return x

```

The key operations are `.copy_()` (overwrite contents), `.add_(tensor, alpha=scalar)` (scaled accumulate), `.sub_()` (scaled subtract), and `.mul_()` (scale in place). The underscore suffix always means “modify this tensor directly, do not allocate a new one.”

Intuition

Why this matters. A naive implementation that writes `x = x + alpha * p` allocates a new tensor for `alpha * p`, another for the sum, then discards the old `x`. At 200,000 nodes in `float64`, that is 1.6 MB per line, hundreds of lines per timestep, millions of timesteps — terabytes of transient allocations that the garbage collector must clean up. In-place operations produce zero allocations after the initial workspace setup.

NumPy interoperability

PyTorch tensors and NumPy arrays can share the same memory on the CPU, making conversion essentially free. You will use this constantly for I/O — loading meshes, saving output, calling Matplotlib:

```

import numpy as np

# NumPy → PyTorch (shares memory – no copy on CPU)
arr = np.array([1.0, 2.0, 3.0])
t = torch.from_numpy(arr)
t[0] = 99.0
print(arr[0])      # 99.0 ← arr changed too! (shared memory)

# PyTorch → NumPy (CPU tensors only)
t2 = torch.tensor([4.0, 5.0, 6.0])
arr2 = t2.numpy()

# GPU tensor → NumPy (must go through CPU first)
if torch.cuda.is_available():
    t_gpu = torch.randn(5, device='cuda')
    arr3 = t_gpu.detach().cpu().numpy()    # the canonical pattern

```

The three-step chain `.detach().cpu().numpy()` appears throughout Engine V5.4 whenever simulation results are saved to disk or passed to plotting libraries. `.detach()` disconnects from the autograd tape (even though the engine disables gradients, this call is a safety net), `.cpu()` copies from GPU to CPU memory, and `.numpy()` wraps the CPU data as a NumPy array.

Coordinate grids with `torch.meshgrid`

Setting up a structured mesh requires coordinate arrays for every node. `torch.meshgrid` creates these from 1-D coordinate vectors — the same idea as NumPy’s `np.meshgrid`:

```
# A 3x4 grid from 0 to 1 cm in x, 0 to 1.5 cm in y
x = torch.linspace(0, 1.0, 3, dtype=torch.float64) # [0.0, 0.5, 1.0]
y = torch.linspace(0, 1.5, 4, dtype=torch.float64) # [0.0, 0.5, 1.0, 1.5]

xx, yy = torch.meshgrid(x, y, indexing='ij')
print(xx.shape) # torch.Size([3, 4])

# xx holds the x-coordinate of each node:
# tensor([[0.0, 0.0, 0.0, 0.0],
#         [0.5, 0.5, 0.5, 0.5],
#         [1.0, 1.0, 1.0, 1.0]])

# To get a flat list of (x,y) pairs:
nodes = torch.stack([xx.flatten(), yy.flatten()], dim=1) # shape [12, 2]
```

△ Key Insight

Always use `indexing='ij'`. The default in older PyTorch versions is `'xy'` (Cartesian convention), which transposes the output. Scientific computing expects matrix indexing (`'ij'`), where the first axis is rows (x) and the second is columns (y). Engine V5.4 passes `indexing='ij'` in every `meshgrid` call.

Fourier transforms with `torch.fft`

The FFT solver (Chapter 11, §11.6) exploits the fact that the discrete Laplacian is diagonal in the frequency domain. PyTorch provides a full FFT library:

```
# 1-D FFT
x = torch.randn(64, dtype=torch.float64)
X = torch.fft.fft(x) # complex-valued frequency coefficients
x_back = torch.fft.ifft(X) # inverse FFT recovers the original

# 2-D FFT (for a 2-D voltage grid)
V_grid = torch.randn(32, 32, dtype=torch.float64)
V_freq = torch.fft.fft2(V_grid)
V_back = torch.fft.ifft2(V_freq)

# Wavenumber grid (for building spectral operators)
k = torch.fft.fftfreq(32, d=0.025) # frequencies for dx=0.025 cm
```

Engine V5.4's FFT solver computes the eigenvalues of the discrete Laplacian analytically, transforms the right-hand side with a DCT (discrete cosine transform), divides by the eigenvalues in frequency space, and transforms back — solving the implicit diffusion step in $O(N \log N)$ time with no iterative convergence checks.

D.10 Automatic Differentiation (Autograd)

Automatic differentiation is PyTorch's signature feature and the reason it dominates machine learning. The idea: PyTorch can record every arithmetic operation you perform on a tensor, building an invisible **computation graph**. When you ask for gradients, it walks backward through the graph applying the chain rule — automatically, exactly, and efficiently.

Think of it like a receipt tape at a cash register. Every operation (add, multiply, exp, matmul) prints a line on the tape. When you call `.backward()`, PyTorch reads the tape in reverse, computing $\partial(\text{output})/\partial(\text{each input})$ at every step.

```
import torch

# Tell PyTorch to track operations on w
w = torch.tensor(2.0, requires_grad=True)
```

```

x = torch.tensor(3.0)
y = torch.tensor(5.0)

L = (w * x - y) ** 2          # L = (2*3 - 5)2 = 1.0

L.backward()                  # walk the tape backward

print(w.grad)                 # tensor(6.)
# Check by hand: dL/dw = 2(wx - y) · x = 2(6-5)(3) = 6 ✓

```

The `requires_grad=True` flag turns on recording. The `.backward()` call traverses the graph and deposits the gradient in `w.grad`. This is the same mechanism that trains neural networks with millions of parameters — the only difference is scale.

When you do *not* need gradients (e.g., running a forward simulation), you can disable the tape entirely to save memory and speed:

```

with torch.no_grad():
    # Everything inside this block runs without recording the computation graph.
    V_new = V + dt * KV      # fast, no tape overhead

```

D.11 Building Neural Networks with `nn.Module`

PyTorch’s neural network library lives in `torch.nn`. The core abstraction is `nn.Module`: a class where you define your network’s layers in `__init__` and the forward pass in `forward`.

```

import torch
import torch.nn as nn

class TinyNet(nn.Module):
    def __init__(self, n_in, n_hidden, n_out):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_in, n_hidden),      # first layer: n_in → n_hidden
            nn.Tanh(),                      # activation function
            nn.Linear(n_hidden, n_hidden),  # second layer
            nn.Tanh(),
            nn.Linear(n_hidden, n_out),     # output layer
        )

    def forward(self, x):
        return self.net(x)

```

`nn.Linear(m, n)` is a fully connected layer: it stores an $n \times m$ weight matrix $\mathbf{W}$ and an n -vector bias $\mathbf{b}$, and computes $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$. `nn.Tanh()` applies the hyperbolic tangent element-wise — this introduces the nonlinearity that lets a network approximate complex functions.

The training loop. Training a neural network is a four-step cycle repeated many times:

1. **Forward pass.** Feed inputs through the network, get predictions.
2. **Loss.** Measure how wrong the predictions are (e.g., mean squared error).
3. **Backward pass.** Compute gradients of the loss with respect to every weight (autograd).
4. **Optimizer step.** Nudge every weight in the direction that reduces the loss.

Here is a complete worked example: training a small network to approximate the TTP06 sodium current $I_{Na}(V, m, h, j) = G_{Na} \cdot m^3 \cdot h \cdot j \cdot (V - E_{Na})$ from Chapter 5:

```

# The "true" function we want the network to learn
def true_INa(V, m, h, j):
    G_Na = 14.838      # nA/pF
    E_Na = 67.0        # mV

```

```

    return G_Na * m**3 * h * j * (V - E_Na)

# Generate 10,000 random training samples
N = 10000
V = torch.rand(N, 1) * 160 - 100      # V in [-100, 60] mV
m = torch.rand(N, 1)                  # gating variables in [0, 1]
h = torch.rand(N, 1)
j = torch.rand(N, 1)
X = torch.cat([V, m, h, j], dim=1)     # shape [10000, 4]
y = true_INa(V, m, h, j)              # shape [10000, 1]

# Create network, loss function, and optimizer
model = TinyNet(n_in=4, n_hidden=64, n_out=1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

# Training loop
for epoch in range(2000):
    y_pred = model(X)                  # 1. forward
    loss = loss_fn(y_pred, y)          # 2. loss
    optimizer.zero_grad()              # clear old gradients
    loss.backward()                    # 3. backward
    optimizer.step()                   # 4. update weights

    if epoch % 500 == 0:
        print(f"Epoch {epoch:4d} Loss: {loss.item():.6f}")
# After ~2000 epochs: loss ≈ 0.001, relative error < 1%

```

⚠ Key Insight

Surrogates are approximations, not replacements. A trained neural network evaluates in microseconds per batch — much faster than the original formula when that formula involves expensive sub-expressions (like the GHK flux in I_{CaL}). But the surrogate introduces approximation error that accumulates over time. For production simulations where accuracy matters, always validate against the original model.

D.12 Connecting to Engine V5.4

If you have followed sections B.1 through B.11, you now understand every PyTorch concept that Engine V5.4 uses. The engine does not use neural networks or automatic differentiation — it uses PyTorch purely as a **GPU-accelerated numerical backend**. Here is the map from what you just learned to the engine’s source code:

You learned	Section	Engine V5.4 uses it in	What it does
Tensors, dtypes	B.1– B.4	Everywhere	All state variables, including <code>float64</code>
Element-wise ops	B.2, B.5	Ionic model solvers	Gate update: $m^{n+1} = n$ — all elements
Reductions	B.5	LBM collision step	$V_m = \sum_i$ distribution voltage
<code>linspace</code> , <code>clamp</code>	B.5	<code>cardiac_sim/ionic/lut.py</code>	Lookup table on a fine grid via <code>clamp</code>
Sparse COO tensors	B.7	<code>cardiac_sim/simulation/classical/discretization_scheme/fdm.py</code>	Stiffness matrix COO, implicit $A = \chi C_m$
<code>torch.sparse.mm</code>	B.7	All diffusion solvers	The sparse $K \cdot V$ at each step
Device management	B.8	<code>cardiac_sim/utils/backend.py</code>	The Backend GPU/CPU component
<code>torch.no_grad()</code>	B.10	All solver loops	Disables autograd; has no learning recording

Notice what is *missing* from the engine: `nn.Module`, `nn.Linear`, `loss.backward()`, optimizers. Engine V5.4 solves the monodomain and bidomain equations with classical numerical methods (finite differences, finite elements, Crank-Nicolson, LBM). Every coefficient comes from the physics, not from training data. PyTorch is used as a numerical computing library, not a machine-learning framework — and that is perfectly fine. It is, in fact, the cleanest way to get GPU-accelerated sparse linear algebra in Python.

⚙ Engine V5.4 Connection

The Backend class. Every simulation in Engine V5.4 begins by instantiating `Backend(device='auto')`. This checks `torch.cuda.is_available()` and stores a `torch.device`. A helper function `ensure_tensor(x, device, dtype)` converts NumPy arrays, Python lists, and existing tensors to the correct device and dtype in a single call — this is the most-called utility in the engine. When you see `.to(device)` in the source, you now know exactly what it does.

⚙ Engine V5.4 Connection

The LookupTable class. The ionic solvers in `cardiac_sim/ionic/lut.py` use `torch.linspace` to create a voltage grid of 16,001 points from **−100** to **+60** mV, then precompute all gate steady-states and time constants on that grid. At each timestep, a single vectorized lookup (using `clamp` + integer indexing + linear interpolation) retrieves all 18 parameters for every node — replacing 4.7 million transcendental function evaluations per timestep with memory reads.

D.13 Where Machine Learning Meets Cardiac Modeling

The neural network in B.11 was a toy example, but the idea of combining machine learning with physics-based simulation is an active and rapidly growing research area. Three directions are particularly relevant to this textbook:

Physics-Informed Neural Networks (PINNs). Instead of training on data alone, a PINN embeds the PDE itself into the loss function. For the monodomain equation (eq. 7.1), the loss penalizes violations of $\chi C_m \partial V_m / \partial t = \nabla \cdot (D \nabla V_m) - \chi I_{ion}$. The network learns a solution $V_m(\mathbf{x}, \mathbf{y}, t)$ that approximately satisfies the PDE without ever assembling a stiffness matrix. PINNs are mesh-free and handle irregular geometries naturally, but they struggle with the sharp wavefronts and fast timescales of cardiac action potentials.

Neural ODE surrogates. The most expensive part of a cardiac simulation is the ionic model — eighteen coupled ODEs (TTP06) or forty-one (ORd), evaluated at every node at every timestep. A neural ODE replaces this system with a learned vector field $ds/dt = f_\theta(\mathbf{s}, \mathbf{V})$ trained on trajectories from the original model, yielding 10–50× speedups while preserving action-potential morphology to within a few percent.

ML-based classification. Given simulation output (or clinical recordings), machine learning can classify arrhythmia mechanisms, predict reentry vulnerability, or identify optimal pacing sites. The simulation engine generates training data; the ML model learns the map from tissue parameters to clinical outcomes.

Intuition

When to use ML vs. when to solve the equations. Solve the equations when you need quantitative accuracy for a specific case (drug safety, patient-specific planning). Use ML when you need to explore a vast parameter space quickly (sensitivity analysis, real-time tools) or when the forward model is too expensive to run thousands of times (uncertainty quantification, Bayesian inference). The two approaches are complementary: physics generates ground truth, ML learns to interpolate at a fraction of the cost.

For PyTorch tutorials, see pytorch.org/tutorials. For PINNs in cardiac EP, see Sahli Costabal et al. (2020). For neural ODE surrogates, see Herrero Martin et al. (2022).